

OpenShift Virtualization cookbook

Version 1, 2026-06-12

目次

目次

OpenShift Virtualization のインストールとアップグレードガイド

OpenShift

OpenShift Virtualization のインストールとアップグレードガイド
このガイドは、OpenShift Virtualization のインストールとアップグレードの手順を説明します。

(バージョン: {build-date})

目次

このガイドは、OpenShift Virtualization のインストールとアップグレードの手順を説明します。

- 対象: OpenShift Virtualization のインストールとアップグレード OpenShift 4.18 以降
- コマンド: `oc` `kubectl` `virtctl` `kustomize` `helm` `ansible`
- 形式: YAML または CLI のコマンドを使用して Kubernetes/OpenShift のリソースを管理します。

このガイドは、OpenShift Virtualization のインストールとアップグレードの手順を説明します。

目次

このガイドは、OpenShift Virtualization のインストールとアップグレードの手順を説明します。

- `virtctl` - OpenShift Virtualization の CLI ツール
- `virtctl` - OpenShift Virtualization の CLI ツール
- `virtctl` - OpenShift Virtualization の CLI ツール
- `[vm-configuration:index]` - VM の設定に関するインデックス
- `VM` のインストールとアップグレード - CPU、NUMA、Hugepages のインストールとアップグレード
- `VM` のインストールとアップグレード - OpenShift Virtualization のインストールとアップグレード
- `virtctl` - OpenShift Virtualization の CLI ツール
- `API` のインストールとアップグレード - OpenShift Virtualization の API のインストールとアップグレード

目次

- OpenShift Virtualization
 - `virtctl`
 - `virtctl`
 - `virtctl` (pdf)
 - `virtctl`

◦ [OpenShift](#)

- [KubeVirt](#) [OpenShift](#)
- [Cloud-init](#) [OpenShift](#)
- [OpenShift](#) - [OpenShift](#)

OpenShift Virtualization

OpenShift Virtualization is a container management platform that allows you to run virtual machines (VMs) on OpenShift. For more information, see the [OpenShift Virtualization](#) documentation. [Issue](#) ocp-virt-cookbook@redhat.com

Contributors

OpenShift Virtualization Onboarding Kit contributors: Alexandre Menezes, Andrea Perotti, Daphne Hoover, Josh Manning, Melvin Hillsman, Roshan Warriar, Yash Devan Oza, and the OpenShift Virtualization team.

OpenShift Virtualization is a project of the OpenShift community.

License

OpenShift Virtualization is licensed under the MIT license.



OpenShift Virtualization is a project of the OpenShift community.

OpenShift Virtualization is a project of the OpenShift community. For more information, see the [OpenShift Virtualization](#) documentation.

목차

1

OpenShift Virtualization 소개 OpenShift Virtualization 소개

2

OpenShift Virtualization 소개

- OpenShift Virtualization 소개
- OpenShift Virtualization 소개
- OpenShift Virtualization 소개
- cloud-init VM 소개

3

OpenShift Virtualization 소개

- OpenShift Virtualization Operator OpenShift 4.18+ 소개
- OpenShift Virtualization Operator RBAC 소개
- OpenShift Virtualization CLI 소개: `oc kubectl virtctl`
- Kubernetes/OpenShift 소개

4

OpenShift

OpenShift Virtualization 소개

OpenShift Virtualization 소개

OpenShift Virtualization Operator 소개

Virtctl 소개

OpenShift Virtualization 소개

OpenShift

OpenShift Virtualization 소개

Web VM 소개

OpenShift Web 소개

VM 소개

VM 소개

OpenShift Virtualization VM Cloud-init

cloud-init をインストールして VirtualMachine を cloud-init をインストール

インストール

インストールするパッケージ

- [cloud-init](#) - cloud-init をインストール
- [cloud-init](#) - VM をインストール
- [cloud-init cloud-init](#) - cloud-init をインストール

インストール

インストール

インストールする OpenShift Virtualization をインストール

インストール

OpenShift

- OpenShift 4.18
- OpenShift Virtualization をインストール

インストール

インストール

- **Cluster Admin:** OpenShift のインストール
- **Namespace Admin:** VM をインストール
- **Developer:** RBAC をインストール

CLI

OpenShift CLI (oc)

OpenShift をインストール

インストール: [OpenShift CLI](#)

インストール:

```
oc version
```

Kubernetes CLI (kubectl)

oc 使用 Kubernetes 管理 **kubectl** 使用 Kubernetes

使用 **Kubernetes** 管理

使用:

```
kubectl version
```

Virtctl

virtctl 使用 KubeVirt/OpenShift Virtualization 管理

使用 **OpenShift Virtualization** 管理

使用:

```
virtctl version
```

使用

使用

- **kustomize**: Kubernetes 管理
- **helm**: 管理
- **ansible**: 管理

使用

使用

使用

- 使用 1 个 StorageClass 管理
- VM 管理
- 管理

使用:

```
oc get storageclass
```

使用

使用

- **Container** 容器
- **Pod** 进程
- **VM** 虚拟机

11

[illegible]

```
# Check cluster status
oc get nodes

# Verify OpenShift Virtualization operator
oc get csv -n openshift-cnv

# Check available VM templates
oc get datasources -n openshift-virtualization-os-images
```

□ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- virtctl 管理虚拟机
- 虚拟机生命周期管理

--	--	--	--

- OpenShift Virtualization
- KubeVirt

OpenShift Virtualization

11

```

#####OpenShift#####OpenShift
Virtualization#####OpenShift
Virtualization#####KubeVirt#####

```

- [Installing via Web Console]
- [Installing via Command Line]

□ □ □ □

- **OpenShift 4.18+:** `oc adm upgrade`
- `oc adm upgrade`: `oc adm upgrade`

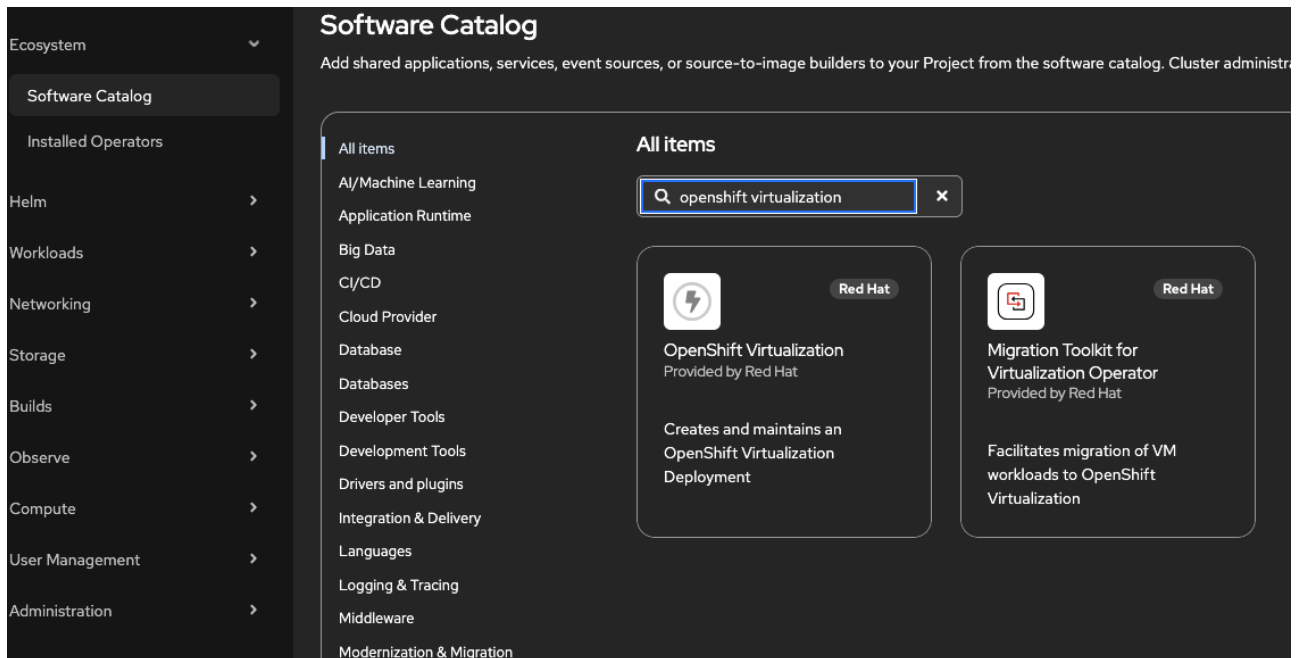
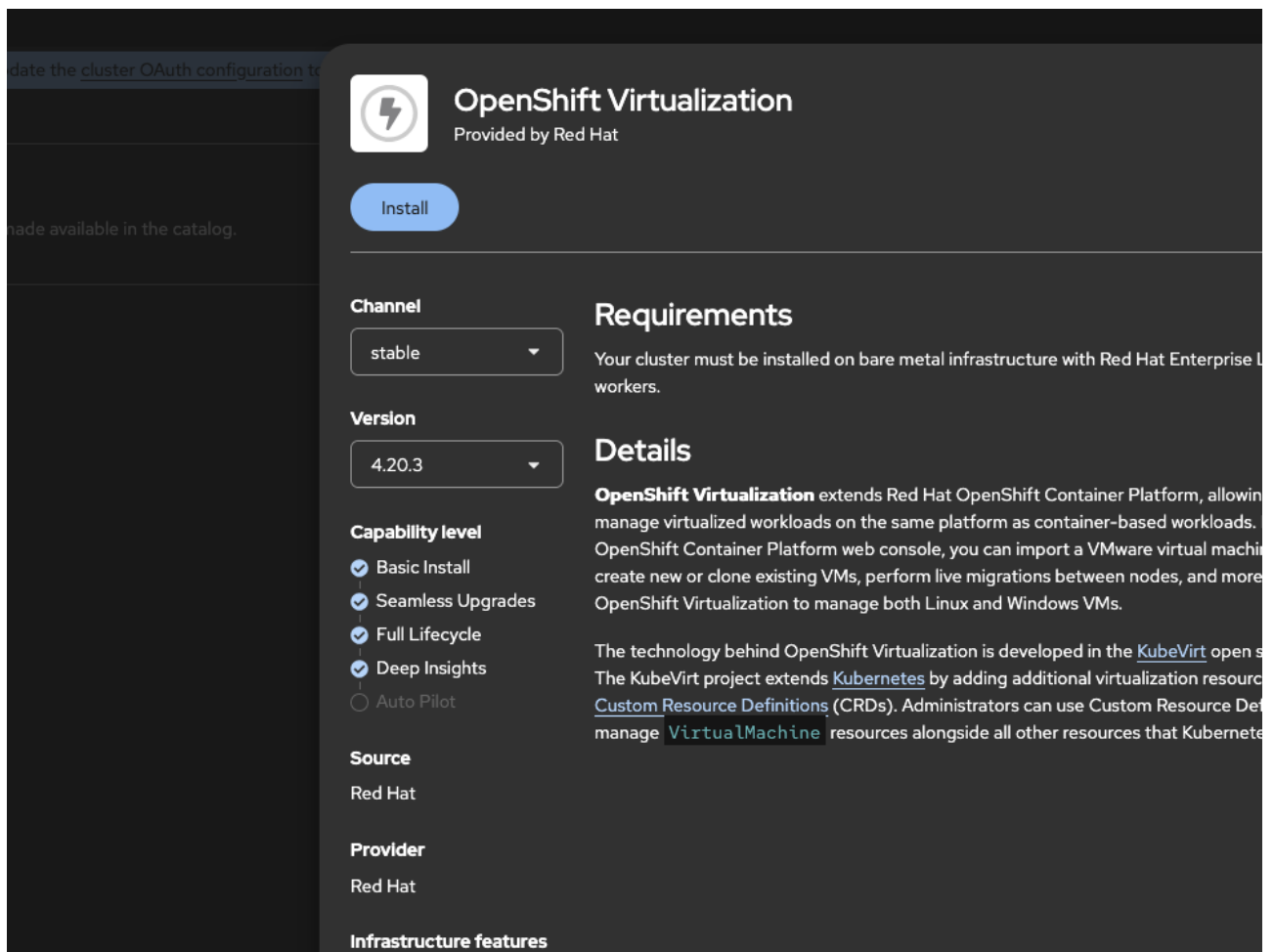
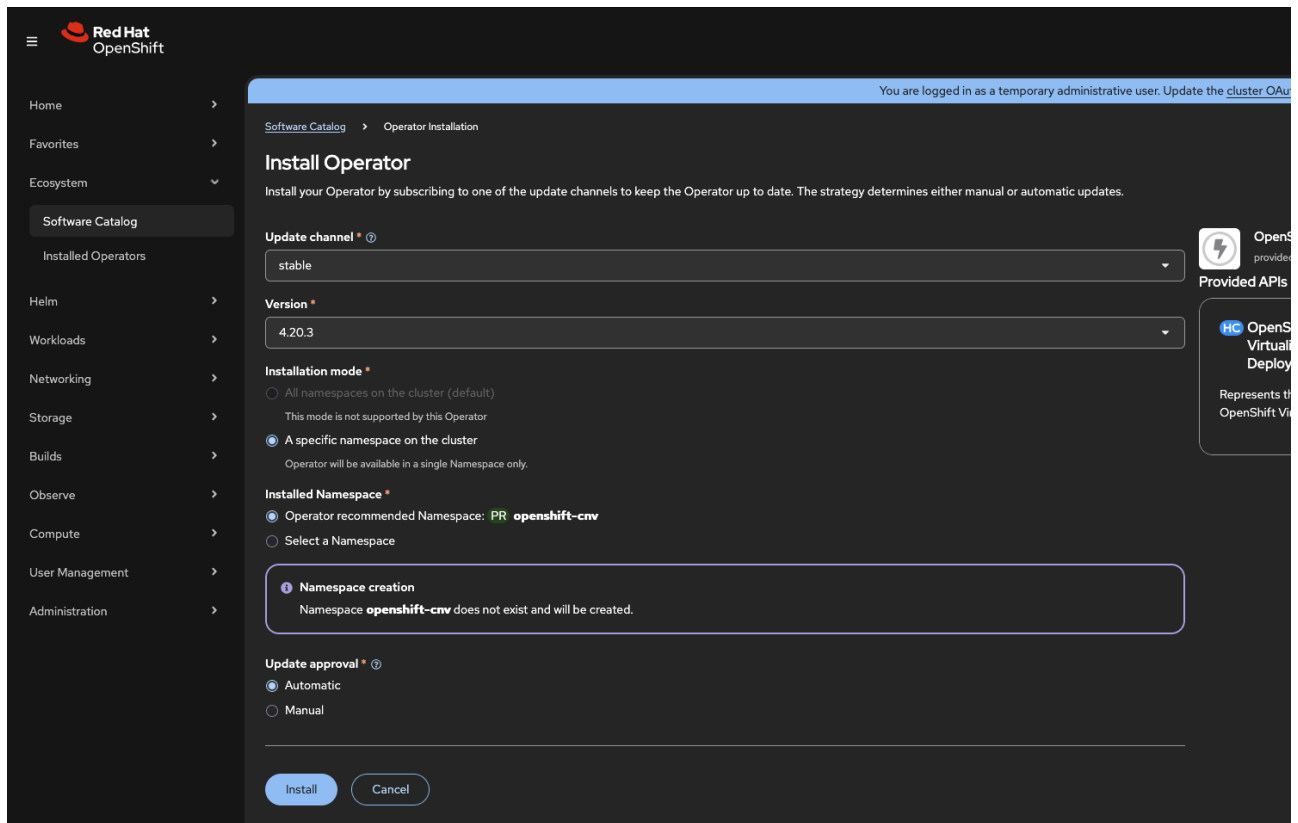


Figure 2: Software Catalog

1. Red Hat **OpenShift Virtualization**
2. **Install**



3. Install Operator **Install**

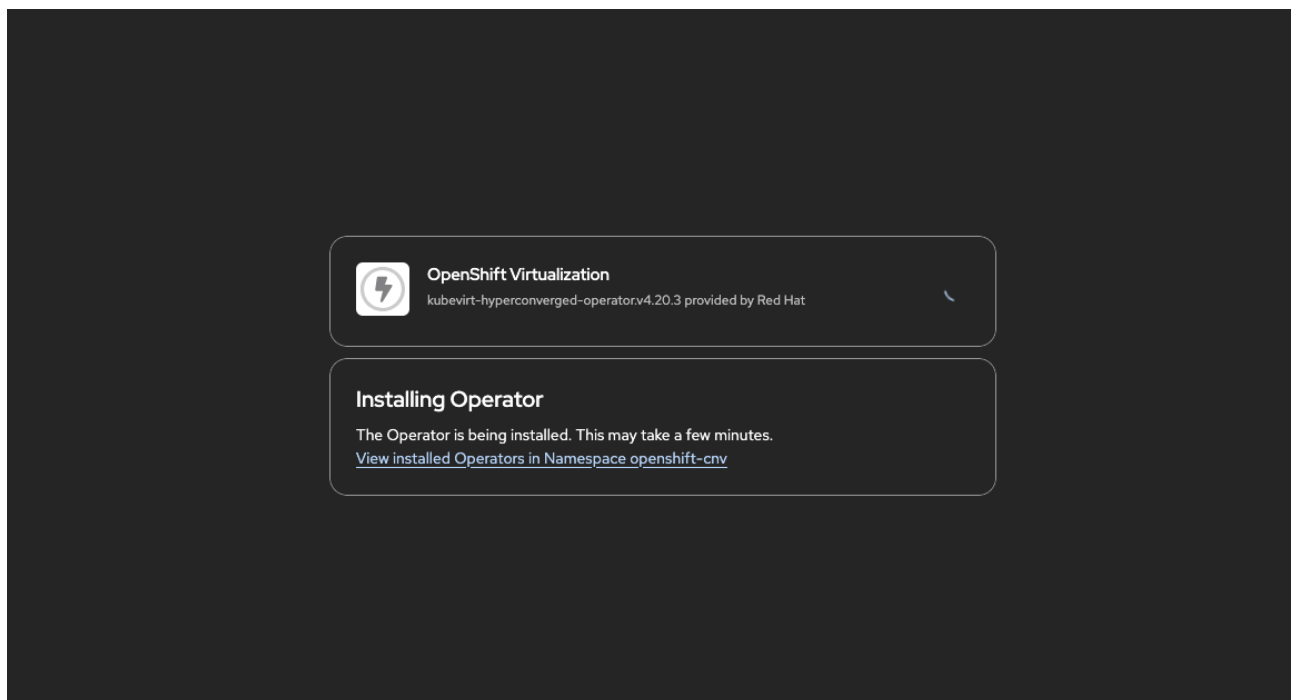


Manual

approval

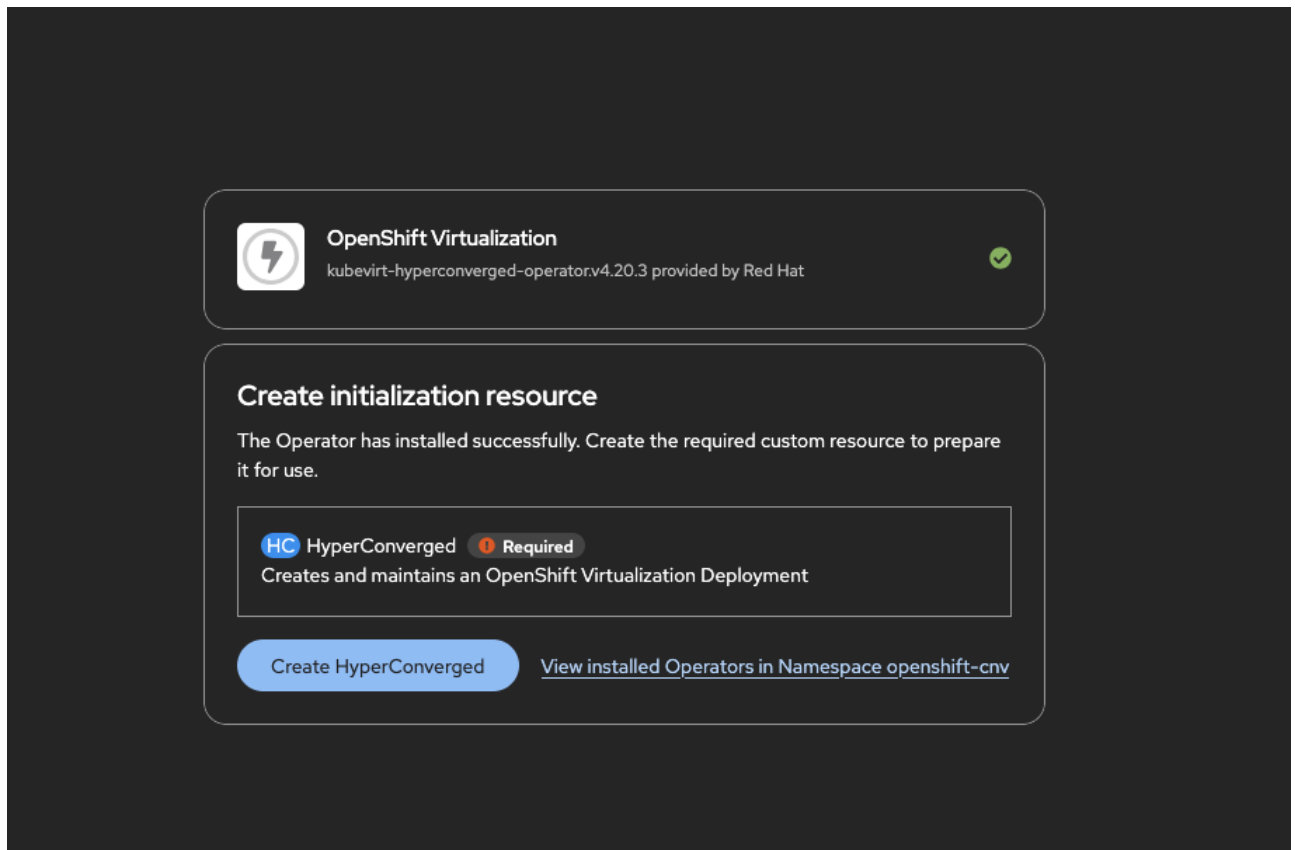
Manual

4.

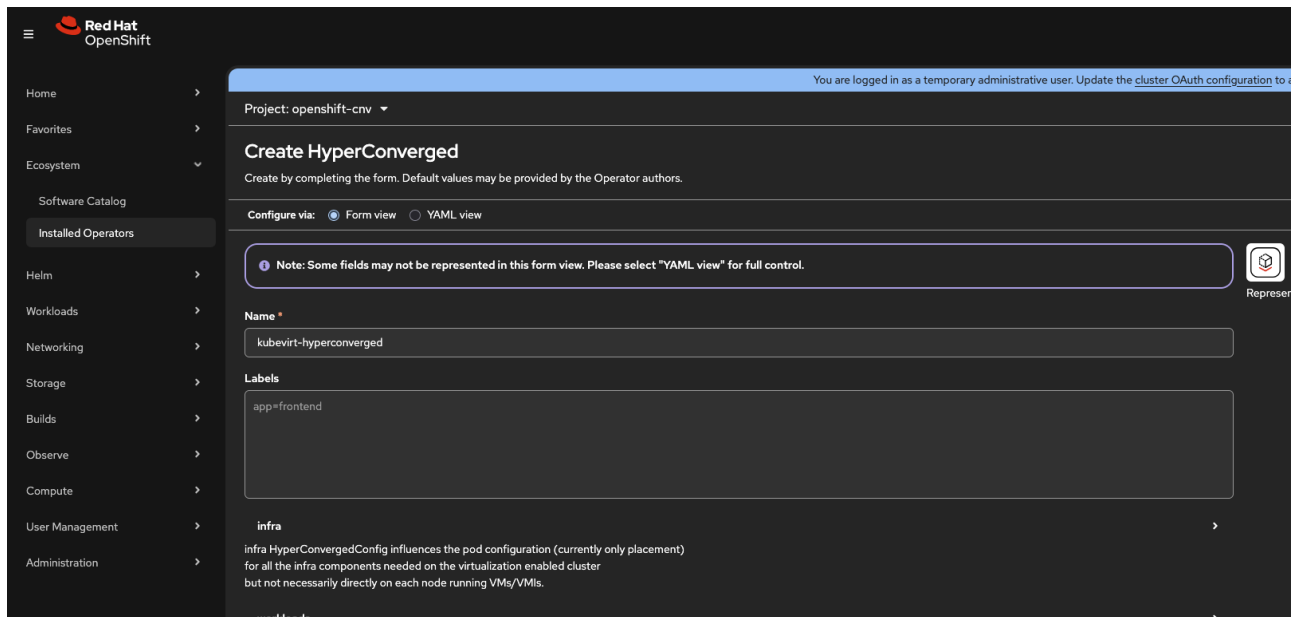


3: HyperConverged

1. Create HyperConverged



2. 確認する



3. Create OpenShift Virtualization

Observe
Compute
User Management
Administration

DataimportCronTemplates holds list of data import cron templates (golden images)

permittedHostDevices

PermittedHostDevices holds information about devices allowed for passthrough

commonBootImageNamespace

CommonBootImageNamespace override the default namespace of the common boot images, in order to hide them.

If not set, HCO won't set any namespace, letting SSP to use the default. If set, use the namespace to create the DataimportCronTemplates and the common image streams, with this namespace. This field is not set by default.

CommonInstancetypesDeployment

CommonInstancetypesDeployment holds the configuration of common-instancetypes deployment within KubeVirt.

resourceRequirements

ResourceRequirements describes the resource requirements for the operand workloads.

filesystemOverhead

FilesystemOverhead describes the space reserved for overhead when using Filesystem volumes. A value is between 0 and 1, if not defined it is 0.055 (5.5 percent overhead)

Create
Cancel

4: HyperConverged

HyperConverged

Project: openshift-cnvm

Installed Operators
Operator details

OpenShift Virtualization
4.20.3 provided by Red Hat

Details
YAML
Subscription
Events
All instances
OpenShift Virtualization Deployment
HostPathProvisioner Deployment

HyperConvergeds
Create HyperConverged

Name
Search by name...

Name	Kind	Status	Labels	Last updated
HC kubevirt-hyperconverged	HyperConverged	Conditions: ReconcileComplete, Available, Upgradable	app=kubevirt-hyperconverged	2 minutes ago

Operators

Red Hat OpenShift
Administrator

Home
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Virtualization
Networking

You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow o

Project: openshift-cnvm

Installed Operators
Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the Understanding Operators documentation. Or create an O

Name
Search by name...

Name	Managed Namespaces	Status	Last updated
OpenShift Virtualization 4.20.3 provided by Red Hat	NS openshift-cnvm	Succeeded Up to date	Jan 28, 2026, 2:45 f

12

kubevirt-hyperconverged-operator.v4.20.1 Succeeded

Succeeded `Ctrl+C`

3: HyperConverged

HyperConverged

```
oc apply -f - <<EOF
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec:
EOF
```



HyperConverged https://docs.openshift.com/container-platform/latest/virt/install/installing-virt.html#virt-customizing-storage-profile_installing-virt [window=_blank]

4:

HyperConverged

```
oc wait hyperconverged kubevirt-hyperconverged -n openshift-cnv
--for=condition=Available --timeout=15m
```

510

5:

```
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o jsonpath='{range
.status.conditions[*]}{.type}: {.status}{"\n"}{end}'
```

```
ReconcileComplete: True
Available: True
Progressing: False
Degraded: False
Upgradeable: True
```

OpenShift Virtualization

OpenShift

Virtualization
OpenShift Virtualization



HyperConverged

Step 1: OpenShift

OpenShift Virtualization VMs

```
# List all VMs across namespaces
oc get vm -A

# Delete VMs (repeat for each namespace)
oc delete vm --all -n <namespace>
```



OpenShift VMs

Step 2: HyperConverged

OpenShift Virtualization

```
oc delete hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

OpenShift

```
# Watch pods being terminated (this may take 5-10 minutes)
oc get pods -n openshift-cnv -w
```

OpenShift Pod

Step 3: OpenShift CSV

```
# Delete the subscription
oc delete subscription hco-operatorhub -n openshift-cnv

# Delete the ClusterServiceVersion
oc delete csv -n openshift-cnv -l operators.coreos.com/kubevirt-
hyperconverged.openshift-cnv
```

ステップ 4: Namespace削除 (確認)

Namespaceが削除されていることを確認します。

```
oc delete namespace openshift-cnv
```

ステップ 5: 確認

Namespaceが削除されていることを確認します。

```
# Should return empty or "No resources found"
oc get pods -n openshift-cnv
oc get csv -n openshift-cnv
```

完了

OpenShift Virtualizationのインストールと設定が完了しました。このガイドに従って、OpenShift Virtualizationをインストールし、設定し、使用します。

インストール

- [virtctl](#) - VMを管理するためのツール
- [OpenShift Virtualization](#) - VMを管理するためのツール
- [VMのIP](#) - cloud-init/DHCPで自動的に割り当てられるSysprepで自動的にIPが割り当てられます。

完了

OpenShift Virtualizationのインストールと設定が完了しました。このガイドに従って、OpenShift Virtualizationをインストールし、設定し、使用します。

- [OpenShift Virtualization](#)
- [OpenShift Virtualization](#)
- [KubeVirt](#)

インストール

完了

OpenShift

Virtualizationのインストールと設定が完了しました。このガイドに従って、OpenShift Virtualizationをインストールし、設定し、使用します。

Stream、Windows、Linux、およびその他のOSをサポートするOpenShift Virtualizationのインストールと設定が完了しました。このガイドに従って、OpenShift Virtualizationをインストールし、設定し、使用します。

OpenShift Virtualizationのインストールと設定が完了しました。このガイドに従って、OpenShift Virtualizationをインストールし、設定し、使用します。

OpenShift Virtualizationのインストールと設定が完了しました。このガイドに従って、OpenShift Virtualizationをインストールし、設定し、使用します。

完了:

前提条件

OpenShift 4.19, 4.20

- **OpenShift**インストール: OpenShift VirtualizationインストールOpenShiftインストール
- **root**権限: `cluster-admin`権限を有するユーザー
- **root**権限: `root`権限を有するユーザー100000
- **root**権限: `root`権限を有するユーザー

ステップ 1: OpenShift Virtualizationインストール

OpenShift Virtualizationインストール

```
oc get sc
```

OpenShift Virtualizationインストール

(default)

OpenShift Virtualizationインストール

OpenShift Virtualizationインストール:

NAME	PROVISIONER	AGE	RECLAIMPOLICY
VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION		
ceph-rbd-sc	rbd.csi.ceph.com		Delete
Immediate	true	2d	
hostpath-provisioner	kubevirt.io/hostpath-provisioner		Delete
WaitForFirstConsumer	false	1d	
local-storage	kubernetes.io/no-provisioner		Delete
WaitForFirstConsumer	false	1d	

OpenShift Virtualizationインストール:

NAME	PROVISIONER	AGE	RECLAIMPOLICY
VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION		
ceph-rbd-sc (default)	rbd.csi.ceph.com		Delete
Immediate	true	2d	
hostpath-provisioner	kubevirt.io/hostpath-provisioner		Delete
WaitForFirstConsumer	false	1d	
local-storage	kubernetes.io/no-provisioner		Delete
WaitForFirstConsumer	false	1d	

OpenShift Virtualizationインストール

1. 確認デフォルトストレージクラス
2. デフォルトストレージクラスを確認

ステップ 2: デフォルトストレージクラスを確認

デフォルトストレージクラスを確認するには、`kubectl get sc` コマンドを実行します。1つのデフォルトストレージクラスが表示されます。

デフォルトストレージクラスを確認するには、`oc get sc` コマンドを実行します。

```
oc annotate sc <the-old-default-storage-class-name> storageclass.kubernetes.io/is-default-class-
```

例: デフォルトストレージクラスを確認

ステップ 3: デフォルトストレージクラスを設定

デフォルトストレージクラスを設定するには、`oc annotate` コマンドを実行します。OpenShiftでは、

デフォルトストレージクラスは: `storageclass.kubernetes.io/is-default-class: "true"`

```
oc annotate sc <your-storage-class-name> storageclass.kubernetes.io/is-default-class="true"
```

デフォルトストレージクラス `ceph-rbd-sc` を例:

```
oc annotate sc ceph-rbd-sc storageclass.kubernetes.io/is-default-class="true"
```

ステップ 4: デフォルトストレージクラスを確認

デフォルトストレージクラスを確認するには、`oc get sc` コマンドを実行します。

```
oc get sc
```

デフォルトストレージクラス:

NAME	PROVISIONER	AGE	RECLAIMPOLICY
VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION		
ceph-rbd-sc (default)	rbd.csi.ceph.com		Delete
Immediate	true	2d	
hostpath-provisioner	kubevirt.io/hostpath-provisioner		Delete
WaitForFirstConsumer	false	1d	
local-storage	kubernetes.io/no-provisioner		Delete
WaitForFirstConsumer	false	1d	

デフォルト (default) の PersistentVolumeClaim

ステップ 5: PersistentVolumeClaim の作成

PersistentVolumeClaim (PVC) の作成

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-default-storage
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
EOF
```

PVC の作成を確認

```
oc get pvc test-default-storage -n default
```

結果:

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
test-default-storage	Pending				lvms-vg1	0s

PVC のステータスが **Pending** の場合は、**lvms-vg1** のストレージクラスを確認

ストレージクラスの作成

PVC の **Pending** ステータスは、**WaitForFirstConsumer** の volumeBindingMode が原因で発生します。

volumeBindingMode	説明	動作
WaitForFirstConsumer	PVC が Pod にバインドされた後、PV を作成する	Pod が LVM を使用してストレージをマウントする
Immediate	PVC が作成された後、PV を作成する	ストレージクラスが NFS である場合は、ephemeral として作成される

ストレージクラスの volumeBindingMode の確認

```
oc get sc <storage-class-name> -o jsonpath='{.volumeBindingMode}'
```

例 6: VM のデフォルトストレージ

この例では、VM のデフォルトストレージを設定します。

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: test-vm-default-storage
  namespace: default
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: test-vm-disk
    spec:
      pvc:
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 10Gi
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
  template:
    metadata:
      labels:
        kubevirt.io/vm: test-vm-default-storage
    spec:
      domain:
        devices:
          disks:
            - name: datavolumedisk
              disk:
                bus: virtio
          resources:
            requests:
              memory: 1Gi
              cpu: 1
          networks:
            - name: default
              pod: {}
          volumes:
            - name: datavolumedisk
              dataVolume:
                name: test-vm-disk
EOF
```


2. `storageclass.kubernetes.io/is-default-class`:
 storageclass.kubernetes.io/is-default-class
3. `storageclass.kubernetes.io/is-default-class`: storageclass.kubernetes.io/is-default-class
4. `storageclass.kubernetes.io/is-default-class`: storageclass.kubernetes.io/is-default-class

□□□

storageclass.kubernetes.io/is-default-class

- storageclass.kubernetes.io/is-default-class
- storageclass.kubernetes.io/is-default-class
- `storageclass.kubernetes.io/is-default-class` storageclass.kubernetes.io/is-default-class
- PVC VM storageclass.kubernetes.io/is-default-class
- OpenShift Virtualization storageclass.kubernetes.io/is-default-class

□□□□

- [OpenShift - Persistent Storage](#)
- [OpenShift Virtualization - Storage](#)

Web□□□□□□VM□□□

□□

storageclass.kubernetes.io/is-default-class OpenShift

Web storageclass.kubernetes.io/is-default-class VM storageclass.kubernetes.io/is-default-class Web storageclass.kubernetes.io/is-default-class VM storageclass.kubernetes.io/is-default-class storageclass.kubernetes.io/is-default-class

□□□□

- OpenShift Virtualization storageclass.kubernetes.io/is-default-class OpenShift 4.18 [OpenShift Virtualization](#) storageclass.kubernetes.io/is-default-class
- VM storageclass.kubernetes.io/is-default-class OpenShift Web storageclass.kubernetes.io/is-default-class
- VM storageclass.kubernetes.io/is-default-class storageclass.kubernetes.io/is-default-class StorageClass
- storageclass.kubernetes.io/is-default-class DataSource

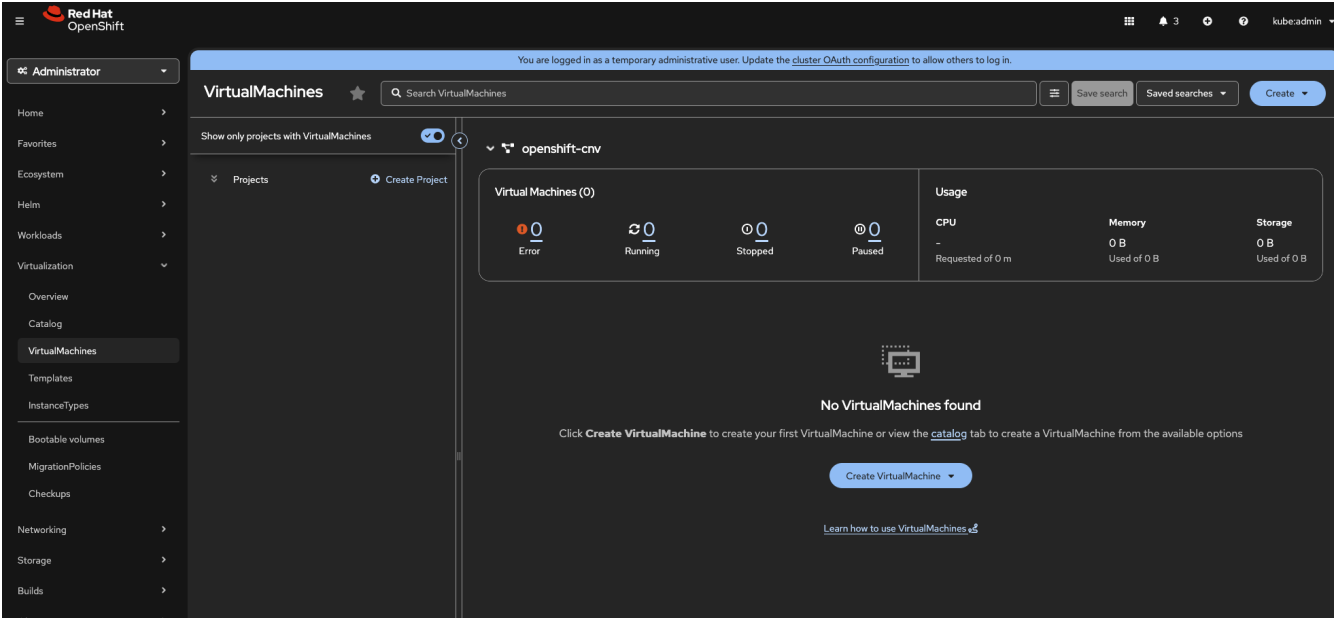
storageclass.kubernetes.io/is-default-class

```
oc get datasources -n openshift-virtualization-os-images
```

□□□□ **1:** storageclass.kubernetes.io/is-default-class

1. OpenShift Web storageclass.kubernetes.io/is-default-class
2. storageclass.kubernetes.io/is-default-class **Virtualization** storageclass.kubernetes.io/is-default-class

3. VirtualMachines

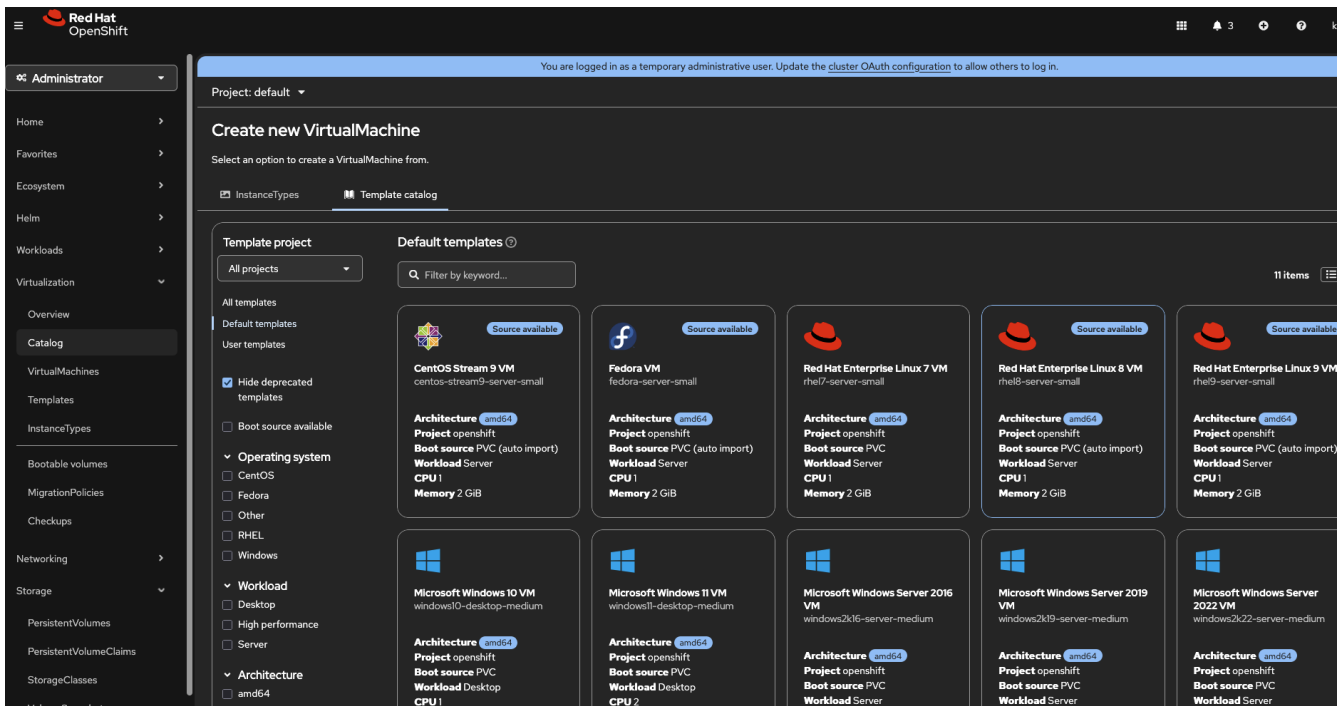


VirtualMachines

2:

OpenShift Virtualization

1. VirtualMachines **Create** → **From template**
2.
 - **Red Hat Enterprise Linux** - RHEL
 - **Fedora** - Fedora
 - **CentOS Stream** - CentOS Stream
 - **Microsoft Windows** - Windows Server



Source available

StorageClass

- CPU
- OS
- DataSource
- Cloud-init sysprep

3: VM

Fedora VM

1. **Fedora**
- 2.
3. **Customize VirtualMachine**

administrative user, L

source available

auto import)

TIVM
medium

Fedora VM
fedora-server-small

Template info

Operating system

Fedora VM

Workload type

Server (default)

Description

Template for Fedora Linux 40 VM or newer. A PVC with the Fedora disk image must be available.

Documentation

[Refer to documentation](#)

CPU | Memory

1 CPU | 2 GiB Memory

Network interfaces (1)

Name	Network	Type
default	Pod networking	Masquerade

Disks (2)

Name	Drive	Size
rootdisk	Disk	30 GiB
cloudinitdisk	Disk	-

Storage

☐ Boot from CD

Disk source

Template default

Disk size

-

30

+

GiB

Drivers

☐ Mount Windows drivers disk

Optional parameters

Quick create VirtualMachine

VirtualMachine name

fedora-plum-angelfish-45

Project

default

Public SSH key

[Not configured](#)

☒ Start this VirtualMachine after creation (RerunOnFailure)

Quick create VirtualMachine

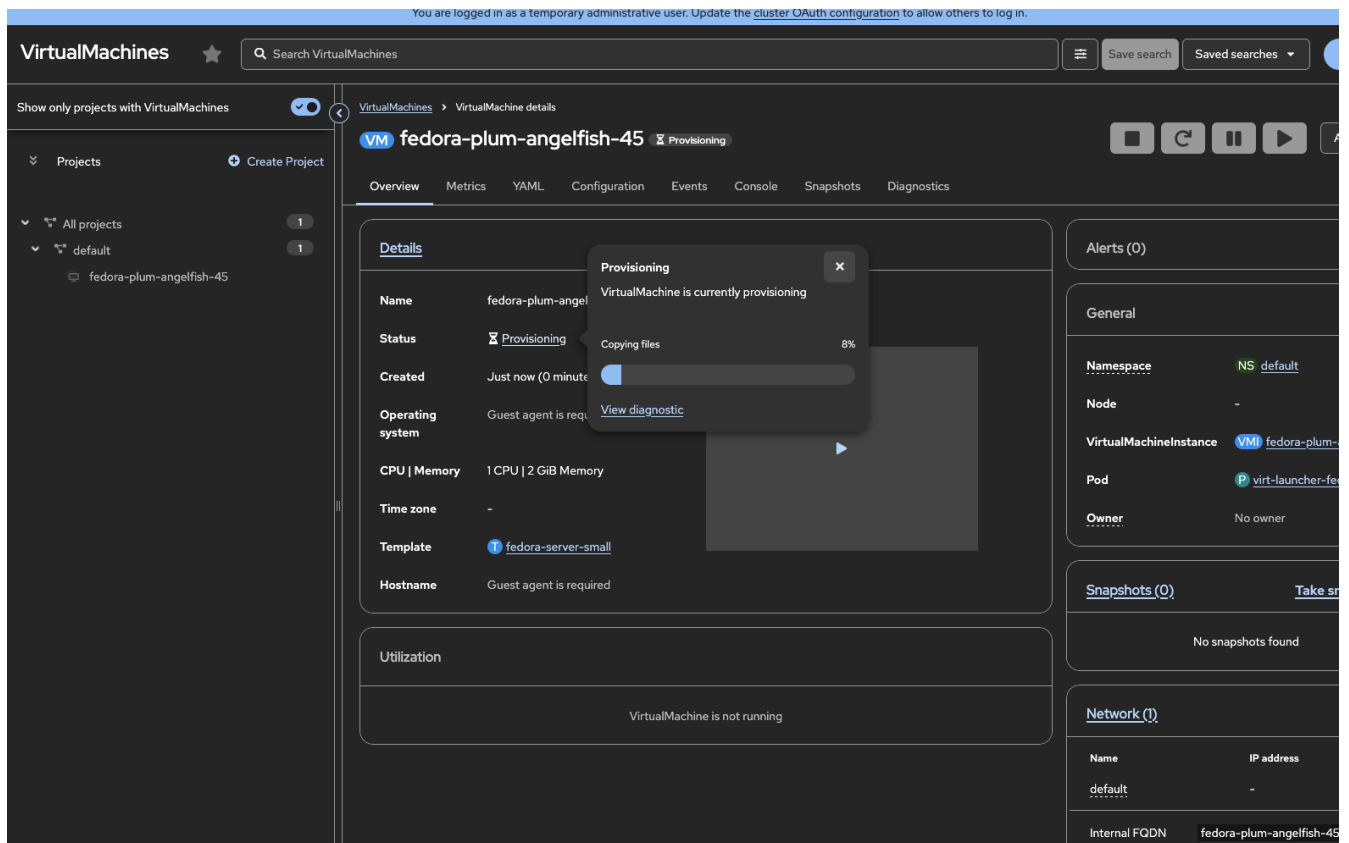
Customize VirtualMachine

Cancel

VM□□□□□

Customize VirtualMachine Overview VM

1. **Name:** VMfedora-test-vm
2. **Project:** VMfedora-test-vm
3. **Boot source:** FedoraDataSourcefedora-test-vm
4. **CPU:** CPUfedora-test-vm1
5. **Memory:** RAMfedora-test-vm1

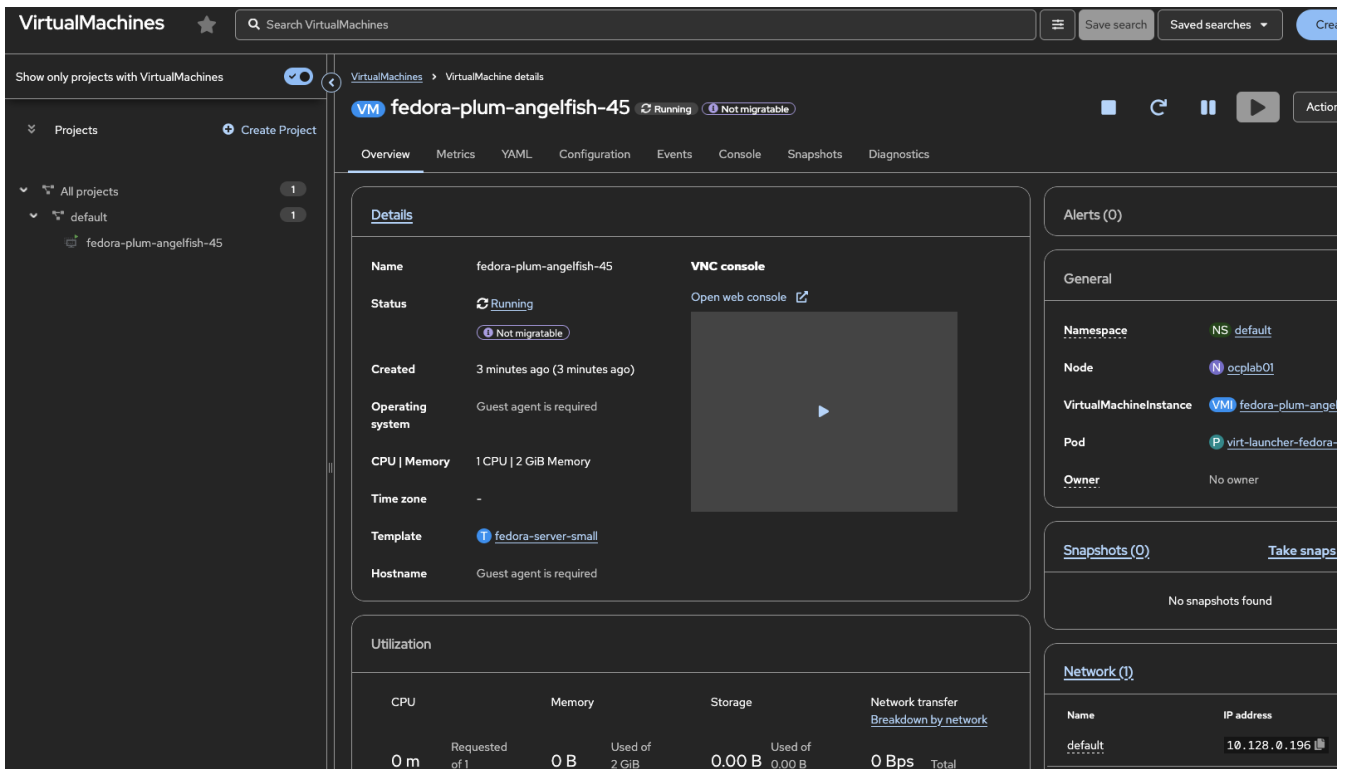


Events

```
# Alternatively, watch events from CLI
oc get events -n <your-namespace> --field-selector involvedObject.name=<vm-name> -w
```

VM

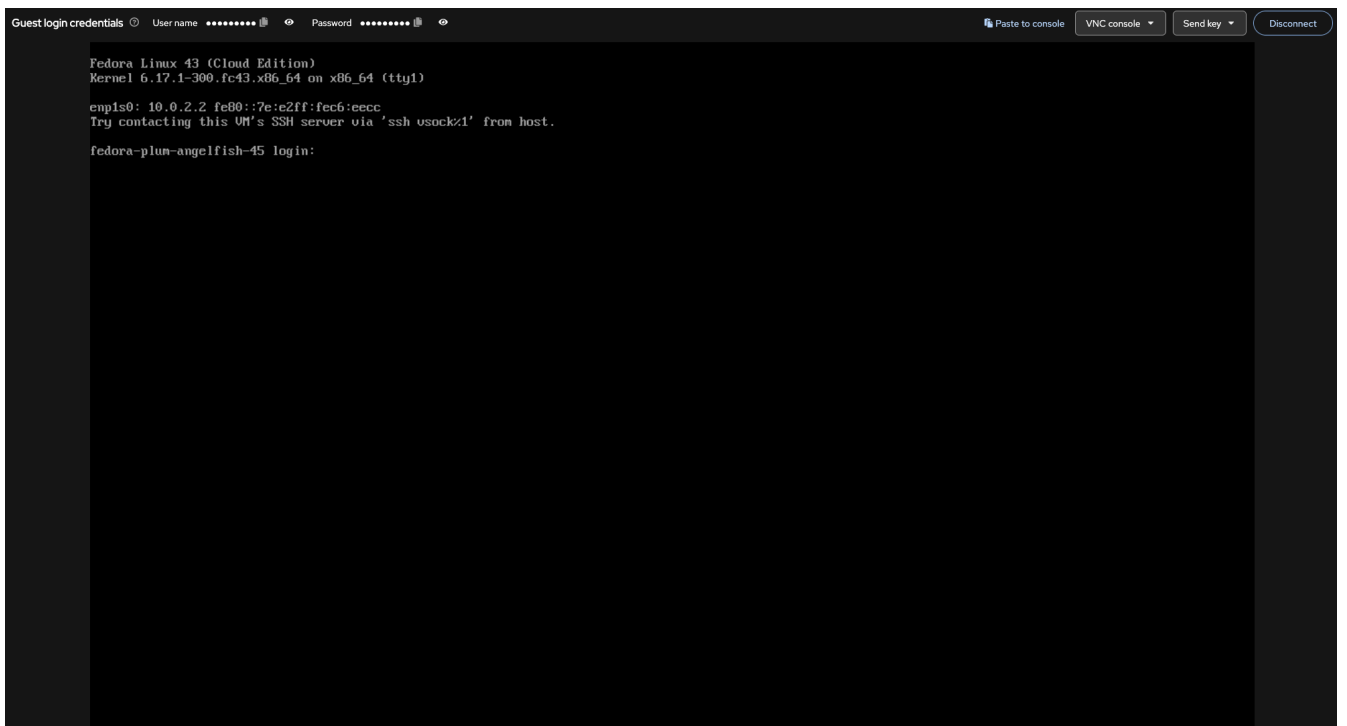
Running

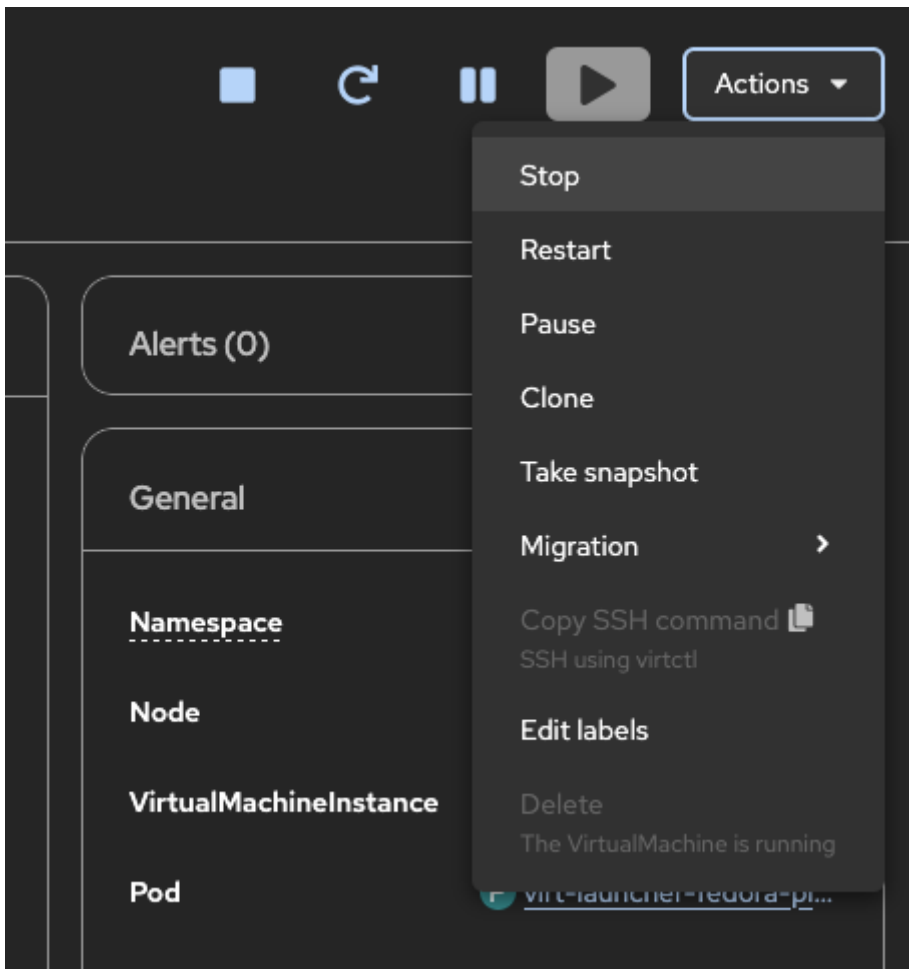


5: VM

Web VM

1. VM **Console**
2. VNC





VMを停止

1. **Actions** → **Stop** をクリック
2. 停止が完了するまで待つ
3. VMを削除する

```
# CLI equivalent
virtctl stop <vm-name>
```

VMを再起動

1. **Actions** → **Restart** をクリック
2. VMを再起動する

```
# CLI equivalent
virtctl restart <vm-name>
```

VMを一時停止

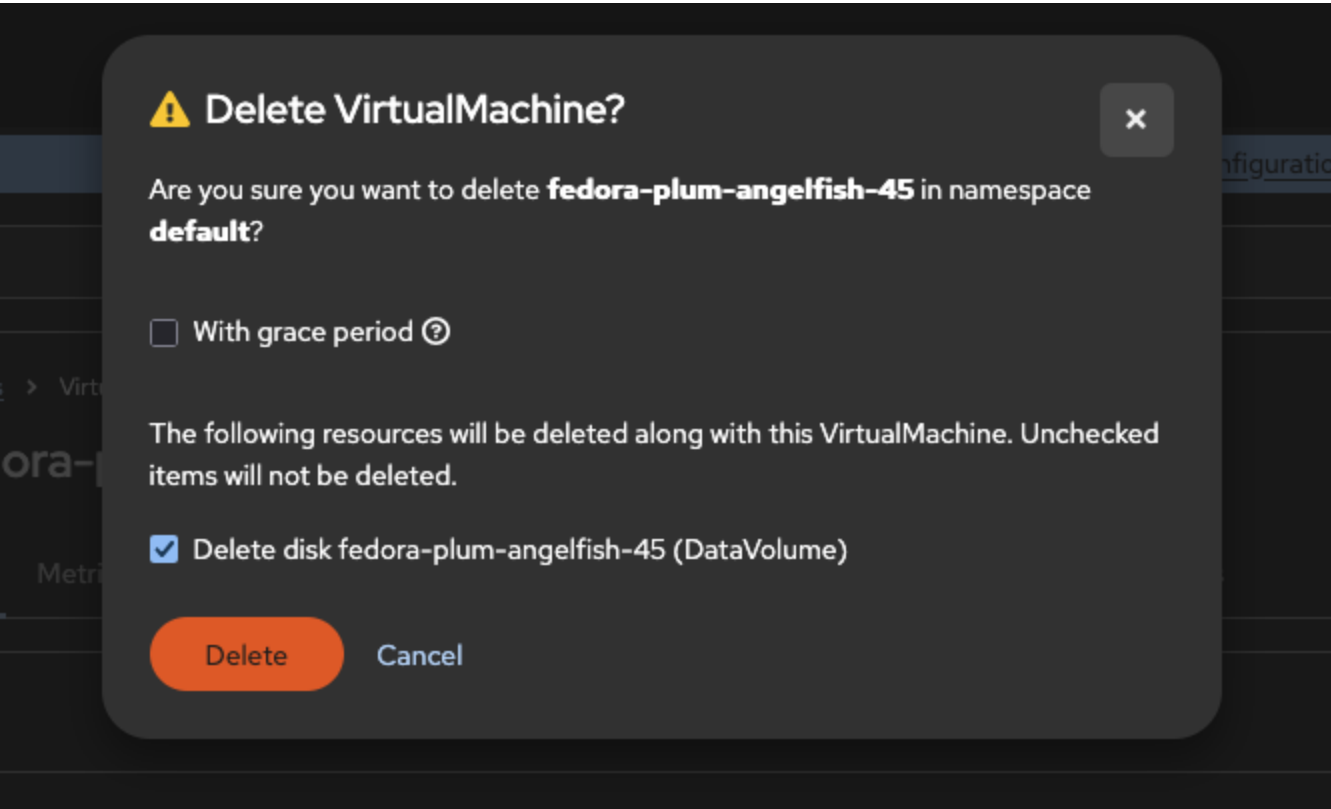
VMを一時停止する

1. **Actions** → **Pause** をクリック

- 2. VM
- 3. Actions → Unpause

VM

- 1. Actions → Delete
- 2. PVC
- 3.



Delete

disk

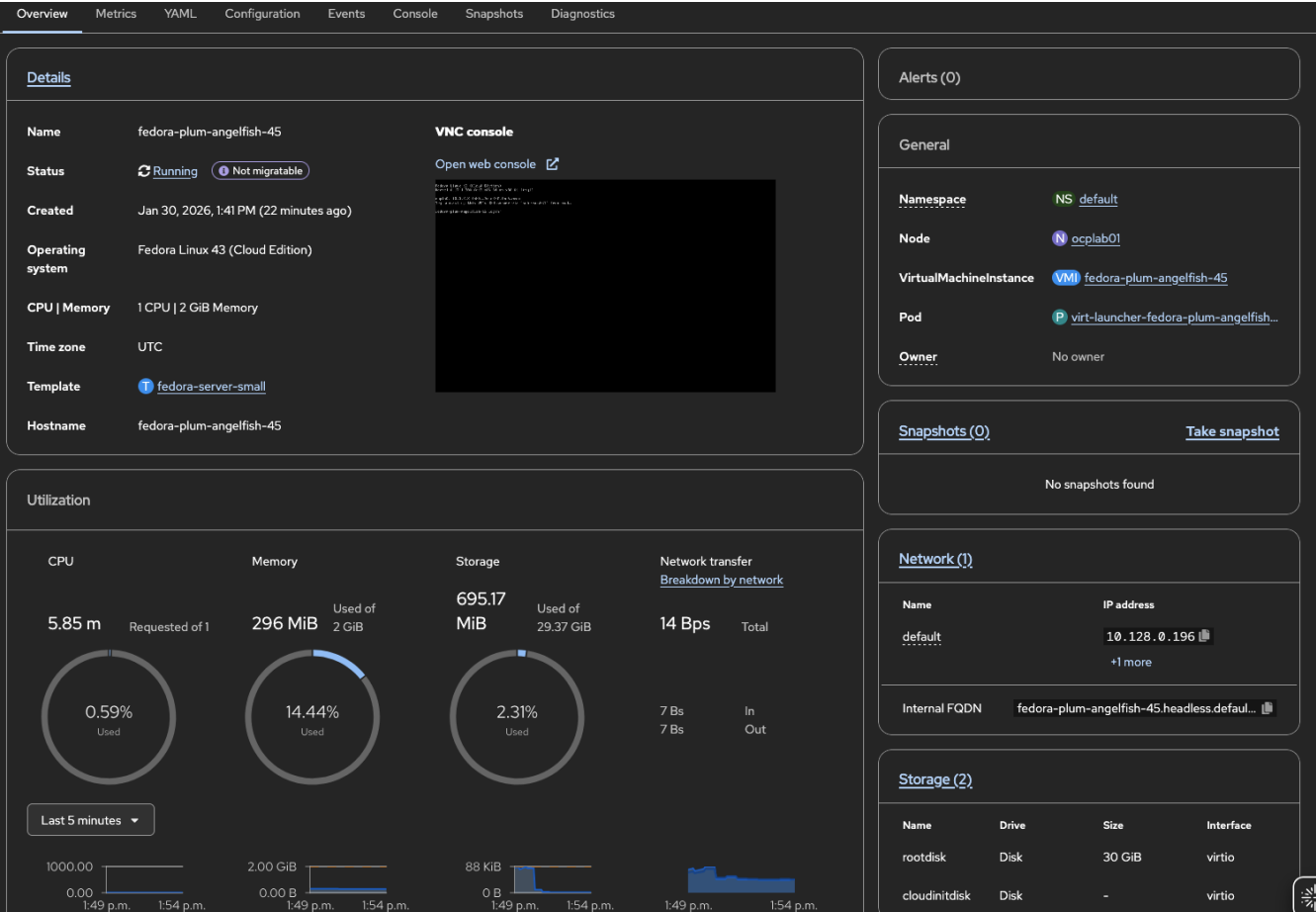
7: VM

VM

Overview

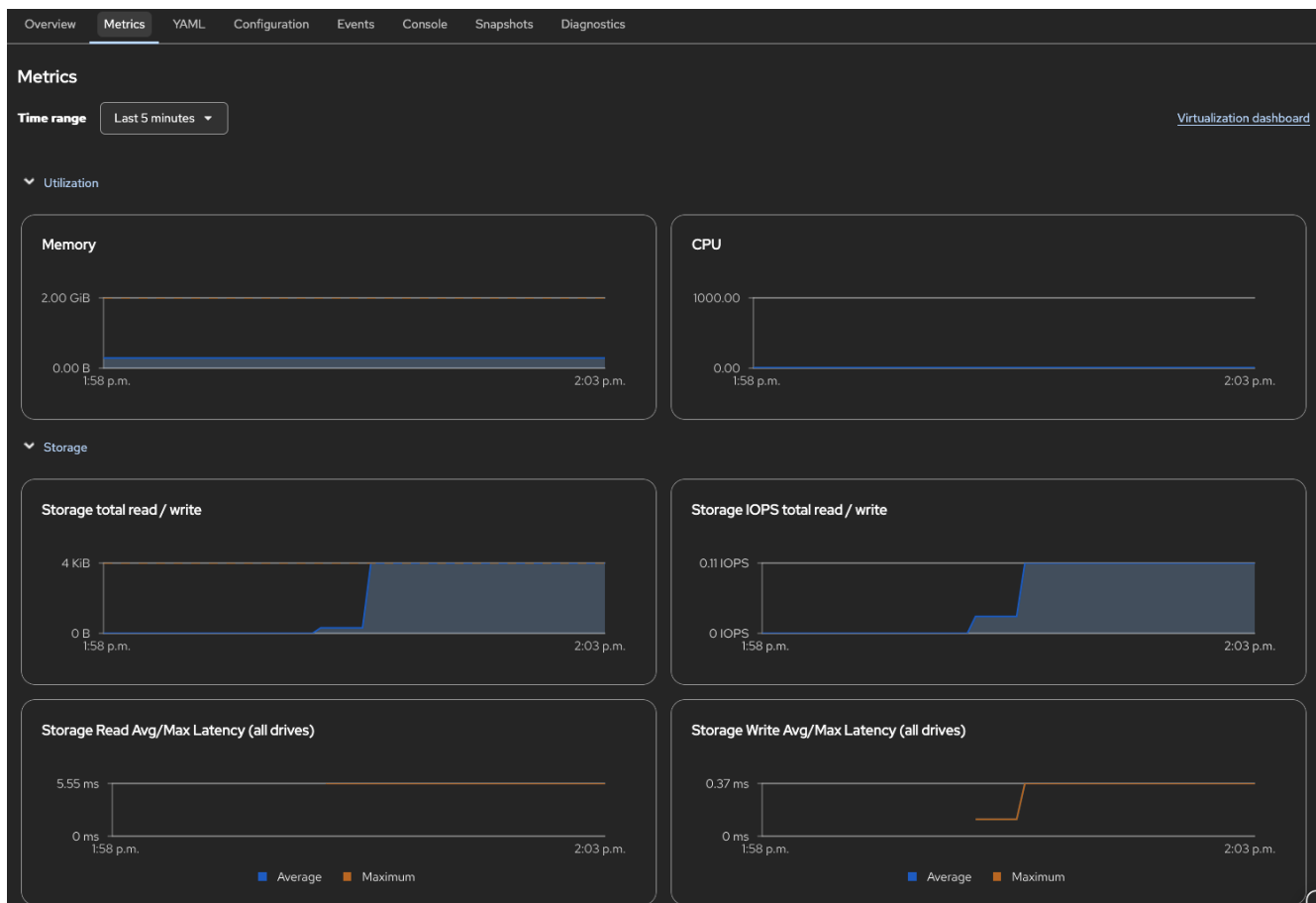
- VM
- IP

- CPU
-



Metrics

- CPU
-
-
- I/O

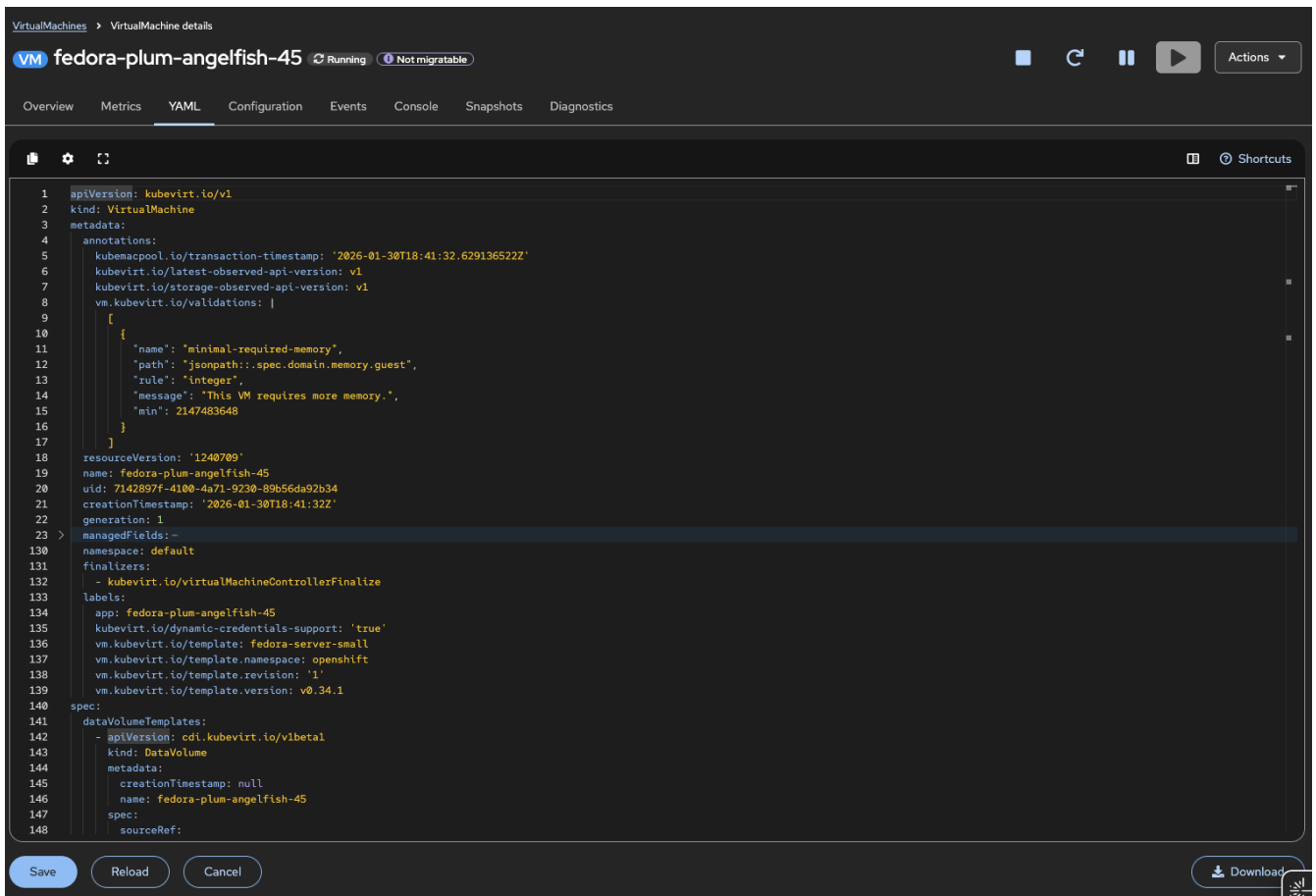


このページは、VMのQEMUに関する情報を提供します。

YAML

VMの構成を定義するためのYAMLファイルを作成します。

1. **YAML** ファイルを作成します。
2. **VirtualMachine** オブジェクトを定義します。
3. **Save** を実行して保存します。



Download `VirtualMachine` YAML

VirtualMachine

VirtualMachine

1. VirtualMachines **Create** → **From template**
- 2.
3. **Customize VirtualMachine** **Quick create VirtualMachine**
4. VM

administrative user, U

source available

auto import)

IVM
medium

Fedora VM
fedora-server-small

Template info

Operating system

Fedora VM

Workload type

Server (default)

Description

Template for Fedora Linux 40 VM or newer. A PVC with the Fedora disk image must be available.

Documentation

[Refer to documentation](#)

CPU | Memory

1 CPU | 2 GiB Memory

Network interfaces (1)

Name	Network	Type
default	Pod networking	Masquerade

Disks (2)

Name	Drive	Size
rootdisk	Disk	30 GiB
cloudinitdisk	Disk	-

Storage

☐ Boot from CD

Disk source

Template default

Disk size

-

30

+

GiB

Drivers

☐ Mount Windows drivers disk

Optional parameters

Quick create VirtualMachine

VirtualMachine name

fedora-plum-angelfish-45

Project

default

Public SSH key

[Not configured](#)

☒ Start this VirtualMachine after creation (RerunOnFailure)

Quick create VirtualMachine

Customize VirtualMachine

Cancel

□□□□□□□□□□□□

- □□□□□□□□
- □□□□□□□□□□□□□□□□
- □□□□□□□□□□

111

□□□□□□□□□□

- Web
- VM

- [OpenShift Virtualization VM](#)
- [VM](#)
- [Web](#) [OpenShift VM](#)
- [OpenShift VM](#)
- [VM](#)

OpenShift

- [virtctl CLI](#) - [OpenShift VM](#)
- [IP](#) - [VM IP](#)
- [SSH](#) - [SSH Linux VM](#)

OpenShift

- [OpenShift - OpenShift](#)
- [OpenShift - OpenShift](#)

VM

OpenShift

OpenShift

Virtualization OpenShift VM VirtualMachine VirtualMachineInstance VM VM VM

OpenShift:

OpenShift 4.20

OpenShift

- [OpenShift Virtualization](#) [OpenShift 4.18](#)
- [OpenShift Web](#) [CLI](#) ([oc](#) [virtctl](#))
- [OpenShift VM](#) [1](#) ([Web](#) [VM](#))

VirtualMachine VirtualMachineInstance

OpenShift Virtualization 2

VirtualMachine (VM)

VirtualMachine VM

- [VM](#)

- CPU
-
-
- (VM)

[illegible]

```
# List VirtualMachine resources
oc get vm -n my-vms
```

□□□:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Running	True

VirtualMachineInstance (VMI)

VirtualMachineInstance

VM VM VM VM VM

- 國際標準 (ISO 9000)
- 國際標準/法
- 國際標準 (ISO 9000)

```
# List VirtualMachineInstance resources
oc get vmi -n my-vms
```

000:

NAME	AGE	PHASE	IP	NODENAME	READY
fedora-app-vm	78s	Running	10.128.0.184	ocplab01	True



```

VirtualMachine
VirtualMachineInstance
VM
VMI
VM
VMI
VM

```

VM□□□□□□□□

[illegible]

1111

번호	상태	설명	VM이름
1	Stopped (중지)	VM이 완전히 종료된 상태입니다.	vm1
2	Pending (대기) (예약)	VM이 예약된 상태입니다 (리소스가 할당되지 않음)	vm2
3	Scheduling (예약) (대기)	Kubernetes가 VM을 스케줄링하고 있습니다.	vm3
4	Starting (시작)	VM이 시작되고 있습니다.	vm4
5	Running (실행) (가동)	VM이 정상적으로 실행 중입니다.	vm5
6	Paused (중지) (대기)	VM이 일시적으로 중지된 상태입니다.	vm6
7	Stopping (중지) (대기)	VM이 종료되고 있습니다.	vm7
8	Terminating (종료) (대기)	VM이 종료되고 있습니다.	vm8 (종료)

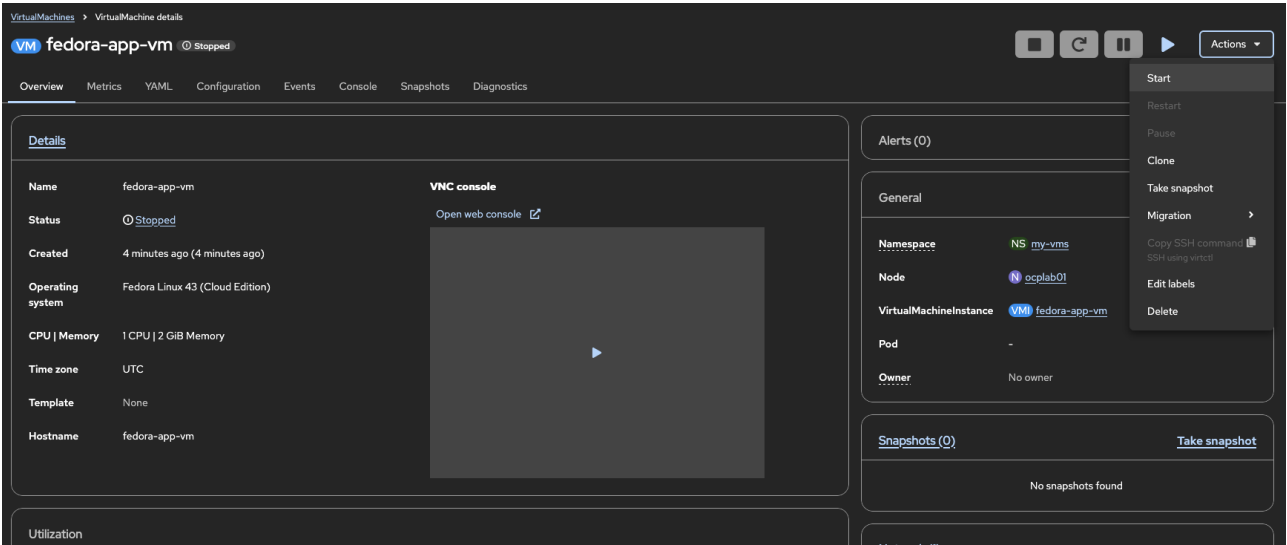
VM이름을 지정합니다

VM을 생성합니다

다음과 같이 VM을 생성합니다

Web 콘솔:

- 1. **Virtualization** → **VirtualMachines** 섹션으로 이동
- 2. 생성할 VM을 선택합니다
- 3. VM의 **Actions** → **Start** 버튼을 클릭합니다
- 4. 생성된 VM (3개) 중 **Start** 버튼을 클릭합니다



CLI:

```
virtctl start fedora-app-vm -n my-vms
```

VM:

```
# Check VM status
oc get vm fedora-app-vm -n my-vms

# Check VMI was created
oc get vmi fedora-app-vm -n my-vms
```

:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Running	True

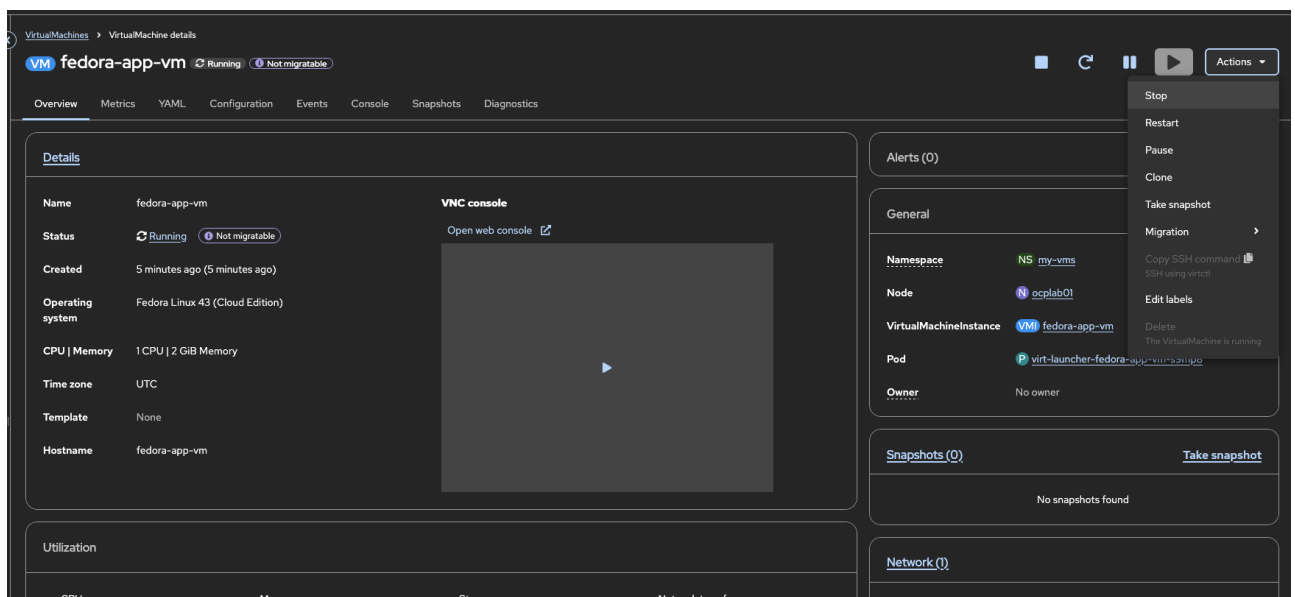
NAME	AGE	PHASE	IP	NODENAME	READY
fedora-app-vm	78s	Running	10.128.0.184	ocplab01	True

VM

VM

Web:

1. VM
2. **Actions** → **Stop**



CLI ():

```
virtctl stop fedora-app-vm -n my-vms
```

CLI (ターミナル):

VMを強制停止する

```
virtctl stop fedora-app-vm -n my-vms --force
```



この操作は、VMのデータを破損する可能性があります。

VMを確認する:

```
# VM should show Stopped status
oc get vm fedora-app-vm -n my-vms

# VMI should no longer exist
oc get vmi fedora-app-vm -n my-vms
```

結果:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Stopped	False

Error from server (NotFound): virtualmachineinstances.kubevirt.io "fedora-app-vm" not found

VMを再開する

VM (Pause) VMを再開する

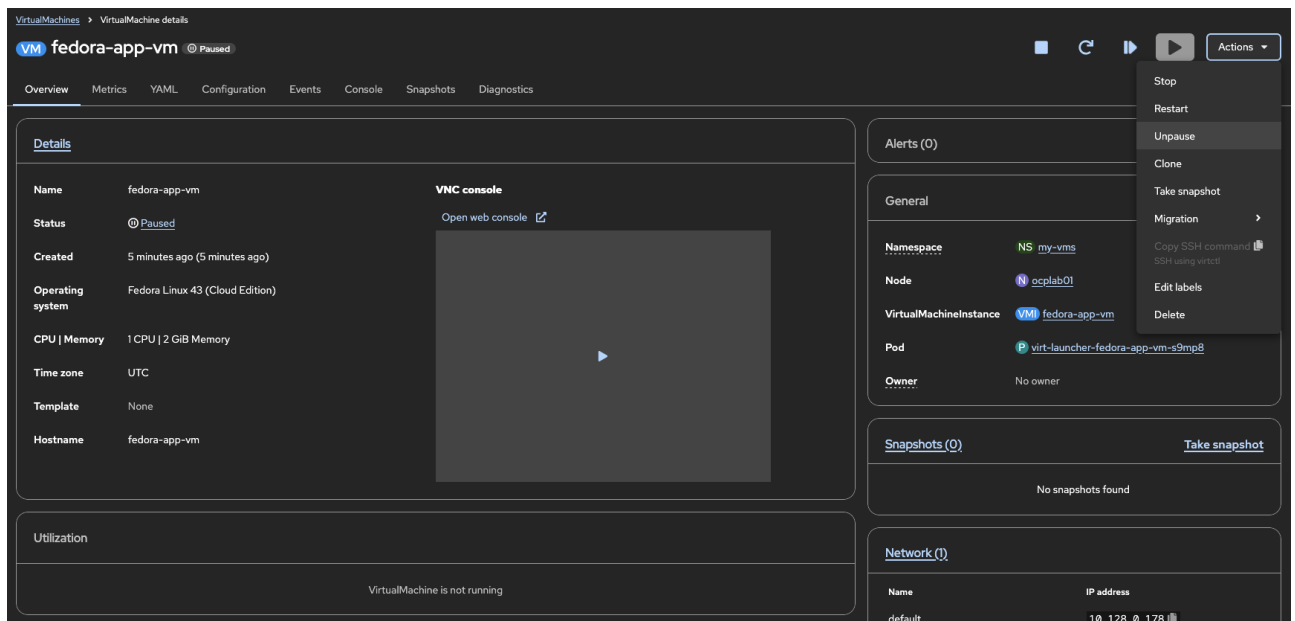
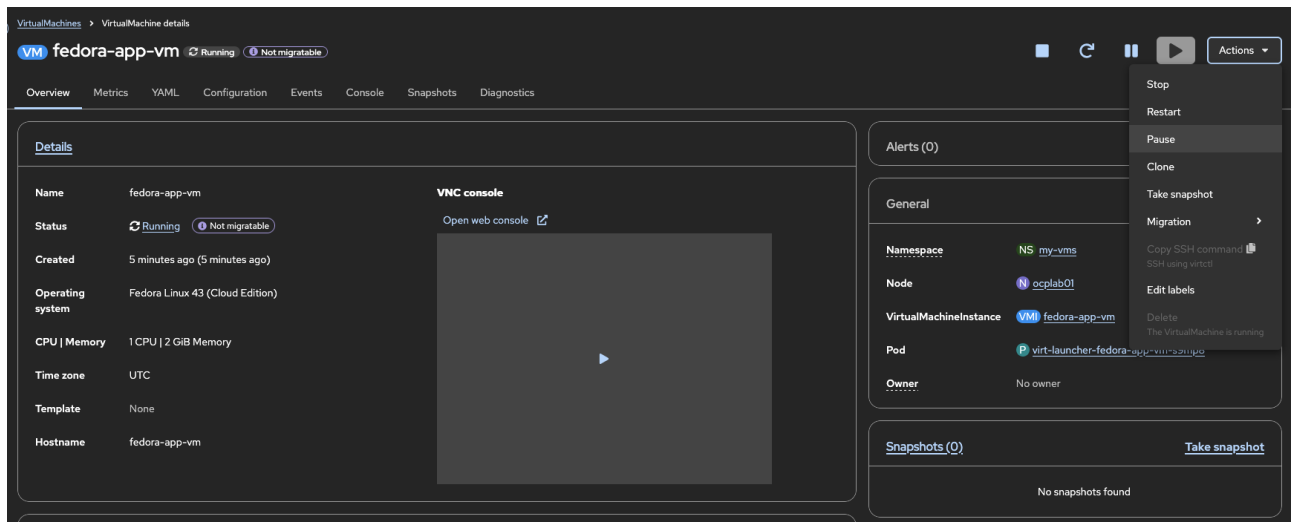
- CPUを再開する
- VMのメモリを解放する
- VMのディスクを解放する



この操作は、VMのデータを破損する可能性があります。

Webコンソール:

1. VMを選択する
2. **Actions** → **Pause** をクリックする
3. **コンソール**: **Actions** → **Unpause** をクリックする



CLI:

```
# Pause a VM
virtctl pause vm fedora-app-vm -n my-vms

# Unpause a VM
virtctl unpause vm fedora-app-vm -n my-vms
```

oc get vm:

```
oc get vm fedora-app-vm -n my-vms
```

Table:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Paused	False

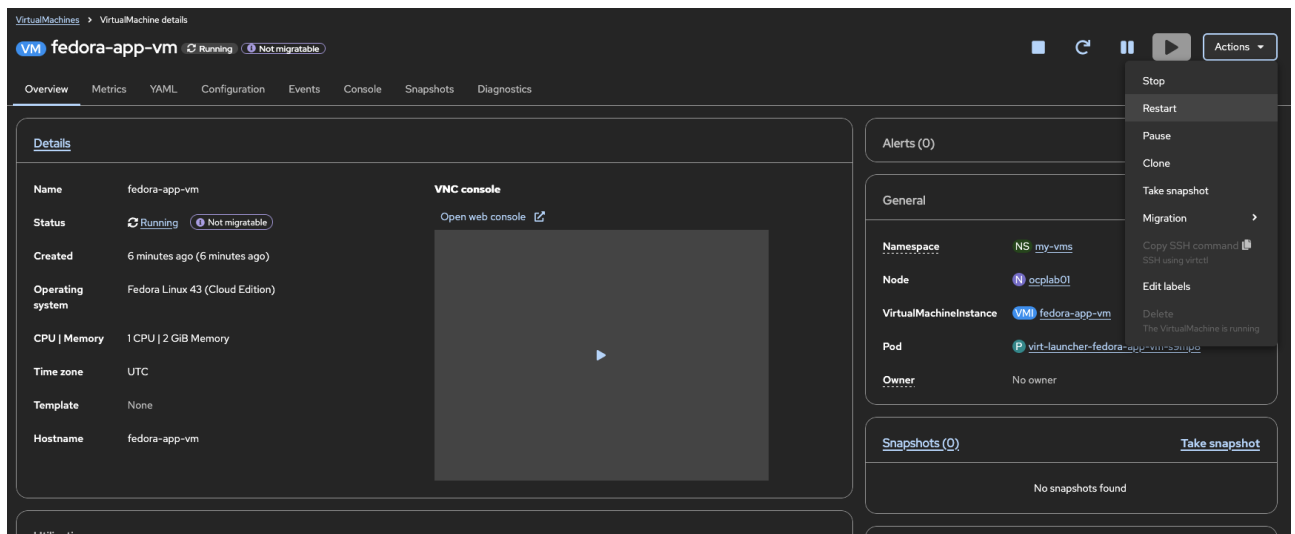
VM

(Restart)

-
- OS
- VM

Web:

1. VM
2. **Actions** → **Restart**



CLI:

```
virtctl restart fedora-app-vm -n my-vms
```



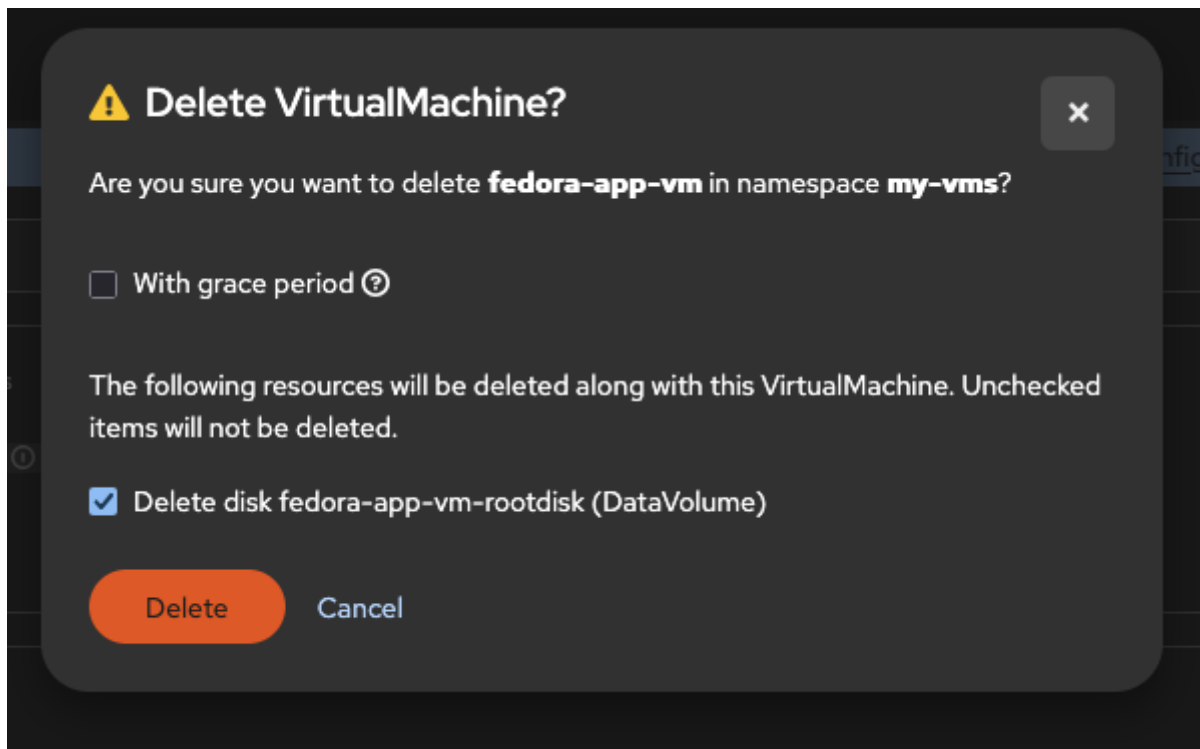
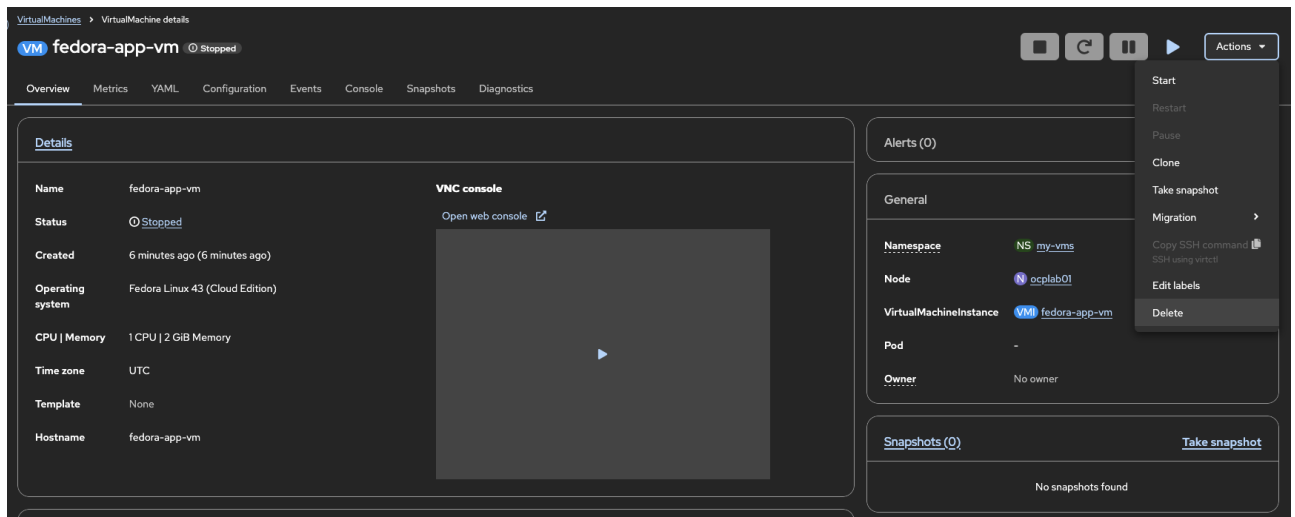
OS

VM

VM (PVC/DataVolume)

Web:

1. VM
2. **Actions** → **Delete**
- 3.
4. **Delete**



CLI (VM):

```
oc delete vm fedora-app-vm -n my-vms
```

VM DataVolume Web

:

```
# VM should not exist
oc get vm fedora-app-vm -n my-vms

# Check if PVCs were preserved (if desired)
oc get pvc -n my-vms
```

:

Error from server (NotFound): virtualmachines.kubevirt.io "fedora-app-vm" not found
No resources found in my-vms namespace.



VM

VM

VM VMI

VM:

```
oc describe vm fedora-app-vm -n my-vms
```

():

```
Name:          fedora-app-vm
Namespace:     my-vms
Labels:        app=fedora-app-vm
...
Status:
  Conditions:
    Last Transition Time:  2026-03-05T17:10:11Z
    Status:                True
    Type:                  Ready
    Message:               All of the VMI's DVs are bound and ready
    Reason:                AllDVsReady
    Status:                True
    Type:                  DataVolumesReady
    Status:                True
    Type:                  AgentConnected
  Printable Status:       Running
  Ready:                  true
  Run Strategy:           Manual
```

:

- **Status.Conditions** -
- **Status.PrintableStatus** -
- **Status.Ready** - VM

VMI:

```
oc describe vmi fedora-app-vm -n my-vms
```

名前 (namespace):

```
Name:      fedora-app-vm
Namespace:  my-vms
...
Status:
  Guest OS Info:
    Id:      fedora
    Kernel Release: 6.17.1-300.fc43.x86_64
    Name:      Fedora Linux
    Pretty Name:  Fedora Linux 43 (Cloud Edition)
    Version Id:  43
  Interfaces:
    Interface Name: enp1s0
    Ip Address:     10.128.0.184
    Mac:            02:7e:e2:0a:7c:df
    Name:            default
  Node Name:      ocplab01
  Phase:          Running
  Phase Transition Timestamps:
    Phase: Pending
    Phase: Scheduling
    Phase: Scheduled
    Phase: Running
```

条件 (conditions):

- **Status.Phase** - 現在のフェーズ
- **Status.Interfaces** - IPアドレス
- **Status.NodeName** - VMの名前
- **Status.GuestOSInfo** - ゲストOSの情報 (OSの種類、バージョンなど)

条件 (Conditions) の確認

VMの条件 (Conditions) を確認するコマンド

```
oc get vm fedora-app-vm -n my-vms -o jsonpath='{.status.conditions}' | jq
```

結果:

```
[
  {
    "lastTransitionTime": "2026-03-05T17:10:11Z",
    "status": "True",
    "type": "Ready"
  },
  {
```


コマンドを実行:

```
oc get events -n my-vms -w
```

結果:

イベント	メッセージ
SuccessfulCreate	VM作成成功
Started	VM起動成功
Stopped	VM停止 (正常)
Killing	VM強制終了
FailedScheduling	VMスケジューリング失敗
FailedCreate	VM作成失敗 (エラー)

実行戦略 (Run Strategies)

実行戦略は、VMが失敗した場合の動作を定義します。VMが失敗すると、実行戦略に基づいてVMが再実行されるか、手動で再実行されるか、または停止されるかが決定されます。

実行戦略の種類:

戦略	動作	説明
Always	VMが失敗すると、常に再実行される (再実行回数制限あり)	VMが失敗すると、常に再実行される
RerunOnFailure	VMが失敗すると、一度だけ再実行される	VMが失敗すると、一度だけ再実行される
Manual	virtctl コマンドまたはWebコンソールから手動で再実行される	手動で再実行される
Halted	VMが失敗すると、永久的に停止される	VMが失敗すると、永久的に停止される

例:

VM YAML:

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  runStrategy: Always # or RerunOnFailure, Manual, Halted
  template:
```

```
# ... VM template spec
```

Web操作步骤:

1. VM部署成功
2. **YAML** 部署成功
3. **spec.runStrategy** 部署成功
4. **Save** 部署成功

VM

fedora-app-vm

Stopped

Overview

Metrics

YAML

Configuration

Events

Console

Snapshots

Diagnostics

```

3  metadata:
94  labels:
96  spec:
97    dataVolumeTemplates:
98      - metadata:
99          creationTimestamp: null
100         name: fedora-app-vm-rootdisk
101       spec:
102         sourceRef:
103           kind: DataSource
104           name: fedora
105           namespace: openshift-virtualization-os-images
106         storage:
107           resources:
108             requests:
109               storage: 30Gi
110         runStrategy: Manual
111       template:
112         metadata:
113           creationTimestamp: null
114           labels:
115             kubevirt.io/domain: fedora-app-vm
116         spec:
117           architecture: amd64
118           domain:
119             cpu:
120               cores: 1
121               sockets: 1
122               threads: 1
123             devices:
124               disks:
125                 - disk:
126                     bus: virtio
127                     name: rootdisk
128                 - disk:
129                     bus: virtio
130                     name: cloudinitdisk
131               interfaces:
132                 - macAddress: '02:7e:e2:0a:7c:df'
133                   masquerade: {}
134                   name: default
135             firmware:

```

Save

Reload

Cancel



`runStrategy` is `running` VM `runStrategy`

Always `RerunOnFailure`

- `virtctl stop` VM

- VM **Halted** **Manual** **shutdown** **-h now** **now**
- VM **shutdown -h now** **now**

VM

VirtualMachine

- VirtualMachine **VirtualMachineInstance**
- VM **VirtualMachineInstance**
- VM **VirtualMachineInstance**
- VM (Conditions) **Events** **VM**
- VM **Run Strategies**

Virtctl

VM

virtctl CLI **OpenShift** **Virtualization** **VM** **VM**

OpenShift:

OpenShift 4.20

OpenShift

- OpenShift **Virtualization** **OpenShift 4.18**
- **virtctl** CLI **OpenShift**

virtctl

virtctl CLI **OpenShift Web** **Virtualization**

1. OpenShift Web **Virtualization** → **Overview**

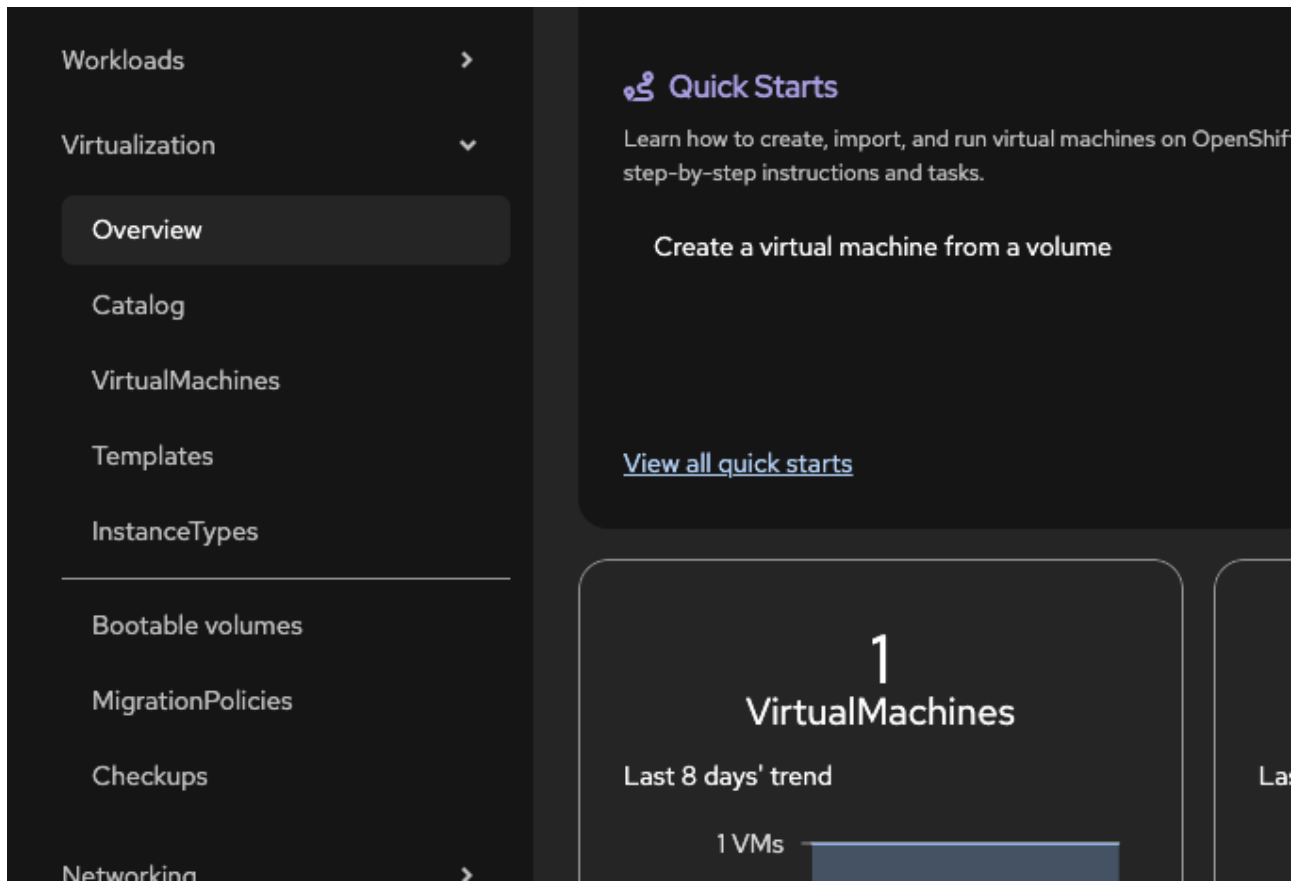


Figure 1. OpenShift Virtualization Overview

2. Overview Download the virtctl command-line utility

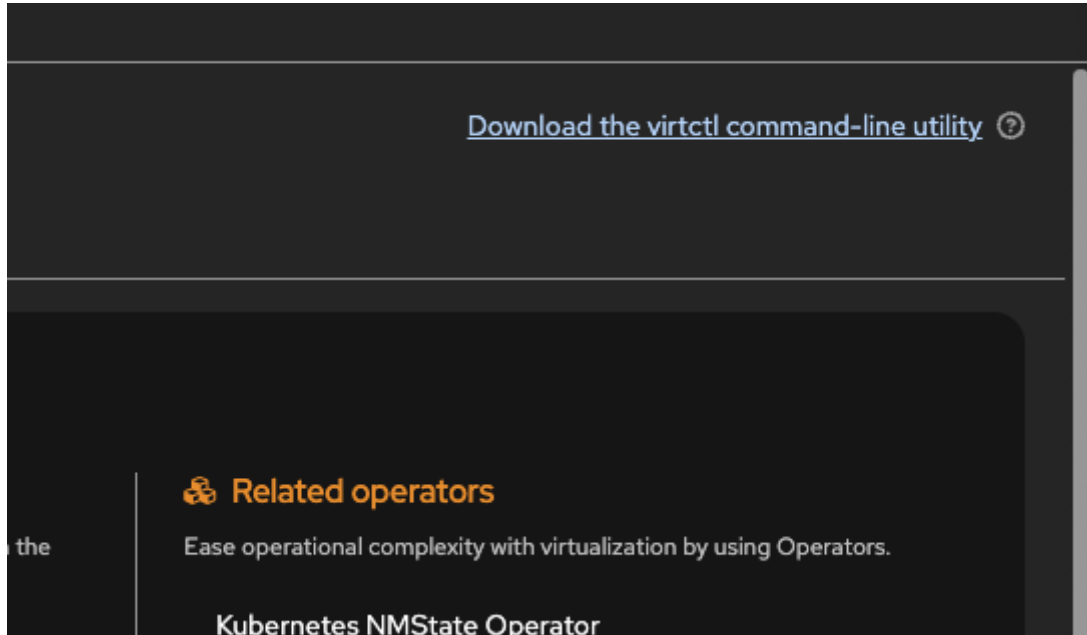


Figure 2. virtctl

3. virtctl

virtctl – KubeVirt command line interface

The virtctl client is a supplemental command-line utility for managing virtualization resources from the command line.

- [Download virtctl for Linux for ARM 64](#)
- [Download virtctl for Linux for IBM Z](#)
- [Download virtctl for Linux for x86_64](#)
- [Download virtctl for Mac for ARM 64](#)
- [Download virtctl for Mac for x86_64](#)
- [Download virtctl for Windows for ARM 64](#)
- [Download virtctl for Windows for x86_64](#)

Figure 3. virtctl のインストール

4. PATH に追加する

```
# Linux/Mac
chmod +x virtctl
sudo mv virtctl /usr/local/bin/

# Verify installation
virtctl version
```

Windows の場合は PATH に追加する

インストール

VM のインストール

VM のインストールに成功したら VM のインストール

```
# List all VMs in a specific namespace
oc get vm -n myapp

# List all VMs across all namespaces
oc get vm -A
```

例:

NAMESPACE	NAME	AGE	STATUS	READY
myapp	fedora-app-vm	13d	Running	True

IP の取得は VM のインストール時に指定する

```
oc get vmi -A
```

111

NAMESPACE	NAME	AGE	PHASE	IP	NODENAME	READY
myapp	fedora-app-vm	13d	Running	10.128.1.225	ocplab01	True



VM 000000000000 oc 000000000000 VM 00000000000000000000 virtctl 00000000

VM □□□□□□□□

VM □□□□□□□□□□□□□□

```
# Start a VM
virtctl start fedora-app-vm -n myapp

# Stop a VM
virtctl stop fedora-app-vm -n myapp

# Restart a VM
virtctl restart fedora-app-vm -n myapp
```

VM 000000

VM 00000000

```
# Console access (press Ctrl+] to exit)
virtctl console fedora-app-vm -n myapp

# SSH access (requires SSH configured)
virtctl ssh user@fedora-app-vm -n myapp

# VNC access
virtctl vnc fedora-app-vm -n myapp
```

VM □□□□□

VM 虚拟机 VM 的 QEMU guest agent 虚拟机代理

```
# Guest OS information
virtctl guestosinfo fedora-app-vm -n myapp
```

□□□:

```
{
  "guestAgentVersion": "10.1.0",
  "hostname": "fedora-app-vm",
```

```
"os": {
  "name": "Fedora Linux",
  "kernelRelease": "6.5.6-300.fc39.x86_64",
  "version": "39",
  "prettyName": "Fedora Linux 39 (Cloud Edition)",
  "versionId": "39",
  "kernelVersion": "#1 SMP PREEMPT_DYNAMIC",
  "machine": "x86_64",
  "id": "fedora"
}
```

```
# List filesystems
virtctl fslist fedora-app-vm -n myapp
```

:

```
{
  "items": [
    {
      "diskName": "vda2",
      "mountPoint": "/boot/efi",
      "fileSystemType": "vfat",
      "usedBytes": 21723136,
      "totalBytes": 104607744
    },
    {
      "diskName": "vda3",
      "mountPoint": "/boot",
      "fileSystemType": "ext4",
      "usedBytes": 91316224,
      "totalBytes": 1902895104
    },
    {
      "diskName": "vda4",
      "mountPoint": "/",
      "fileSystemType": "btrfs",
      "usedBytes": 687783936,
      "totalBytes": 29525868544
    }
  ]
}
```

```
# List logged-in users
virtctl userlist fedora-app-vm -n myapp
```

□□□□□□

virtctl scp □□□□ VM □□□□□□□□□□□□□□□□ VM □ SSH □□□□□□□□□□□□□□□□□□□□□□□□

- **SSH** □□□□□: □□□□ VM □ `~/.ssh/authorized_keys` □□□□□□
- □□□□□: VM □ SSH □□□□□□□□□□□□□□□□□□

```
# Copy to VM (uses default SSH key)
virtctl scp local-file.txt fedora@fedora-app-vm:/tmp/remote.txt -n myapp

# Copy from VM
virtctl scp fedora@fedora-app-vm:/tmp/remote.txt ./local.txt -n myapp

# Specify a different SSH key
virtctl scp -i ~/.ssh/my_key local-file.txt fedora@fedora-app-vm:/tmp/ -n myapp
```



SSH □□□□□□□□□□□□ **virtctl console** □□□□ VM □□□□□□SSH □□□□□□□□□□□□

VM □□□□ SSH □□□□□□□□□□cloud-init □□□□□□□□

```
#cloud-config
user: fedora
ssh_authorized_keys:
  - ssh-rsa AAAAB3NzaC... your-key-here
```

□□□ SSH □□□□□□□□□□□□[\[vm-configuration:remote-access-linux-vms\]](#) □□□□□□□□□□

VM □□□□□

VM □□□□□□□□□□□□□□□□□□□□

```
# Check VM status
oc get vm fedora-app-vm -n myapp
```

□□□□□□□□□:

NAME	AGE	STATUS	READY
fedora-app-vm	7d3h	Running	True

```
# Check VMI (running instance)
oc get vmi fedora-app-vm -n myapp
```

□□□□□□□: □□□□□□□ VMI □□□□□□□□□□□□ VMI □□□□□□□□□□

OpenShift VM

- OpenShift VM -n namespace
- OpenShift VM SSH
- OpenShift VM VM
- VM OpenShift oc VM virtctl
- OpenShift VM Ctrl+]
- OpenShift VM virtctl ssh

OpenShift

- [virtctl Commands Reference](#)

OpenShift

OpenShift VM

- OpenShift Web virtctl CLI
- OpenShift VM VM
- OpenShift SSH VNC VM
- QEMU guest agent OS
- virtctl scp VM

OpenShift

- [\[vm-configuration:remote-access-linux-vms\]](#) - VM

OpenShift Virtualization VM Cloud-init

OpenShift

Cloud-init OpenShift Virtualization VM Cloud-init SSH Cloud-init KubeVirt VM VirtualMachine Cloud-init

OpenShift

- Cloud-init VM
- 3 Cloud-init userData networkData metadata
- KubeVirt NoCloud Cloud-init VM
- cloud-config
- VirtualMachine Cloud-init 2 Secret
- Cloud-init

OpenShift 4.18+

Cloud-init

- OpenShift Virtualization Cloud-init OpenShift 4.18
- oc virtctl CLI Virtctl
- YAML
- VM Web VM

Cloud-init

Cloud-init OS

SSH

Cloud-init Fedora RHEL CentOS Ubuntu Debian Linux

AWS Azure GCP OpenShift

Virtualization KubeVirt

Cloud-init VM 2 Cloud-init

KubeVirt Cloud-init

VM 169.254.169.254

HTTP Cloud-init

KubeVirt NoCloud

NoCloud KubeVirt

1. VirtualMachine Secret Cloud-init
2. ISO
3. virtio VM
4. OS Cloud-init

Cloud-init Kubernetes

```
spec:
  template:
    spec:
      volumes:
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            ssh_pwauth: true
```

```

user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: changeme

```

① **cloudInitNoCloud** KubeVirtNoCloud

Cloud-init

Cloud-init KubeVirt **cloudInitNoCloud**

userData (cloud-config)

userData **cloud-config** **#cloud-config**
 YAML VM

```

cloudInitNoCloud:
  userData: |
    #cloud-config
    ssh_pwauth: true
    user:
      name: fedora
      lock_passwd: false
      plain_text_passwd: s3cur3p4ss
      ssh_authorized_keys:
        - ssh-ed25519 AAAA... user@workstation
    packages:
      - vim
      - tmux

```

1 **#cloud-config** Cloud-init cloud-config

networkData (network-config v2)

VM Netplan **network-config** **version** **2**
 IP DNS

DHCP

```

cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      enp1s0:
        dhcp4: true

```

DNS IP


```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      enp1s0:
        addresses:
          - 192.168.1.100/24 ①
        nameservers:
          addresses:
            - 192.168.1.1
            - 8.8.8.8 ②
        routes:
          - to: default
            via: 192.168.1.1 ③
          - to: 10.0.0.0/8
            via: 192.168.1.254 ④
```

① CIDR形式のIPアドレス

② DNSサーバー

③ デフォルトゲートウェイ

④ 10.0.0.0/8 へのルート

MACアドレスは、cloud-initが自動的に生成します。

```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      primary:
        match:
          macaddress: "52:54:00:12:34:56"
        addresses:
          - 192.168.1.100/24
        routes:
          - to: default
            via: 192.168.1.1
```

例: KubeVirt VMのNetworkAttachmentDefinitionとVirtualMachine

VLANとIPアドレス

metadata

instance-id: local-hostname

例: KubeVirt VirtualMachine

```
cloudInitNoCloud:
  metadata: |
    instance-id: my-vm-001
    local-hostname: my-vm
```

####: #####userData#### hostname #####

Cloud-init#####

Cloud-init#####5#####

####	#####	##
Generator	##### (systemd generator)	Cloud-init#####
Local	#####	networkData#####
Network	#####	#####
Config	Network#####	#####SSH#####
Final	#####	runcmd #####

#####networkData#####userData#####**runcmd**
#####

####Cloud-config####

#####VM##### **#cloud-config** userData#####

#####

```
#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: s3cur3p4ss
  ssh_authorized_keys:
    - ssh-ed25519 AAAA...key-data... user@workstation
```

user ##### **lock_passwd: false** **plain_text_passwd** #####
ssh_pwauth: true #####SSH#####

#####**users** #####

```
#cloud-config
users:
```


[illegible]

VirtualMachineCloud-init

VirtualMachineCloud-init2

□□□□□□ (cloudInitNoCloud)

```
Cloud-initVirtualMachine
```

minimal-cloud-init-vm.yaml □□□□□□

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-cloud-init-demo
  namespace: demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        app: fedora-cloud-init-demo
    spec:
      domain:
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
        resources:
          requests:
            memory: 2Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - dataVolume:
            name: fedora-cloud-init-demo
            name: rootdisk
        - cloudInitNoCloud:
            userData: |
              #cloud-config
              ssh_pwauth: true
```

①

②

```

      user:
        name: fedora
        lock_passwd: false
        plain_text_passwd: s3cur3p4ss
        ssh_authorized_keys:
          - 'ssh-ed25519 AAAA...your-key... user@workstation'
      users:
        - name: userone
          lock_passwd: false
          plain_text_passwd: s3cur3p4ss
          ssh_authorized_keys:
            - 'ssh-ed25519 AAAA...your-key... user@workstation'
      packages:
        - vim
        - git
      runcmd:
        - echo "Cloud-init complete" > /tmp/cloud-init-done
    name: cloudinitdisk
dataVolumeTemplates:
  - metadata:
      name: fedora-cloud-init-demo
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
        resources:
          requests:
            storage: 30Gi

```

① `cloudInitNoCloud` `NoCloud`

② `userData` `#cloud-config` `cloud-config` `YAML`

`Secret`

Secret

`Cloud-init` `Kubernetes Secret` `VirtualMachine`

`cloud-init-secret-vm.yaml`

`Secret`

```

oc create secret generic my-vm-cloud-init \
  --from-literal=userdata='#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false

```

```

plain_text_passwd: s3cur3p4ss
ssh_authorized_keys:
  - ssh-ed25519 AAAA...your-key... user@workstation
users:
  - name: userone
    lock_passwd: false
    plain_text_passwd: s3cur3p4ss
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...your-key... user@workstation
packages:
  - vim
  - git
' \
-n demo

```

VirtualMachine

```

volumes:
  - cloudInitNoCloud:
      secretRef:
        name: my-vm-cloud-init ①
      name: cloudinitdisk

```

① KubeVirt Secret `userdata` Cloud-init `userData`

Cloud-init

VM Cloud-init

VM

```
virtctl console fedora-cloud-init-demo -n demo
```

Cloud-init

```
cloud-init status --long
```

```

status: done
extended_status: done
boot_status_code: enabled-by-generator
detail:
DataSourceNoCloud [seed=/dev/vdb]

```

Cloud-init

```
cat /var/log/cloud-init.log | tail -50
```

Cloud-init

#cloud-config

userData1 #cloud-config Cloud-init
inituserData

YAML

Cloud-configYAMLVM cloud-init devel
schema --config-file /path/to/config.yaml

userData

userData networkData userDatanetwork-config
v2YAML

Cloud-init

Cloud-initcloud-init clean

Secret

secretRef Secret userdata userData user-data

Windows VM Sysprep

Cloud-initLinuxWindows VMOpenShift Virtualization Sysprep
Microsoft System
Preparation

Windows VMWindows VM

```
oc delete vm fedora-cloud-init-demo -n demo
oc delete vm fedora-cloud-init-secret-demo -n demo
oc delete secret my-vm-cloud-init -n demo
oc delete namespace demo
```

- Cloud-initLinux VM

- KubeVirt NoCloud Cloud-init VM virtio
- 3 userdata cloud-config networkData network-config v2 metadata
- Cloud-init /
- 2 cloudInitNoCloud Secret
- `cloud-init status --long` `/var/log/cloud-init.log`

□□□□

- IP - Cloud-init IP DHCP VLAN
- Cloud-init - Cloud-init
- [vm-configuration:remote-access-windows-vm] - Sysprep Windows VM
- Cloud-init
- Cloud-init
- KubeVirt cloudInitNoCloud

OpenShift Virtualization

Overview

OpenShift Virtualization is a container management solution that allows you to run virtual machines (VMs) on OpenShift. It uses LVM Storage Operator and HostPath Provisioner to manage storage.

OpenShift Virtualization uses VMs to run applications. VMs are managed by the OpenShift Virtualization operator.

Architecture

The architecture of OpenShift Virtualization is as follows:

- OpenShift Virtualization uses LVM Storage Operator to manage storage.
- OpenShift Virtualization uses HostPath Provisioner to manage storage.
- OpenShift Virtualization uses Ceph/ODF to manage storage.
- OpenShift Virtualization uses NFS to manage storage.
- OpenShift Virtualization uses AWS EBS/Azure Disk to manage storage.

OpenShift Virtualization Architecture

OpenShift Virtualization architecture is as follows:

- **LVM Storage:** Logical Volume Manager is used to manage storage.
- **HostPath Provisioner:** HostPath Provisioner is used to manage storage.
- **Ceph/ODF:** Ceph/ODF is used to manage storage.
- **NFS:** NFS is used to manage storage.
- **Cloud Storage:** AWS EBS/Azure Disk is used to manage storage.

OpenShift Virtualization Single Node OpenShift (SNO) architecture is as follows:

Architecture

The architecture of OpenShift Virtualization Single Node OpenShift (SNO) is as follows:

- OpenShift 4.18 is used to manage storage.
- OpenShift Virtualization is used to manage storage.
- OpenShift Virtualization uses LVM Storage Operator to manage storage.
- Linux is used to manage storage.

□ □ □ □ □ □

LVM Operator

```

#####LVM Storage Operator#####

```

[storage:lvm-troubleshooting]

[illegible]

[storage:hostpath-provisioner]

```

#####HostPath Provisioner#####

```

[storage:multi-disk-configuration]

StorageClass VM

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

■■■■■■■■■■■■■■■■■■■■OpenShift Virtualization■■■■■■■■■■■■■■

- **DataSource OS**: DataSource OS is the operating system that runs the VM.
- **VM**: The virtual machine that runs on the DataSource OS.
- **VM**: The virtual machine that runs on the DataSource OS.
- **VM**: The virtual machine that runs on the DataSource OS.

■■■■■■■■LVM Operator■■■■ Step 5: Set Default StorageClass ■■■■■■■■

□ □ □ □ □ □

[illegible]

- 虚拟机
- 虚拟机

□ □ □ □

- [OpenShift LVM Storage](#) [OpenShift LVM Storage](#)

LVM Operator

11

```

OpenShift
Operator
Logical
Manager
LVM
OpenShift
SNO
LVM Storage Operator
Volume

```



```
root:
```

```
PV VG Fmt Attr PSize PFree /dev/sda vg1 lvm2 a-- 558.37g <55.84g
```

```
root`/dev/sda` oc create -f - --namespace=lvm-operator --forceWipeDevicesAndDestroyAllData: true`
oc create -f - --namespace=lvm-operator --forceWipeDevicesAndDestroyAllData: true`

[source,bash,role=execute]
```

```
oc debug node/<node-name> -- chroot /host ls -la /dev/disk/by-path/ | grep -E "pci.*scsi"
```

```
root:
```

```
lrwxrwxrwx. 1 root root 9 Oct 6 13:30 pci-0000:03:00.0-scsi-0:2:1:0 -> ../../sda
```

```
root`/dev/disk/by-path/pci-0000:03:00.0-scsi-0:2:1:0` oc create -f - --namespace=lvm-operator
== 2: LVM Storage Operator
oc create -f - --namespace=lvm-operator --forceWipeDevicesAndDestroyAllData: true`
LVM Operator
xref:attachment$lvm-operator/lvm-namespace.yaml[lvm-namespace.yaml]
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: Namespace metadata: name: openshift-lvm-storage EOF
```

```
OperatorGroup
xref:attachment$lvm-operator/lvm-operatorgroup.yaml[lvm-operatorgroup.yaml]
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: operators.coreos.com/v1 kind: OperatorGroup metadata: name:
openshift-lvm-storage-operatorgroup namespace: openshift-lvm-storage spec: targetNamespaces: -
openshift-lvm-storage EOF
```

```
OperatorSubscription
xref:attachment$lvm-operator/lvm-subscription.yaml[lvm-subscription.yaml]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: operators.coreos.com/v1alpha1 kind: Subscription metadata: name: lvms-operator namespace: openshift-lvm-storage spec: channel: stable-4.20 name: lvms-operator source: redhat-operators sourceNamespace: openshift-marketplace installPlanApproval: Automatic EOF
```

```
Operator
```

```
[source,bash,role=execute]
```

Wait for the operator to be installed (may take 2-3 minutes)

```
oc get csv -n openshift-lvm-storage
```

Check operator pod is running

```
oc get pods -n openshift-lvm-storage
```

```
OperatorLVMLcluster:
```

```
NAME READY STATUS RESTARTS AGE lvms-operator-864b6975bb-kxt4t 1/1 Running 0 2m
```

```
NOTE: 'lvms-operator' Pod 'vg-manager'
Pod3LVMLcluster
```

```
3LVMLcluster:
```

```
NAME READY STATUS RESTARTS AGE lvms-operator-864b6975bb-kxt4t 1/1 Running 0 5m vg-
manager-xxxxx 1/1 Running 0 1m
```

```
== 3: LVMLcluster
```

```
LVM LVMLcluster
```

```
===
```

```
'sda' 'forceWipeDevicesAndDestroyAllData: true'
```

```
NOTE: 'forceWipeDevicesAndDestroyAllData: true'
```

```
=====
```

```
xref:attachment$lvm-operator/lvmcluster-with-wipe.yaml[lvmcluster-with-wipe.yaml  
=====]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: lvm.topolvm.io/v1alpha1 kind: LVMCluster metadata: name:  
lvmcluster namespace: openshift-lvm-storage spec: storage: deviceClasses: - name: vg1 default: true  
thinPoolConfig: name: thin-pool-1 sizePercent: 90 overprovisionRatio: 10 nodeSelector:  
nodeSelectorTerms: - matchExpressions: - key: node-role.kubernetes.io/worker operator: In values:  
- "" deviceSelector: paths: - /dev/sda forceWipeDevicesAndDestroyAllData: true EOF
```

```
=== =====
```

```
=====:
```

```
xref:attachment$lvm-operator/lvmcluster-clean-disk.yaml[lvmcluster-clean-disk.yaml  
=====]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: lvm.topolvm.io/v1alpha1 kind: LVMCluster metadata: name:  
lvmcluster namespace: openshift-lvm-storage spec: storage: deviceClasses: - name: vg1 default: true  
thinPoolConfig: name: thin-pool-1 sizePercent: 90 overprovisionRatio: 10 nodeSelector:  
nodeSelectorTerms: - matchExpressions: - key: node-role.kubernetes.io/worker operator: In values:  
- "" deviceSelector: paths: - /dev/sda EOF
```

```
=== =====
```

```
* **`name: vg1`**: =====  
* **`default: true`**: deviceClass =====  
** **`**: deviceClass =====  
** **`**: PVC=====1  
** **`**: deviceClass =====1  
* **`thinPoolConfig`**: =====  
** `sizePercent: 90`: 90%  
** `overprovisionRatio: 10`: 10=====  
* **`nodeSelector`**: =====  
* **`deviceSelector.paths`**: =====  
** `/dev/sda`/dev/disk/by-path/...`/dev/disk/by-id/...` =====  
* **`forceWipeDevicesAndDestroyAllData: true`**: =====  
** **`**: =====  
** **`**: LVM=====  
** **`**: - =====  
** **`**: /=====
```

```
== 4: LVMCluster=====
```

```
LVMCluster
```

```
[source,bash,role=execute]
```

Check LVMCluster status

```
oc get lvmcluster -n openshift-lvm-storage
```

Verify volume group was created

```
oc debug node/<node-name> — chroot /host vgs
```

Verify thin pool was created

```
oc debug node/<node-name> — chroot /host lvs
```

```
root@node1:~#
```

```
* LVMCluster 1 'STATUS: Ready'
* VG 'vg1'
* thin pool 'thin-pool-1' /dev/mapper/
```

```
xref:lvm-troubleshooting.adoc[LVM Storage ]
```

```
[[_step_5_set_default_storageclass]]
== 5: 
```

```
LVM Operator 'lvms-vg1'
VirtualizationVMDataVolume
```

```
=== 
```

```
[source,bash,role=execute]
```

```
oc get storageclass
```

```
root@node1:~#
```

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
lvms-vg1 topolvm.io Delete WaitForFirstConsumer true 5m
```

```
NOTE: '(default)'
```

```
===  {{{{
```

```
LVM{{{
```

```
[source,bash,role=execute]
```

```
oc patch storageclass lvms-vg1 -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

```
===  {{{{
```

```
[source,bash,role=execute]
```

```
oc describe sc | grep IsDefaultClass
```

```
#####:
```

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
lvms-vg1 (default) topolvm.io Delete WaitForFirstConsumer true 6m
```

```
`(default)` #####
```

```
=== OpenShift Virtualization#####
```

```
#####OpenShift Virtualization#####
```

```
==== DataSource OS#####
```

```
OpenShift VirtualizationDataSource OS#####FedoraRHELCentOS
StreamWindows#####
```

```
[source,bash,role=execute]
```

Check available OS images

```
oc get datasources -n openshift-virtualization-os-images
```

```
#####DataSource#####VM#####
```

```
==== VM#####PVC
```

```
`dataVolumeTemplates` #####VM#####`storageClassName` #####
```



```
[source,yaml]
```

```
dataVolumeTemplates: - metadata: name: fedora-volume spec: pvc: accessModes: - ReadWriteOnce
resources: requests: storage: 30Gi sourceRef: kind: DataSource name: fedora namespace: openshift-
virtualization-os-images
```

```
#####`lvms-vg1`#####

==== VM#####

#####

* VM#####
* DataSource#####
* PVC `storageClassName` #####
* #####

==== #####

#####OpenShift Virtualization OperatorOS#####

[source,bash,role=execute]
```

Check boot source preparation status

```
oc get DataImportCron -n openshift-virtualization-os-images
```

```
=== #####VM#####

#####VM#####

[source,yaml]
```

```
dataVolumeTemplates: - metadata: name: fedora-volume spec: pvc: accessModes: - ReadWriteOnce
resources: requests: storage: 30Gi storageClassName: lvms-vg1 # Must be explicit sourceRef: kind:
DataSource name: fedora namespace: openshift-virtualization-os-images
```

```
NOTE: #####OpenShift Virtualization#####

== 6: #####

#####PVC#####
```


Expected output: LVM Storage Test

```
#####
```

```
[source,bash,role=execute]
```

oc delete pod lvms-test-pod oc delete pvc lvms-test-pvc

```
== #####
```

```
LVM#####xref:lvn-troubleshooting.adoc[LVM Storage #####]
```

```
#####
```

- * #####
- * #####
- * #####
- * PVC#####
- * #####
- * #####

```
== forceWipeDevicesAndDestroyAllData #####
```

```
=== #####
```

```
`forceWipeDevicesAndDestroyAllData: true` #####
```

- . **#####**: GPTMBR#####
- . **#####**: ext4xfsbtrfs#####
- . **LVM#####**: #####
- . **#####**: #####
- . **#####**: LVM Operator#####

```
=== #####
```

```
`forceWipeDevicesAndDestroyAllData: true` #####
```

- * #####
- * #####LVM#####PV/VG/LV#####
- * #####
- * #####

```
#####
```

- * #####
- * #####
- * #####


```

storage[日本語,window=_blank]

:!navtitle:

:docname: lvm-troubleshooting
:page-module: storage
:page-relative-src-path: lvm-troubleshooting.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: LVM 日本語
[#storage:lvm-troubleshooting]
=== LVM 日本語

[discrete#storage:lvm-troubleshooting::_]
====

日本語OpenShift 日本語 LVM Storage Operator 日本語LVM
日本語 (PVC) 日本語

[discrete#storage:lvm-troubleshooting::_日本語]
==== 日本語

LVM 日本語

[source,bash,role=execute]

```

1. Check LVMCluster status

```
oc get lvmcluster -n openshift-lvm-storage
```

2. Check all pods are running

`oc get pods -n openshift-lvm-storage`

3. Check StorageClass exists

`oc get storageclass | grep lvms`

4. Check volume group on node

```
oc debug node/<node-name> — chroot /host vgs
```

5. Check thin pool

oc debug node/<node-name> — chroot /host lvs

```
root@node1:~#

* LVMCluster: `STATUS: Ready`
* Pods: 1/1 `Running`
* StorageClass: `lvms-vg1`
* VG: lvms-vg1
* LV: lvms-vg1/lvms-vg1

[discrete#storage:lvm-troubleshooting::_root@node1:~#]
==== root@node1:~#

[discrete#storage:lvm-troubleshooting::_root@node1:~#]
===== LVMCluster 1/1

[discrete#storage:lvm-troubleshooting::_root@node1:~#]
===== root@node1:~#

[source,bash,role=execute]
```

oc get lvmcluster -n openshift-lvm-storage

```
root@node1:~#
```

NAME STATUS lvmcluster Ready

```
root@node1:~#

* lvmcluster `NotReady` `Degraded` `Error`
* lvmcluster

[discrete#storage:lvm-troubleshooting::_root@node1:~#]
===== root@node1:~#

[source,bash,role=execute]
```

oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml

```
root@node1:~#

[source,yaml]
```

status: conditions: - status: "True" type: ResourcesAvailable message: Reconciliation is complete and all the resources are available - status: "True" type: VolumeGroupsReady message: All the VGs are ready deviceClassStatuses: - name: vg1 nodeStatus: - devices: - /dev/sda status: Ready ready: true state: Ready

```
#####:

* `status.conditions`: `ResourcesAvailable: True`   `VolumeGroupsReady: True`
#####
* `deviceClassStatuses[].nodeStatus[].status`: `Ready`   #####
* `deviceClassStatuses[].nodeStatus[].devices`: #####
* `ready: true`   `state: Ready`   #####

[discrete#storage:lvm-troubleshooting::_#####]
=====   #####

[source,bash,role=execute]
```

oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 50 "deviceClassStatuses"

```
###:

[source,yaml]
```

deviceClassStatuses: - name: vg1 nodeStatus: - devices: - /dev/sda excluded: - name: /dev/sdc reasons: - /dev/sdc has children block devices and could not be considered - /dev/sdc is not part of the device selector status: Ready

```
#####

[discrete#storage:lvm-troubleshooting::_pod_####]
===== Pod   ####

[discrete#storage:lvm-troubleshooting::_####_pod_####]
=====   #### Pod   ####

[source,bash,role=execute]
```

oc get pods -n openshift-lvm-storage

```
####:
```

NAME READY STATUS RESTARTS AGE lvms-operator-864b6975bb-kxt4t 1/1 Running 0 10m vg-manager-xxxxxx 1/1 Running 0 5m

Pod:

* **lvms-operator**: LVM

* **vg-manager**: 1

* **topolvm-controller**: Single Node OpenShift (SNO)

[discrete#storage:lvm-troubleshooting::_pod_]

==== Pod

[source,bash,role=execute]

Get pod details

`oc describe pod -n openshift-lvm-storage <pod-name>`

Check specific pod logs

```
oc logs -n openshift-lvm-storage <pod-name>
```

Check vg-manager logs (most useful for device issues)

oc logs -n openshift-lvm-storage daemonset/vg-manager

#####:

```
* `"device is in use"` - #####  
* `"cannot wipe device"` - `forceWipeDevicesAndDestroyAllData: true` #####  
* `"device not found"` - #####  
* `"no devices found"` - #####
```

```
[discrete#storage:lvm-troubleshooting::_operator_#####]  
===== Operator #####
```

```
[source,bash,role=execute]
```

Check ClusterServiceVersion

```
oc get csv -n openshift-lvm-storage
```


Check subscription

```
oc get subscription -n openshift-lvm-storage
```

Check install plan

oc get installplan -n openshift-lvm-storage

```
[discrete#storage:lvm-troubleshooting::_storageclass_000]
```

```
===== 0000000000
```

```
[discrete#storage:lvm-troubleshooting::_storageclass_000]
```

```
===== StorageClass 000
```

```
[source,bash,role=execute]
```

List all StorageClasses

oc get storageclass

Get specific LVM StorageClass

```
oc get storageclass lvms-vg1 -o yaml
```

```
Annotations:
```

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
lvms-vg1 topolvm.io Delete WaitForFirstConsumer true 10m
```

```
Annotations:
```

```
* 'PROVISIONER: topolvm.io' - topolvm.io
* 'VOLUMEBINDINGMODE: WaitForFirstConsumer' - WaitForFirstConsumer
* 'ALLOWVOLUMEEXPANSION: true' - true

[discrete#storage:lvms-troubleshooting::_storageclass_Annotations]
===== StorageClass Annotations

[source,bash,role=execute]
```

```
oc get storageclass lvms-vg1 -o jsonpath='{.metadata.annotations.storageclass\kubernetes\io\is-
default-class}'
```

```
Annotations:
```

```
* 'true' - true StorageClass Annotations
* Annotations 'false' - false

[discrete#storage:lvms-troubleshooting::_persistentvolumes_Annotations]
===== PersistentVolumes Annotations

[source,bash,role=execute]
```

List all PVs

oc get pv

Check PVs using LVM storage

oc get pv | grep topolvm

```
[discrete#storage:lvm-troubleshooting::_root]
===== root

[discrete#storage:lvm-troubleshooting::_root]
===== root

[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host vgs

```
root:
```

VG #PV #LV #SN Attr VSize VFree vg1 1 1 0 wz--n- 558.37g <55.84g

```
root:

* VG root LVMCluster root (root: `vg1`)
* `VSize` root
* `VFree` root
* `Attr` root `wz--n-` (root) root

[discrete#storage:lvm-troubleshooting::_root]
===== root

[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host vgdisplay vg1

```
[discrete#storage:lvm-troubleshooting::_root]
root:
```

--- Volume group --- VG Name vg1 System ID Format lvm2 Metadata Areas 1 Metadata Sequence No 2 VG Access read/write VG Status resizable MAX LV 0 Cur LV 1 Open LV 0 Max PV 0 Cur PV 1 Act PV 1 VG Size 558.37 GiB PE Size 4.00 MiB Total PE 142942 Alloc PE / Size 128517 / 502.04 GiB Free PE / Size 14425 / 56.35 GiB VG UUID abc123...

```
root:

* `VG Status: resizable`
* `VG Access: read/write`
```

```
* 'Free PE / Size' 0000000000000000
```

```
=== 0000000000000000
```

```
==== 000000000000000000000000
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host lvs

```
000000:
```

LV VG Attr LSize Pool Origin Data% Meta% thin-pool-1 vg1 twi-a-tz-- <502.04g 0.00 6.76

```
000000 (twi-a-tz--):
```

```
* 't' - thin pool
```

```
* 'w' - writeable
```

```
* 'i' - inherited
```

```
* 'a' - active
```

```
* 't' - thin
```

```
* 'z' - zero
```

```
Data% 000 Meta% 000000:
```

```
* **Data%***: 00000000000000000000
```

```
** '< 80%' - 00
```

```
** '80-90%' - 00000000
```

```
** '> 90%' - 000000000000
```

```
* **Meta%***: 0000000000
```

```
** '< 80%' - 00
```

```
** '> 80%' - 00000000000000000000
```

```
==== 00000000000000000000
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host lvdisplay vg1/thin-pool-1

```
=== 000000000000
```

```
==== 000000000000
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host pvs

```
#####:
```

```
PV VG Fmt Attr PSize PFree /dev/sda vg1 lvm2 a-- 558.37g <55.84g
```

```
##### (a--):
```

```
* `a` - allocatable
* `-' - not exported
* `-' - not missing
```

```
==== #####
```

```
[source,bash,role=execute]
```

```
oc debug node/<node-name> — chroot /host pvdisplay /dev/sda
```

```
==== #####
```

```
[source,bash,role=execute]
```

Check device exists

```
oc debug node/<node-name> — chroot /host ls -la /dev/sda
```

Check device is a block device

```
oc debug node/<node-name> — chroot /host lsblk /dev/sda
```

Check device is not mounted

```
oc debug node/<node-name> — chroot /host mount | grep /dev/sda
```

```
##: #####LVM #####
```

```
=== #####
```

```
==== #####
```

```
[source,bash,role=execute]
```

```
oc          debug          node/<node-name> — chroot          /host          lsblk          -o
NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL
```



```
root:
```

```
NAME SIZE TYPE FSTYPE MOUNTPOINT MODEL sda 558.4G disk LVM2_member PERC H730 Adp
sdc 558.4G disk PERC H730 Adp |-sdc1 1M part |-sdc2 127M part vfat |-sdc3 384M part ext4 /boot `
sdc4 557.9G part xfs /sysroot
```

```
root@node1:~#
```

```
* MOUNTPOINT / - LVM2_member
* FSTYPE xfs 'LVM2_member' (LVM2_member)
* MOUNTPOINT / - LVM2_member
* LVM2_member (1) - LVM2_member
```

```
==== LVM2_member
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host lsblk -d -o NAME,SIZE,SERIAL

```
root@node1:~#
```

```
==== LVM2_member
```

```
[source,bash,role=execute]
```

By path

oc debug node/<node-name> — chroot /host ls -la /dev/disk/by-path/ | grep sda

By ID

oc debug node/<node-name> — chroot /host ls -la /dev/disk/by-id/ | grep sda

```
== LVM2_member
```

```
=== 1: LVMcluster 1 "NotReady" LVM2_member
```

```
root:
```

NAME STATUS lvmcluster NotReady

```
root:
```

```
[source,bash,role=execute]
```

Check detailed status

```
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 20 "conditions"
```

Check vg-manager logs

```
oc logs -n openshift-lvm-storage daemonset/vg-manager
```

```
root@node01:~#
```

```
====
```

```
[source,bash,role=execute]
```

Verify device exists

```
oc debug node/<node-name> -- chroot /host ls -la /dev/sda
```

```
root@node01:~# LVMCluster
```

```
====
```

```
[source,bash,role=execute]
```

Check for existing filesystem

```
oc debug node/<node-name> -- chroot /host lsblk -f /dev/sda
```

```
root@node01:~# deviceSelector `forceWipeDevicesAndDestroyAllData: true`
```

```
====
```

```
[source,bash,role=execute]
```

Check if mounted

oc debug node/<node-name> — chroot /host mount | grep /dev/sda

```
root: ~
===== 2: ~

root:

* LVMCluster ~ `vgs` ~
* ~ deviceClassStatuses ~

root:

[source,bash,role=execute]
```

Check if disk was wiped

oc debug node/<node-name> — chroot /host pvs

Check vg-manager logs for errors

oc logs -n openshift-lvm-storage daemonset/vg-manager --tail=100

Check device visibility

oc debug node/<node-name> — chroot /host lsblk -o NAME,SIZE,TYPE,FSTYPE

```
root:

=====

* deviceSelector ~
* ~ (deviceClassStatuses.excluded ~)

=====

* vg-manager Pod ~
* SELinux ~

=====

* `forceWipeDevicesAndDestroyAllData: true` ~
```

```
* 0Operator
```

```
=== 3: PVC Pending
```

```
:
```

```
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS AGE test-pvc Pending lvms-vg1
5m
```

```
:
```

```
[source,bash,role=execute]
```

Check PVC events

```
oc describe pvc <pvc-name>
```

Check if pod using PVC exists

```
oc get pods --all-namespaces -o wide | grep <pvc-name>
```

Check available space in VG

```
oc debug node/<node-name> — chroot /host vgs
```

```
:
```

```
==== WaitForFirstConsumer - Pod
```

```
* **PVC**: PVC Pod PVC Pending
```

```
* **Pod**: PVC Pod
```

```
====
```

```
[source,bash,role=execute]
```

Check VG free space

```
oc debug node/<node-name> — chroot /host vgdisplay vg1 | grep "Free"
```

```
: PV
```

```
==== 000000000000000000000000
```

```
[source,bash,role=execute]
```

Check thin pool usage

oc debug node/<node-name> — chroot /host lvs | grep thin-pool

```
000: 0000000000000000000000000000000000000000000000000000000000000000
```

```
==== 000000000000
```

```
[source,bash,role=execute]
```

Check pod node selector vs LVMCluster node selector

oc get pod <pod-name> -o yaml | grep nodeSelector oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 5 nodeSelector

```
=== 00 4: 00000000000000 Data% 000 Meta% 000
```

```
00:
```

LV VG Attr LSize Data% Meta% thin-pool-1 vg1 twi-a-tz-- 502.04g 92.00 85.00

```
00:
```

```
[source,bash,role=execute]
```

Check thin pool details

oc debug node/<node-name> — chroot /host lvs -a | grep thin-pool

List all thin volumes

oc debug node/<node-name> — chroot /host lvs -a -o lv_name,lv_size,data_percent,pool_lv | grep thin-pool

Check which PVs are consuming space

```
oc get pv -o custom-columns=NAME:.metadata.name,SIZE:.spec.capacity.storage,STORAGECLASS:.spec.storageClassName | grep lvms
```

```
NAME:
===== NAME PVC SIZE
[source,bash,role=execute]
```

Find PVCs not in use

```
oc get pvc --all-namespaces
```

Delete PVC

```
oc delete pvc <pvc-name> -n <namespace>
```

```
===== NAME (VG SIZE)
[source,bash,role=execute]
```

Check VG free space

```
oc debug node/<node-name> -- chroot /host vgs
```

Extend thin pool (if needed manually)

Ensure forceWipeDevicesAndDestroyAllData is set

oc edit lvmcluster lvmcluster -n openshift-lvm-storage

Add under deviceSelector:

forceWipeDevicesAndDestroyAllData: true

```
=== 6: vg-manager Pod  CrashLooping
```

```
:
```

```
NAME READY STATUS RESTARTS AGE vg-manager-xxx 0/1 CrashLoopBackOff 5 3m
```

```
:
```

```
[source,bash,role=execute]
```

Check pod logs

oc logs -n openshift-lvm-storage <vg-manager-pod> --previous

Check pod events

oc describe pod -n openshift-lvm-storage <vg-manager-pod>

Check node resources

oc describe node <node-name>

```
ResourceList:
```

```
==== ResourceList
```

```
* vg-manager  ResourceList
```

```
* SELinux  ResourceList
```



```

==== 检查

* 检查存储类名称
* 检查存储类名称

==== 检查

* 检查存储类 CPU 使用率
* 检查存储类名称

=== 第 7: StorageClass 检查

检查:

* storageClassName 检查 PVC 使用率
* 检查: "no default deviceClass was specified"

检查:

[source,bash,role=execute]

```

Check if any StorageClass is default

oc get storageclass | grep default

Check lvms StorageClass annotations

oc get storageclass lvms-vg1 -o yaml | grep is-default-class

```

检查:

==== LVM StorageClass 检查

[source,bash,role=execute]

```

oc patch storageclass lvms-vg1 -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

```

==== 检查 LVMCluster 检查

[source,bash,role=execute]

```

```
oc edit lvmcluster lvmcluster -n openshift-lvm-storage
```

Add under deviceClasses:

- name: vg1 default: true

```
==  []

LVM  []

[source,bash,role=execute]
```

```
#!/bin/bash
```

```
NAMESPACE="openshift-lvm-storage" NODE_NAME="<your-node-name>"
```

```
echo "=== LVM Storage Diagnostic Report ===" echo ""
```

```
echo "1. LVMCluster Status:" oc get lvmcluster -n $NAMESPACE echo ""
```

```
echo "2. Pods Status:" oc get pods -n $NAMESPACE echo ""
```

```
echo "3. StorageClass:" oc get storageclass | grep lvms echo ""
```

```
echo "4. PVs:" oc get pv | grep topolvm echo ""
```

```
echo "5. Volume Groups:" oc debug node/$NODE_NAME -- chroot /host vgs echo ""
```

```
echo "6. Logical Volumes:" oc debug node/$NODE_NAME -- chroot /host lvs echo ""
```

```
echo "7. Physical Volumes:" oc debug node/$NODE_NAME -- chroot /host pvs echo ""
```

```
echo "8. Block Devices:" oc debug node/$NODE_NAME -- chroot /host lsblk -o
NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT echo ""
```

```
echo "9. Recent vg-manager logs:" oc logs -n $NAMESPACE daemonset/vg-manager --tail=20 echo ""
```

```
echo "=== End of Diagnostic Report ==="
```

```
==  []

[]

* LVMCluster Pod StorageClass []
* LVMCluster  NotReady [] Pending  [] PVC  []
* `vgs` `lvs` `pvs`  []
*  []vg-manager  CrashLoopBackOff StorageClass  []
*  []LVM  []
```

```
== 00
```

```
* xref:lvm-operator.adoc[LVM Operator 00000000000000000000] - 00000000000000000000
```

```
:!navtitle:
```

```
:docname: hostpath-provisioner
```

```
:page-module: storage
```

```
:page-relative-src-path: hostpath-provisioner.adoc
```

```
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
```

```
:page-origin-start-path:
```

```
:page-origin-refname: main
```

```
:page-origin-reftype: branch
```

```
:page-origin-refhash: (worktree)
```

```
:navtitle: HostPath Provisioner
```

```
[#storage:hostpath-provisioner]
```

```
=== HostPath Provisioner
```

```
[discrete#storage:hostpath-provisioner::_00]
```

```
===== 00
```

HostPath Provisioner (HPP) 00OpenShift Virtualization 00HPP

0 OpenShift Virtualization 000 (Persistent Volume)

00000000

```
[discrete#storage:hostpath-provisioner::_hostpath_provisioner_00000]
```

```
===== HostPath Provisioner 00000
```

HPP 00000000000000000000

* **000000000000** : 000

* **000000000000** : 000

* **0000000000** : 000

* **000000000000** : 000

```
[discrete#storage:hostpath-provisioner::_lvm_storage_00000]
```

```
===== LVM Storage 0000
```

```
[cols="2,2,2"]
```

```
|==
```

```
|00 |HostPath Provisioner |LVM Storage
```

```
|000000000000
```

```
|0000 (000000000000)
```

```
|000 (000000000000)
```

```
|000000000000
```

```
|00
```

```
|00
```

```
|000000000000
```

```

|
|

|
| ( )
| ( )

|
| ( )
|

|
| /
|

|===

xref:storage:lvm-operator[LVM Storage Operator]

[discrete#storage:hostpath-provisioner::_]
=====

* HostPathProvisioner
* HPP StorageClass
*
* PVC VM

:

```

OpenShift 4.21+ OpenShift Virtualization 4.21+

```

[discrete#storage:hostpath-provisioner::_]
=====

* **OpenShift 4.21+** (OpenShift Virtualization )
* **:: HostPathProvisioner StorageClass
* **:: VM
* **oc CLI**:
* **virtctl CLI**: VM

OpenShift Virtualization

[source,bash,role=execute]

```

oc get csv -n openshift-cnv | grep kubevirt

```


```



```
xref:attachment$hostpath-provisioner/hpp-hostpathprovisioner.yaml[hpp-
hostpathprovisioner.yaml]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: hostpathprovisioner.kubevirt.io/v1beta1 kind: HostPathProvisioner
metadata: name: hostpath-provisioner spec: imagePullPolicy: IfNotPresent storagePools: - name:
local path: /var/hpvolumes workload: nodeSelector: kubernetes.io/os: linux EOF
```

```
===
```

```
* **`storagePools`**: 1
** `name`: StorageClass
** `path`:
* **`workload.nodeSelector`**: HPP CSI
* **`imagePullPolicy`**: 
```

```
=== HostPathProvisioner
```

```
HostPathProvisioner
```

```
[source,bash,role=execute]
```

```
oc get hostpathprovisioner hostpath-provisioner
```

```
:
```

```
NAME AGE hostpath-provisioner 30s
```

```
CSI
```

```
[source,bash,role=execute]
```

```
oc get pods -n openshift-cnv -l k8s-app=hostpath-provisioner
```

```
1
```

```
NAME READY STATUS RESTARTS AGE hostpath-provisioner-csi-xxxxxx 4/4 Running 0 2m hostpath-
provisioner-csi-yyyyyy 4/4 Running 0 2m hostpath-provisioner-csi-zzzzz 4/4 Running 0 2m
```

```
== 3: StorageClass
```

```
HostPath Provisioner StorageClass
```

```
xref:attachment$hostpath-provisioner/hpp-storageclass.yaml[hpp-storageclass.yaml]
```

```
#####]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: hostpath-csi
provisioner:  kubevirt.io/hostpath-provisioner  reclaimPolicy:  Delete  volumeBindingMode:
WaitForFirstConsumer parameters: storagePool: local EOF
```

```
=== #####
```

```
* **`provisioner`**: `kubevirt.io/hostpath-provisioner` #####
* **`reclaimPolicy`**: `Delete` #####PVC ##### PV ##### `Retain` #####
* **`volumeBindingMode`**: `WaitForFirstConsumer` ##### PVC ##### PV
#####
* **`parameters.storagePool`**: HostPathProvisioner #####
```

```
=== StorageClass []
```

```
[source,bash,role=execute]
```

```
oc get storageclass hostpath-csi
```

```
{}:
```

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
hostpath-csi kubevirt.io/hostpath-provisioner Delete WaitForFirstConsumer false 10s
```

```
=== {}: ##### StorageClass []
```

```
HPP ##### StorageClass #####/#####
```

```
[source,bash,role=execute]
```

```
oc patch storageclass hostpath-csi -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-
default-class":"true"}}}'
```

```
{}: ##### StorageClass 1 #####
```

```
[source,bash,role=execute]
```

StorageClass

```
oc      patch      storageclass      <existing-default>      -p      '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
```



```
== 4: 
```

```
PVC HPP 
```

```
xref:attachment$hostpath-provisioner/hpp-test-pvc.yaml[hpp-test-pvc.yaml ]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: PersistentVolumeClaim metadata: name: hpp-test-pvc
namespace: default spec: accessModes: - ReadWriteOnce resources: requests: storage: 5Gi
storageClassName: hostpath-csi EOF
```

```
PVC 
```

```
[source,bash,role=execute]
```

```
oc get pvc hpp-test-pvc
```

```
`WaitForFirstConsumer` PVC `Pending` 
```

```
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS
AGE hpp-test-pvc Pending hostpath-csi <unset> 5s
```

```
 
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: Pod metadata: name: hpp-test-pod namespace: default spec:
containers: - name: test image: registry.access.redhat.com/ubi9/ubi:latest command: ["sleep",
"3600"] volumeMounts: - name: data mountPath: /data volumes: - name: data
persistentVolumeClaim: claimName: hpp-test-pvc EOF
```

```
PVC 
```

```
[source,bash,role=execute]
```

```
oc get pvc hpp-test-pvc
```

```
:
```

```
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS
AGE hpp-test-pvc Bound pvc-abc12345-6789-def0-1234-567890abcdef 5Gi RWO hostpath-csi <unset>
30s
```

```
[source,bash,role=execute]
```

```
oc exec hpp-test-pod -- sh -c "echo 'HPP Storage Test' > /data/test.txt && cat /data/test.txt"
```

000 :

HPP Storage Test

11111111111111111111

```
[source,bash,role=execute]
```

```
oc delete pod http-test-pod oc delete pvc http-test-pvc
```

== 0000 5: 0000000000

HPP 0000000000000000 VM 00000000

```
root@kali:~# cat /etc/cloud-init/cloud-init.conf
```

```
xref:attachment$hostpath-provisioner/hpp-test-vm.yaml[hpp-test-vm.yaml 00000000]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: hpp-test-vm
namespace: default spec: runStrategy: Always template: metadata: labels: kubevirt.io/vm: hpp-test-vm spec: domain: cpu: cores: 1 devices: disks: - name: rootdisk disk: bus: virtio - name: cloudinitdisk
disk: bus: virtio interfaces: - name: default masquerade: {} resources: requests: memory: 1Gi
networks: - name: default pod: {} volumes: - name: rootdisk dataVolume: name: hpp-test-vm-
rootdisk - name: cloudinitdisk cloudInitNoCloud: userData: | #cloud-config user: fedora password:
fedora chpasswd: expire: false dataVolumeTemplates: - metadata: name: hpp-test-vm-rootdisk spec:
storage: accessModes: - ReadWriteOnce resources: requests: storage: 10Gi storageClassName:
hostpath-csi source: registry: url: docker://quay.io/containerdisks/fedora:latest EOF
```

```
VM  DataVolume  
```

```
[source,bash,role=execute]
```

DataVolume □□□□□□□□□□

```
oc get dv -n default http-test-vm-rootdisk -w
```

```
□□□□□□:
```

```
NAME PHASE PROGRESS RESTARTS AGE http-test-vm-rootdisk ImportScheduled N/A 5s http-test-vm-  
rootdisk ImportInProgress 0.00% 10s http-test-vm-rootdisk ImportInProgress 45.50% 30s http-test-  
vm-rootdisk Succeeded 100.0% 1m
```

```
VM □□□□□□□□□□□□
```

```
[source,bash,role=execute]
```

```
oc get vm http-test-vm oc get vmi http-test-vm
```

```
□□□:
```

```
NAME AGE STATUS READY http-test-vm 2m Running True
```

```
NAME AGE PHASE IP NODENAME READY http-test-vm 2m Running 10.x.x.x workerXX True
```

```
VM □□□□□□□□□□□□
```

```
[source,bash,role=execute]
```

```
virtctl console http-test-vm
```

```
VM □□□□□□□□□□□□
```

```
[source,bash,role=execute]
```

```
oc delete vm http-test-vm
```

```
== □□□□□□□□□□
```

```
=== PVC □ Pending □□□□□□
```

```
□□□□□□ PVC □ Pending □□□:
```

```
[source,bash,role=execute]
```

PVC 問題解決

oc describe pvc <pvc-name>

HPP 問題解決

oc logs -n openshift-cnv -l k8s-app=hostpath-provisioner -c hostpath-provisioner

```
root@:
* **SELinux 問題解決***: SELinux '/var/hpvolumes' 問題解決
* **SELinux 問題解決***: 'chcon -Rt container_file_t /var/hpvolumes' 問題解決
* **HPP 問題解決***: HPP CSI 問題解決

=== 問題 (Permission Denied) 問題

問題解決:

[source,bash,role=execute]
```

SELinux 問題解決

oc debug node/<worker-node-name> — chroot /host ls -laZ /var/hpvolumes

SELinux 問題解決

oc debug node/<worker-node-name> — chroot /host chcon -Rt container_file_t /var/hpvolumes

```
=== HPP 問題解決

HostPathProvisioner 問題解決

[source,bash,role=execute]
```

oc describe hostpathprovisioner hostpath-provisioner

openshift-cnv 問題解決

oc get events -n openshift-cnv --sort-by='.lastTimestamp' | tail -20

```
=== 問題解決
```

```
[source,yaml]
```



```
* *VolumeBindingMode:*
* *AllowVolumeExpansion:* true PVC
```

```
[discrete#storage:creating-managing-storage-class::_]
====
(LVM/HostPath)
```

A. Immediate

```
* : Pod PVC
* : ('AWS us-east-1a, 1b, 1c') A Pod B
* :
```

B. WaitForFirstConsumer ()

```
* : PVC Pod
* : Pod PVC Pod (B) B
* : (LVM)HostPath (Multi-AZ)
```

```
[discrete#storage:creating-managing-storage-class::_reclaim_policies]
==== (Reclaim Policies)
PersistentVolumeClaim (PVC)
```

A. Delete ()

```
* : PVC PV (AWS EBS)
* : CI/CD
```

B. Retain

```
* : PVC PV Released
* : PV
* :
```

```
[discrete#storage:creating-managing-storage-class::_storageclass_]
==== StorageClass
StorageClass storageClassName PVC
[source,bash,role=execute]
```

```
oc patch storageclass <your-class-name> -p '{"metadata":{"storageclass.kubernetes.io/is-default-class": "true"}}'
```

```
:
[source,bash,role=execute]
```

```
oc patch storageclass <old-default-name> -p '{"metadata":{"storageclass.kubernetes.io/is-default-class": "false"}}'
```

```
name: default-storage-class SC 1
link:../getting-started/default-storage-class.html[Default Storage Class]
```

```
[discrete#storage:creating-managing-storage-class::_storageclass_]
==== StorageClass
```

```
A: AWS EBS (gp3)
CSI gp3
```

```
[source,yaml]
```

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: gp3-fast provisioner:
ebs.csi.aws.com volumeBindingMode: WaitForFirstConsumer allowVolumeExpansion: true
reclaimPolicy: Delete parameters: type: gp3 iops: "3000" throughput: "125" encrypted: "true"
```

```
OpenShift
https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storag
e/using-container-storage-interface-csi#persistent-storage-csi-ebs
```

```
B: / LVM (Local Storage Operator LVM Storage Operator)
```

```
[source,yaml]
```

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: local-lvm-storage provisioner:
topolvm.io # Depends on the specific operator used volumeBindingMode: WaitForFirstConsumer #
Critical for local storage! reclaimPolicy: Retain parameters: "csi.storage.k8s.io/fstype": "xfs"
```

```
name: local-lvm-storage LVM
```

```
C: NFS (NFS / RWX)
NFS CSI Driver NFS
```

```
[source,yaml]
```

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: nfs-shared provisioner:
nfs.csi.k8s.io parameters: server: nfs-server.example.com share: /exports/data reclaimPolicy: Delete
volumeBindingMode: Immediate mountOptions: - nfsvers=4.1
```

```
OpenShift
https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storag
e/persistent-storage-using-local-storage
```

```
[discrete#storage:creating-managing-storage-class::_storageclass_]
====
```



```
[source,bash,role=execute]
```

□□□□□□□□ :

```
[source,bash,role=execute]
```

□□□□□□□□□□: □□□□ PVC □□□□□□:

```
[source,yaml]
```

OpenShift 部署:

https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/configuring-persistent-storage

```
[discrete#storage:creating-managing-storage-class::_00000000000000000000]
```

=====

```
* ** 1:** PVC  Pending
```

```
**  A: volumeBindingMode  WaitForFirstConsumer  PVC  Pod
```

```
*** 000: 00 PVC 00000000 Pod 0000000000
```

** B: Pod

```
*** root: oc get pods -l app=Pod -o Pending --help
```

```
* ** 2:** Pod ( )
```

```
** 000: Volume node affinity conflict
```

```
** 00: 000000000000 volumeBindingMode: Immediate 0000000000000000 A 0000000000Pod 0000 B
```

□□□□□□□□□□□□

```
** 000: PVC 00000WaitForFirstConsumer 00000 StorageClass 0000000000000000
```

```
* ** 3:** "Provisioner not found"
```

```
**   : provisioner: CSI 
```

```
** 000: 000000000000 (0: openshift-cluster-csi-drivers) 000000000000 Pod 000000000000
```

* ** 4: **

```
** []: StorageClass [] allowVolumeExpansion [] false [] [] [] []
```

```
** 000: StorageClass 00000 true 0000000000
```

00 : 000


```
* **\oc\ CLI**  ## **\virtctl\ CLI** #####
* **StorageClass** ##### (?: \lvms-vg1')
```

```
#####:
```

OpenShift 4.20, 4.21

```
[discrete#storage:multi-disk-configuration::_0000_1_#####]
```

```
====  ## 1: #####
```

```
KubeVirtVM#####
```

```
* **\spec.template.spec.domain.devices.disks\**: #####OS#####
```

```
* **\spec.template.spec.volumes\**:
```

```
#####PVCDataVolumecontainerDiskcloudInitNoCloud#####
```

```
* ##### \name\ ##### \name\ #####
```

```
#####OS## \dev/vda\, \dev/vdb\ #####
```

```
[discrete#storage:multi-disk-configuration::_0000_2_#####]
```

```
====  ## 2: #####
```

```
[source,bash,role=execute]
```

oc create namespace multi-disk-demo

```
[discrete#storage:multi-disk-configuration::_0000_3_2###storageclass#####]
```

```
====  ## 3: 2###StorageClass#####
```

```
#####StorageClass1#####StorageClass#####2###
StorageClass#####
```

```
LVM##### \lvms-vg1\
```

```
StorageClass#####\Retain\#####2###StorageClass#####
```

```
link:attachment$multi-disk-configuration/sc-data-retain.yaml[sc-data-retain.yaml
```

```
#####role=unresolved]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: lvms-vg1-retain provisioner: topolvm.io reclaimPolicy: Retain volumeBindingMode: WaitForFirstConsumer allowVolumeExpansion: true EOF
```

```
###StorageClass#####
```

```
[source,bash,role=execute]
```

```
oc get storageclass
```

□□□□□□ :

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
lvms-vg1 (default) topolvm.io Delete WaitForFirstConsumer true 30d lvms-vg1-retain topolvm.io
Retain WaitForFirstConsumer true 5s
```

```
[discrete#storage:multi-disk-configuration::_disk_4_vm]
=== disk 4: VM
3VM

* **rootdisk**: 30GiStorageClassFedora
* **datadisk**: 10Gi2StorageClass
* **cloudinitdisk**: Cloud-init

: cloud-initSSH

link:attachment$multi-disk-configuration/vm-multi-disk.yaml[vm-multi-disk.yaml
,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: multi-disk-vm
namespace: multi-disk-demo spec: runStrategy: Always template: metadata: labels:
kubevirt.io/domain: multi-disk-vm spec: domain: devices: disks: - name: rootdisk disk: bus: virtio
bootOrder: 1 - name: datadisk disk: bus: virtio - name: cloudinitdisk disk: bus: virtio interfaces: -
name: default masquerade: {} resources: requests: memory: 1Gi networks: - name: default pod: {}
volumes: - name: rootdisk dataVolume: name: multi-disk-rootdisk - name: datadisk dataVolume:
name: multi-disk-datadisk - name: cloudinitdisk cloudInitNoCloud: userData: | #cloud-config
password: redhat chpasswd: expire: false dataVolumeTemplates: - metadata: name: multi-disk-
rootdisk spec: pvc: accessModes: - ReadWriteOnce resources: requests: storage: 30Gi
storageClassName: lvms-vg1 source: registry: pullMethod: node url:
docker://quay.io/containerdisks/fedora:latest - metadata: name: multi-disk-datadisk spec: pvc:
accessModes: - ReadWriteOnce resources: requests: storage: 10Gi storageClassName: lvms-vg1-
retain source: blank: {} EOF
```

```
[discrete#storage:multi-disk-configuration::_00000000]
===== 00000000

* rootdisk00 **`bootOrder: 1`** 00VMOS000000000000000000000000
* rootdisk00 **`source.registry`** 00Fedora000000000000000000000000
* datadisk00 **`source.blank: {}`** 00000000000000000000000000000000
* 0 `dataVolumeTemplates` 0000000000 `storageClassName` 0000000000
* `devices.disks` 00000000 `volumes` 000000000000000000000000 `dataVolumeTemplates`
000000000000000000000000
```


[source,bash]

lsblk

□□□□□□:

```
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
vda      252:0    0   30G  0 disk
|-vda1   252:1    0    1M  0 part
|-vda2   252:2    0   1.9G  0 part [SWAP]
|-vda3   252:3    0    1G  0 part /boot
`-vda4   252:4    0  27.1G  0 part /
vdb      252:16    0   10G  0 disk
vdc      252:32    0    1M  0 disk
```

□□□□□□ **vdb** 10G□□□□□□□□□□□□□□□□XFS□□□□□□□□□□

```
sudo mkfs.xfs /dev/vdb
```

□□□□□□□□□□□□□□□□□□□□□□

```
sudo mkdir -p /mnt/data
sudo mount /dev/vdb /mnt/data
```

□□□□□□□□□□

```
df -h /mnt/data
```

□□□□□□:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/vdb         10G   105M   9.9G   2% /mnt/data
```

□□□□□□□□□□□□

```
sudo sh -c "echo 'Multi-disk test data' > /mnt/data/test.txt"
```

□□□□□□□□□□

```
cat /mnt/data/test.txt
```

□□□□□□□□□□□□□□□□**/etc/fstab** □□□□□□□□

```
sudo sh -c "echo '/dev/vdb /mnt/data xfs defaults 0 0' >> /etc/fstab"
```

Ctrl+]

7:

PVCStorageClassPersistentVolume

```
oc get pvc multi-disk-rootdisk -n multi-disk-demo -o jsonpath='{.spec.storageClassName}, Volume: {.spec.volumeName}, Capacity: {.status.capacity.storage}'
```

```
oc get pvc multi-disk-datadisk -n multi-disk-demo -o jsonpath='{.spec.storageClassName}, Volume: {.spec.volumeName}, Capacity: {.status.capacity.storage}'
```

VM

	SSD (OS +)	HDD ()
	(OS)	SSD ()
VM	(OS)	()
/	(OS)	()

VMPVCDataVolume

```
oc delete namespace multi-disk-demo
```

2StorageClass

```
oc delete storageclass lvms-vg1-retain
```

lvms-vg1-retain reclaimPolicy: Retain
PVCPersistentVolumeReleased PV

```
oc get pv | grep Released
```


[illegible]

```
oc delete pv <pv-name>
```

111

□ □

- `StorageClass` 儲存空間的規格
- `dataVolumeTemplates` 儲存空間的規格
- `VM` 虛擬機器
- `OS` 作業系統
- `VM` 虛擬機器

11

- [Disks and Volumes - KubeVirt User Guide](#)
- [Configuring Storage Profiles - OpenShift Documentation](#)
- [Storage Profiles - OpenShift Documentation](#)
- [LVM Storage Operator](#)

Network Architecture

Overview

OpenShift Virtualization User Defined Networks (UDN) localnet 2 VLAN Linux OpenShift Virtualization VM

Components

Network components

- cloud-init DHCP IP
- cloud-init IP
- User Defined Networks (UDN) 2
- localnet
- VLAN
- 2
- VM Linux
- PCI VM SR-IOV
- OVS
-

OpenShift Virtualization Network Architecture

OpenShift Virtualization Network Architecture

- **User Defined Networks (UDN):** namespace 2
- **Localnet:**
- **Layer 2:** 2
- **Linux Bridges:** VM
- **SR-IOV:** PCI VM SR-IOV
- **OVN-Kubernetes Overlay:** Pod

OpenShift Virtualization Network Architecture

OpenShift Virtualization Network Architecture

- OpenShift Virtualization OpenShift 4.19+
- NMState

- 配置VLAN
- 配置IP

配置IP

IP配置

cloud-init DHCP QEMU Windows VM Sysprep IP配置

Cloud-init 配置

cloud-init IP配置

UDN 配置

User Defined Networks 2配置

[networking:localnet-secondary]

ClusterUserDefinedNetwork localnet

[networking:localnet-vlan]

NetworkAttachmentDefinition VLAN localnet

[networking:cudn-localnet-vlan]

ClusterUserDefinedNetwork VLAN localnet

[networking:layer2-secondary]

VM 2配置

[networking:linux-bridges]

VM Linux

[networking:sriov-secondary]

Virtual Functions VM PCI SR-IOV

[networking:ovs-bridge-troubleshooting]

localnet OVS

[networking:vm-ingress-routes]

OpenShift Ingress VM HTTP HTTPS

配置Pod

- Pod
- Pod

配置Linux

localnet Linux

- 配置网络接口
- 配置VLAN
- 配置网络策略

网络配置

网络配置包括以下内容：

- 配置网络接口
- 配置VLAN

网络策略

- [Internal DNS for VMs](#) - VM内部DNS配置
- [OpenShift Virtualization Networking](#)

IP地址

IP

OpenShift

Virtualization配置VM IP地址，配置DHCP，配置IP地址。

配置5个IP地址。

- **Cloud-init `networkData`** — 配置Linux VM
- **IP地址UDN** — OVN-Kubernetes IP地址
- **DHCP** — DHCP MAC地址IP地址
- **`nmcli`** 配置QEMU IP地址
- **Sysprep Windows VM** — `unattend.xml` 配置IP地址

配置网络策略。

网络策略

- OpenShift Virtualization IP地址
- Cloud-init `networkData` IP地址
- IP地址UDN
- MAC地址DHCP
- QEMU `nmcli` IP地址
- Sysprep Windows VM IP地址

OpenShift Virtualization Linux VM

1. VM Cloud-init NoCloud `networkData`
2. Cloud-init
- 3.

Cloud-init Netplan `ethernets` `bonds` `bridges` `vlan` `addresses` `dhcp4/dhcp6` `routes` `nameservers` `addresses` `search` `mtu` `match` `macaddress` `name` `driver`

```
networkData: |
  version: 2
  ethernets:
    enp1s0:
      dhcp4: true
    enp2s0:
      dhcp4: false
      dhcp6: false
      addresses:
        - 192.168.100.50/24
      nameservers:
        addresses:
          - 192.168.100.1
```

`enp1s0` Pod DHCP `enp2s0` IP

`gateway4` `gateway6` Netplan 0.103 Cloud-init 23.1 `routes`

```
networkData: |
  version: 2
  ethernets:
    enp2s0:
      dhcp4: false
      addresses:
        - 192.168.100.50/24
      routes:
        - to: 192.168.100.0/24
          via: 192.168.100.1
      nameservers:
```

```
addresses:
- 192.168.100.1
search:
- example.com
```

search DNS routes to: default

1. networkData
- 2.
3. DHCP

KubeVirt

KubeVirt VM PCI

- enp1s0 — Pod
- enp2s0 — 2
- enp3s0 — 3

eth0 VM enpXs0

cloudInitNoCloud networkData

network:

KubeVirt networkData version: 2 network: NoCloud networkData

userData

userData Cloud-init networkData userData #cloud-config userData

Secret networkData

networkDataSecretRef Cloud-init networkData networkDataBase64

IP

IP Pod IP 10.0.2.0/24 NAT Pod IP

enp1s0 DHCP enp2s0 IP

配置: DHCP + 静态IP

Pod DHCP VLAN IP

0000: 00000000000000000000000000000000: localnet VLAN 100 NetworkAttachmentDefinition
 (NAD) Localnet VLAN 00000000 0000000000

□ □

```
oc create namespace static-ip-demo
```

fedora-multi-network-vm.yaml 0000000

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-multi-vm
  namespace: static-ip-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: fedora-multi-vm-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-multi-vm
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
```



```
virtctl console fedora-multi-vm -n static-ip-demo
```

VM00000000000000

```
ip addr show
```

□ □ □ □ □ □ □

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:7e:e2:23:d7:da brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.2/24 brd 10.0.2.255 scope global dynamic noprefixroute enp1s0
        valid_lft 86313438sec preferred_lft 86313438sec
    inet6 fe80::7e:e2ff:fe23:d7da/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:7e:e2:23:d7:db brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.50/24 brd 192.168.100.255 scope global noprefixroute enp2s0
        valid_lft forever preferred_lft forever
    inet6 fe80::7e:e2ff:fe23:d7db/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

```
ip route show
```

□ □ □ □ □ □ □

```
default via 10.0.2.1 dev enp1s0 proto dhcp src 10.0.2.2 metric 100
10.0.2.0/24 dev enp1s0 proto kernel scope link src 10.0.2.2 metric 100
192.168.100.0/24 dev enp2s0 proto kernel scope link src 192.168.100.50 metric 101
```

```
0:      enp1s0DHCP2enp2s0IP
192.168.100.50
```

Cloud-init

Cloud-

```
init VM VirtualMachineInstance VM IP QEMU
```


2. DHCPサーバーがMACアドレスからIPアドレスを割り当てる

3. VMがDHCPサーバーからIPアドレスを割り当てる

結論

- DHCPサーバーがMACアドレスからIPアドレスを割り当てる
- VMがDHCPサーバーからIPアドレスを割り当てる
- VMがDHCPサーバーからIPアドレスを割り当てるMACアドレスを割り当てる

MACアドレス

DHCPサーバーがVMがMACアドレスを割り当てるKubeVirtがMACアドレスを割り当てる

fedora-dhcp-reserved-vm.yaml の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-dhcp-reserved-vm
  namespace: static-ip-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: fedora-dhcp-reserved-vm-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-dhcp-reserved-vm
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
```

```

        name: rootdisk
      - disk:
          bus: virtio
          name: cloudinitdisk
    interfaces:
      - masquerade: {}
        name: default
      - bridge: {}
        name: vlan-network
        macAddress: "02:00:00:01:00:01"
    rng: {}
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
    - name: vlan-network
      multus:
        networkName: localnet-vlan-100
  volumes:
    - dataVolume:
        name: fedora-dhcp-reserved-vm-disk
        name: rootdisk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd: { expire: False }
          ssh_pwauth: True
        name: cloudinitdisk

```

Cloud-init `macAddress` `networkData`

DHCP

DHCP MAC DHCP

ISC DHCP (`dhcpd.conf`):

```

host fedora-dhcp-reserved-vm {
    hardware ethernet 02:00:00:01:00:01;
    fixed-address 192.168.100.60;
}

```

dnsmasq:

```
dhcp-host=02:00:00:01:00:01,192.168.100.60,fedora-dhcp-reserved-vm
```

00000000VM0000000000

```
oc apply -f fedora-dhcp-reserved-vm.yaml
```

11

VMIP

```
virtctl console fedora-dhcp-reserved-vm -n static-ip-demo
```

VM00:

```
ip addr show enp2s0
```

DHCP MAC IP

□ □ □ □ □ □ □

```
3: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group
default qlen 1000
    link/ether 02:00:00:01:00:01 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.60/24 brd 192.168.100.255 scope global dynamic enp2s0
        valid_lft 3600sec preferred_lft 3600sec
```

DHCP 00000000000000000000000000000000 IPv6 00000000000000000000000000000000 MAC 02:00:00:01:00:01 00macAddress
 00000000000000000000000000000000

□: 02: □□□□MAC□□□□□□□□□□□□□□□□□□□□□□□□□□□□MAC□□□□□□□□□□□□

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

IPVM

- **Network**: **localnet** **Linux** **Network**:
VM **Network** **MAC** **IP** **Network** **MAC** **Network**:
MAC **Cloud-init** **DHCP** **nmcli** **IP**
- **SR-IOV**: **SR-IOV** **OpenShift** **Virtualization**

Pod	Pod IP	OS
pod1	10.1.1.1	Linux
pod2	10.1.1.2	Linux
pod3	10.1.1.3	Linux
pod4	10.1.1.4	Linux
pod5	10.1.1.5	Linux
pod6	10.1.1.6	Linux
pod7	10.1.1.7	Linux
pod8	10.1.1.8	Linux
pod9	10.1.1.9	Linux
pod10	10.1.1.10	Linux
pod11	10.1.1.11	Linux
pod12	10.1.1.12	Linux
pod13	10.1.1.13	Linux
pod14	10.1.1.14	Linux
pod15	10.1.1.15	Linux
pod16	10.1.1.16	Linux
pod17	10.1.1.17	Linux
pod18	10.1.1.18	Linux
pod19	10.1.1.19	Linux
pod20	10.1.1.20	Linux
pod21	10.1.1.21	Linux
pod22	10.1.1.22	Linux
pod23	10.1.1.23	Linux
pod24	10.1.1.24	Linux
pod25	10.1.1.25	Linux
pod26	10.1.1.26	Linux
pod27	10.1.1.27	Linux
pod28	10.1.1.28	Linux
pod29	10.1.1.29	Linux
pod30	10.1.1.30	Linux
pod31	10.1.1.31	Linux
pod32	10.1.1.32	Linux
pod33	10.1.1.33	Linux
pod34	10.1.1.34	Linux
pod35	10.1.1.35	Linux
pod36	10.1.1.36	Linux
pod37	10.1.1.37	Linux
pod38	10.1.1.38	Linux
pod39	10.1.1.39	Linux
pod40	10.1.1.40	Linux
pod41	10.1.1.41	Linux
pod42	10.1.1.42	Linux
pod43	10.1.1.43	Linux
pod44	10.1.1.44	Linux
pod45	10.1.1.45	Linux
pod46	10.1.1.46	Linux
pod47	10.1.1.47	Linux
pod48	10.1.1.48	Linux
pod49	10.1.1.49	Linux
pod50	10.1.1.50	Linux
pod51	10.1.1.51	Linux
pod52	10.1.1.52	Linux
pod53	10.1.1.53	Linux
pod54	10.1.1.54	Linux
pod55	10.1.1.55	Linux
pod56	10.1.1.56	Linux
pod57	10.1.1.57	Linux
pod58	10.1.1.58	Linux
pod59	10.1.1.59	Linux
pod60	10.1.1.60	Linux
pod61	10.1.1.61	Linux
pod62	10.1.1.62	Linux
pod63	10.1.1.63	Linux
pod64	10.1.1.64	Linux
pod65	10.1.1.65	Linux
pod66	10.1.1.66	Linux
pod67	10.1.1.67	Linux
pod68	10.1.1.68	Linux
pod69	10.1.1.69	Linux
pod70	10.1.1.70	Linux
pod71	10.1.1.71	Linux
pod72	10.1.1.72	Linux
pod73	10.1.1.73	Linux
pod74	10.1.1.74	Linux
pod75	10.1.1.75	Linux
pod76	10.1.1.76	Linux
pod77	10.1.1.77	Linux
pod78	10.1.1.78	Linux
pod79	10.1.1.79	Linux
pod80	10.1.1.80	Linux
pod81	10.1.1.81	Linux
pod82	10.1.1.82	Linux
pod83	10.1.1.83	Linux
pod84	10.1.1.84	Linux
pod85	10.1.1.85	Linux
pod86	10.1.1.86	Linux
pod87	10.1.1.87	Linux
pod88	10.1.1.88	Linux
pod89	10.1.1.89	Linux
pod90	10.1.1.90	Linux
pod91	10.1.1.91	Linux
pod92	10.1.1.92	Linux
pod93	10.1.1.93	Linux
pod94	10.1.1.94	Linux
pod95	10.1.1.95	Linux
pod96	10.1.1.96	Linux
pod97	10.1.1.97	Linux
pod98	10.1.1.98	Linux
pod99	10.1.1.99	Linux
pod100	10.1.1.100	Linux


```
ipv4.method manual \
ipv4.addresses 192.168.100.70/24 \
ipv4.dns 192.168.100.1
```

[illegible]

```
sudo nmcli connection up "Wired connection 2"
```

VM0000000000000000

```
ip addr show enp2s0
```

```
VirtualMachineInstance[] VMs[]
```

```
oc get vmi fedora-dhcp-reserved-vm -n static-ip-demo -o
jsonpath='{.status.interfaces[*].ipAddress}'
```

NetworkManager VM

□ □ □ □ □ □ □ □

- 虚拟机VM的启动顺序
- Cloud-init的启动顺序
- 网络启动的顺序
- Cloud-init的启动顺序

```
00: VMCloud-init networkData Cloud-init nmcli Cloud-init
01: VM networkData Cloud-init
```

5: Windows VM Sysprep

Windows unattend.xml Microsoft

Windows unattend.xml Microsoft

```
Sysprep Windows OOBespecialize IP
```

1. 設定ファイル `unattend.xml` を作成する
2. 設定ファイル `sysprep` を実行してVMを作成する
3. Windowsをインストールする

unattend.xml 0000000000000000

```

specialize
Ethernet
IP
NIC
Windows
Identifier
VM

```

```
<?xml version="1.0" encoding="utf-8"?>
<unattend xmlns="urn:schemas-microsoft-com:unattend">
  <settings pass="specialize">
    <component name="Microsoft-Windows-TCPIP"
      processorArchitecture="amd64"
      publicKeyToken="31bf3856ad364e35"
      language="neutral"
      versionScope="nonSxS"
      xmlns:wcm="http://schemas.microsoft.com/WMIConfig/2002/State">
      <Interfaces>
        <Interface wcm:action="add">
          <Identifier>Ethernet</Identifier>
          <Ipv4Settings>
            <DhcpEnabled>>false</DhcpEnabled>
          </Ipv4Settings>
          <UnicastIpAddresses>
            <IpAddress wcm:action="add" wcm:keyValue="1">192.168.100.80/24</IpAddress>
          </UnicastIpAddresses>
          <Routes>
            <Route wcm:action="add">
              <Identifier>0</Identifier>
              <NextHopAddress>192.168.100.1</NextHopAddress>
              <Prefix>0.0.0.0/0</Prefix>
            </Route>
          </Routes>
        </Interface>
      </Interfaces>
    </component>
    <component name="Microsoft-Windows-DNS-Client"
      processorArchitecture="amd64"
      publicKeyToken="31bf3856ad364e35"
      language="neutral"
      versionScope="nonSxS"
      xmlns:wcm="http://schemas.microsoft.com/WMIConfig/2002/State">
      <Interfaces>
        <Interface wcm:action="add">
          <Identifier>Ethernet</Identifier>
          <DNSServerSearchOrder>
            <IpAddress wcm:action="add" wcm:keyValue="1">8.8.8.8</IpAddress>
          </DNSServerSearchOrder>
        </Interface>
      </Interfaces>
    </component>
  </settings>
</unattend>
```

unattend.xml ConfigMap VM sysprep Sysprep Windows
VM Sysprep Windows VM OpenShift

```
oc create configmap windows-sysprep --from-file=unattend.xml -n static-ip-demo
```

```
volumes:
  - name: sysprep
    sysprep:
      configMap:
        name: windows-sysprep
```

```
devices:
  disks:
    - cdrom:
        bus: sata
        name: sysprep
```

- IPWindows Server VM
- Windows
- VMwareHyper-VWindows

□ □

項目	Cloud-init networkData	UDN の IP	DHCP	インストール / nmcli	Sysprep
OS	Linux	Linux / Windows	Linux / Windows	Linux	Windows
インストール	インストール	インストール	DHCP		Oobe
インストール			DHCP		

項目	Cloud-init networkData	UDN 項目IP	DHCP項目	ネットワーク / nmcli	Sysprep
ネットワーク	項目	項目 (IPAMClaim)	項目DHCP項目	項目 (NetworkManager)	項目
ネットワーク	項目 (項目)	項目項目	項目 (項目)	項目 (項目)	項目 (項目)
ネットワーク	項目	項目項目	項目	項目	項目
項目 / 項目項目	項目	項目 (UDN項目)	項目 (MAC項目)	項目	項目 (ConfigMap)
項目項目	VM項目項目	項目項目	DHCP項目項目	項目VM項目	項目 (RDP/PowerShell)

項目項目項目項目

network: 項目項目項目 **networkData**

項目: **network:** 項目項目項目項目Cloud-init項目 **networkData** 項目項目項目

項目: KubeVirt項目 **cloudInitNoCloud** 項目項目項目**networkData** 項目項目項目 **version: 2** 項目項目項目項目項目項目項目
network: 項目項目項目項目項目項目項目項目 **network:\n version: 2\n ethernet:\n** ... 項目項目**version:**
2\nethernet:\n ... 項目

項目項目項目項目項目項目項目

項目: **networkData** 項目 **enp1s0** 項目 **eth0** 項目項目

項目: 項目OS項目項目項目項目項目項目項目項目項目項目Linux項目項目項目項目RHEL 8+項目Fedora項目Ubuntu
16.04+項目 **enp1s0**項目**enp2s0** 項目項目項目項目項目項目項目項目項目項目項目項目項目

項目項目項目項目項目項目項目項目項目

項目: 項目項目項目項目DHCP項目項目項目項目項目

項目: Cloud-init項目 **networkData** 項目IP項目項目項目項目 **dhcp4: false** 項目 **dhcp6: false** 項目項目項目項目

項目項目項目項目項目項目項目項目項目

項目: 項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目

項目: 項目項目項目項目項目項目項目項目項目項目 **enp1s0**項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目

Cloud-init項目項目項目項目

項目: YAML項目Cloud-init項目項目項目項目項目

項目: VM項目 **/var/log/cloud-init.log** 項目Cloud-init項目項目項目項目項目YAML項目項目項目項目**cloud-init status**
項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目項目

DNS

OS:

OS: networkData
DNS

gateway4

OS: networkData gateway4: 192.168.100.1 Cloud-init23.1

OS: gateway4 routes routes: [{to: default, via: 192.168.100.1}]
gateway4 gateway6 Netplan 0.103

DHCP

OS: DHCP VM IP

OS: VM macAddress
MAC KubeVirt MAC DHCP MAC

Cloud-init Cloud-init

VM

```
oc delete vm fedora-multi-vm -n static-ip-demo --ignore-not-found
```

```
oc delete vm fedora-dhcp-reserved-vm -n static-ip-demo --ignore-not-found
```

```
oc delete namespace static-ip-demo
```

- OpenShift Virtualization IP
- Linux VM Cloud-init networkData IP
- IP UD N
- MAC DHCP
- QEMU nmcli IP

- 11

- # Cloud-init

11

-

```
oc get vm <vm-name> -n <namespace>
oc get vmi <vm-name> -n <namespace>
```

VM IP 確認

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.interfaces[*].ipAddress}'
```

Guest Agent 確認

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.guestOSInfo}'
```

Guest Agent 確認 [Guest Agent](#) 確認

cloud-init 確認

VM 確認

```
virtctl console <vm-name> -n <namespace>
```

cloud-init 確認

VM 確認:

```
# cloud-init 確認
cloud-init status

# 確認:
# status: done

# 確認
cloud-init status --long
```

cloud-init 確認

```
# cloud-init 確認
cat /var/log/cloud-init.log

# 確認 (確認)
cat /var/log/cloud-init-output.log

# 確認
grep -i error /var/log/cloud-init.log
grep -i failed /var/log/cloud-init.log
```

cloud-init 確認

```
# userdata 確認
```

```

sudo cat /var/lib/cloud/instance/user-data.txt

# 네트워크 (networkData)
sudo cat /var/lib/cloud/instance/network-config.txt

# cloud-init
sudo cat /var/lib/cloud/instance/cloud-config.txt

```

네트워크 확인

```

# IP 주소 확인
ip addr show

# 라우팅 테이블 확인
ip route show

# DNS 확인
cat /etc/resolv.conf

# NetworkManager (Fedora, RHEL)
nmcli device status
nmcli connection show

```

인터넷 연결

```

# ping 테스트
ping -c 3 8.8.8.8

# DNS 테스트
nslookup google.com

# Pod 테스트
curl http://<vm-ip>

```

네트워크 설정

1: DHCP

DHCP VM IP

:

```

$ oc get vmi fedora-dhcp-vm -n vms -o jsonpath='{.status.interfaces[0].ipAddress}'
# IP

```

:


```
# VM 起動
virtctl console fedora-dhcp-vm -n vms

# ネットワークを確認
ip addr show enp1s0

# DHCP ログを確認
journalctl -u NetworkManager | grep -i dhcp
```

確認:

- DHCP 設定を確認
- ネットワーク設定を確認
- cloud-init が DHCP を取得

手順:

1. VM 内で cloud-init DHCP 設定:

```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernet:
      enp1s0:
        dhcp4: true
```

2. ネットワーク接続を確認:

```
sudo nmcli connection down "Wired connection 1"
sudo nmcli connection up "Wired connection 1"
```

3. NetworkManager ログを確認:

```
sudo journalctl -u NetworkManager --since "5 minutes ago"
```

エラー: **cloud-init** 実行エラー

エラー: 設定ファイルに device eth0 not found

確認:

```
# VM 内 - ネットワークを確認
ip link show
```

□□□□□:

- □□ OS □□□□□□□□□□□□□□□□□□ (eth0, eth1) □□□□□□

□□□:

1. □□ OS □□□□□□□□□□□□□□□□□□ Linux □□□□□□□□□□ (RHEL 8+, Fedora, Ubuntu 16.04+) □□□□□□□□□□□□□□□□□□
 - □□□□□□□□□□: enp1s0
 - 2□□□□□□□□□□: enp2s0
 - 3□□□□□□□□□□: enp3s0
2. cloud-init □ networkData □□□□□□□□□□□□□□□□:

```
networkData: |
  version: 2
  ethernet:
    enp1s0: # eth0 □□□□
    dhcp4: true
```

□□□□ 2: □□ IP □□

□□: □□ IP □□□□□□□□

□□: □□□□□□ IP □□□□□□VM □ DHCP IP □□□□□□□□□□

□□:

```
# VM □
ip addr show enp1s0

# □□□□□□ cloud-init □□□□□
grep -i "network" /var/log/cloud-init.log
```

□□□□□□:

- networkData □□□□□□□□□□□□□□□□□□
- DHCP □□□□□□□□□□□□
- networkData YAML □□□□□□□□
- cloud-init □ networkData □□□□□□□□□□

□□□:

1. networkData □□□ (□□□□□□□□YAML □□) □□□:

```
networkData: |
```

```

version: 2
ethernets:
  enp1s0:
    dhcp4: false # DHCP
    addresses:
      - 192.168.1.100/24
    routes:
      - to: default
        via: 192.168.1.1
    nameservers:
      addresses:
        - 8.8.8.8
        - 8.8.4.4

```

2. cloud-init 確認:

```

cloud-init status --long
cat /var/log/cloud-init.log | grep -A 5 -B 5 "network"

```

3. ネットワーク設定:

```

sudo nmcli connection modify "Wired connection 1" \
  ipv4.method manual \
  ipv4.addresses 192.168.1.100/24 \
  ipv4.gateway 192.168.1.1 \
  ipv4.dns "8.8.8.8 8.8.4.4"

sudo nmcli connection up "Wired connection 1"

```

確認: IP 設定

確認:

```

# VM
ip addr show enp1s0
# IP
ping 192.168.1.1
# Network unreachable

```

確認:

```

#
ip route show
#

```

```
ping -c 1 192.168.1.1
```

□□□□□□:

- □□□□□□□□□□□□□□□□
- □□□□□□□□□□
- □□ IP □□□□□□□□□□□□□□□□
- □□□□□□□□ VLAN □□□□

□□□:

1. cloud-init □□□□□□□□□□□□
2. □□□□□□□□□□□□□□□□□□□□□□
3. □□□□□□□□□□□□□□□□ IP □□□□□□□□□□
4. VLAN □□□□□□□□□□NetworkAttachmentDefinition (NAD) □□□□□□□□□□□□□□□□

□□□□ 3: □□□□□□□□□□□□□□□□

□□: □□□□□□□□□□□□□□□□□□□□

□□: □□□□□□□□□□□□□□ IP □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□:

```
# VM □  
ip addr show  
  
# NetworkManager □□□□□  
nmcli connection show
```

□□□□□□:

- networkData □□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□

□□□:

1. VM □□□□□□□□□□□□□□□□□□□□□□:

```
spec:  
  template:  
    spec:  
      networks:  
        - name: default  
          pod: {}
```

```
- name: secondary-net
  multus:
    networkName: my-nad
```

2. networkData 設定:

```
networkData: |
  version: 2
  ethernet:
    enp1s0:
      dhcp4: true
    enp2s0: # 設定
      dhcp4: false
      addresses:
        - 192.168.100.10/24
```

3. NAD 設定:

```
oc get network-attachment-definition -n <namespace>
oc describe network-attachment-definition <nad-name> -n <namespace>
```

例: 設定

例: 設定

例:

```
# VM
ip route show

# 設定
# default via 10.128.0.1 dev enp1s0
# default via 192.168.100.1 dev enp2s0
```

例:

- 設定
- DHCP 設定
- 設定

例:

1. networkData 設定:

```
enp2s0:
  dhcp4: false
```

```
addresses:
  - 192.168.100.10/24
# 'routes' 設定 - 設定
```

2. 設定:

```
enp1s0:
  dhcp4: true
  dhcp4-overrides:
    route-metric: 100 # 設定

enp2s0:
  dhcp4: false
  addresses:
    - 192.168.100.10/24
  routes:
    - to: default
      via: 192.168.100.1
      metric: 200 # 設定 = 設定
```

4: **VLAN** 設定

VLAN 設定

VLAN (IP: **enp2s0.100**) 設定

設定:

```
# VM
ip link show

# cloud-init 設定
grep -i vlan /var/log/cloud-init.log
```

設定:

- **networkData** 設定
- 設定
- **VLAN** 設定

設定:

1. **networkData** 設定:

```
networkData: |
  version: 2
  ethernets:
```

```

    enp2s0: {} # 物理ネットワークカード
  vlans:
    vlan100:
      id: 100
      link: enp2s0
      addresses:
        - 192.168.100.10/24

```

2. VLAN 設定:

```

sudo nmcli connection add type vlan \
  con-name vlan100 \
  ifname enp2s0.100 \
  dev enp2s0 \
  id 100 \
  ip4 192.168.100.10/24

```

確認: VLAN 設定

確認: VLAN 設定

確認:

```

# VLAN 設定確認
ip addr show enp2s0.100

# ネットワークパケットキャプチャ
tcpdump -i enp2s0 -n vlan 100

```

確認:

- ネットワークカード
- ネットワークカード VLAN 設定
- ネットワーク VLAN ID 設定
- NAD の VLAN 設定

確認:

1. ネットワークカード VLAN 100 設定
2. NAD の VLAN 設定:

```
oc get network-attachment-definition <nad-name> -n <namespace> -o yaml
```

"vlan": 100 設定

3. localnet の OVN 設定:

```
oc get nncp -A
```

□□□□ 5: IPv6 □□

□□: IPv6 □□□□□□□□□□

□□: IPv4 □□□□□□□□IPv6 □□□□□□□□

□□:

```
# VM □  
ip -6 addr show
```

□□□□□□:

- cloud-init □ IPv6 □□□□□□□□□□
- OS □ IPv6 □□□□□□□□□□

□□□:

1. networkData □ IPv6 □□□□:

```
networkData: |  
  version: 2  
  ethernets:  
    enp1s0:  
      dhcp4: true  
      dhcp6: true  
      # □□□□ IPv6:  
      addresses:  
        - 2001:db8::10/64
```

2. IPv6 □□□□□□□□□□□□□□:

```
cat /proc/sys/net/ipv6/conf/all/disable_ipv6  
# 0 (□□) □□□□□□□□□□
```

□□□□ 6: DNS □□

□□: DNS □□□□□□

□□:

```
# VM □  
ping 8.8.8.8  
# □□
```



```
ping google.com
# Unknown host
```

□□:

```
cat /etc/resolv.conf

# DNS □□□□
nslookup google.com
```

□□□□□□:

- cloud-init □□□□□□□□□□□□□□□□
- DHCP □ DNS □□□□□□□□□□□□
- **/etc/resolv.conf** □□□□□□□□□□□□

□□□:

1. networkData □□□□□□□□□□□□:

```
networkData: |
  version: 2
  ethernets:
    enp1s0:
      dhcp4: false
      addresses:
        - 192.168.1.100/24
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
      search:
        - example.com
```

2. DNS □□□□□□:

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

□□□□□□

□□□□□□□□□□□□□□□□

```
# VM □ - □□□□□□□□□□
echo "=== Interfaces ==="
```

```

ip addr show

echo "=== Routes ==="
ip route show

echo "=== DNS ==="
cat /etc/resolv.conf

echo "=== Cloud-init Status ==="
cloud-init status --long

echo "=== Cloud-init Network Config ==="
sudo cat /var/lib/cloud/instance/network-config.txt

echo "=== NetworkManager Connections ==="
nmcli connection show

echo "=== Connectivity Tests ==="
ping -c 2 8.8.8.8 && echo "IP connectivity: OK" || echo "IP connectivity: FAILED"
nslookup google.com && echo "DNS: OK" || echo "DNS: FAILED"

```

OpenShift ☐☐☐☐☐☐☐☐☐☐

```

# VM ☐☐☐☐☐☐☐☐
oc get vm,vmi <vm-name> -n <namespace>

# VM IP ☐☐☐☐☐☐☐☐
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.interfaces}' | jq

# ☐☐☐☐☐☐☐☐☐☐☐☐
oc get vm <vm-name> -n <namespace> -o jsonpath='{.spec.template.spec.networks}' | jq

# NADs ☐☐☐
oc get network-attachment-definition -n <namespace>

# VM ☐☐☐☐☐☐☐☐
oc get events -n <namespace> --field-selector involvedObject.name=<vm-name> --sort
-by='.lastTimestamp'

# VM ☐☐☐ cloud-init ☐☐☐☐☐☐☐☐☐☐
oc get vm <vm-name> -n <namespace> -o yaml | grep -A 50 cloudInitNoCloud

```

cloud-init ☐☐☐☐ (☐☐☐☐☐)

```

# VM ☐ - cloud-init ☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐ (☐☐☐☐)
sudo cloud-init clean --logs
sudo cloud-init init
sudo cloud-init modules --mode config

```

```
sudo cloud-init modules --mode final
```

```
#
```

```
cloud-init status --long
```

VM Linux PCI

Linux PCI

1 (Pod)	enp1s0	(DHCP)
2 (Multus/NAD)	enp2s0	
3 (Multus/NAD)	enp3s0	3
4 (Multus/NAD)	enp4s0	4

OS Linux (RHEL 8+ Fedora Ubuntu 16.04+) enpXsY eth0 eth1

cloud-init

cloud-init v1 vs v2

Version 1 ():

```
networkData: |
  version: 1
  config:
    - type: physical
      name: eth0
      subnets:
        - type: dhcp
```

Version 2 ():

```
networkData: |
  version: 2
  ethernets:
    enp1s0:
      dhcp4: true
```

: 2

- [Cloud-init -](#)

- [Netplan](#) [링크](#)
- [OpenShift Virtualization](#) [문서](#)
- [KubeVirt](#) [문서](#)
- [OVS](#) [문서](#)

❏: **ocplab01** [링크](#)

SNO [링크](#) (ocplab01) [링크](#):

```
$ oc get vmi fedora-dhcp-vm -n cloud-init-test -o jsonpath='{.status.interfaces}' | jq
[
  {
    "infoSource": "domain, guest-agent, multus-status",
    "interfaceName": "eth0",
    "ipAddress": "10.128.0.165",
    "ipAddresses": [
      "10.128.0.165",
      "fd02::5e"
    ],
    "mac": "02:92:c2:00:00:05",
    "name": "default",
    "queueCount": 1
  }
]
```

❏: VMI [링크](#) **interfaceName** [링크](#) **eth0** [링크](#) (KubeVirt [링크](#)) Fedora VM [링크](#) **enp1s0**
[cloud-init](#) [링크](#) OS [링크](#)

VM [링크](#):

```
[fedora@fedora ~]$ ip addr show enp1s0
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group
default qlen 1000
    link/ether 02:92:c2:00:00:05 brd ff:ff:ff:ff:ff:ff
    inet 10.128.0.165/23 brd 10.128.1.255 scope global dynamic noprefixroute enp1s0
        valid_lft 86295sec preferred_lft 86295sec
    inet6 fd02::5e/64 scope global dynamic noprefixroute
        valid_lft 86296sec preferred_lft 14296sec
```

cloud-init [링크](#):

```
[fedora@fedora ~]$ cloud-init status
status: done
```

[링크](#) **DHCP** [링크](#)

UDN 网络配置

网络

OpenShift Virtualization 中 User Defined Networks (UDN) 配置 2
UDN 配置 IP
IP
(Network Attachment Definition)
5

配置:

OpenShift 4.19, 4.20, 4.21

配置

- OpenShift Virtualization 配置 OpenShift 4.19 配置
- OVN-Kubernetes (OpenShift 配置)
- UserDefinedNetwork 配置
- CLI 命令: `oc virtctl`

配置 1: UDN 配置

UDN 配置 User Defined Network 配置 k8s.ovn.org/primary-user-defined-network OpenShift 配置

配置: 配置

`udn-primary-namespace.yaml` 配置

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: udn-primary-vm-guests
  labels:
    k8s.ovn.org/primary-user-defined-network: ""
EOF
```

配置 2: UserDefinedNetwork 配置

配置 2 配置 UDN 配置 IP 配置 IPAM
Pod 配置

`udn-primary.yaml` 配置

```
oc apply -f - <<EOF
apiVersion: k8s.ovn.org/v1
kind: UserDefinedNetwork
metadata:
  name: udn-primary
  namespace: udn-primary-vm-guests
spec:
  topology: Layer2
  layer2:
    role: Primary
    subnets:
      - "192.168.100.0/24"
  ipam:
    lifecycle: Persistent
EOF
```

Figure 3: UDN Performance

0000 UDN 0000000000000000 Python HTTP 0000000000000000000000000000UDN 0 IPAM 0000
 192.168.100.0/24 000000000000 IP 0000000000UDN 0000000000000000000000000000

00: cloud-init 00000000000000000000000000000000 SSH 0000000000

fedora-vm-with-udn.yaml 0000000

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-vm-with-udn
  namespace: udn-primary-vm-guests
  labels:
    app: fedora-web-vm
    network-type: udn-primary
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-udn-volume
    spec:
      pvc:
        accessModes:
        - ReadWriteOnce
      resources:
        requests:
          storage: 33Gi
      sourceRef:
        kind: DataSource
        name: fedora
```

```

    namespace: openshift-virtualization-os-images
template:
  metadata:
    labels:
      app: fedora-web-vm
  spec:
    domain:
      devices:
        disks:
          - name: datavolumedisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        interfaces:
          - name: default
            binding:
              name: l2bridge
      resources:
        requests:
          memory: 2Gi
          cpu: 1
    networks:
      - name: default
        pod: {}
    volumes:
      - name: datavolumedisk
        dataVolume:
          name: fedora-udn-volume
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            write_files:
              - path: /etc/resolv.conf
                content: |
                  nameserver 172.30.0.10      # cluster DNS
                  search udn-primary-vm-guests.svc.cluster.local svc.cluster.local
                  options ndots:5
            user: fedora
            password: fedora
            chpasswd: { expire: False }
            packages:
              - python3
            runcmd:
              - echo "<h1>Welcome to OpenShift Virtualization !!!</h1>" >
                /root/index.html
              - cd /root && nohup python3 -m http.server 80 > /dev/null 2>&1 &
EOF

```

ステップ 4: VM をデプロイする

Kubernetes を使用して LoadBalancer を作成し、VM に Python HTTP サーバをデプロイする

例: AWS を使用して LoadBalancer を作成する場合は、[MetalLB](#) を使用して LoadBalancer を作成する

fedora-vm-web-service.yaml を作成する

```
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: fedora-vm-web-service
  namespace: udn-primary-vm-guests
  labels:
    app: fedora-web-vm
spec:
  selector:
    app: fedora-web-vm
  ports:
  - name: http
    port: 80
    targetPort: 80
    protocol: TCP
  type: LoadBalancer
EOF
```

ステップ 5: サービスを確認する

LoadBalancer が作成されたことを確認し、AWS コンソールで IP アドレスを確認する:

```
oc get svc
```

例:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	fedora-vm-web-service	LoadBalancer
172.30.72.140	aa32981df19ad4959ae9a82fb9990db9-1996929606	ca-central-1.elb.amazonaws.com	80:32291/TCP	45m			

```
curl http://172.30.72.140:
```

```
[source,bash,role=execute]
```

```
curl aa32981df19ad4959ae9a82fb9990db9-1996929606.ca-central-1.elb.amazonaws.com
```



~~~~~:

<h1>Welcome to OpenShift Virtualization !!!</h1>

== ~~~

~~~~~:

- * `k8s.ovn.org/primary-user-defined-network` ~~~~ UDN ~~~~~
- * ~~~ 2 ~~~~~ `UserDefinedNetwork` ~~~~~
- * ~~~ IPAM ~~~UDN ~~~~~ VM ~~~~~
- * UDN ~~~~~ VM ~ LoadBalancer ~~~~~
- * UDN ~~~~~ IP ~~~~~

== ~~~

*

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/multiple_networks/understanding-multiple-networks[OpenShift - ~~~~~,window=_blank]

*

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/multiple_networks/primary-networks[~~~~~,window=_blank]

:!navtitle:

:docname: localnet-secondary

:page-module: networking

:page-relative-src-path: localnet-secondary.adoc

:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

:page-origin-refhash: (worktree)

:navtitle: CUDN~~~~~Localnet~~~~~

[#networking:localnet-secondary]

=== CUDN~~~~~Localnet~~~~~

~~~~~:

## OpenShift 4.20

~~~~~OpenShift Virtualization~~~~~ClusterUserDefinedNetwork (CUDN)

~~~~~localnet~~~~~Localnet~~~~~Pod~~~~~VM~~~~~  
~~~~~2~~~~~

[discrete#networking:localnet-secondary::~_~~~~]

==== ~~~

```
* **NMState Operator**: 
* **NMState CR**: Operatornmstate
* ****: 
```

```
:
```

OpenShift 4.19

```
[discrete#networking:localnet-secondary::_1_vm]
=== 1: VM

localnetVM

link:attachment$localnet-secondary/localnet-secondary-namespace.yaml[localnet-
secondary-namespace.yaml ,role=unresolved]

[source,bash,role=execute]
```

oc apply -f - <<EOF apiVersion: v1 kind: Namespace metadata: name: vm-guests-localnet EOF

```
[discrete#networking:localnet-secondary::_2_]
=== 2: 

physnet `br-ex` localnetOVN-
Kubernetes

link:attachment$localnet-secondary/localnet-bridge-mapping-nncp.yaml[localnet-bridge-
mapping-nncp.yaml ,role=unresolved]

[source,bash,role=execute]
```

oc apply -f - <<EOF apiVersion: nmstate.io/v1 kind: NodeNetworkConfigurationPolicy metadata: name: localnet-bridge-mapping spec: nodeSelector: node-role.kubernetes.io/worker: " desiredState: ovn: bridge-mappings: - localnet: localnet1 bridge: br-ex state: present EOF

```
****: `localnet1` `br-ex` VM

[discrete#networking:localnet-secondary::_3_clusteruserdefinednetwork]
=== 3: ClusterUserDefinedNetwork

localnetClusterUserDefinedNetwork

link:attachment$localnet-secondary/cudn-localnet.yaml[cudn-localnet.yaml
,role=unresolved]

[source,bash,role=execute]
```


VM□□□□□□

oc get vm -n vm-guests-localnet

□□□□**VM**□□□□□□□□□□

oc get vmi -n vm-guests-localnet

VM起動時の初期化 (cloud-initでfedora/fedora)

`virtctl console -n vm-guests-localnet fedora-localnet-vm`

VM起動時の初期化

[source,bash,role=execute]

Podman - 本地网络 (Podman) 本地网络 (localnet)

ip addr show

□□□□□□ - **2**□□□□□□□□□□□□□□
(□□□□□□□□□□□□□□)

□□□□□□□□□□□□: □□□□□ **100**, □□□□□□:
□□□□□ **101**

ip route show

□□□□□□□□:

default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.172 metric 100 default via 192.168.2.1 dev
enp2s0 proto dhcp src 192.168.2.128 metric 101



ping 8.8.8.8

Web□□□□□□□□□□□□□□□□

curl localhost:8080

localnet (<localnet-ip> IP)

curl <localnet-ip>:8080

```
[discrete#networking:localnet-secondary::_]
```

```
====
```

- * `NodeNetworkConfigurationPolicy` OVN-Kubernetes `br-ex`
- * localnet `ClusterUserDefinedNetwork`
- * Pod localnet VM
- * IP
- * CUDN localnet

```
[discrete#networking:localnet-secondary::_]
```

```
====
```

*

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/multiple_networks/understanding-multiple-networks[OpenShift - ,window=_blank]

*

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/virtualization/networking#virt-creating-secondary-localnet-udn_virt-connecting-vm-to-secondary-udn[OCP Virtualization Localnet ,window=_blank]

:!navtitle:

:docname: localnet-vlan

:page-module: networking

:page-relative-src-path: localnet-vlan.adoc

:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

:page-origin-refhash: (worktree)

:navtitle: NAD VLAN localnet

[#networking:localnet-vlan]

```
=== NAD VLAN localnet
```

OpenShift Virtualization NetworkAttachmentDefinition (NAD)

VLAN localnet VLAN Pod VM VLAN VLAN

```
[discrete#networking:localnet-vlan::_]
```

```
====
```

```

* **NMState Operator**:
* **NMState CR**: Operator
* **:
* **VLAN**: VLAN
* **: OpenShift VLAN **

```

:

OpenShift 4.19

```

[discrete#networking:localnet-vlan::_]
===

[discrete#networking:localnet-vlan::__]
===== :

OpenShift localnet VLAN **:

* **: VLAN
* **VLAN**: VLAN ID: 100, 200, 300
* **: VLAN

[discrete#networking:localnet-vlan::__1_vm_]
=== 1: VM

VLAN localnet VM

link:attachment$localnet-vlan/localnet-vlan-namespace.yaml[localnet-vlan-namespace.yaml ,role=unresolved]

[source,bash,role=execute]

```

oc apply -f - <<EOF apiVersion: v1 kind: Namespace metadata: name: vm-guests-vlan EOF

```

[discrete#networking:localnet-vlan::__2_]
=== 2:

`localnet-vlan` `br-ex` OVN-
Kubernetes VLAN localnet VM VLAN

link:attachment$localnet-vlan/localnet-vlan-bridge-mapping-nncp.yaml[localnet-vlan-bridge-mapping-nncp.yaml ,role=unresolved]

[source,bash,role=execute]

```

oc apply -f - <<EOF apiVersion: nmstate.io/v1 kind: NodeNetworkConfigurationPolicy metadata:

```
name: localnet-vlan-bridge-mapping spec: nodeSelector: node-role.kubernetes.io/worker: "
desiredState: ovn: bridge-mappings: - localnet: localnet-vlan bridge: br-ex state: present EOF
```

```
[discrete#networking:localnet-vlan::_03_vlan#networkattachmentdefinition#]
==== 03: VLAN#NetworkAttachmentDefinition#

#####VLAN#####localnet#####NetworkAttachmentDefinition#####

link:attachment$localnet-vlan/localnet-vlan-100-nad.yaml[localnet-vlan-100-nad.yaml
#####,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: "k8s.cni.cncf.io/v1" kind: NetworkAttachmentDefinition metadata:
name: localnet-vlan-100 namespace: vm-guests-vlan spec: config: '{ "cniVersion": "0.3.1", "name":
"localnet-vlan-100", "type": "ovn-k8s-cni-overlay", "topology": "localnet", "netAttachDefName": "vm-
guests-vlan/localnet-vlan-100", "physicalNetworkName": "localnet-vlan", "vlanID": 100, "subnets":
"192.168.100.0/24", "excludeSubnets": "192.168.100.1/32" }' EOF
```

```
**#####:

* `physicalNetworkName: "localnet-vlan"`: #NetworkAttachmentDefinition (NAD)
#NCP#####bridge-mappings# `localnet` #####
* `vlanID: 100`: #####VLAN#100#####
* `subnets`: OVN-Kubernetes IP#IP#####
* `excludeSubnets`: #####IP#####/#####
* `topology: "localnet"`: #####localnet#####

[discrete#networking:localnet-vlan::_04_vlan#####vm#####]
==== 04: VLAN#####VM#####

Pod#####VLAN####localnet#####VM#####

WARNING: ###cloud-init#####SSH#####

link:attachment$localnet-vlan/fedora-vlan-vm.yaml[fedora-vlan-vm.yaml
#####,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <a href="EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-
vlan-vm namespace: vm-guests-vlan labels: app: fedora-vlan-vm spec: runStrategy: Always
dataVolumeTemplates: - metadata: name: fedora-vlan-volume spec: pvc: accessModes: -
ReadWriteOnce resources: requests: storage: 33Gi sourceRef: kind: DataSource name: fedora
namespace: openshift-virtualization-os-images template: metadata: labels: app: fedora-vlan-vm
spec: domain: devices: disks: - name: datavolumedisk disk: bus: virtio - name: cloudinitdisk disk:
bus: virtio interfaces: - name: default bridge: {} - name: vlan_network bridge: {} resources: requests:
memory: 2Gi cpu: 1 networks: - name: default pod: {} - name: vlan_network multus: networkName:
```

```

localnet-vlan-100 volumes: - name: datavolumedisk dataVolume: name: fedora-vlan-volume - name:
cloudinitdisk cloudInitNoCloud: userData: | .pdf#cloud-config user: fedora password: fedora
chpasswd: { expire: False } packages: - python3 - tcpdump - net-tools runcmd: - echo
"&lt;h1&gt;Welcome to OpenShift Virtualization VLAN Network!&lt;/h1&gt;" &gt; /root/index.html -
echo "&lt;p&gt;VLAN ID: 100&lt;/p&gt;" ">EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine
metadata: name: fedora-vlan-vm namespace: vm-guests-vlan labels: app: fedora-vlan-vm spec:
runStrategy: Always dataVolumeTemplates: - metadata: name: fedora-vlan-volume spec: pvc:
accessModes: - ReadWriteOnce resources: requests: storage: 33Gi sourceRef: kind: DataSource
name: fedora namespace: openshift-virtualization-os-images template: metadata: labels: app:
fedora-vlan-vm spec: domain: devices: disks: - name: datavolumedisk disk: bus: virtio - name:
cloudinitdisk disk: bus: virtio interfaces: - name: default bridge: {} - name: vlan_network bridge: {}
resources: requests: memory: 2Gi cpu: 1 networks: - name: default pod: {} - name: vlan_network
multus: networkName: localnet-vlan-100 volumes: - name: datavolumedisk dataVolume: name:
fedora-vlan-volume - name: cloudinitdisk cloudInitNoCloud: userData: | .pdf</a> /root/index.html -
echo "&lt;p&gt;Network: 192.168.100.0/24&lt;/p&gt;" &gt;&gt; /root/index.html - cd /root
&amp;&amp; nohup python3 -m http.server 8080 &gt; /dev/null 2&gt;&amp;1 &amp; EOF

```

```

[discrete#networking:localnet-vlan::_5_vlan]
==== 5: VLAN

VMVLAN

[source,bash,role=execute]

```


Check the VM status

```
oc get vm -n vm-guests-vlan
```

Check the running VM instance

```
oc get vmi -n vm-guests-vlan
```


Check network interfaces - primary (pod network) and secondary (VLAN network)

`ip addr show`

**Check routes - two default gateways
(lower metric prioritized)**

Primary network: metric 100, Secondary VLAN: metric 101

ip route show

Expected output example:

```
default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.172 metric 100 default via 192.168.100.1  
dev enp2s0 proto dhcp src 192.168.100.50 metric 101
```

Test external connectivity

ping 8.8.8.8

Verify the web service is running

```
curl localhost:8080
```

Test VLAN interface connectivity (replace <vlan-ip> with actual IP)

```
curl <vlan-ip>:8080
```

Verify VLAN tagging (if tcpdump is available)

This will show VLAN-tagged traffic on the secondary interface

```
sudo tcpdump -i enp2s0 -nn vlan 100
```

```
[discrete#networking:localnet-vlan::_000000000000]
```

```
==== 000000000000
```

```
[discrete#networking:localnet-vlan::_00000000]
```

```
===== 00000000
```

```
. **VM VLAN000000000000**:
```

```
* 0000000000 VLAN000000000000000000000000
```

```
* 000000000000000000000000000000000000
```

```
* VLAN ID000000000000000000000000000000000000
```

```
. **IP000000000000**:
```

```
* VLAN00000000 DHCP000000000000000000000000
```

```
. **000000000000**:
```

```
* 000000000000000000000000000000000000000000
```

```
* 000000000000000000000000000000000000000000
```

```
[discrete#networking:localnet-vlan::_00000000]
```

```
===== 00000000
```

```
[source,bash,role=execute]
```

Check NetworkAttachmentDefinition status

```
oc get network-attachment-definitions -n vm-guests-vlan
```

Verify bridge mapping on worker nodes

```
oc get nncp localnet-vlan-bridge-mapping -o yaml
```

Check VM network status

```
oc get vmi -n vm-guests-vlan -o yaml | grep -A 10 networks
```

View OVN-Kubernetes logs for troubleshooting

```
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node
```

```
[discrete#networking:localnet-vlan:::_000]
```

== ==

:

```
* 配置 `NodeNetworkConfigurationPolicy` 配置VLAN配置
* localnet配置VLAN配置OVN-Kubernetes IPAM配置 `NetworkAttachmentDefinition` 配置
* VLAN配置
* Pod配置VLAN配置localnet配置VM配置
* VLAN配置VLAN配置
```

```
[discrete#networking:localnet-vlan:::_00]
```

==

*

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/multiple_networks/understanding-multiple-networks[OpenShift - 理解多个网络,window=_blank]

*

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/virtualization/networking#virt-creating-secondary-localnet-udn_virt-connecting-vm-to-secondary-udn[OCP Virtualization localnet,window= blank]

```
:!navtitle:
```

```
:docname: cudn-localnet-vlan
```

```
:page-module: networking
```

```
:page-relative-src-path: cudn-localnet-vlan.adoc
```

```
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
```

```
:page-origin-start-path:
```

```
:page-origin-refname: main
```

```
:page-origin-reftype: branch
```

```
:page-origin-refhash: (worktree)
```

```
:navtitle: Localnet VLAN-CUDN
```

```
[#networking: cudn-localnet-vlan]
```

=== Localnet VLAN+CuDN

OpenShift Virtualization Cluster User Defined Network (CUDN)

```

#####VLAN#####Localnet#####VLAN#####

```

Pod (VM)

```

#####VLAN#####CUDN#####VLAN#####

```

□□□□□

```
[discrete#networking:cdn-localnet-vlan:: 0000]
```



```
====  
```

```
* **NMState Operator**:    
* **NMState CR**: Operator  nmstate    
* ** **:    
* **VLAN **:  VLAN    
* ** **: OpenShift  VLAN  ** **
```

```
  :
```

OpenShift 4.19

```
[discrete#networking:cudn-localnet-vlan::_ ]
```

```
====  
```

```
[discrete#networking:cudn-localnet-vlan::_ _trunk_mode]
```

```
=====  (TRUNK MODE)
```

```
Localnet VLAN OpenShift **
```

```
* ** **:  VLAN    
* **VLAN **:  VLAN ID ( : 100, 200, 300)    
* ** **:  VLAN
```

```
[discrete#networking:cudn-localnet-vlan::_1_vm ]
```

```
====  1: VM
```

```
VLAN Localnet VM ClusterUserDefinedNetwork (CUDN)
```

```
[source,bash,role=execute]
```

Create first namespace

```
oc apply -f - <<EOF apiVersion: v1 kind: Namespace metadata: name: vm-guests-vlan-prod EOF
```

Create second namespace

```
oc apply -f - <<EOF apiVersion: v1 kind: Namespace metadata: name: vm-guests-vlan-dev EOF
```

```
[discrete#networking:cudn-localnet-vlan::_0000_2_00nic00000000ovs00000000]
```

```
====  2:  NICOOVS
```

VLAN (NIC) OVS

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: nmstate.io/v1 kind: NodeNetworkConfigurationPolicy metadata:
name: br-vlan-creation spec: nodeSelector: node-role.kubernetes.io/worker: " desiredState:
interfaces: - name: enp11s0f0np0 # Replace with your physical interface name type: ethernet state:
up ipv4: enabled: false ipv6: enabled: false description: "Physical interface for VLAN localnet" -
name: br-vlan type: ovs-bridge state: up bridge: port: - name: enp11s0f0np0 description: "OVS
bridge for VLAN traffic" EOF
```

[illegible]

```
[source,bash,role=execute]
```

oc debug node/<node-name> — ip link show

1000000000000000000

```
[source,bash,role=execute]
```

oc get nncp br-vlan-creation # Expected output: STATUS: Available

```
[discrete#networking:cdn-localnet-vlan:::_0000_3_000000000000]
```

==== 0000 3: 00000000000000

```

##### 'localnet-vlan' ##### 'br-vlan'

```

| | | | |
|-----|------|----------|----------------|
| OVS | VLAN | Localnet | OVN-Kubernetes |
|-----|------|----------|----------------|

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: nmstate.io/v1 kind: NodeNetworkConfigurationPolicy metadata:
name: cudn-localnet-vlan-bridge-mapping spec: nodeSelector: node-role.kubernetes.io/worker: "
desiredState: ovn: bridge-mappings: - localnet: localnet-vlan bridge: br-vlan state: present EOF
```

```
[discrete#networking:cudn-localnet-vlan::: 4 vlan clusteruserdefinednetwork]
```

```
==== 4: VLANClusterUserDefinedNetwork
```

```
#####VLAN#####Localnet#####ClusterUserDefinedNetwork#####
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: k8s.ovn.org/v1 kind: ClusterUserDefinedNetwork metadata: name:
cudn-localnet-vlan-100 spec: namespaceSelector: matchExpressions: - key:
kubernetes.io/metadata.name operator: In values: ["vm-guests-vlan-prod", "vm-guests-vlan-dev"]
network: topology: Localnet localnet: role: Secondary physicalNetworkName: localnet-vlan vlanID:
100 ipam: mode: Disabled EOF
```

```
**#####**:
```

```
* `vlanID: 100`: #####VLAN#100#
* `physicalNetworkName`: ##### (`localnet-vlan`) #
* `role: Secondary`: ##### (VM#Pod#####)
* `namespaceSelector`: #####
* **IPAM Disabled**: IP#####cloud-init (#) #####DHCP#####
```

```
[discrete#networking:cudn-localnet-vlan::_#_5_vlan#####ip###vm###]
==== # 5: VLAN#####IP###VM#####
```

```
#####1#Pod#####VLAN#Localnet#####VM#####
```

```
#: ###cloud-init#####SSH#####
```

```
[source,yaml]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-cudn-
vlan-vm namespace: vm-guests-vlan-prod labels: app: fedora-cudn-vlan-vm spec: runStrategy:
Always dataVolumeTemplates: - metadata: name: fedora-cudn-vlan-volume spec: pvc: accessModes:
- ReadWriteOnce resources: requests: storage: 33Gi sourceRef: kind: DataSource name: fedora
namespace: openshift-virtualization-os-images template: metadata: labels: app: fedora-cudn-vlan-
vm spec: domain: devices: disks: - name: datavolumedisk disk: bus: virtio - name: cloudinitdisk
disk: bus: virtio interfaces: - name: default bridge: {} - name: cudn_vlan_network bridge: {}
resources: requests: memory: 2Gi cpu: 1 networks: - name: default pod: {} - name:
cudn_vlan_network multus: networkName: cudn-localnet-vlan-100 volumes: - name:
datavolumedisk dataVolume: name: fedora-cudn-vlan-volume - name: cloudinitdisk
cloudInitNoCloud: networkData: | version: 2 ethernet: enp2s0: addresses: ["192.168.100.10/24"]
userData: | #cloud-config user: fedora password: fedora chpasswd: { expire: False } packages: -
python3 - tcpdump - net-tools write_files: - path: /root/index.html content: | <h1>Welcome to
OpenShift Virtualization CUDN VLAN Network!</h1> <p>VLAN ID: 100</p> <p>Static IP:
192.168.100.10/24</p> <p>Network: CUDN Localnet</p> <p>Namespace: vm-guests-vlan-prod</p>
permissions: '0644' runcmd: - nohup python3 -m http.server 8080 --directory /root > /dev/null 2>&1
& EOF
```


Check VMs in production namespace

```
oc get vm -n vm-guests-vlan-prod oc get vmi -n vm-guests-vlan-prod
```

Check VMs in development namespace

```
oc get vm -n vm-guests-vlan-dev oc get vmi -n vm-guests-vlan-dev
```

Access production VM console (credentials: fedora/fedora via cloud-init)

```
virtctl console -n vm-guests-vlan-prod fedora-cudn-vlan-vm
```


Access development VM console (credentials: fedora/fedora via cloud-init)

```
virtctl console -n vm-guests-vlan-dev fedora-cudn-vlan-vm-dev
```

```
VMVLAN
```

```
[source,bash]
```

Check network interfaces - primary (pod network) and secondary (VLAN network)

`ip addr show`

Check routes

ip route show

Expected output example (primary network via DHCP):

default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.174 metric 100

Test external connectivity

ping 8.8.8.8

Verify the web service is running

```
curl localhost:8080
```

Verify static IP assignment on VLAN interface

`ip addr show enp2s0` # Expected: inet 192.168.100.10/24 (prod) or 192.168.100.20/24 (dev)

Test VLAN interface connectivity between VMs

From production VM (192.168.100.10) ping development VM:

ping 192.168.100.20

Verify VLAN tagging (if tcpdump is available)

This will show VLAN-tagged traffic on the secondary interface

```
sudo tcpdump -i enp2s0 -nn vlan 100
```

```
[discrete#networking:cudn-localnet-vlan::_0000_8_0000000000000000]
```

```
==== 0000 8: 0000000000000000
```

```
000000000000VM00000000CUDN VLAN0000000000000000000000000000
```

```
[source,bash]
```

From production VM (192.168.100.10), test connectivity to development VM

```
ping 192.168.100.20
```

Test web service connectivity between namespaces

`curl 192.168.100.20:8080`

From development VM (192.168.100.20), test connectivity to production VM

ping 192.168.100.10 curl 192.168.100.10:8080

```
[discrete#networking: cudn-localnet-vlan::_]
=====
```

OVSxref:networking:ovs-bridge-troubleshooting::_clusteruserdefinednetwork_cudn_verification[OVS]

```
[discrete#networking: cudn-localnet-vlan::_]
=====
```

```
[discrete#networking: cudn-localnet-vlan::_vlanvm]
===== VLANVM
```

* VLAN

*

* VLAN ID

* OVS ('br-vlan') : 'oc get nncp br-vlan-creation'

*

```
[discrete#networking: cudn-localnet-vlan::_ip]
===== IP
```

* cloud-initnetworkDataIP

* (2enp2s0)

* VMcloud-init: 'cat /var/log/cloud-init.log'

```
[discrete#networking: cudn-localnet-vlan::_]
=====
```

* VLAN

* VLAN

*

```
[discrete#networking: cudn-localnet-vlan::_]
=====
```

* CUDN

* values

* NAD

```
[discrete#networking:cudn-localnet-vlan::_cudn]
```

```
===== CUDN
```

```
* CUDN `physicalNetworkName` ('localnet-vlan') 
```

```
* CUDN NAD: `oc get network-attachment-definitions -A | grep cudn-localnet-vlan-100`
```

```
* IPAM (cloud-initIP `Disabled`)
```

```
[discrete#networking:cudn-localnet-vlan::_]
```

```
=====
```

```
[source,bash,role=execute]
```

Check ClusterUserDefinedNetwork status

```
oc get clusteruserdefinednetwork
```


Verify bridge mapping on worker nodes

```
oc get nncp cudn-localnet-vlan-bridge-mapping -o yaml
```

Check VM network status in both namespaces

```
oc get vmi -n vm-guests-vlan-prod -o yaml | grep -A 10 networks oc get vmi -n vm-guests-vlan-dev -o  
yaml | grep -A 10 networks
```

View OVN-Kubernetes logs for troubleshooting

```
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node
```

Check CUDN network attachment across namespaces

```
oc get network-attachment-definitions -A | grep cudn-localnet-vlan-100
```

```
[discrete#networking:cudn-localnet-vlan::_cudn-networkattachmentdefinition]
==== CUDNNetworkAttachmentDefinition
```

```
|====
|  | CUDN | NetworkAttachmentDefinition
```

```
| ****
| 
| 
| 
```

```
| **
| 
| 
```

```
| **VLAN**
|  vlanID
|  vlanID
```

```
| **IP**
|  /Disabled
|  IPAM/DHCP
```

```
| ****
| 
| 
|====
```

```
[discrete#networking:cudn-localnet-vlan::_]
====
```

- * NIC OVS VLAN
- * VLAN `ClusterUserDefinedNetwork`
- * CUDN VLAN cloud-init IP VM
- * CUDN VLAN VM
- * VLAN CUDNNetworkAttachmentDefinition

```
[discrete#networking:cudn-localnet-vlan::_]
====
```

```
*
```

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/m

```

multiple_networks/understanding-multiple-networks[OpenShift - 
( ),window=_blank]
*
link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/v
irtualization/networking#virt-creating-secondary-localnet-cudn_virt-connecting-vm-to-
secondary-cudn[ClusterUserDefinedNetwork( ),window=_blank]

:!navtitle:

:docname: layer2-secondary
:page-module: networking
:page-relative-src-path: layer2-secondary.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Layer 2 
[#networking:layer2-secondary]
=== Layer 2 

[discrete#networking:layer2-secondary::_]
==== 

VirtualMachines` layer2` 

VirtualMachines` Pod 

* VM 
* VM 
* VM 
* 
* 
* 

:

```

OpenShift 4.20,4.21

```

[discrete#networking:layer2-secondary::_]
==== 

* OpenShift Virtualization OpenShift 4.20 
* `NetworkAttachmentDefinition` 
* CLI : `oc`, `virtctl`
* Kubernetes 

[discrete#networking:layer2-secondary::_]
==== 

```

```
#####3#####
```

```
* <<networking:layer2-secondary::creating_a_namespace,#####>>
* <<networking:layer2-secondary::creating_a_nad,NetworkAttachmentDefinition ###>>
* <<networking:layer2-secondary::attaching_vms,VirtualMachines #####>>
** <<networking:layer2-secondary::creating_first_vm,### VirtualMachine ###>>
** <<networking:layer2-secondary::creating_second_vm,2### VirtualMachine ###>>
* <<networking:layer2-secondary::testing_secondary_network,#####>>
```

```
[discrete#networking:layer2-secondary::creating_a_namespace]
==== ##### [[networking:layer2-secondary::creating_a_namespace]]
```

```
##### 'NetworkAttachmentDefinition' ##### 'VirtualMachines'
#####
```

```
YAML #####`oc` #####
```

```
[source,bash,role=execute]
```

oc create namespace example-l2-secondary

```
##### kube #####OpenShift #####
```

```
[source,bash,role=execute]
```

oc project example-l2-secondary

```
#####<<networking:layer2-
secondary::creating_a_nad,NetworkAttachmentDefinition ###>> #####
```

```
[discrete#networking:layer2-secondary::creating_a_nad]
==== NetworkAttachmentDefinition ### [[networking:layer2-secondary::creating_a_nad]]
```

```
2##### `layer2` ##### `NetworkAttachmentDefinition' #####
```

```
`l2-network.nad.yaml` ##### YAML #####
```

```
[source,yaml]
```

```
apiVersion: k8s.cni.cncf.io/v1 kind: NetworkAttachmentDefinition metadata: name: l2-network
namespace: example-l2-secondary spec: config: |- { "cniVersion": "0.3.1", "name": "l2-network",
"type": "ovn-k8s-cni-overlay", "topology":"layer2", "mtu": 1400, "netAttachDefName": "example-l2-
secondary/l2-network" }
```

```
`oc apply` #####OpenShift ##### `example-l2-secondary` 
`NetworkAttachmentDefinition' #####
```

```
[source,bash,role=execute]
```

```
oc apply -f l2-network.nad.yaml
```

```
`NetworkAttachmentDefinition` .....`l2-network`  `example-l2-secondary`  
.....
```

```
[source,bash,role=execute]
```

```
oc get net-attach-def l2-network -n example-l2-secondary
```

```
`l2-network` .....<<networking:layer2-secondary::attaching_vms,VirtualMachines .....>>  
.....
```

```
[discrete#networking:layer2-secondary::attaching_vms]  
==== VirtualMachines ..... [[networking:layer2-secondary::attaching_vms]]
```

```
.....`VirtualMachines` ..... `layer2` .....  
..... OpenShift (Kubernetes) .....`VirtualMachine` API .....  
.....
```

```
.....`l2-network` .....2..... VM .....
```

```
[discrete#networking:layer2-secondary::creating_first_vm]  
===== ..... VirtualMachine ..... [[networking:layer2-secondary::creating_first_vm]]
```

```
..... VirtualMachine  YAML .....  
..... `fedora-l2-secondary.vm.yaml` .....
```

```
..: ..... cloud-init ..... SSH .....
```

```
[source,yaml]
```

```
apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-l2-secondary labels: app:  
fedora-l2-secondary spec: dataVolumeTemplates: - metadata: name: fedora-l2-secondary-root spec:  
sourceRef: kind: DataSource name: fedora namespace: openshift-virtualization-os-images storage:  
resources: requests: storage: 30Giinstancetype: name: u1.large preference: name: fedora  
runStrategy: Always template: metadata: labels: app: fedora-l2-secondary spec: domain: devices:  
interfaces: - name: default masquerade: {} - name: secondary bridge: {} resources: {}  
terminationGracePeriodSeconds: 180 networks: - name: default pod: {} - name: secondary multus:  
networkName: l2-network volumes: - name: fedora-l2-secondary-root dataVolume: name: fedora-l2-  
secondary-root - name: cloudinitdisk cloudInitNoCloud: userData: | #cloud-config user: fedora  
password: fedora chpasswd: expire: False networkData: | version: 2 ethernet: enp2s0: addresses: -  
192.168.20.10/24
```

```
[NOTE]
```

```
.名前 VM 名前空間
====
名前空間のspec.template.spec` 名前空間

**domain.devices.interfaces** - 名前空間の2名前空間

* **name** - 名前空間 `networks` 名前空間 `secondary` 名前空間
* **bridge** 名前 **masquerade** - 名前空間 `bridge` 名前空間 Pod
名前空間 (`{}`) 名前空間

**networks** - 名前空間 `interfaces` 名前空間2名前空間

* **name** - `default` 名前空間 `secondary` 名前空間
* **pod** 名前 **multus** - 名前空間 `multus` 名前空間 `pod`
名前空間 `multus` 名前 `networkName` 名前空間
** **networkName** (`multus` 名前) - 名前空間 `NetworkAttachmentDefinition` 名前空間 `l2-
network` 名前空間

**volumes** - `fedora-l2-secondary-root` 名前空間 VM 名前空間
`cloudinitdisk` 名前空間2名前空間
====

`oc create` 名前空間OpenShift 名前空間 (`example-l2-secondary`) 名前空間 VirtualMachine
名前空間

[source,bash,role=execute]
```

oc create -f fedora-l2-secondary.vm.yaml

```
名前空間 `fedora-l2-secondary` VM 名前 `example-l2-secondary` 名前空間 `Provisioning`
名前空間

[source,bash,role=execute]
```

oc get vm fedora-l2-secondary -n example-l2-secondary

```
[IMPORTANT]
====
.名前空間
`VirtualMachine` 名前 `Pending` 名前空間VM 名前空間2名前空間

* 名前空間 `DataVolume` 名前空間
* `l2-network` 名前 `NetworkAttachmentDefinition` 名前空間
* OpenShift 名前空間 vCPU 名前空間VM 名前空間2名前空間

名前空間 `vmi` `datavolume` `pvc` 名前空間 `oc describe` 名前 `oc get -o yaml`
名前空間
名前空間 Events (`oc describe`) 名前 Status (`oc get -o yaml`) 名前空間
====
```



```
* `VirtualMachine`: `fedora-l2-secondary2` (NIC IP: 192.168.20.20)

VM ping

`virtctl` `VirtualMachine` **:`fedora`**:`fedora`
`ENTER`

[source,bash,role=execute]
```

virtctl console fedora-l2-secondary

```
`VirtualMachine` IP `enp2s0` `192.168.20.10`

[source,bash]
```

ip addr show

```
VM `fedora-l2-secondary2` (IP: `192.168.20.20`) VM ping
`CTRL+C` ping

[source,bash]
```

ping 192.168.20.20

```
VM `CTRL+]`

`example-l2-secondary`

[source,bash,role=execute]
```

oc project default oc delete namespace example-l2-secondary

```
[discrete#networking:layer2-secondary::_]
===

* `ovn-k8s-cni-overlay` CNI `layer2` `NetworkAttachmentDefinition`
* masquerade bridge VirtualMachines
* cloud-init `networkData` IP
* OVN VM Layer 2
* Linux localnet Layer 2

[discrete#networking:layer2-secondary::_]
===
```

```

*
link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/v
irtualization/networking#virt-connecting-vm-to-ovn-secondary-network[OpenShift
Virtualization - OVN ,window=_blank]
*
link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/m
ultiple_networks/understanding-multiple-networks[OpenShift - ,window=_blank]

:!navtitle:

:docname: linux-bridges
:page-module: networking
:page-relative-src-path: linux-bridges.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Linux
[#networking:linux-bridges]
=== Linux

:

```

OpenShift 4.20,4.21

```

Linuxhttps://nmstate.io/kubernetes-nmstate/[Kubernetes NMState
Operator,_window=blank]Linuxhttps://github.com/k8snetworkplumbin
gwg/multus-cni[Multus
CNI,_window=_blank]https://www.cni.dev/plugins/current/main/bridge/[bridge
CNI,_window=blank]Pod`VirtualMachine` (VM)

LinuxPodVM2
LinuxVMPod

Linux

* Openshift bridge CNI
* VMNAT/
* `VirtualMachine` VLAN

: Linux `NetworkAttachmentDefinition` `VirtualMachine` VLAN

[discrete#networking:linux-bridges::_]
===

* OpenShift VirtualizationOpenShift 4.20
* NMState
* NodeNetworkConfigurationPolicyNetworkAttachmentDefinition
* CLI: `oc`, `virtctl`

```

```

*
[discrete#networking:linux-bridges::_]
====

networking:linux-bridges::create_linux_bridge_nncp, Linux
networking:linux-bridges::create_net_attach_def, NetworkAttachmentDefinition Pod VM
networking:linux-bridges::attach_virtual_machines, VirtualMachine Linux
** <<networking:linux-bridges::create_first_vm, VirtualMachine>>
** <<networking:linux-bridges::create_second_vm, 2 VirtualMachine>>
* <<networking:linux-bridges::test_secondary_network, Linux>>

[discrete#networking:linux-bridges::create_linux_bridge_nncp]
==== Linux [[networking:linux-bridges::create_linux_bridge_nncp]]

NodeNetworkConfigurationPolicy` (`nncp`) Linux
`nncp`
`nncp`

[IMPORTANT]
====
.:
* `NodeNetworkConfigurationPolicy` Kubernetes NMState Operator
* `nncp` VM
====

NodeNetworkConfigurationPolicy` lnbr0-bridge.nncp.yaml`
2

[source,yaml]

```

```

apiVersion: nmstate.io/v1 kind: NodeNetworkConfigurationPolicy metadata: name: lnbr0-bridge
spec: desiredState: interfaces: - name: lnbr0 description: layer2-only linux bridge with an optional
port type: linux-bridge state: up ipv4: enabled: false bridge: options: stp: enabled: false #
Uncomment below and change the interface name # to match the desired physical host port #port:
#- name: ens1

```

```

.:
* *`lnbr0`* (`linux bridge 0`): Linux
* *`ipv4`*: IP
* *`options.stp.enabled`*:
* *`port`* (): 1
** *`name`*: ( `ens1`
: https://nmstate.io/devel/yaml_api.html[NMState

```

```
API[window=_blank]
```

```
`lnbr0-bridge.nncp.yaml` `nncp` OpenShift
```

```
[source,bash,role=execute]
```

oc create -f lnbr0-bridge.nncp.yaml

```
`lnbr0`
```

```
`NodeNetworkConfigurationPolicy`  
`NodeNetworkConfigurationEnactment` (`nncp`)
```

```
[source,bash,role=execute]
```

oc get nncp

```
`Progressing` `Available`
```

```
`lnbr0` `NodeNetworkState` `ip addr show`
```

```
`NodeNetworkState` `nns` `grep`
```

```
[source,bash,role=execute]
```

oc describe nns | grep lnbr0

```
`nmstate` NMState Operator `nncp`  
`nncp`
```

```
Linux<<networking:linux-  
bridges::create_net_attach_def,NetworkAttachmentDefinition>>
```

```
[discrete#networking:linux-bridges::create_net_attach_def]  
=== LinuxNetworkAttachmentDefinition [[networking:linux-  
bridges::create_net_attach_def]]  
Pod/VM2 `NetworkAttachmentDefinition`  
`NetworkAttachmentDefinition` /Pod/VM
```

```
IMPORTANT: `NetworkAttachmentDefinition` `VirtualMachine`  
`nncp`  
NICVM
```

```
VM
```

```
[source,bash,role=execute]
```

```
oc create namespace bridge-demo
```

□□□□□□□□□□□□

```
[source,bash,role=execute]
```

oc project bridge-demo

```
00000000 `NetworkAttachmentDefinition` 00000000 \n br0-bridge.net-attach-def.yaml`  
00000000000000000000
```

'nncsp' 00000000000000000000000000000000 'vlan' IP000000000

```
`nncp`   00000000000000000000000000000000 `vlan` ID000000000000
```

```
[source,yaml]
```

```
[source,yaml]
```

```
apiVersion: k8s.cni.cncf.io/v1 kind: NetworkAttachmentDefinition metadata: name: lnbr0-bridge
namespace: bridge-demo spec: config: '{ "name": "lnbr0-bridge", "cniVersion": "0.3.1", "type":
"bridge", "bridge": "lnbr0", "ipam": {}, "vlan": 1, "macspoofchk": false, "disableContainerInterface":
false, "portIsolation": false }'
```

[illegible]

```
* **metadata.name**: 'NetworkAttachmentDefinition'  Kubernetes
* **metadata.namespace**: Pod / VM 'bridge-demo'
```

```
* *`metadata.namespace`: Pod/VM`bridge-demo`
* *`spec.config`: JSON
** *`"name"`: `metadata.name`
```

```
* *`spec.config`: 配置信息JSON
** *`"name"`: 元数据名称
** *`"cniVersion"`: CNI版本 `0.3.1`
```

```
** *\'name\'*: 000000\'metadata.name\' 0000000000000000
** *\'cniVersion\'*: 000000000000CNI000000 \'0.3.1\' 000000
** *\'type\'* (00): Linux000000000000000000000000 \'bridge\' 000000000000
```

```
** *\"cniVersion\"*: 0000000000CNI000000 \"0.3.1\" 000000
** *\"type\"* (00): Linux00000000000000000000 \"bridge\" 000000000000
** *\"bridge\"* (00): NNCP0000000000/0000000000000000000000000000
```

```
** *`"type"*` (0): Linux  "bridge"  0
** *`"bridge"*` (0): NNCP  /  0
** *`"ipam"*` (0): OpenShift Virtualization  bridge
CNI  IP  `layer2`  0
```

```
** *"  
** *"  
** *"  
CNI  
** *"  
** *"
```

```
** *`ipam`* (配置): OpenShift Virtualization bridge
CNI配置IP配置layer2 配置
** *`vlan`* (配置): 配置VLAN ID `1`
** *`macspoofchk`* (配置): MAC配置MAC配置
```

```
CNIIP`layer2`
** *`vlan`* (): VLAN ID `1`
** *`macspoofchk`* (): MACMAC
`false`
```

```
** *`"vlan"* (vlan): 指定VLAN ID`1`
** *`"macspoofchk"* (macspoofchk): MAC地址欺骗检查`false`
** *`"disableContainerInterface"* (disableContainerInterface): 禁用veth接口
```

```
** *`"macspoofchk"`* (布尔型): MAC地址欺骗功能是否启用。默认为false。
`false`
** *`"disableContainerInterface"`* (布尔型): 是否禁用容器接口veth。默认为false。
`false`
** *`"portIsolation"`* (布尔型):
```

```
`false`  
** *"disableContainerInterface"* (bool): 是否禁用容器接口  
`false`
```

```
** *`"disableContainerInterface"* (bool): 是否禁用veth接口
`false`

** *`"portIsolation"* (bool):
是否禁用端口隔离 `false`
```

```
`false`  
** *`"portIsolation"`* (****):  
***** `false`
```

```

** *`"portIsolation"* (bool):
    是否启用端口隔离功能。默认为 false。

Linux CNI 支持 OpenShift

```

LinuxCNIOpenShift
Virtualizationhttps://www.cni.dev/plugins/current/main/bridge/[bridge
CNIwindows - block]

```
LinuxCNIOpenShift
Virtualizationhttps://www.cni.dev/plugins/current/main/bridge/[bridge
CNI,window=_blank]
https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virt
```

```
Virtualizationhttps://www.cni.dev/plugins/current/main/bridge/[bridge
CNI,window=_blank]
https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virt
ualization/networking#virt-creating-linux-bridge-nad-cli_virt-connecting-vm-to-linux-
bridge[OpenShift,window=_blank]
```

```
CNI[OpenShift,window=_blank]
https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virt
ualization/networking#virt-creating-linux-bridge-nad-cli_virt-connecting-vm-to-linux-
bridge[OpenShift,window=_blank]
```

```
https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virt
ualization/networking#virt-creating-linux-bridge-nad-cli_virt-connecting-vm-to-linux-
bridge[OpenShift,window=_blank]
```

[NOTE]

====

.IP

* `VirtualMachine` `NetworkAttachmentDefinition` IP IPAM OpenShift

Virtualization`"ipam": {}`

* LinuxVMIP

** *cloud-init networkData*: VMcloud-initIP

** *DHCP*: DHCPVMIP

** **: VM `nmcli` IP

* OpenShift

Virtualizationhttps://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/configuring_and_managing_networking/configuring-a-network-bond#upstream-switch-configuration-depending-on-the-bonding-modes[Linux,window=_blank]

056

====

`lnbr0-bridge.net-attach-def.yaml` `network-attachment-definition`
OpenShift

[source,bash,role=execute]

oc create -f lnbr0-bridge.net-attach-def.yaml

`network-attachment-definition`

[source,bash,role=execute]

oc get net-attach-def lnbr0-bridge

32 <<networking:linux-bridges::attach_virtual_machines,VirtualMachine>>

[discrete#networking:linux-bridges::attach_virtual_machines]

==== VirtualMachine [[networking:linux-bridges::attach_virtual_machines]]

2VirtualMachine

* `kubernetes-nmstate-operator` `nmstate`

* `NodeNetworkConfigurationPolicy` OpenShiftLinux

* `NodeNetworkConfigurationEnactment`

* `NodeNetworkState`

* (Multus) `NetworkAttachmentDefinition` `VirtualMachine`

Linux

<<networking:linux-bridges::create_first_vm,VirtualMachine>>

[discrete#networking:linux-bridges::create_first_vm]

```

===== VirtualMachine [[networking:linux-bridges::create_first_vm]]

YAML`kind`: 'VirtualMachine`oc apply`

VirtualMachineYAML`fedora-bridge-demo.vm.yaml`

WARNING: cloud-initSSH

[source,yaml]

```

```

apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-bridge-demo namespace:
bridge-demo labels: app: fedora-bridge-demo spec: runStrategy: Always dataVolumeTemplates: -
metadata: name: fedora-bridge-demo-root spec: sourceRef: kind: DataSource name: fedora
namespace: openshift-virtualization-os-images storage: resources: requests: storage: 30Gi
instancetype: name: u1.large preference: name: fedora template: metadata: labels: app: fedora-
bridge-demo spec: domain: devices: disks: - name: fedora-bridge-demo-root disk: bus: virtio - name:
cloudinitdisk disk: bus: virtio interfaces: - masquerade: {} name: default - bridge: {} name: nic-linux-
bridge networks: - name: default pod: {} - multus: networkName: lnbr0-bridge name: nic-linux-
bridge volumes: - name: fedora-bridge-demo-root dataVolume: name: fedora-bridge-demo-root -
name: cloudinitdisk cloudInitNoCloud: networkData: | version: 2 ethernet: enp2s0: addresses: -
192.168.50.10/24 userData: | #cloud-config user: fedora password: fedora chpasswd: { expire: False }

```

```

`spec.template.spec`

* `domain.devices.interfaces`: `default` `nic-linux-bridge`
2
** `bridge`: VMNIC21IP `masquerade`
vm-to-container`lnbr0` container-to-node
** `name`: `network`
* `networks`: `interfaces`
** `name`: `interfaces`
** `multus.networkName`: Pod/VM `NetworkAttachmentDefinition` (NAD)

* `volumes`: 2 `datavolumedisk`1 `cloudinitdisk`
** `CloudInitNoCloud`: cloud-
inithttps://cloudinit.readthedocs.io/en/latest/reference/datasources/nocloud.htm
l[NoCloud,window=_blank]KubeVirt
*** `networkData`: (`enp2s0`) IP (`192.168.50.10/24`)
**** `ethernets`: `enp2s0` KubeVirt2

: (`enp1s0`) PodIP

`VirtualMachine` `fedora-bridge-demo.vm.yaml` OpenShift

[source,bash,role=execute]

```

oc apply -f fedora-bridge-demo.vm.yaml


```
'VirtualMachine' VM 'DataVolume'
VM 'Running' 2 VM

[source,bash,role=execute]
```

oc get vm fedora-bridge-demo

```
[discrete#networking:linux-bridges::create_second_vm]
==== 2 VirtualMachine [[networking:linux-bridges::create_second_vm]]

'VirtualMachine' 'vm' IP
YAML 'fedora-bridge-demo2.vm.yaml'

[source,yaml]
```

```
apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-bridge-demo2 namespace:
bridge-demo labels: app: fedora-bridge-demo2 spec: runStrategy: Always dataVolumeTemplates: -
metadata: name: fedora-bridge-demo2-root spec: sourceRef: kind: DataSource name: fedora
namespace: openshift-virtualization-os-images storage: resources: requests: storage: 30Gi
instancetype: name: u1.large preference: name: fedora template: metadata: labels: app: fedora-
bridge-demo2 spec: domain: devices: disks: - name: fedora-bridge-demo2-root disk: bus: virtio -
name: cloudinitdisk disk: bus: virtio interfaces: - masquerade: {} name: default - bridge: {} name:
nic-linux-bridge networks: - name: default pod: {} - multus: networkName: lnbr0-bridge name: nic-
linux-bridge volumes: - name: fedora-bridge-demo2-root dataVolume: name: fedora-bridge-demo2-
root - name: cloudinitdisk cloudInitNoCloud: networkData: | version: 2 ethernet: enp2s0:
addresses: - 192.168.50.20/24 userData: | #cloud-config user: fedora password: fedora chpasswd: {
expire: False }
```

```
'fedora-bridge-demo2.vm.yaml' 2 'VirtualMachine' OpenShift

[source,bash,role=execute]
```

oc create -f fedora-bridge-demo2.vm.yaml

```
VM 'VirtualMachineInstance' ('vmi') 'interfaces' 'status'

[source,bash,role=execute]
```

```
oc get vm -n bridge-demo oc get vmi fedora-bridge-demo -n bridge-demo -o
jsonpath='{.status.interfaces}' | jq . oc get vmi fedora-bridge-demo2 -n bridge-demo -o
jsonpath='{.status.interfaces}' | jq .
```

```
2 VM <<networking:linux-bridges::test_secondary_network,>>
```

```
[discrete#networking:linux-bridges:::test_secondary_network]
=== Linux [[networking:linux-bridges:::test_secondary_network]]

Linux PodVM

Linux 'VirtualMachine' 'VirtualMachine'

'bridge-demo'

* 'NetworkAttachmentDefinition': 'lnbr0-bridge'
* 'VirtualMachine': 'fedora-bridge-demo' (NIC IP: 192.168.50.10)
* 'VirtualMachine': 'fedora-bridge-demo2' (NIC IP: 192.168.50.20)

VM ping

'virtctl' VirtualMachine **user**:'fedora' **password**:'fedora'
'ENTER'

[source,bash,role=execute]
```

virtctl console fedora-bridge-demo

```
VirtualMachineIP`enp2s0` `192.168.50.10`

[source,bash]
```

ip addr show

```
VM`fedora-bridge-demo2` (IP: `192.168.50.20`) `VirtualMachine`
ping`CTRL+C` ping

[source,bash]
```

ping 192.168.50.20

```
VM`CTRL+]`

`bridge-demo`

[source,bash,role=execute]
```

oc project default oc delete namespace bridge-demo

```
[discrete#networking:linux-bridges:::_]
===
```

#####

```
* NMState Operator##### `NodeNetworkConfigurationPolicy` #####Linux#####
* bridge CNI##### `NetworkAttachmentDefinition` #####VM#####
* #####VirtualMachine#####
* cloud-init `networkData` #####IP#####
* Linux#####VM#####2#####
```

```
[discrete#networking:linux-bridges::_]
```

```
====
```

```
*
```

```
link:https://docs.redhat.com/en/documentation/openshift\_container\_platform/4.20/html/virtualization/networking#virt-connecting-vm-to-linux-bridge\_virt-connecting-vm-to-linux-bridge[OpenShift Virtualization - VMLinux,window=_blank]
* link:https://nmstate.io/devel/yaml\_api.html[NMState API,window=_blank]
```

```
:!navtitle:
```

```
:docname: sriov-secondary
:page-module: networking
:page-relative-src-path: sriov-secondary.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook\_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: SR-IOV #####
[#networking:sriov-secondary]
=== SR-IOV #####
```

```
#####:
```

OpenShift 4.21

SR-IOV (Single Root I/O Virtualization) ##### (VF) ##### (VM) #####

(OVN ##### Linux #####) #####SR-IOV #####VM

#####SR-IOV ##### (NFV)#####

#####SR-IOV ##### VLAN ##### VLAN #####

SR-IOV VF ##### VLAN ##### VM #####

(UPF) ##### VLAN ##### (Telco)

#####SR-IOV Network Operator #####SR-IOV ##### VLAN 2 SR-IOV ##### SR-IOV ##### VM #####


```
00: SR-IOV   000000000000 NIC   0000000000000000  
000000000000Machine Config Operator 0000 SR-IOV Network Operator 0000000000000000000000  
0000000000000000000000000000
```

```
==== 0000 1: SR-IOV Operator 00000000
```

```
[source,bash,role=execute]
```

□□□□□□ (□□□□□□□□□□□□) :

Pod _____:

```
[source,bash,role=execute]
```

```
===== 0000 2: SR-IOV 0000000000
```

```
[source,bash,role=execute]
```

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator
```

```

:

[source,bash,role=execute]
```

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o yaml
```

```
`status.interfaces`  `totalvfs`  0
PCI  ID VF
Mellanox ConnectX-4 Lx :
```

- deviceID: "1015" driver: mlx5_core linkSpeed: 25000 Mb/s linkType: ETH name: enp11s0f0np0 pciAddress: "0000:0b:00.0" totalvfs: 6 vendor: "15b3"

```

: IOMMU
IOMMU :

[source,bash,role=execute]
```

```
oc debug node/<node-name> — chroot /host ls /sys/kernel/iommu_groups/ | wc -l
```

```
0 (IOMMU) IOMMU
0 BIOS/UEFI IOMMU MachineConfig `intel_iommu=on iommu=pt`
(Intel) `amd_iommu=on iommu=pt` (AMD)

[discrete#networking:sriov-secondary::_3_sriovnetworknodepolicy_]
=== 3: SriovNetworkNodePolicy

`SriovNetworkNodePolicy` (PF) VF
OpenShift Virtualization VM `deviceType: vfio-pci` VF VFIO
PCI

4 VF
VF VLAN `SriovNetwork`
VF VLAN

link:attachment$sriov-secondary/sriov-network-node-policy.yaml[sriov-network-node-policy.yaml,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: sriovnetwork.openshift.io/v1 kind: SriovNetworkNodePolicy
metadata: name: mlx-sriov-policy namespace: openshift-sriov-network-operator spec:
resourceName: mlxsriovnic nodeSelector: kubernetes.io/hostname: <node-name> numVfs: 4
```

nicSelector: vendor: "15b3" deviceID: "1015" pfNames: - enp11s0f0np0 deviceType: vfio-pci isRdma: false EOF

```
<node-name> SR-IOV NIC ([: 'ocplab01'])

:

* 'resourceName' -- VF Kubernetes ([: 'openshift.io/mlxsriovnic')
* 'numVfs' -- VF ([: 'totalvfs' ]) VM 2 VF (VLAN 1 )
2 VM
* 'nicSelector.vendor' / 'deviceID' -- NIC Mellanox ConnectX-4 Lx
'15b3' / '1015' Intel NIC ID
* 'nicSelector.pfNames' -- VF
* 'deviceType: vfio-pci' -- VM Pod 'netdevice'

: SR-IOV VF
(drain)
Machine Config Operator

:

[source,bash,role=execute]
```

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o
jsonpath='{.items[0].status.syncStatus}'
```

```
Succeeded'

[discrete#networking:sriov-secondary::_4_]
=== 4:

( ) VF

[source,bash,role=execute]
```

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o yaml
```

```
'.status.interfaces' PF 'numVfs: 4' VF PCI

:

[source,bash,role=execute]
```

```
oc debug node/<node-name> — chroot /host cat /sys/class/net/enp11s0f0np0/device/sriov_numvfs
```

```
'numVfs' ([: '4')

-----
```

```
[source,bash,role=execute]
```

□□□□□□□□ :

```
[discrete#networking:sriov-secondary::_sriov_5_vlan_sr_iov_]
=== 5:  VLAN  SR-IOV

VLAN
`SriovNetwork` VLAN `NetworkAttachmentDefinition` (NAD)
NAD VF VM VLAN ID OS
VM:
```

```
link:attachment$sriov-secondary/sriov-demo-namespace.yaml[sriov-demo-namespace.yaml,role=unresolved]

[source,bash,role=execute]
```

```
VLAN 100 000000000000000000000000:

link:attachment$sriov-secondary/sriov-net-data.yaml[sriov-net-data.yaml
00000000,role=unresolved]

[source,bash,role=execute]
```

```
VLAN 200 0000000000000000:
```

```
link:attachment$sriov-secondary/sriov-net-mgmt.yaml[sriov-net-mgmt.yaml  
00000000,role=unresolved]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: sriovnetwork.openshift.io/v1 kind: SriovNetwork metadata: name:
sriov-net-mgmt namespace: openshift-sriov-network-operator spec: networkNamespace: sriov-
demo resourceName: mlxsrnivnic vlan: 200 spoofChk: "off" trust: "on" EOF
```

```
#####:
```

```
* `vlan` -- ##### VF ##### 802.1Q VLAN ID NIC ##### OS
#####
* `networkNamespace` -- NAD ##### VM #####
* `resourceName` -- `SriovNetworkNodePolicy` `resourceName` ##### VF
#####
* `spoofChk: "off"` `trust: "on"` -- OS MAC IP VM #####
```

```
## NAD #####:
```

```
[source,bash,role=execute]
```

```
oc get net-attach-def -n sriov-demo
```

```
#####:
```

```
NAME AGE sriov-net-data 10s sriov-net-mgmt 5s
```

```
[discrete#networking:sriov-secondary::_6_sr_iov_vm_]
=== ## 6: ## SR-IOV VM
```

```
##### Pod #####VLAN 100 ##### SR-IOV #####VLAN 200 ##### SR-IOV ##### 3
##### VM
#####
```

```
##: ## cloud-init
##### SSH
```

```
link:attachment$sriov-secondary/fedora-sriov-vm.yaml[fedora-sriov-vm.yaml
,role=unresolved]
```

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-sriov-
vm namespace: sriov-demo labels: app: fedora-sriov-vm spec: runStrategy: Always
dataVolumeTemplates: - metadata: name: fedora-sriov-volume spec: pvc: accessModes: -
ReadWriteOnce resources: requests: storage: 33Gi sourceRef: kind: DataSource name: fedora
namespace: openshift-virtualization-os-images template: metadata: labels: app: fedora-sriov-vm
spec: domain: devices: disks: - name: datavolumedisk disk: bus: virtio - name: cloudinitdisk disk:
bus: virtio interfaces: - name: default masquerade: {} - name: data-plane sriov: {} - name: mgmt-
plane sriov: {} resources: requests: memory: 4Gi cpu: 2 networks: - name: default pod: {} - name:
data-plane multus: networkName: sriov-net-data - name: mgmt-plane multus: networkName: sriov-
```



```
net-mgmt volumes: - name: datavolumedisk dataVolume: name: fedora-sriov-volume - name:
cloudinitdisk cloudInitNoCloud: userData: | #cloud-config user: fedora password: fedora chpasswd:
{ expire: False } EOF
```

```
VM 3 :

* `default` (masquerade: {}) -- IP (Pod)
* `data-plane` (sriov: {}) -- VLAN 100 VF
* `mgmt-plane` (sriov: {}) -- VLAN 200 VF

SR-IOV 1 VF VM 4 VF 2 VM

VM :

[source,bash,role=execute]
```

```
oc get vm -n sriov-demo
```

```
[source,bash,role=execute]
```

```
oc get vmi -n sriov-demo
```

```
VMI `Running` READY: True

[discrete#networking:sriov-secondary::_7_vlan_sr_iov_]
=== 7: VLAN SR-IOV

SR-IOV VF PCI VLAN

[discrete#networking:sriov-secondary::_pci_]
===== PCI

VM :

[source,bash,role=execute]
```

```
virtctl console -n sriov-demo fedora-sriov-vm
```

```
`fedora` / `fedora` VF PCI :

[source,bash]
```

```
lspci | grep -i ethernet
```

```
Pod virtio NIC 2 Mellanox (Intel) VF
```

ip link show

```
* `enp1s0` -- Pod ネットワーク (virtio ネット)
* `enp7s0` -- SR-IOV VF (ネットワークVLAN 100)
* `enp8s0` -- 2 SR-IOV VF (ネットワークVLAN 200)
```

例: ネットワークカードPCI ID/ネットワークカード `udev` のネットワークカード名を調べる

```
`ip link show` を実行して SR-IOV VF を確認する (virtio ネットワーク Mellanox または Intel
ネットワークカードの場合)
```

```
[discrete#networking:sriov-secondary::_ip_ネットワーク]
```

```
===== IP アドレス
```

SR-IOV ネットワーク IP アドレス
ネットワーク VLAN ネットワークカード名

ネットワーク (VLAN 100) ネットワーク:

```
[source,bash]
```

```
sudo ip addr add 10.100.0.10/24 dev enp7s0
```

```
[source,bash]
```

```
sudo ip link set enp7s0 up
```

```

# 200 (VLAN 200) の IP アドレス:
[source,bash]

```

```
sudo ip addr add 10.200.0.10/24 dev enp8s0
```

```
[source,bash]
```

```
sudo ip link set enp8s0 up
```

```
#####:
```

```
[source,bash]
```

```
ip -br addr show
```

```
#####:
```

```
lo UNKNOWN 127.0.0.1/8 ::1/128 enp1s0 UP 10.0.2.2/24 enp7s0 UP 10.100.0.10/24 enp8s0 UP 10.200.0.10/24
```

```
###: ##### `ip addr add` #####  
#####:
```

```
* **DHCP** --  VLAN DHCP #####NetworkManager #####  
* **Cloud-init** -- VM cloud-init ##### `networkData` ##### IP #####  
xref:networking:static-ip-configuration[] #####  
* **NetworkManager #####** -- cloud-init `write_files` ##### NetworkManager #####  
`.nmconnection` #####
```

```
[discrete#networking:sriov-secondary::_vlan_#####]  
===== VLAN #####
```

```
##### VLAN ##### VLAN ##### ping #####
```

```
##### VLAN (100) #####:
```

```
[source,bash]
```

```
ping -c 3 -I enp7s0 10.100.0.1
```

```
## VLAN (200) #####:
```

```
[source,bash]
```

```
ping -c 3 -I enp8s0 10.200.0.1
```

```
### ping #####VLAN ##### VF ##### VLAN #####
```

```
##### VLAN 200 ##### (#####) #####
```

```
#####`Ctrl+]` #####
```

```
[discrete#networking:sriov-secondary::_#####]  
===== #####
```

```

[discrete#networking:sriov-secondary::_vf_ ]
===== VF

* `SriovNetworkNodeState` `Failed` NIC
* BIOS/UEFI IOMMU
* `nicSelector` (IDPF)

[discrete#networking:sriov-secondary::_vm_]
===== VM

* VF : `oc get node <node-name> -o jsonpath='{.status.allocatable}'`
* 0 VF
* `deviceType: vfio-pci` VM VFIO
* `Insufficient openshift.io/mlxsriovnic` VM VF VM SR-IOV
`numVfs`

[discrete#networking:sriov-secondary::_vm_sr_iov_]
===== VM SR-IOV

* `SriovNetwork` `resourceName` `SriovNetworkNodePolicy` `resourceName`
* NAD : `oc get net-attach-def -n sriov-demo`
* virt-launcher Pod : `oc describe pod -n sriov-demo -l app=fedora-sriov-vm`

[discrete#networking:sriov-secondary::_sr_iov_]
===== SR-IOV

* NIC : `oc debug node/<node-name> -- chroot /host ip link show <pf-name>`
* VLAN 100 200
* Mellanox NIC

[discrete#networking:sriov-secondary::_vlan_vm_]
===== VLAN VM

* `SriovNetwork` `vlan` VLAN
* ( ) VLAN ID
* VF `SriovNetwork` `spooofChk` `"off"` `trust` `"on"`

[discrete#networking:sriov-secondary::_]
=====

:

[source,bash,role=execute]

```

oc delete vm fedora-sriov-vm -n sriov-demo

```
[source,bash,role=execute]
```

```
oc delete sriovnetwork sriov-net-data -n openshift-sriov-network-operator
```

```
[source,bash,role=execute]
```

```
oc delete sriovnetwork sriov-net-mgmt -n openshift-sriov-network-operator
```

```
[source,bash,role=execute]
```

```
oc delete sriovnetworknodepolicy mlx-sriov-policy -n openshift-sriov-network-operator
```

SR-IOV ユーザーガイド (VF ユーザーガイド):

```
[source,bash,role=execute]
```

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o
jsonpath='{.items[0].status.syncStatus}'
```

XXXXXXXXXXXXXXXXXXXX :

```
[source,bash,role=execute]
```

```
oc delete namespace sriov-demo
```

```
[discrete#networking:sriov-secondary::_000]
==== 000
```

=====

[illegible]

* SR-IOV □ PCI □□□□□□ VF □□ VM □□□□□□□□□□□□□□□□□□□□□□□□

* `SriovNetworkNodeState` 结构体 SR-IOV 网络节点状态

```
* VM 0000000000000000 'deviceType: vfio-pci' 00000000 'SriovNetworkNodePolicy' 00000000
```

```
* SR-IOV 口 VLAN 配置 (1 个 SR-IOV 口 VLAN 配置 `SriovNetwork` 配置 VF 口 VLAN 配置)
```

* 物理网口 (VLAN 100) 物理网口 (VLAN 200) 物理网口 'SriovNetwork' 物理网口

* VLAN SR-IOV VM

* 00000 PCI 000000 VLAN 00000000000000000000

```
[discrete#networking:sriov-secondary::_00]
==== 00
```

=====

link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html/n

```

networking/hardware-networks[OpenShift 仮想ネットワーク,window=_blank]
*
link:https://docs.redhat.com/en/documentation/openshift\_container\_platform/4.18/html/virtualization/networking#virt-attaching-vm-to-sriov-network[VM SR-IOV
仮想ネットワーク,window=_blank]
* link:https://kubevirt.io/user-guide/network/interfaces\_and\_networks/#sriov[KubeVirt
SR-IOV 仮想ネットワーク,window=_blank]
* xref:networking:localnet-secondary[Localnet 仮想ネットワーク] --
仮想ネットワークを二次的に接続する
* xref:networking:localnet-vlan[Localnet VLAN 仮想ネットワーク] -- localnet 仮想ネットワーク VLAN 仮想ネットワーク
(仮想ネットワーク)
* xref:networking:linux-bridges[Linux 仮想ネットワーク] -- Linux 仮想ネットワーク

```

:!navtitle:

```

:docname: vm-ingress-routes
:page-module: networking
:page-relative-src-path: vm-ingress-routes.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook\_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: VM Ingress 仮想ネットワーク
[#networking:vm-ingress-routes]
=== VM Ingress 仮想ネットワーク

```

[discrete#networking:vm-ingress-routes::_[仮想ネットワーク](#)]

==== [仮想ネットワーク](#)

[仮想ネットワーク](#) OpenShift Virtualization VM [仮想ネットワーク](#) OpenShift [仮想ネットワーク](#) (Routes) [仮想ネットワーク](#) (Services) [仮想ネットワーク](#) Web [仮想ネットワーク](#) VM [仮想ネットワーク](#) ClusterIP [仮想ネットワーク](#) TLS [仮想ネットワーク](#) OpenShift [仮想ネットワーク](#)

[仮想ネットワーク](#) OpenShift Ingress [仮想ネットワーク](#) VM [仮想ネットワーク](#) Pod

[discrete#networking:vm-ingress-routes::_[仮想ネットワーク](#)]

==== [仮想ネットワーク](#)

```

* **OpenShift 4.18+** (OpenShift Virtualization 仮想ネットワーク)
* **仮想ネットワーク** (`system:admin` 仮想ネットワーク)
* **`oc` CLI** 仮想ネットワーク **`virtctl` CLI** 仮想ネットワーク
* **仮想ネットワーク StorageClass** 仮想ネットワーク
* **Ingress 仮想ネットワーク** 仮想ネットワーク (OpenShift 仮想ネットワーク)

```

[仮想ネットワーク](#):

```
[discrete#networking:vm-ingress-routes::_0000_1_00000000]
==== 0000 1: 00000000

[source,bash,role=execute]
```

```
oc create namespace ingress-demo
```

```
[discrete#networking:vm-ingress-routes::_2_web_vm]
==== 2: Web VM

cloud-init Nginx Web Fedora VM VM
WARNING: cloud-init SSH
link:attachment$vm-ingress-routes/vm-webserver.yaml[vm-webserver.yaml
,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: web-vm
namespace: ingress-demo spec: runStrategy: Always template: metadata: labels:
kubevirt.io/domain: web-vm app: web-vm spec: domain: devices: disks: - name: rootdisk disk: bus:
virtio - name: cloudinitdisk disk: bus: virtio interfaces: - name: default masquerade: {} resources:
requests: memory: 1Gi networks: - name: default pod: {} volumes: - name: rootdisk containerDisk:
image: quay.io/containerdisks/fedora:latest - name: cloudinitdisk cloudInitNoCloud: userData: |
#cloud-config password: redhat chpasswd: expire: false packages: - nginx write_files: - path:
/usr/share/nginx/html/index.html content: | <html> <head><title>OpenShift Virt - Ingress
Demo</title></head> <body> <h1>Hello from OpenShift Virtualization</h1> <p>This page is served
by Nginx inside a VM, exposed via an OpenShift Route.</p> </body> </html> runcmd: - systemctl
enable nginx - systemctl start nginx EOF
```

```
'spec.template.metadata.labels' : 'app: web-vm' virt-launcher Pod
VM [source,bash,role=execute]
```

```
oc get vm -n ingress-demo -w
```

```
VM is `Running` and `Ready=True` .....`Ctrl+C` .....  
  
[discrete#networking:vm-ingress-routes::_0000_3_web_.....]  
==== 0000 3: Web .....
```

```
ccloud-init ... Nginx ..... (VM is Running ..... 102 .....)
```


NAME ENDPOINTS AGE web-vm-svc 10.128.x.x:80 5s

```
##### IP  VM  Pod IP ##### VM #####
```

```
#####
```

```
[source,bash,role=execute]
```

oc run curl-test --rm -it --restart=Never --image=registry.access.redhat.com/ubi9/ubi:latest -n ingress-demo — curl -s <http://web-vm-svc.ingress-demo.svc.cluster.local>

```
HTML #####
```

```
[discrete#networking:vm-ingress-routes::_5_openshift_#####]
```

```
==== 5: OpenShift #####
```

```
[discrete#networking:vm-ingress-routes::_http_#####]
```

```
===== HTTP  (#####)
```

```
 TLS #####OpenShift #####
```

```
link:attachment$vm-ingress-routes/route-web-vm.yaml[route-web-vm.yaml  
#####,role=unresolved]
```

```
[source,bash,role=execute]
```

oc apply -f - <<EOF apiVersion: route.openshift.io/v1 kind: Route metadata: name: web-vm-route namespace: ingress-demo spec: to: kind: Service name: web-vm-svc weight: 100 port: targetPort: 80 tls: termination: edge insecureEdgeTerminationPolicy: Redirect wildcardPolicy: None EOF

```
[discrete#networking:vm-ingress-routes::_#####]
```

```
=====
```

```
* **`spec.to.name`**: 4 #####
```

```
* **`spec.port.targetPort`**: (80) #####
```

```
* **`tls.termination: edge`**: TLS ##### VM ##### HTTP #####
```

```
* **`insecureEdgeTerminationPolicy: Redirect`**: HTTP ##### HTTPS #####
```

```
[discrete#networking:vm-ingress-routes::_#####]
```

```
=====
```

```
[source,bash,role=execute]
```

oc get route web-vm-route -n ingress-demo

```
#####:
```

| NAME | HOST/PORT | PATH | SERVICES | PORT | TERMINATION | WILDCARD |
|--------------|---|------------|----------|---------------|-------------|----------|
| web-vm-route | web-vm-route-ingress-demo.apps.ocplab01.example.com | web-vm-svc | 80 | edge/Redirect | None | |

```
[discrete#networking:vm-ingress-routes::_]
=====
```

1111111111111111

```
[source,bash,role=execute]
```

```
ROUTE_HOST=$(oc get route web-vm-route -n ingress-demo -o jsonpath='{.spec.host}') echo "Route URL: <a href="https://$ROUTE_HOST"" class="bare">https://$ROUTE_HOST"</a>
```

```
'curl'  []
```

```
[source,bash,role=execute]
```

```
curl -sk https://$ROUTE_HOST
```

VM 01 Nginx 000000 HTML 00000000000000000000

TIP: DNS 問題が解決しない場合は、Ingress IP を `/etc/hosts` に追加して `curl --resolve` を使ってください。

```
[discrete#networking:vm-ingress-routes::_0000_6_00000_tls_00000]
==== 0000 6: 00000 TLS (00000)
```

VM TLS VM

```
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: route.openshift.io/v1 kind: Route metadata: name: web-vm-
passthrough namespace: ingress-demo spec: to: kind: Service name: web-vm-svc weight: 100 port:
targetPort: 443 tls: termination: passthrough wildcardPolicy: None EOF
```

NOTE: `openssl` VM `openssl` `https` `nginx` `http`
`openssl` `tls` `nginx`

```
[discrete#networking:vm-ingress-routes::_000]
===== 000
```

OpenShift VM

- ```
. Kubernetes のデフォルトの Ingress Controller は Nginx である。
. OpenShift Ingress Controller (HAProxy ベース) が TLS をサポートする (OpenSSL)。
. OpenShift Ingress Controller ClusterIP サービスに 80 ポートでアクセスする。
. OpenShift Ingress Controller virt-launcher Pod に IP アドレス 80 を割り当てる。
```

```
. virt-launcher Pod
VM Nginx

VM OpenShift

[discrete#networking:vm-ingress-routes::_]
====

[discrete#networking:vm-ingress-routes::_503_service_unavailable_]
===== 503 Service Unavailable

[source,bash,role=execute]
```

oc get endpoints web-vm-svc -n ingress-demo

```
VM

[source,bash,role=execute]
```

oc get pods -n ingress-demo -l app=web-vm

```
[discrete#networking:vm-ingress-routes::_504_gateway_timeout_]
===== 504 Gateway Timeout

VM VM

[source,bash,role=execute]
```

virtctl console web-vm -n ingress-demo

```
VM Nginx

[source,bash]
```

systemctl status nginx curl -s <http://localhost>

```
[discrete#networking:vm-ingress-routes::_dns_]
===== DNS

Ingress DNS

[source,bash,role=execute]
```

oc get ingresscontroller default -n openshift-ingress-operator -o jsonpath='{.status.domain}'

```
'<route-name>-<namespace>.<ingress-domain>'
```

```
[discrete#networking:vm-ingress-routes::_]
====
```

```
#####
```

```
[source,bash,role=execute]
```

oc delete namespace ingress-demo

```
[discrete#networking:vm-ingress-routes::_]
====
```

```
#####
```

- \* cloud-init ##### Web ##### VM #####
- \* Kubernetes ##### VM #####
- \* ##### HTTPS ##### TLS ##### OpenShift #####
- \* ##### VM ##### TLS #####
- \* VM #####

```
[discrete#networking:vm-ingress-routes::_]
====
```

\* link:[https://docs.openshift.com/container-platform/4.18/virt/vm\\_networking/virt-exposing-vm-with-service.html](https://docs.openshift.com/container-platform/4.18/virt/vm_networking/virt-exposing-vm-with-service.html)[Exposing a VM by Using a Service - OpenShift  
#####,window=\_blank]

\*

link:[https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.18/html/ingress\\_and\\_load\\_balancing/index](https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html/ingress_and_load_balancing/index)[Route Configuration - OpenShift #####,window=\_blank]

```
:!navtitle:
```

```
:docname: ovs-bridge-troubleshooting
```

```
:page-module: networking
```

```
:page-relative-src-path: ovs-bridge-troubleshooting.adoc
```

```
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
```

```
:page-origin-start-path:
```

```
:page-origin-refname: main
```

```
:page-origin-reftype: branch
```

```
:page-origin-refhash: (worktree)
```

```
:navtitle: Localnet#####
```

```
[#networking:ovs-bridge-troubleshooting]
```

```
=== Localnet#####
```

```
#####:
```

```
#####OpenShift Virtualization#####Localnet#####OVS (Open vSwitch)
#####
```

```
[discrete#networking:ovs-bridge-troubleshooting::_ovs#####linux#####]
==== OVS#####Linux#####
```

```
:: OVS#####Linux#####
```

```
* **OVS**`ovs-vsctl`#####
* **OVS**`ip addr show` `ip link show`#####
* **#####**OVS#####Linux#####OVS#####
```

```
[discrete#networking:ovs-bridge-troubleshooting::_#####]
==== #####
```

```
[discrete#networking:ovs-bridge-
troubleshooting::_1_nodenetworkconfigurationpolicy#####]
===== 1: NodeNetworkConfigurationPolicy#####
```

```
#####NNCP#####
```

```
[source,bash,role=execute]
```

# □□□□NNCP□□□□□

oc get nncp

# NNCP

oc get nncp br-vlan-creation

```
**:
```

NAME STATUS REASON br-vlan-creation Available SuccessfullyConfigured

```
[discrete#networking:ovs-bridge-troubleshooting::_2_nncp]
```

```
===== 2: NNCP
```

```
NNCP
```

```
[source,bash,role=execute]
```

oc get nncp br-vlan-creation -o yaml

```
**:
```

```
[source,yaml]
```

status: conditions: - lastHeartbeatTime: "2025-10-06T20:38:57Z" lastTransitionTime: "2025-10-06T20:38:57Z" message: 1/1 nodes successfully configured reason: SuccessfullyConfigured status: "True" type: Available - reason: SuccessfullyConfigured status: "False" type: Degraded

```
* `Available: True` = `SuccessfullyConfigured` =
```

```
* `Degraded: False` =
```

```
* `message: X/X nodes successfully configured` =
```

```
[discrete#networking:ovs-bridge-troubleshooting::_3_ovs]
```

```
===== 3: OVS
```

```
**OVS:
```

```
[source,bash,role=execute]
```



oc get nodes



# ○○○○OVS○○○○○○○○○○

oc debug node/<node-name> — chroot /host ovs-vsctl list-br

```
○○○:
```

Starting pod/ocplab01-debug-k7bfc ... To use host binaries, run **chroot /host** br-ex br-int br-vlan

Removing debug pod ...

```
○○○○○ `br-vlan` ○○○○○○○○○○○○○○○○○○○○○○○○

[discrete#networking:ovs-bridge-troubleshooting::_○○○_4_○○○○○○○○○○○○]
===== ○○○ 4: ○○○○○○○○○○○

○○○OVS○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host ovs-vsctl show

```
○○○ (○○○):
```

Bridge br-vlan Port enp11s0f0np0 Interface enp11s0f0np0 type: system

```
○○○:

* ○○○○ (`br-vlan`) ○○○○○○○○
* ○○○○○○○○ ([: `enp11s0f0np0`) ○○○○○○○○○○○○○○○○
* ○○○○○○○○○○○○○○○○○○○ `system` ○○○○

[discrete#networking:ovs-bridge-troubleshooting::_○○○_5_○○○○○○○○○○○○○○]
===== ○○○ 5: ○○○○○○○○○○○○○○○○

○○○○○○○○○○○○○○○○○○○○UP○○○○○○○○○○○○○○

[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host ip link show <interface-name>

```
○○○:
```

4: enp11s0f0np0: <BROADCAST,MULTICAST,UP,LOWER\_UP> mtu 1500 qdisc mq master ovs-system  
state UP mode DEFAULT group default qlen 1000 link/ether e8:eb:d3:13:06:16 brd ff:ff:ff:ff:ff:ff

```
[]:
```

```
* `UP,LOWER_UP` - []
```

```
* `master ovs-system` - []OVS[]
```

```
* `state UP` - []
```

```
[discrete#networking:ovs-bridge-troubleshooting::_6_`ip_addr_show_br_vlan`]
==== [] 6: [] "ip addr show br-vlan" []
```

```
[]:
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host ip addr show br-vlan

```
[]:
```

Device "br-vlan" does not exist.

```
[]:
```

```
* OVS[]Linux[]OVS[]
```

```
* [] `ip` []
```

```
* []
```

```
* OVS[] `ovs-vsctl` []
```

```
[discrete#networking:ovs-bridge-troubleshooting::_]
==== []
```

```
[discrete#networking:ovs-bridge-troubleshooting::_1_ovs_vsctl_list_br`]
==== [] 1: ovs-vsctl list-br []
```

```
[]:
```

```
* NNCP[] `Available: True` [] `ovs-vsctl list-br` []
```

```
[]:
```

```
[source,bash,role=execute]
```

# NNCP□□□□□□

```
oc get nncp br-vlan-creation -o jsonpath='{.status.conditions[?(@.type=="Available")]}' | jq
```

# NMState

```
oc logs -n openshift-nmstate -l app=kubernetes-nmstate-operator
```

```
oc get nns -A
```

272

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

oc debug node/<node-name> — chroot /host ovs-vsctl list-ports br-vlan

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

oc debug node/<node-name> — chroot /host ip link show <interface-name>

# NNCP

oc get nncp br-vlan-creation -o yaml | grep -A 5 "name: <interface-name>"

\*\*:

- . NNCP
- .
- .

[discrete#networking:ovs-bridge-troubleshooting::\_3\_]  
==== 3:

\*\*:

\* NNCP

\*\*:

[source,bash,role=execute]





oc get nodes --show-labels

# NNCP

```
oc get nncp br-vlan-creation -o jsonpath='{.spec.nodeSelector}'
```

```
**:
```

```
* OpenShift (SNO) `control-plane,master,worker`
```

```
* `node-role.kubernetes.io/worker: ""` NNC
```

```
* NNC "X/X nodes successfully configured"
```

```
[discrete#networking:ovs-bridge-troubleshooting::_4_]
```

```
==== 4:
```

```
**:
```

```
*
```

```
**:
```

```
[source,bash,role=execute]
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

oc debug node/<node-name> — chroot /host cat /sys/class/net/<interface-name>/carrier

□ □ □ □ □ □ □ □ □ □ □ □ □ □

oc debug node/<node-name> — chroot /host ethtool -S <interface-name>

# 故障排除

oc debug node/<node-name> — chroot /host ethtool <interface-name> | grep "Link detected"

**\*\*配置:\*\***

- \* 配置
- \* 配置
- \* 配置/配置

[discrete#networking:ovs-bridge-troubleshooting::\_localnet配置]  
==== Localnet配置

OVS配置OVN-Kubernetes配置

[discrete#networking:ovs-bridge-troubleshooting::\_配置]  
===== 配置

配置\*\* (NetworkAttachmentDefinitions) \*\*OVS\*\*:

- \* 配置: `localnet-vlan`
- \* 配置: `br-vlan`
- \* 配置: `localnet-vlan:br-vlan`

[discrete#networking:ovs-bridge-troubleshooting::\_配置\_1\_配置nncp]  
===== 配置 1: 配置NNCP

配置NNCP:

[source,bash,role=execute]

# □□□□**NNCP**□□□□□

oc get nncp | grep mapping

□□□□□□□□□□NNCP□□□

```
oc get nncp localnet-vlan-bridge-mapping
```

**\*\*□□□□□□ : \*\***

localnet-vlan-bridge-mapping Available SuccessfullyConfigured

```
[discrete#networking:ovs-bridge-troubleshooting::_0000_2_0000000000000000]
```

===== 0000 2: 00000000000000000000

```
[source,bash,role=execute]
```

```
oc get nncp localnet-vlan-bridge-mapping -o yaml
```

**\*\*XXXXXXXXXXXXXXXXXXXXXXXXXXXX: \*\***

```
[source,yaml]
```

```
spec: desiredState: ovn: bridge-mappings: - bridge: br-vlan localnet: localnet-vlan state: present
nodeSelector: node-role.kubernetes.io/worker: "" status: conditions: - message: 1/1 nodes
successfully configured reason: SuccessfullyConfigured status: "True" type: Available
```

```
[discrete#networking:ovs-bridge-troubleshooting::_0000_3_00000ovn000000000000]
```

===== 0000 3: 00000OVN000000000000000

```

OVN-Kubernetes:
```

```
[source,bash,role=execute]
```

```
oc debug node/<node-name> — chroot /host ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings
```

\*\*\*:\*\*\*

```
"localnet-vlan:br-vlan,physnet:br-ex"
```

\*\*\*:\*\*\*

```
* `localnet-vlan:br-vlan` = 00000000
```

```
* `physnet:br-ex` = 00000000OpenShift00000000
```

\* □□□□□□□□□□□□□□□□□□□□□□□□

```
[source,bash,role=execute]
```

```
oc debug node/<node-name> --chroot /host ovs-vsctl list Open_vSwitch | grep -A 2 external_ids
```

\*\*\*

```
external_ids : {hostname=ocplab01, ovn-bridge-mappings="localnet-vlan:br-vlan,physnet:br-ex",
ovn-bridge-remote-probe-interval="0", ovn-enable-lflow-cache="true", ovn-encap-
ip="192.168.2.241", ovn-encap-type=geneve, ovn-is-interconn="true", ovn-memlimit-lflow-cache-
kb="1048576", ovn-monitor-all="true", ovn-ofctrl-wait-before-clear="0", ovn-
remote="unix:/var/run/ovn/ovnsb_db.sock", ovn-remote-probe-interval="180000", ovn-set-local-
ip="true", rundir="/var/run/openvswitch", system-id="1b41fc3f-f597-4cd9-b508-e8247c209d97"}
```

\*\*\*:

```
* 'ovn-bridge-mappings' []
```

```
* 'hostname' 0000000000
```

```
* 'ovn-encap-type=geneve' 00000000
```

===== 5: =====

```
[source,bash,role=execute]
```

```
oc debug node/<node-name> — chroot /host ovs-vsctl list-ports br-vlan
```

\*\*\*

enp11s0f0np0

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

=====

XXXXXXXXXXXXXXXXXXXXXXXXXXXX:

[illegible]



eth1 (物理ネットワークインターフェース) は、ネットワークカードに接続されている物理ポートを表します。このポートは、ネットワークに接続するための物理的なインターフェースです。

▼

NetworkAttachmentDefinition は、Kubernetes の NetworkAttachmentDefinition (NAD) の一種です。この NAD は、Kubernetes の Pod にネットワーク接続を提供するための設定を定義します。

NetworkAttachmentDefinition の - name: localnet-vlan-100 は、この NAD の名前を指定しています。- topology: localnet は、この NAD が localnet という topology を使用することを示しています。- vlanID: 100 は、この NAD が VLAN 100 を使用することを示しています。

▼

OVN-Kubernetes (Open Virtual Network - Kubernetes) は、Kubernetes のネットワークプラグインです。このプラグインは、Kubernetes の Pod にネットワーク接続を提供するための設定を定義します。

localnet-vlan → br-vlan

▼

OVS Bridge: br-vlan は、Open vSwitch (OVS) のブリッジです。このブリッジは、Kubernetes の Pod にネットワーク接続を提供するための設定を定義します。

VLAN 100 は、このブリッジの VLAN ID を指定しています。この VLAN ID は、Kubernetes の Pod にネットワーク接続を提供するための設定を定義します。

▼

Physical Interface: enp11s0f0np0 は、物理ネットワークインターフェースの名前です。このインターフェースは、Kubernetes の Pod にネットワーク接続を提供するための設定を定義します。

VLAN は、このインターフェースの VLAN ID を指定しています。この VLAN ID は、Kubernetes の Pod にネットワーク接続を提供するための設定を定義します。

```
[discrete#networking:ovs-bridge-troubleshooting::_0_0_0_0_0_0_0_0_0_0]
```

```
==== 0_0_0_0_0_0_0_0_0_0
```

```
[discrete#networking:ovs-bridge-troubleshooting::_0_1_0_0_0_0_0_0_0_0]
```

```
===== 0_1: 0_0_0_0_0_0_0_0_0_0
```

```
0_1:
```

```
* NNCP`ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings`
0_0_0_0_0_0_0_0_0_0
```

```
0_0_0_0_0_0_0_0:
```

```
[source,bash,role=execute]
```

# NNCP□□□□□□□□

oc get nncp localnet-vlan-bridge-mapping -o yaml

# NMState

```
oc delete pod -n openshift-nmstate -l app=kubernetes-nmstate-operator
```

# NMState

oc logs -n openshift-nmstate -l component=kubernetes-nmstate-handler

\*\* : \*\*

\* NNC  
\* NMState  
\* OVS

\*\* : \*\*

NNC

[discrete#networking:ovs-bridge-troubleshooting::\_2\_]  
==== 2:

\*\* : \*\*

\*

\*\* : \*\*

[source,bash,role=execute]

□ □ □ □ □ □ □ □ □ □

oc debug node/<node-name> — chroot /host ovs-vsctl list-br



□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc get nncp localnet-vlan-bridge-mapping -o jsonpath='{.spec.desiredState.ovn.bridge-mappings[0].localnet}'
```

# NetworkAttachmentDefinition ClusterUserDefinedNetwork (CUDN)

## NetworkAttachmentDefinition

oc get network-attachment-definitions <nad-name> -n <namespace> -o yaml

```
例:
```

```
NetworkAttachmentDefinition ClusterUserDefinedNetwork
NetworkAttachmentDefinition
```

```
[discrete#networking:ovs-bridge-troubleshooting::_4_]
===== 4:
```

```
例:
```

```
*
```

```
例:
```

```
[source,bash,role=execute]
```



# OC Troubleshooting

oc debug node/<node-name> — chroot /host ovs-vsctl get Open\_vSwitch . external\_ids:ovn-bridge-mappings

**\*\*root:\*\***

"mapping1:bridge1,mapping2:bridge2,mapping3:bridge3"

**\*\*root:\*\***

#####OVN#####

[discrete#networking:ovs-bridge-troubleshooting::\_#####]

==== #####

#####OVS#####:

[discrete#networking:ovs-bridge-troubleshooting::\_#####]

===== #####

\* [ ] NNCP##### `Available: True` 和 `SuccessfullyConfigured` #####

\* [ ] NNCP#####

\* [ ] `ovs-vsctl list-br` #####

\* [ ] `ovs-vsctl show` #####

\* [ ] ##### `UP,LOWER\_UP` #####

\* [ ] ##### `master ovs-system` #####

\* [ ] ##### (#####)

\* [ ] NMState#####

[discrete#networking:ovs-bridge-troubleshooting::\_#####]

===== #####

\* [ ] #####NNCP `Available: True` #####

\* [ ] `ovs-vsctl get Open\_vSwitch . external\_ids:ovn-bridge-mappings` #####

\* [ ] #####: `logical-name:bridge-name`

\* [ ] ##### NetworkAttachmentDefinition (NAD) / CUDN #####

\* [ ] #####OVS#####

\* [ ] ##### (`ovs-vsctl list-ports`)

\* [ ] #####

[discrete#networking:ovs-bridge-troubleshooting::\_#####]

==== #####

[discrete#networking:ovs-bridge-troubleshooting::\_ovs#####]

===== OVS#####

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host systemctl status openvswitch

```
[discrete#networking:ovs-bridge-troubleshooting::_ovs]
===== OVS
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host journalctl -u openvswitch -n 50

```
[discrete#networking:ovs-bridge-troubleshooting::_ovs]
===== OVS
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host ovs-vsctl list interface

```
[discrete#networking:ovs-bridge-troubleshooting::_ovs]
===== OVS
```

```
[source,bash,role=execute]
```

oc debug node/<node-name> — chroot /host ovs-vsctl list bridge

```
[discrete#networking:ovs-bridge-troubleshooting::_ovs]
===== OVS (
```

```
[source,bash,role=execute]
```



# □□□□**CUDN**□□□□□

oc get clusteruserdefinednetwork

# □□□ CUDN □□□□□

```
oc get clusteruserdefinednetwork <cuda-name> -o yaml
```

\*\*\*:\*\*\*

NAME AGE cudn-localnet-vlan-100 5m

```
[discrete#networking:ovs-bridge-troubleshooting::_0000_2_cudn00000]
```

===== 2: CUDN

CUDN spec

```
[source,bash,role=execute]
```

```
oc get clusteruserdefinednetwork cudn-localnet-vlan-100 -o yaml
```

\*\* : \*\*

```
[source,yaml]
```

```
spec: namespaceSelector: matchExpressions: - key: kubernetes.io/metadata.name operator: In
values: ["vm-guests-vlan-prod", "vm-guests-vlan-dev"] network: topology: Localnet localnet: role:
Secondary physicalNetworkName: localnet-vlan # 物理ネットワーク名 vlanID: 100 ipam: mode:
Disabled # IPAMモード
```

```
[discrete#networking:ovs-bridge-
```

```
troubleshooting::::_3_00000000networkattachmentdefinition000]
```

```
==== 3: NetworkAttachmentDefinition
```

CUDNNetworkAttachmentDefinition:

```
[source,bash,role=execute]
```

# □□□□□□□□**NAD**□□□□□

oc get network-attachment-definitions -A | grep <udn-name>



```
oc get network-attachment-definitions -n <namespace>
```

**\*\*[REDACTED]:\*\***

| NAMESPACE | NAME              | AGE | vm-guests-vlan-dev | cudn-localnet-vlan-100 | 5m | vm-guests-vlan-prod | cudn- |
|-----------|-------------------|-----|--------------------|------------------------|----|---------------------|-------|
|           | localnet-vlan-100 | 5m  |                    |                        |    |                     |       |

```
[discrete#networking:ovs-bridge-troubleshooting::_0000_4_physicalNetworkName_]==== 0000 4: physicalNetworkName CUDN `physicalNetworkName` [source,bash,role=execute]
```

# CUDN物理NetworkName

oc                      get                      clusteruserdefinednetwork                      < cudn-name>                      -o  
jsonpath='{.spec.network.localnet.physicalNetworkName}'



□ □ □ □ □ □ □ □ □ □ □ □ □ □ □

oc get nncp <bridge-mapping-nncp> -o jsonpath='{.spec.desiredState.ovn.bridge-mappings[0].localnet}'

# OVN

oc debug node/<node-name> — chroot /host ovs-vsctl get Open\_vSwitch . external\_ids:ovn-bridge-mappings

```
** (id: 'localnet-vlan') **
```

```
[discrete#networking:ovs-bridge-troubleshooting::_5_]
```

```
==== 5:
```

```
VM CUDN:
```

```
[source,bash,role=execute]
```

# VM

```
oc get vmi -n vm-guests-vlan-prod -o jsonpath='{.items[*].status.interfaces[?(@.name=="cudn_vlan_network")].ipAddress}'
```

## 2 VM

```
oc get vmi -n vm-guests-vlan-dev -o jsonpath='{.items[*].status.interfaces[?(@.name=="cudn_vlan_network")].ipAddress}'
```

VM CUDN IP

```
[discrete#networking:ovs-bridge-troubleshooting::_cudn]
==== CUDN
```

```
. **NAD**:
* ***: CUDN`oc get network-attachment-definitions -n <namespace>`
* ***:
* ***: CUDN
+
[source,bash,role=execute]
```

□ □ □ □ □ □ □ □ □

```
oc get namespace <namespace-name> -o jsonpath='{.metadata.labels}'
```

# CUDN

```
oc get clusteruserdefinednetwork <cu-dn-name> -o jsonpath='{.spec.namespaceSelector}'
```

```
. **VM "failed bridge mapping validation" **:
* **: CUDN VM
* **: CUDN `physicalNetworkName`
* **: ()
+
[source,bash,role=execute]
```

□□□□□

```
oc get clusteruserdefinednetwork <cluserdef-net> -o
jsonpath='{.spec.network.localnet.physicalNetworkName}' oc get nncp <bridge-mapping-nncp> -o
jsonpath='{.spec.desiredState.ovn.bridge-mappings[0].localnet}'
```

```
. **IPAM**:
 * **VM**: VMIP
 * ****: IPAMcloud-init
 * **cloud-initIP**: CUDN `ipam.mode: Disabled`
 * **OVN IPAM**: `staticIPAMConfig` `ipam.mode: Static`
 * **:
+
[source,bash,role=execute]
```

# IPAM□□□□□□

```
oc get clusteruserdefinednetwork <cudn-name> -o jsonpath='{.spec.network.localnet.ipam.mode}'
```



# VM cloud-init

virtctl console -n <namespace> <vm-name> # VM: cat /var/log/cloud-init.log

```
[discrete#networking:ovs-bridge-troubleshooting::_localnet_vlan]
==== Localnet VLAN

[discrete#networking:ovs-bridge-troubleshooting::_]
=====

. **VLANVM**:
 * VLAN
 *
 * VLAN ID
 * NICUP
 * OVS

. **IP**:
 * DHCPVLAN

. **:
 * VLAN
 * VLAN
 *

. **:
 * : OVS `ip addr show` `ovs-vsctl list-br`
 *
 * NMState

. **:
 *
 * OVN: `ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings`
 * localnetNAD

. **VM "failed bridge mapping validation"**:
 * **: VM: `failed to find OVN bridge-mapping for network: "localnet-vlan-100"`
 * **: NetworkAttachmentDefinition `physicalNetworkName`
 * **: NAD `physicalNetworkName: "localnet-vlan"`
 * **: NAD `oc delete` `oc apply`

[discrete#networking:ovs-bridge-troubleshooting::_]
=====

[source,bash,role=execute]
```

# NetworkAttachmentDefinition□□□□

□□□□□

oc get network-attachment-definitions -n vm-guests-vlan

# NodeNetworkConfigurationPolicies

oc get nncp

□□□□□□□□□□□□□□ (□□□□□□□□□□□□)

oc get nncp br-vlan-creation -o yaml

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

oc get nncp localnet-vlan-bridge-mapping -o yaml



```
oc debug node/<worker-node-name> — chroot /host ip link show ens224
```

# OVS□□□□□□□□

oc debug node/<worker-node-name> — chroot /host ovs-vsctl show

# OVN

```
oc debug node/<worker-node-name> — chroot /host ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings
```



□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

oc debug node/<worker-node-name> — chroot /host ovs-vsctl list-ports br-vlan

# VM

```
oc get vmi -n vm-guests-vlan -o yaml | grep -A 10 networks
```

# OpenShift OVN-Kubernetes

```
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node
```

# NetworkManagerState

oc logs -n openshift-nmstate -l app=kubernetes-nmstate-operator

```
[discrete#networking:ovs-bridge-troubleshooting::_]
====

:

* OVSLinux `ip addr show`
* `ovs-vsctl` OVS
* OVS OVN
*
* ClusterUserDefinedNetwork (CUDN) NAD

[discrete#networking:ovs-bridge-troubleshooting::_]
====

* link:https://nmstate.io/[NMState Documentation,window=_blank]
* link:http://www.openvswitch.org/support/dist-docs/[Open vSwitch
Documentation,window=_blank]
*
link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/n
etworking_operators/k8s-nmstate-about-the-k8s-nmstate-operator[OpenShift NMState
Operator,window=_blank]

:!navtitle:

:docname: index
:page-module: vm-configuration
:page-relative-src-path: index.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle:
[#vm-configuration:index]
==

[discrete#vm-configuration:index::_]
====

OpenShift
VirtualizationDNSVM

[discrete#vm-configuration:index::_]
```



```

* xref:getting-started:index[] -
* xref:storage:index[] - VM
* xref:networking:index[] -
* xref:performance:index[] - CPUNUMAHugepagesVM
* xref:vm-lifecycle:index[VM] -

[discrete#vm-configuration:index::_]
===

*
link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/virtualization/about#about-virt[OpenShift Virtualization ,window=_blank]
* link:https://cloudinit.readthedocs.io/[Cloud-init ,window=_blank]
* link:https://kubevirt.io/user-guide/[KubeVirt ,window=_blank]

:!navtitle:

:docname: internal-dns-for-vms
:page-module: vm-configuration
:page-relative-src-path: internal-dns-for-vms.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: VMDNS
[#vm-configuration:internal-dns-for-vms]
=== VMDNS

[discrete#vm-configuration:internal-dns-for-vms::_]
====

OpenShift VirtualizationDNSKubernetes
ServiceVMDNSIPPodVMVMVM

[discrete#vm-configuration:internal-dns-for-vms::_]
====

* Kubernetes ServiceVM
* VMDNS
* OpenShift VirtualizationVMDNS

[discrete#vm-configuration:internal-dns-for-vms::_]
====

* OpenShift VirtualizationOpenShift 4.18
* CLI: `oc` `virtctl`
* Kubernetes ServiceDNS

[discrete#vm-configuration:internal-dns-for-vms::_1_]

```

```

===== pod1: podtemplate

[discrete#vm-configuration:internal-dns-for-vms::_pod1_1_podtemplate]
===== pod 1: podtemplate

podtemplatetemplate

[source,bash,role=execute]

```

oc new-project myapp

```

[discrete#vm-configuration:internal-dns-for-vms::_pod1_2_fedora_vmtemplate]
===== pod 2: Fedora VMtemplate

Fedora VMtemplate

WARNING: podcloud-inittemplateSSHtemplate

link:attachment$internal-dns-for-vms/fedora-app-vm.yaml[fedora-app-vm.yaml
template,role=unresolved]

[source,bash,role=execute]

```

```

oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-app-vm
namespace: myapp labels: app: myapp spec: runStrategy: Always dataVolumeTemplates: -
metadata: name: fedora-app-vm-volume spec: sourceRef: kind: DataSource name: fedora
namespace: openshift-virtualization-os-images storage: resources: requests: storage: 30Gi template:
metadata: labels: kubevirt.io/domain: fedora-app-vm app: myapp spec: domain: devices: disks: -
disk: bus: virtio name: rootdisk - disk: bus: virtio name: cloudinitdisk interfaces: - name: default
masquerade: {} resources: requests: memory: 2Gi networks: - name: default pod: {} volumes: -
dataVolume: name: fedora-app-vm-volume name: rootdisk - cloudInitNoCloud: userData: |- #cloud-
config user: fedora password: fedora chpasswd: { expire: false } ssh_pwauth: true name:
cloudinitdisk EOF

```

```

VMtemplate

[source,bash]

```

watch 'oc get vm,vmi,dv -n myapp'

```

VM `Running` pod `Ready: True` template `Ctrl+C`

[discrete#vm-configuration:internal-dns-for-vms::_pod1_3_vmService]
===== pod 3: VMService

VMServiceService

```

```
link:attachment$internal-dns-for-vm/headless-service.yaml[headless-service.yaml
,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: Service metadata: name: myapp-service namespace: myapp
spec: clusterIP: None selector: kubevirt.io/domain: fedora-app-vm ports: - name: ssh protocol: TCP
port: 22 targetPort: 22 EOF
```

```
ServiceEndpointSlice

[source,bash,role=execute]
```

```
oc get svc myapp-service -n myapp oc get endpointslice -n myapp -l kubernetes.io/service-
name=myapp-service
```

```

:

Service `TYPE: ClusterIP` `CLUSTER-IP: None` ()

EndpointSliceENDPOINTSVMPod IP

[discrete#vm-configuration:internal-dns-for-vm::_4_dns]
===== 4: DNS

DNSPod

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: Pod metadata: name: debug-dns namespace: myapp spec:
containers: - name: debug image: registry.access.redhat.com/ubi9/ubi command: ["sleep", "3600"]
securityContext: allowPrivilegeEscalation: false runAsNonRoot: true runAsUser: 1000 capabilities:
drop: - ALL seccompProfile: type: RuntimeDefault restartPolicy: Never EOF
```



# Pod□□□□□□□□□□

oc wait --for=condition=Ready pod/debug-dns -n myapp --timeout=60s

# DNS□□□□□□

oc exec -n myapp debug-dns — getent hosts myapp-service.myapp.svc.cluster.local



**\*\*Step 4: ServiceVM Pod IP\*\***

Service`clusterIP: None`CoreDNSEndpointSliceVM Pod IPClusterIP  
ServiceServiceIP

**\*\*Step 5: \*\***

IPVM

[discrete#vm-configuration:internal-dns-for-vms::\_]

====

\* **\*\*DNSService\*\***: CoreDNSKubernetes ServiceDNS

\* **\*\*VM DNS\*\***: VM `metadata.name` KubernetesDNS

\* **\*\*ServiceDNS\*\***: Service `metadata.name` `

\* **\*\*VM IP\*\***: Service`clusterIP: None`VM Pod IPVM

\* **\*\*ClusterIP ServiceIPVM\*\***: ClusterIP ServiceIPVM

\* **\*\*`/etc/resolv.conf` `search`**  
ServiceFQDN `myapp-service`

[discrete#vm-configuration:internal-dns-for-vms::\_3]

==== 3:

[discrete#vm-configuration:internal-dns-for-vms::\_]

====

**\*\*ServiceEndpointSlice\*\***

[source,bash,role=execute]

# Service□□□□□

```
oc get svc myapp-service -n myapp
```

# EndpointSlice (VM Pod IP)

```
oc get endpointslice -n myapp -l kubernetes.io/service-name=myapp-service
```

# Service□□□□□□□

oc describe svc myapp-service -n myapp

**\*\*DNS□□□□□□□□□□:\*\***

[source,bash,role=execute]

# Pod

```
oc apply -f - <<EOF apiVersion: v1 kind: Pod metadata: name: debug-dns namespace: myapp spec:
containers: - name: debug image: registry.access.redhat.com/ubi9/ubi command: ["sleep", "3600"]
securityContext: allowPrivilegeEscalation: false runAsNonRoot: true runAsUser: 1000 capabilities:
drop: - ALL seccompProfile: type: RuntimeDefault restartPolicy: Never EOF
```



# Pod

```
oc wait --for=condition=Ready pod/debug-dns -n myapp --timeout=60s
```

# DNS□□□□□□

oc exec -n myapp debug-dns — getent hosts myapp-service.myapp.svc.cluster.local

□□□□□□□□□□

oc delete pod debug-dns -n myapp

```
VM□□□□□□□□□□:

[source,bash,role=execute]
```

# VM Service

```
oc get vm fedora-app-vm -n myapp -o jsonpath='{.metadata.labels}' oc get svc myapp-service -n myapp -o jsonpath='{.spec.selector}'
```

```
[discrete#vm-configuration:internal-dns-for-vms::_]
```

```
=====
```

```
** : DNS **
```

```
** : **
```

```
[source,bash,role=execute]
```

```
oc get svc myapp-service -n myapp oc get endpointslice -n myapp -l kubernetes.io/service-name=myapp-service
```

```
** : **
```

```
* Service VM
```

```
* VM EndpointSlice
```

```
*
```

```
** : **
```

```
1. Service VM `kubevirt.io/domain`
```

```
2. VM: `oc get vmi -n myapp`
```

```
3. Service VM
```

```
** : DNS **
```

```
** : **
```

```
[source,bash,role=execute]
```

# Pod

```
oc new-project test-namespace 2>/dev/null || true
```

```
oc apply -f - <<EOF apiVersion: v1 kind: Pod metadata: name: debug-cross-ns namespace: test-namespace spec: containers: - name: debug image: registry.access.redhat.com/ubi9/ubi command: ["sleep", "300"] securityContext: allowPrivilegeEscalation: false runAsNonRoot: true runAsUser: 1000 capabilities: drop: - ALL seccompProfile: type: RuntimeDefault restartPolicy: Never EOF
```

□□□□□□□

```
oc wait --for=condition=Ready pod/debug-cross-ns -n test-namespace --timeout=60s oc exec -n test-namespace debug-cross-ns -- \ getent hosts myapp-service.myapp.svc.cluster.local
```



oc delete pod debug-cross-ns -n test-namespace

**\*\*概要:\*\***

- \* FQDN
- \*

**\*\*:\*\***

1. FQDN: ``<service>.<namespace>.svc.cluster.local``
2. NetworkPolicy

[discrete#vm-configuration:internal-dns-for-vms::\_]  
=====

\* link:<https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>[Kubernetes DNS for Services and Pods,window=\_blank]  
\*

link:[https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/virtualization/networking#virt-accessing-vm-internal-fqdn](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/virtualization/networking#virt-accessing-vm-internal-fqdn)[FQDN VM - OpenShift Virtualization,window=\_blank]  
\*

link:[https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/virtualization/networking#virt-accessing-vm-secondary-network-fqdn](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/virtualization/networking#virt-accessing-vm-secondary-network-fqdn)[FQDN VM - OpenShift Virtualization,window=\_blank]

[discrete#vm-configuration:internal-dns-for-vms::\_]  
=====

- \* DNS Kubernetes Service VM
- \* CoreDNS Service VM Pod IP
- \* VM IP Cluster IP IP
- \* FQDN `getent hosts` Pod DNS
- \* DNS Service

[discrete#vm-configuration:internal-dns-for-vms::\_]  
=====

- \* xref:networking:static-ip-configuration[IP]
- \* xref:vm-configuration:remote-access-linux-vms[Linux VM]

:!navtitle:

:docname: remote-access-linux-vms  
:page-module: vm-configuration

```

:page-relative-src-path: remote-access-linux-vms.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Linux
[#vm-configuration:remote-access-linux-vms]
=== Linux

[discrete#vm-configuration:remote-access-linux-vms::_]
===

OpenShift
VirtualizationLinuxVM

```

| cols="1,2,1" |
|--------------|
| SSH          |
| VNC          |
| GUI          |
| virtctl SSH  |
| NodePort     |
|              |
|              |
|              |
|              |
|              |

```

[discrete#vm-configuration:remote-access-linux-vms::_]
===

```



```
* OpenShift VirtualizationOpenShift 4.18
* CLI: 'oc' 'virtctl' 'kubectl'
* Linux VM1
* SSH

[discrete#vm-configuration:remote-access-linux-vms::_vm]
=== : VM

Fedora VM

[source,bash,role=execute]
```

oc create namespace vm-access-demo

```
SSHVM

WARNING: cloud-initSSH

link:attachment$remote-access-linux-vms/fedora-access-vm.yaml[fedora-access-vm.yaml
,role=unresolved]

[source,yaml]
```

```
apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-access-vm namespace: vm-
access-demo spec: dataVolumeTemplates: - apiVersion: cdi.kubevirt.io/v1beta1 kind: DataVolume
metadata: name: fedora-access-vm-disk spec: sourceRef: kind: DataSource name: fedora
namespace: openshift-virtualization-os-images storage: resources: requests: storage: 30Gi
runStrategy: RerunOnFailure template: metadata: labels: kubevirt.io/domain: fedora-access-vm
spec: domain: cpu: cores: 2 devices: disks: - disk: bus: virtio name: rootdisk - disk: bus: virtio name:
cloudinitdisk interfaces: - masquerade: {} name: default rng: {} machine: type: pc-q35-rhel9.4.0
memory: guest: 4Gi networks: - name: default pod: {} volumes: - dataVolume: name: fedora-access-
vm-disk name: rootdisk - cloudInitNoCloud: userData: |- #cloud-config user: fedora password:
fedora123 chpasswd: { expire: false } ssh_pwauth: true ssh_authorized_keys: - ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQ... your-public-key-here name: cloudinitdisk
```

```
: 'your-public-key-here' SSH

VM

[source,bash,role=execute]
```

oc apply -f fedora-access-vm.yaml

```
VM
```

```
[source,bash,role=execute]
```

```
oc get vmi -n vm-access-demo
```

```
[discrete#vm-configuration:remote-access-linux-vms::_001_virtctl]
==== 001: virtctl

VM

[discrete#vm-configuration:remote-access-linux-vms::_]
=====

[source,bash,role=execute]
```

```
virtctl console fedora-access-vm -n vm-access-demo
```

```
[discrete#vm-configuration:remote-access-linux-vms::_]
=====

`Ctrl+]`

[discrete#vm-configuration:remote-access-linux-vms::_]
=====

* VM
*
* VM SSH
*

[discrete#vm-configuration:remote-access-linux-vms::_]
=====

* /
*
*
* 1

[discrete#vm-configuration:remote-access-linux-vms::_002_vnc]
==== 002: VNC

VM

[discrete#vm-configuration:remote-access-linux-vms::_vnc]
===== VNC

[source,bash,role=execute]
```

```
virtctl vnc fedora-access-vm -n vm-access-demo
```

#####VM#####VNC#####

```
[discrete#vm-configuration:remote-access-linux-vm:::_00000000_2]
===== 00000000
```

- \* #####Linux VM#####
- \* GUI#####
- \* GUI#####
- \* #####

```
[discrete#vm-configuration:remote-access-linux-vm:::_0000_2]
===== 0000
```

- \* #####VNC#####
- \* #####GNOMEKDE#####VM

```
[discrete#vm-configuration:remote-access-linux-vm:::_003_virtctlssh]
===== 003: virtctlssh
```

`virtctl` #####SSH#####VMSSH#####

```
[discrete#vm-configuration:remote-access-linux-vm:::_0000_3]
===== 0000
```

- \* VMSSH#####cloud-init
- \* VMSSH#####

```
[discrete#vm-configuration:remote-access-linux-vm:::_ssh]
===== SSH
```

```
[source,bash,role=execute]
```

virtctl ssh fedora@fedora-access-vm -n vm-access-demo

```
[discrete#vm-configuration:remote-access-linux-vm:::_0000000000000000]
===== 0000000000000000
```

```
[source,bash,role=execute]
```

virtctl ssh fedora@fedora-access-vm -n vm-access-demo --local-ssh-opts="-o StrictHostKeyChecking=no"

```
[discrete#vm-configuration:remote-access-linux-vm:::_00000000_3]
===== 00000000
```

- \* #####
- \* #####
- \* #####

```
* 'oc' 'kubectl'
[discrete#vm-configuration:remote-access-linux-vms::_0000_2]
=====
*
*
[discrete#vm-configuration:remote-access-linux-vms::_004_nodeportssh]
===== 004: NodePortSSH
NodePortSSH
[discrete#vm-configuration:remote-access-linux-vms::_nodeport]
===== NodePort
link:attachment$remote-access-linux-vms/ssh-nodeport-service.yaml[ssh-nodeport-
service.yaml ,role=unresolved]
[source,yaml]
```

```
[source,bash,role=execute]
```

```
[discrete#vm-configuration:remote-access-linux-vm:::_]
=====
[source,bash,role=execute]
```

```
[discrete#vm-configuration:remote-access-linux-vm::_ssh_2]
===== SSH

IP

[source,bash,role=execute]
```

```
NodePort
```

```
[source,bash,role=execute]
```

```
ssh fedora@<node-ip> -p 30022
```

```
[discrete#vm-configuration:remote-access-linux-vms::_4]
```

```
=====
```

- \*
- \*
- \* /
- \*

```
[discrete#vm-configuration:remote-access-linux-vms::_]
```

```
=====
```

- \* SSH
- \* SSH
- \*
- \*
- \*

```
[discrete#vm-configuration:remote-access-linux-vms::_5_ssh]
```

```
==== 5: SSH
```

```
`kubectl` `oc port-forward` SSH
```

```
[discrete#vm-configuration:remote-access-linux-vms::_vm_pod]
```

```
===== VM Pod
```

```
[source,bash,role=execute]
```

```
oc get pods -n vm-access-demo
```

```
:
```

```
[source,text]
```

```
NAME READY STATUS RESTARTS AGE virt-launcher-fedora-access-vm-xxxxx 2/2 Running 0 10m
```

```
[discrete#vm-configuration:remote-access-linux-vms::_ssh]
```

```
===== SSH
```

```
[source,bash,role=execute]
```

```
oc port-forward -n vm-access-demo virt-launcher-fedora-access-vm-xxxxx 2222:22
```

```
[discrete#vm-configuration:remote-access-linux-vms::_ssh]
===== SSH

[source,bash,role=execute]
```

```
ssh fedora@localhost -p 2222
```

```
[discrete#vm-configuration:remote-access-linux-vms::_5]
=====

*
*
* VM
* NodePort

[discrete#vm-configuration:remote-access-linux-vms::_3]
=====

*
* 1
*
*

[discrete#vm-configuration:remote-access-linux-vms::_6_6]
===== 6:

VMIP

[discrete#vm-configuration:remote-access-linux-vms::_4]
=====

* VMVLAN
*
* IPDHCP

[discrete#vm-configuration:remote-access-linux-vms::_vm]
===== VM

link:attachment$remote-access-linux-vms/fedora-vlan-access-vm.yaml[fedora-vlan-access-vm.yaml,role=unresolved]

[source,yaml]
```

```
apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: fedora-vlan-vm namespace: vm-
access-demo spec: dataVolumeTemplates: - apiVersion: cdi.kubevirt.io/v1beta1 kind: DataVolume
metadata: name: fedora-vlan-vm-disk spec: sourceRef: kind: DataSource name: fedora namespace:
```

```

openshift-virtualization-os-images storage: resources: requests: storage: 30Gi runStrategy:
RerunOnFailure template: metadata: labels: kubevirt.io/domain: fedora-vlan-vm spec: domain: cpu:
cores: 2 devices: disks: - disk: bus: virtio name: rootdisk - disk: bus: virtio name: cloudinitdisk
interfaces: - masquerade: {} name: default - bridge: {} name: vlan-network rng: {} machine: type:
pc-q35-rhel9.4.0 memory: guest: 4Gi networks: - name: default pod: {} - name: vlan-network multus:
networkName: localnet-vlan-100 volumes: - dataVolume: name: fedora-vlan-vm-disk name: rootdisk
- cloudInitNoCloud: userData: |- #cloud-config user: fedora password: fedora123 chpasswd: {
expire: false } ssh_pwauth: true ssh_authorized_keys: - ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQBAQC... your-public-key-here networkData: |- version: 2
ethernets: enp1s0: dhcp4: true enp2s0: dhcp4: false dhcp6: false addresses: - 192.168.100.100/24
name: cloudinitdisk

```

```

[discrete#vm-configuration:remote-access-linux-vms::_ssh_3]
===== SSH

[source,bash,role=execute]

```

ssh fedora@192.168.100.100

```

[discrete#vm-configuration:remote-access-linux-vms::_6]
=====

*
*
* VM
*
* VM

[discrete#vm-configuration:remote-access-linux-vms::_]
=====

*
*
*
*
*

[discrete#vm-configuration:remote-access-linux-vms::_ssh]
===== SSH

[discrete#vm-configuration:remote-access-linux-vms::_ssh]
===== SSH

[source,bash,role=execute]

```

ssh-keygen -t rsa -b 4096 -f ~/.ssh/ocp-virt-key

```
[discrete#vm-configuration:remote-access-linux-vm:::_cloud_init]
===== Cloud-init

VM `userData`

[source,yaml]
```

ssh\_authorized\_keys: - ssh-rsa AAAAB3NzaC... your-key-here

```
[discrete#vm-configuration:remote-access-linux-vm:::_]
=====

root cloud-init `userData`

[source,yaml]
```

volumes: - cloudInitNoCloud: userData: |- #cloud-config user: fedora ssh\_pwauth: false  
disable\_root: true ssh\_authorized\_keys: - ssh-rsa AAAAB3NzaC... your-key-here name: cloudinitdisk

```
:

* `ssh_pwauth: false` - SSH
* `disable_root: true` - SSH root
* SSH

[discrete#vm-configuration:remote-access-linux-vm:::_]
=====

[discrete#vm-configuration:remote-access-linux-vm:::_vm]
===== VM

[source,bash,role=execute]
```



# VirtualMachine

```
oc get vm -n vm-access-demo
```

# VirtualMachineInstance

`oc get vmi -n vm-access-demo`

# VM□□□□□

oc describe vmi fedora-access-vm -n vm-access-demo

```
[discrete#vm-configuration:remote-access-linux-vm:::_vm□ssh□□□□□□]
```

```
===== VM□SSH□□□□□□□□
```

```
□□□□□□□□ :
```

```
[source,bash,role=execute]
```

virtctl console fedora-access-vm -n vm-access-demo

```
VM□□□□ :
```

```
[source,bash]
```

# SSH 接続確認

```
sudo systemctl status sshd
```

# SSH 接続確認

```
sudo ss -tlnp | grep :22
```

□□□□□□□□□□

sudo firewall-cmd --list-all

```
[discrete#vm-configuration:remote-access-linux-vm:::_□□□□□□□□□□]
```

```
===== □□□□□□□□□□
```

```
[source,bash]
```

# VM□□□□

```
ping -c 3 8.8.8.8
```



```
ping -c 3 192.168.100.100
```

```
[discrete#vm-configuration:remote-access-linux-vms::_]
====
xref:vm-configuration:remote-access-linux-troubleshooting[]

[discrete#vm-configuration:remote-access-linux-vms::_virtctl]
==== : virtctl

****: VM

****: VMVM

[discrete#vm-configuration:remote-access-linux-vms::_ssh]
==== : SSH

****: SSH

***:

* sshd: `sudo systemctl status sshd`
* : `sudo systemctl start sshd`
* : `sudo firewall-cmd --list-services`

[discrete#vm-configuration:remote-access-linux-vms::_virtctl_ssh]
==== : virtctl ssh

****: SSH

***:

* VMSSH
* SSH
* VM
* SSH

[discrete#vm-configuration:remote-access-linux-vms::_nodeport]
==== : NodePort

***:

***: NodePort

[discrete#vm-configuration:remote-access-linux-vms::_ip]
==== : IP
```



**\*\*注意\*\*:** 配置前请检查网络配置

**\*\*注意\*\*:** 配置前请检查网络配置VLAN/子网配置是否正确

[discrete#vm-configuration:remote-access-linux-vm:::\_init]

==== 初始化

[discrete#vm-configuration:remote-access-linux-vm:::\_init]

===== 配置

\* SSH配置

\* 配置SSH

\* 配置SSH

\* 配置SSH

[discrete#vm-configuration:remote-access-linux-vm:::\_init]

===== 配置

\* 配置NodePort和IP

\* 配置VM

\* VM配置

\* 配置

[discrete#vm-configuration:remote-access-linux-vm:::\_init]

===== 配置

\* virtctl RBAC配置

\* 配置

\* VM配置

\* 配置

[discrete#vm-configuration:remote-access-linux-vm:::\_init]

===== 配置

配置完成

[source,bash,role=execute]

# VM□□□

oc delete vm fedora-access-vm fedora-vlan-vm -n vm-access-demo

□ □ □ □ □ □ □

```
oc delete svc fedora-ssh-nodeport -n vm-access-demo
```



```
oc delete namespace vm-access-demo
```

```
[discrete#vm-configuration:remote-access-linux-vm:::_]
```

== == == ==      |||

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
* OpenShift Virtualization Linux VM
```

\* □□□□□□□□□□

```
* cloud-init[SSH]
```

\* VM

\*

\* 仮想マシン

\* VM

\* □□□□□□□□□□□□□□□□□□□□

```
[discrete#vm-configuration:remote-access-linux-vm::_[]
```

== 00

\*

link:[https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.19/html/virtualization/managing-vms](https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/virtualization/managing-vms)[Managing VMs,window=\_blank]

```
:!navtitle:
```

```
:docname: remote-access-linux-troubleshooting
```

```
:page-module: vm-configuration
```

```
:page-relative-src-path: remote-access-linux-troubleshooting.adoc
```

```
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
```

```
:page-origin-start-path:
```

```
:page-origin-refname: main
```

```
:page-origin-reftype: branch
```

```
:page-origin-refhash: (worktree)
```

```
:navtitle: Linux VM □□□□□□□□□□□□□□□□
```

```
[#vm-configuration:remote-access-linux-troubleshooting]
```

### === Linux VM [][][][]

# OpenShift Virtualization Linux VM

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_[]
```

==

- \* <<Quick Health Checks>>

```
* <<Method 1: Console Access>>
```

\* <<Method 2: VNC Access>>

- \* <<Method 3: SSH via virtctl>>

\* <<Method 4: SSH via NodePort Service>>

```
* <<Method 5: SSH via LoadBalancer>>
```

```
* <<Method 6: SSH via Port Forward>>
* <<Method 7: Direct Access via Secondary Network>>
* <<Diagnostic Commands>>
```

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_]
====
```

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_vm_]
===== VM
[source,bash,role=execute]
```

```
oc get vm <vm-name> -n <namespace> oc get vmi <vm-name> -n <namespace>
```

```
VM :
[source,text]
```

```
NAME AGE STATUS READY my-vm 5m Running True
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_vm_ip_]
===== VM IP
[source,bash,role=execute]
```

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.interfaces[*].ipAddress}'
```

```
1 IP
```

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_vm_pod_]
===== VM Pod
[source,bash,role=execute]
```

```
oc get pods -n <namespace> -l kubevirt.io/domain=<vm-name>
```

```
:
[source,text]
```

```
NAME READY STATUS RESTARTS AGE virt-launcher-<vm>-xxxxx 3/3 Running 0 5m
```

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_]
=====
[source,bash,role=execute]
```

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.guestOSInfo.id}'
```

```

#####:
#####VNC#####SSH#####NodePort#####LoadBalancer##### OS
SSH

[discrete#vm-configuration:remote-access-linux-troubleshooting::_1_]
==== 1: #####

[discrete#vm-configuration:remote-access-linux-troubleshooting::__]
===== : #####

::
[source,bash]

```

\$ virtctl console my-vm -n my-namespace Error: unable to connect to console

```

::
[source,bash,role=execute]

```

# Check VM is running

```
oc get vm my-vm -n my-namespace
```

# Check VMI exists

```
oc get vmi my-vm -n my-namespace
```



# Check virt-launcher pod

```
oc get pods -n my-namespace -l kubevirt.io/domain=my-vm
```

```
[]:
- VM []
- VMI []
- virt-launcher Pod []
- [] API []
```

```
[]:
```

```
. VM []:
+
[source,bash,role=execute]
```

```
virtctl start my-vm -n my-namespace
```

```
. VMI []:
+
[source,bash,role=execute]
```

```
oc wait --for=condition=Ready vmi/my-vm -n my-namespace --timeout=60s
```

```
. virt-launcher Pod []:
+
[source,bash,role=execute]
```

```
oc logs -n my-namespace -l kubevirt.io/domain=my-vm
```

```
. virtctl []:
+
[source,bash,role=execute]
```

```
virtctl version oc get kubevirt -n openshift-cnv -o
jsonpath='{.items[0].status.observedKubeVirtVersion}'
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_[]_
===== []: []
```

```
[]:
```

```
[]
```

**\*\*[]\*\*:**

[source,bash]

# Try sending Enter key

# Press Enter multiple times

```
[redacted]:
- VM [redacted]
- VM [redacted]
- ttyS0 [redacted] systemd-getty [redacted]
```

```
[redacted]:
```

```
. VM [redacted] (VMI [redacted]):
+
[source,bash,role=execute]
```

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.phase}'
```

```
. [redacted] OS [redacted]:
- RHEL/Fedora [redacted]: `systemctl status serial-getty@ttyS0.service`
- [redacted]
. [redacted]:
+
[source,bash,role=execute]
```

# Press Ctrl+] to disconnect

```
virtctl console my-vm -n my-namespace
```

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_00_000000000000]
===== 00: 00000000000000

00:
000000000000000000000000000000000000

000000:
- 00000000000000000000000000
- cloud-init 0 SSH 0000000000000000
- root 0000000000000000

000:
00: 0000 cloud-init 000000000000000000000000000000000000SSH 0000000000000000

. 000000000000cloud-init 0000000000000000:
+
[source,yaml]
```

```
cloudInitNoCloud: userData: | #cloud-config user: fedora password: mypassword chpasswd: expire:
false
```

```
. 安装SSH 安装ssh-server和ssh-client
. VM 安装 cloud-init 安装cloud-init

[discrete#vm-configuration:remote-access-linux-troubleshooting::_安装_2_vnc_安装]
==== 安装 2: VNC 安装

[discrete#vm-configuration:remote-access-linux-troubleshooting::_安装_virtctl_vnc_安装]
==== 安装: virtctl vnc 安装

安装:
```

```
$ virtctl vnc my-vm -n my-namespace Error: VNC is not configured for this VM
```

```
[]:
[source,bash,role=execute]
```

# Check if graphics is enabled

```
oc get vm my-vm -n my-namespace -o
jsonpath='{.spec.template.spec.domain.devices.autoattachGraphicsDevice}'
```

# Check VM devices configuration

```
oc get vm my-vm -n my-namespace -o yaml | grep -A 10 "devices:"
```

```
[]:
- `autoattachGraphicsDevice: false` []
- VM []

[]:
. []OpenShift Virtualization
[]`autoattachGraphicsDevice: false` []`true` []

[discrete#vm-configuration:remote-access-linux-troubleshooting::_vnc_[]]
==== []: VNC []

[]:
virtctl vnc []VNC []

[]:
[source,bash,role=execute]
```

# Check virtctl output for port number

`virtctl vnc my-vm -n my-namespace` # Shows: "VNC connection on port 5900"



# Check if port is listening

netstat -tuln | grep 5900

\*\*[redacted]\*\*:

- [redacted] VNC [redacted]
- VNC [redacted]
- [redacted]

\*\*[redacted]\*\*:

. VNC [redacted]:

+

[source,bash]

# Linux

```
sudo dnf install tigervnc
```

# macOS

brew install tiger-vnc

```
. localhost:5900 通过 127.0.0.1:5900 连接成功
. 通过virtctl 连接5901, 5902 连接成功

[discrete#vm-configuration:remote-access-linux-troubleshooting::_vnc_连接成功]
===== 通过 VNC 连接成功

注意:
VNC 连接成功

注意:
- VM 连接成功
- 通过virtctl 连接成功
- VM 连接成功GUI 通过

注意:

. VM 连接成功
. VM 通过 GUI 连接成功:
+
[source,bash]
```

# Inside VM

systemctl status gdm # For GNOME systemctl status lightdm # For other DEs

```
. GUI VM
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_3_virtctl_ssh]
=== 3: virtctl SSH

[discrete#vm-configuration:remote-access-linux-
troubleshooting::_virtctl_ssh_]
==== : virtctl ssh

**::
[source,bash]
```

\$ virtctl ssh fedora@my-vm -n my-namespace Error: SSH is not available for this VM

```
**::
[source,bash,role=execute]
```

# Check VM has IP address

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}'
```

# Check SSH service in VM (via console)

`virtctl console my-vm -n my-namespace # Inside VM: systemctl status sshd`

**\*\*[redacted]\*\*:**

- VM [redacted] SSH [redacted]
- VM [redacted] IP [redacted]
- VM [redacted] SSH [redacted]
- [redacted]

**\*\*[redacted]\*\*:**

. VM [redacted] SSH [redacted]:

+

[source,bash]

# Inside VM via console

```
sudo dnf install openssh-server sudo systemctl enable --now sshd
```

```
. VM IP :
+
[source,bash,role=execute]
```

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces[0].ipAddress}'
```

```
. SSH :
+
[source,bash,role=execute]
```

# Try port-forward method first

```
oc port-forward vmi/my-vm -n my-namespace 2222:22 & ssh -p 2222 fedora@localhost
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_virtctl_ssh_]
==== : virtctl ssh
```

```
**:
[source,bash]
```

```
$ virtctl ssh fedora@my-vm -n my-namespace fedora@my-vm: Permission denied (publickey,gssapi-
keyex,gssapi-with-mic).
```

```
**:
[source,bash,role=execute]
```



# Check SSH keys

```
ls -la ~/.ssh/
```

# Try with password authentication

```
virtctl ssh --local-ssh-opts="-o PreferredAuthentications=password" fedora@my-vm -n my-namespace
```

```
[redacted]:
- SSH [redacted] VM [redacted] authorized_keys [redacted]
- [redacted]
- SSH [redacted]

[redacted]:

. cloud-init [redacted] SSH [redacted]:
+
[source,yaml]
```

```
cloudInitNoCloud: userData: | #cloud-config ssh_authorized_keys: - ssh-rsa AAAAB3NzaC1yc2EA...
user@host
```

```
. [redacted]VM [redacted]:
+
[source,bash]
```

```
mkdir -p ~/.ssh echo "ssh-rsa AAAAB3..." >> ~/.ssh/authorized_keys chmod 600 ~/.ssh/authorized_keys
```

```
. [redacted]Fedora [redacted] `fedora`[redacted]RHEL [redacted] `cloud-user` [redacted] `ec2-user` [redacted]

[discrete#vm-configuration:remote-access-linux-troubleshooting::[_redacted_virtctl_ssh_[redacted]]
===== [redacted]: virtctl ssh [redacted]

[redacted]:
[redacted] 30 [redacted]

[redacted]:
- DNS [redacted]
- GSSAPI [redacted]

[redacted]:
[redacted] SSH [redacted]:
[source,bash,role=execute]
```

```
virtctl ssh --local-ssh-opts="-o GSSAPIAuthentication=no -o UseDNS=no" fedora@my-vm -n my-namespace
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_4_nodeport_service_ssh]
```

```
==== 4: NodePort Service SSH
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_nodeport_service_]
```

```
===== : NodePort Service
```

```
::
```

```
[source,bash]
```

```
$ ssh -p 30022 fedora@<node-ip> ssh: connect to host <node-ip> port 30022: Connection refused
```

```
::
```

```
[source,bash,role=execute]
```

# Check service exists

```
oc get svc -n my-namespace
```

# Check service endpoints

```
oc get endpoints my-vm-ssh -n my-namespace
```

# Check NodePort assigned

```
oc get svc my-vm-ssh -n my-namespace -o jsonpath='{.spec.ports[0].nodePort}'
```

```
[]:
- []
- NodePort []
- VM Pod []
- [] NodePort []
- VM [] SSH []listen[]

[]:

. []:
+
[source,yaml]
```

apiVersion: v1 kind: Service metadata: name: my-vm-ssh spec: type: NodePort selector: kubevirt.io/domain: my-vm ports: - protocol: TCP port: 22 targetPort: 22 nodePort: 30022 # Optional, will be auto-assigned if omitted

```
. [] VM []:
+
[source,bash,role=execute]
```

```
oc get vm my-vm -n my-namespace -o jsonpath='{.spec.template.metadata.labels}'
```

```
. VM [] SSH []:
+
[source,bash,role=execute]
```

```
virtctl console my-vm -n my-namespace # Inside VM: systemctl status sshd
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_[]_nodeport_[]_vm_[]]
==== []: NodePort [] VM []

[]:
SSH [] VM []

[]:
[source,bash,role=execute]
```

# Check service endpoints

```
oc describe svc my-vm-ssh -n my-namespace
```

# Look at Endpoints field - should show VM pod IP

```
Output:
- kubevirt.io/domain: my-vm # Unique per VM # OR app: my-specific-vm-app
- kubevirt.io/ssh: true

YAML:

. kubevirt.io/ssh: true
+
[source,yaml]
```

selector: kubevirt.io/domain: my-vm # Unique per VM # OR app: my-specific-vm-app

```
. kubevirt.io/ssh: true
+
[source,bash,role=execute]
```

oc get endpoints my-vm-ssh -n my-namespace -o yaml

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_node_ip_]
==== Node IP

Output:
NodePort Node IP

YAML:
[source,bash,role=execute]
```



# Get node IPs

```
oc get nodes -o wide
```

**Use any node's INTERNAL-IP or  
EXTERNAL-IP (if available)**

# Or get specific node where VM is running

```
VM_NODE=$(oc get vmi my-vm -n my-namespace -o jsonpath='{.status.nodeName}') oc get node $VM_NODE -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}'
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_5_loadbalancer_ssh]
==== 5: LoadBalancer SSH

[discrete#vm-configuration:remote-access-linux-
troubleshooting::_loadbalancer_service_ip_]
===== : LoadBalancer Service IP

**::
[source,bash]
```

```
$ oc get svc my-vm-ssh -n my-namespace NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) my-vm-ssh LoadBalancer 172.30.x.x <pending> 22:30022/TCP
```

```
**::
[source,bash,role=execute]
```

# Check if LoadBalancer provider is available

```
oc get svc -A | grep LoadBalancer
```

# Check cluster has LoadBalancer support

oc describe svc my-vm-ssh -n my-namespace

```
Output:
- LoadBalancer 10.128.0.1:31280 (MetalLB10.128.0.1:31280)
- LoadBalancer 10.128.0.1:31280
- LoadBalancer 10.128.0.1:31280

Note:

.
link:https://docs.redhat.com/en/documentation/openshift_container_platfo
rm/4.19/html/ingress_and_load_balancing/load-balancing-with-
metallb[MetalLB,window=_blank]
.
.
. LoadBalancer 10.128.0.1:31280:
+
[source,bash,role=execute]
```

oc describe svc my-vm-ssh -n my-namespace

```
. 10.128.0.1:31280: LoadBalancer 10.128.0.1:31280 NodePort 10.128.0.1:31280

[discrete#vm-configuration:remote-access-linux-
troubleshooting::_loadbalancer_ip_10.128.0.1:31280]
==== 10.128.0.1: LoadBalancer IP 10.128.0.1:31280

Note:
10.128.0.1 IP 10.128.0.1:31280 SSH 10.128.0.1:31280

Note:
[source,bash,role=execute]
```

# Check LoadBalancer IP

```
LB_IP=$(oc get svc my-vm-ssh -n my-namespace -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
```

# Try ping

```
ping -c 3 $LB_IP
```

# Check from within cluster

```
oc run test-curl --image=curlimages/curl --rm -it --restart=Never -- curl -v telnet://$LB_IP:22
```

```
[]:
- LoadBalancer IP []
- []
- LoadBalancer []
- VM [] SSH []

[]:

. LoadBalancer IP []
. [] LoadBalancer IP []
. VM [] SSH [] 4 []

[discrete#vm-configuration:remote-access-linux-troubleshooting::_6_{}_ssh]
==== [] 6: [] SSH

[discrete#vm-configuration:remote-access-linux-
troubleshooting::_{}_oc_port_forward_{}]
===== []: oc port-forward []

[]:
[source,bash]
```

```
$ oc port-forward vmi/my-vm -n my-namespace 2222:22 Error: unable to forward port
```

```
[]:
[source,bash,role=execute]
```



# Check VMI exists

```
oc get vmi my-vm -n my-namespace
```

# Check local port not in use

`netstat -tuln | grep 2222 lsof -i :2222`

```
~~~~:
- ~~~~~
- VMI ~~~~~
- ~~~~~
```

```
~~~:
```

```
. ~~~~~:
+
[source,bash,role=execute]
```

`oc port-forward vmi/my-vm -n my-namespace 12345:22`

```
. ~~~~~:
+
[source,bash]
```

# Find process

```
lsof -ti :2222 | xargs kill -9
```

```
. VMI :
+
[source,bash,role=execute]
```

```
oc wait --for=condition=Ready vmi/my-vm -n my-namespace
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_ssh_]
===== : SSH
::
[source,bash]
```

```
$ oc port-forward vmi/my-vm -n my-namespace 2222:22 & Forwarding from 127.0.0.1:2222 → 22
```

```
$ ssh -p 2222 fedora@localhost ssh: connect to host localhost port 2222: Connection refused
```

```
::
[source,bash,role=execute]
```

# Check SSH is listening in VM

`virtctl console my-vm -n my-namespace # Inside VM: ss -tuln | grep :22`

**\*\*Output\*\*:**

- VM is SSH listening
- SSH is listening
- VM is SSH listening

**\*\*Note\*\*:**

. VM is SSH listening:

+

[source,bash]

# Inside VM via console

```
sudo dnf install openssh-server sudo systemctl enable --now sshd
```

```
. SSH 0000000000:
```

```
+
```

```
[source,bash]
```

# Inside VM

`sudo ss -tuln | grep ssh`

```
. SSH 0.0.0.0:
+
[source,bash]
```

# Inside VM

```
sudo cat /etc/ssh/sshd_config | grep ^Port
```

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_00_0000000000000000]
===== 00: 000000000000000000

00:
```

000000000000000000000000

```
000000:
```

- 000000000000
- API 00000000000000
- 000000000000

```
000:
```

. SSH 000 keepalive 000000:

+

```
[source,bash]
```

```
ssh -o ServerAliveInterval=60 -p 2222 fedora@localhost
```

```

. [source,bash,role=execute]
+

```

```
while true; do oc port-forward vmi/my-vm -n my-namespace 2222:22 sleep 5 done &
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_0000_7_00000000000000000000]
===== 0000 7: 0000000000000000000000

[discrete#vm-configuration:remote-access-linux-
troubleshooting::_00_0000000000000000_vm_00000000]
===== 00: 00000000000000 VM 00000000

00:
```

```
$ ping 192.168.100.10 Request timeout
```

```
[]:
[source,bash,role=execute]
```

# Check VM has secondary interface configured

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}' | jq
```



# Check NetworkAttachmentDefinition (NAD) exists

```
oc get network-attachment-definition -n my-namespace
```

# Check VM network configuration

```
oc get vm my-vm -n my-namespace -o jsonpath='{.spec.template.spec.networks}' | jq
```

```
[]:
- NetworkAttachmentDefinition []
- VM []
- [] IP []
- []
- VLAN []
```

```
[]:

. NAD []:
+
[source,bash,role=execute]
```

```
oc describe network-attachment-definition my-nad -n my-namespace
```

```
. VM []:
+
[source,yaml]
```

```
spec: template: spec: networks: - name: default pod: {} - name: secondary-net multus:
networkName: my-nad domain: devices: interfaces: - name: default masquerade: {} - name:
secondary-net bridge: {}
```

```
. VM [] IP []:
+
[source,bash,role=execute]
```

```
virtctl console my-vm -n my-namespace # Inside VM: ip addr show enp2s0 # Secondary interface
```

```
. cloud-init [] IP []cloud-init IP []
. VLAN []OVS []
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting:::_[]_[]_vm_[]]
==== []: [] VM []
```

```
[]:
[] (enp1s0) []enp2s0 []
```

```
[]:
[source,bash,role=execute]
```

# Inside VM

ip link show

# Check from OpenShift

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}' | jq
```

```
[]:
- NAD []
- VM []
- Multus CNI []

[]:

. NAD []:
+
[source,bash,role=execute]
```

```
oc get network-attachment-definition -n my-namespace
```

```
. VM []
. multus []:
+
[source,bash,role=execute]
```

```
oc logs -n openshift-multus -l app=multus
```

```
. virt-launcher Pod []:
+
[source,bash,role=execute]
```

```
POD=$(oc get pods -n my-namespace -l kubevirt.io/domain=my-vm -o name) oc describe $POD -n my-namespace
```

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_]
====

[discrete#vm-configuration:remote-access-linux-troubleshooting::_vm_
===== VM

[source,bash,role=execute]
```

```
#!/bin/bash NAMESPACE="my-namespace" VM_NAME="my-vm"
```

```
echo "=== VM Status ===" oc get vm $VM_NAME -n $NAMESPACE oc get vmi $VM_NAME -n $NAMESPACE
```

```
echo "" echo "=== VM Pod Status ===" oc get pods -n $NAMESPACE -l
```

```
kubevirt.io/domain=$VM_NAME
```

```
echo "" echo "=== VM IP Addresses ===" oc get vmi $VM_NAME -n $NAMESPACE -o
jsonpath='{.status.interfaces}' | jq
```

```
echo "" echo "=== Guest Agent Status ===" oc get vmi $VM_NAME -n $NAMESPACE -o
jsonpath='{.status.guestOSInfo.id}' echo ""
```

```
echo "" echo "=== VM Networks ===" oc get vm $VM_NAME -n $NAMESPACE -o
jsonpath='{.spec.template.spec.networks}' | jq
```

```
echo "" echo "=== Services for VM ===" oc get svc -n $NAMESPACE -l
kubevirt.io/domain=$VM_NAME
```

```
echo "" echo "=== NetworkAttachmentDefinitions ===" oc get network-attachment-definition -n
$NAMESPACE
```

```
echo "" echo "=== Recent Events ===" oc get events -n $NAMESPACE --field-selector
involvedObject.name=$VM_NAME --sort-by='.lastTimestamp' | tail -10
```

```
[discrete#vm-configuration:remote-access-linux-troubleshooting::_vm_]
===== VM

[source,bash]
```

```
#!/bin/bash echo "=== Network Interfaces ===" ip addr show
```

```
echo "" echo "=== Routing Table ===" ip route show
```

```
echo "" echo "=== DNS Configuration ===" cat /etc/resolv.conf
```

```
echo "" echo "=== SSH Service Status ===" systemctl status sshd
```

```
echo "" echo "=== SSH Listening Ports ===" ss -tln | grep :22
```

```
echo "" echo "=== Guest Agent Status ===" systemctl status qemu-guest-agent
```

```
echo "" echo "=== Firewall Status ===" systemctl status firewalld 2>/dev/null || echo "Firewall not
active"
```

```
echo "" echo "=== Connectivity Test ===" ping -c 2 8.8.8.8 && echo "External connectivity: OK" ||
echo "External connectivity: FAILED"
```

```
[discrete#vm-configuration:remote-access-linux-
troubleshooting::_]
===== -

[source,text]
```

Need access to VM? □ □ VM not running? → virtctl start VM □ □ Just need console? → virtctl console □ □ Not working? → Check VM running, check virt-launcher pod □ □ Need GUI? → virtctl vnc □ □ Not working? → Check VNC graphics device, install VNC viewer □ □ Need SSH? □ □ From workstation with virtctl? → virtctl ssh □ □ Not working? → Check SSH service running, SSH keys configured □ □ □ □ Persistent access needed? → NodePort Service □ □ Not working? → Check service selector, NodePort number, node firewall □ □ □ □ Public access needed? → LoadBalancer Service □ □ Not working? → Check LoadBalancer provider installed, IP assigned □ □ □ □ Temporary access? → oc port-forward □ □ Not working? → Check local port free, VMI running, SSH in VM □ □ □ □ Direct network access? → Secondary Network □ □ Not working? → Check NAD, VM secondary interface, IP config, network routing

```
[discrete#vm-configuration:remote-access-linux-troubleshooting:: _00000000]
```

==      □□□□□□□□

\* link:[https://docs.openshift.com/container-platform/latest/virt/vm\\_networking/virt-about-vm-networking.html](https://docs.openshift.com/container-platform/latest/virt/vm_networking/virt-about-vm-networking.html)[OpenShift Virtualization Networking,window=\_blank]

```
* link:https://kubevirt.io/user-
guide/virtual_machines/accessing_virtual_machines/[KubeVirt Network
Access,window= blank]
```

```
* link:https://kubevirt.io/user-guide/operations/virtctl_client_tool/[Virtctl CLI Reference,window= blank]
```

```
* link:https://kubernetes.io/docs/concepts/services-networking/service/[Kubernetes Services>window= blank]
```

\*

```
link:https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/ingress_and_load_balancing/load-balancing-with-metallb[MetalLB>window= blank]
```

```
* xref:networking:cloud-init-troubleshooting[Cloud-init IP Configuration Troubleshooting]
```

\* xref:networking:ovs-bridge-troubleshooting[OVS Bridge Verification]

```
[discrete#vm-configuration:remote-access-linux-troubleshooting:: 00000 ocplab01 00000]
```

```
==== 00000: ocp1ab01 00000
```

SNO 00000 (ocplab01) 00000:

```
[source,bash]
```

```
$ oc get vmi fedora-access-vm -n vm-access-test -o jsonpath='{.status.interfaces}' | jq [{
"infoSource": "domain, guest-agent, multus-status", "interfaceName": "eth0", "ipAddress":
"10.128.0.171", "ipAddresses": ["10.128.0.171", "fd02::62"], "mac": "02:92:c2:00:00:08", "name":
"default", "queueCount": 1 }]
```

[illegible]

```
[source,bash]
```

```
$ oc get vmi fedora-access-vm -n vm-access-test -o jsonpath='{.status.guestOSInfo}' | jq { "id":
"fedora", "kernelRelease": "6.11.4-301.fc41.x86_64", "kernelVersion": "#1 SMP PREEMPT_DYNAMIC
Thu Oct 10 16:16:06 UTC 2024", "name": "Fedora Linux", "prettyName": "Fedora Linux 41 (Cloud
```

Edition)", "version": "41", "versionId": "41" }

Windows VM OpenShift OS SSH

:!navtitle:

:docname: remote-access-windows-vm

:page-module: vm-configuration

:page-relative-src-path: remote-access-windows-vm.adoc

:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook\_ja

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

:page-origin-refhash: (worktree)

:navtitle: RDP Windows VM

[#vm-configuration:remote-access-windows-vm]

=== RDP Windows VM

[discrete#vm-configuration:remote-access-windows-vm::\_]

===

OpenShift Windows 11 (RDP)

Windows VM VNC

[discrete#vm-configuration:remote-access-windows-vm::\_]

===

Red Hat OpenShift Virtualization Windows VM Microsoft

[discrete#vm-configuration:remote-access-windows-vm::\_]

===

[discrete#vm-configuration:remote-access-windows-vm::\_]

=====

IP

\* \*\*10.0.2.2\*: Windows OS VM IP

\* \*\*10.128.1.132\*: OpenShift Pod IP

\* \*\* -

[discrete#vm-configuration:remote-access-windows-vm::\_]

===

\* OpenShift Windows 11 VM

\* OpenShift Web 'oc' CLI

\* VNC VM

\* Windows 11 VM

```

* 1. Windows VM

[discrete#vm-configuration:remote-access-windows-vm::_1]
=====

[discrete#vm-configuration:remote-access-windows-vm::_1]
=====

[discrete#vm-configuration:remote-access-windows-vm::_1_windows_11]
===== 1: Windows 11

. **VM**
+
OpenShift VNC Windows 11 VM
. **
+
**
+
**
. **
+
**
+
**
+
**
+
**PC
[discrete#vm-configuration:remote-access-windows-vm::_2_vm]
===== 2: VM

RDP 3389

[discrete.h7#vm-configuration:remote-access-windows-vm::_a_openshift_web_]
===== A: OpenShift Web (0)

. **
+
[source, text]

```

OpenShift Console → Your Project → Networking → Services

```

. **
+
**
+

```



```

|===
| | | |
|
| **
| `LoadBalancer`
| IP
|
| **
| `win11-rdp-service`
|
| **
| `vm.kubevirt.io/name: <your-vm-name>`
| ** : VM
|
| **
| `3389`
|
| **
| `3389`
| Windows VM RDP
|
| **
| `TCP`
| RDP
|===
+
NOTE: LoadBalancer `NodePort` (30000-32767)
. **
+
**
+
NodePort
[discrete.h7#vm-configuration:remote-access-windows-vm:::_b_ _cli]
===== B: (CLI)
. **
+
`rdp-service.yaml`
+
[source,yaml]

```

apiVersion: v1 kind: Service metadata: name: win11-rdp-service spec: selector: vm.kubevirt.io/name: your-vm-name # REPLACE with your VM's actual name ports: - protocol: TCP port: 3389 targetPort: 3389 type: LoadBalancer # or NodePort

```

. **
+

```

```
[source,bash,role=execute]
```

```
oc apply -f rdp-service.yaml
```

```
[discrete#vm-configuration:remote-access-windows-vms::_0000_3_00000000]
===== 0000 3: 00000000

[discrete.h7#vm-configuration:remote-access-windows-vms::_loadbalancer_00000000]
===== LoadBalancer 00000000:

. **0000000000 → 0000** 00000000
. `win11-rdp-service` 00000000
. **00 IP** 000000000000

[discrete.h7#vm-configuration:remote-access-windows-vms::_nodeport_00000000]
===== NodePort 00000000:

. 0000000000000000000000 **NodePort** (0: `31234`) 00000000
. 000000000000 IP 00000000
+
[source,bash,role=execute]
```

```
oc get nodes -o wide
```

```

+
EXTERNAL-IP []

[discrete#vm-configuration:remote-access-windows-vms::_4_]
===== 4: []

. **[]**
+
[] Windows []
. **[]**
+
|===
| [] | [] | []

| **LoadBalancer**
| '<External-IP>'
| '203.0.113.5'

| **NodePort**
| '<Node-External-IP>:<NodePort>'
| '203.0.113.10:31234'
|===
. **[]**
+
[][]

```

+

Windows 11 VM

[discrete#vm-configuration:remote-access-windows-vms::\_]

====

NOTE: RDP VPN Windows (NLA)

[discrete#vm-configuration:remote-access-windows-vms::\_]

====

[discrete#vm-configuration:remote-access-windows-vms::\_]

=====

- \* Windows 11
- \* VPN
- \* (NLA)

[discrete#vm-configuration:remote-access-windows-vms::\_]

=====

|====  
| | |

| \*\*  
|   
| IP

| \*\*  
| Windows RDP  
| Windows

| \*\*  
| VM  
| VNC

| \*\* IP  
| LoadBalancer  
| NodePort  
|====

[discrete#vm-configuration:remote-access-windows-vms::\_]

====

- \* OpenShift Virtualization Windows 11 VM (RDP)
- \* LoadBalancer NodePort VM RDP (3389)
- \* (IP NodePort IP)

```
* 仮想マシンをインストールする Windows VM
* RDP 仮想マシンからリモートデスクトップ接続する
```

```
[discrete#vm-configuration:remote-access-windows-vms::_]
====
```

```
* link:https://docs.openshift.com/container-platform/latest/virt/about-
virt.html[OpenShift Virtualization Documentation,window=_blank]
* link:https://kubevirt.io/user-guide/[KubeVirt User Guide,window=_blank]
* link:https://learn.microsoft.com/en-us/windows-server/remote/remote-desktop-
services/[Windows Remote Desktop Best Practices,window=_blank]
```

```
:!navtitle:
```

```
:docname: windows-vm-virtio-drivers
:page-module: vm-configuration
:page-relative-src-path: windows-vm-virtio-drivers.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: VirtIO仮想マシンWindows VM
[#vm-configuration:windows-vm-virtio-drivers]
=== VirtIO仮想マシンWindows VM
```

```
[discrete#vm-configuration:windows-vm-virtio-drivers::_]
====
```

VirtIO仮想マシンOpenShift  
Virtualization仮想マシンVM仮想マシンVirtIO仮想マシンVM仮想マシンI/O  
仮想マシン

Windows VM仮想マシンWindows仮想マシンVirtIO仮想マシンOpenShift  
Virtualization仮想マシン

```
[discrete#vm-configuration:windows-vm-virtio-drivers::_]
====
```

```
* **OpenShift 4.18+** OpenShift Virtualization
* **CLI**: 'oc' 'virtctl'
* **WindowsISO** (Windows 10/11 Server 2019/2022)
* **VirtIOISO** (Red HatFedora)
* **Windows**
* ****: 4GBRAM60GB
```

```
仮想マシン:
```

```
[discrete#vm-configuration:windows-vm-virtio-drivers::_virtio_virtio]
=== 1: VirtIOISO
FedoraVirtIOISO
[source,bash,role=execute]
```

```
curl -L -O https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-virtio/virtio-win.iso
```

```
VirtIOISODataVolume

link:attachment$windows-vm-virtio-drivers/virtio-drivers-dv.yaml[virtio-drivers-
dv.yaml ,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: cdi.kubevirt.io/v1beta1 kind: DataVolume metadata: name: virtio-
drivers-iso annotations: cdi.kubevirt.io/storage.bind.immediate.requested: "true" spec: storage:
resources: requests: storage: 1Gi accessModes: - ReadWriteOnce source: upload: {} EOF
```

```
VirtIO000000ISO000000000000
```

```
[source,bash,role=execute]
```

```
virtctl image-upload dv virtio-drivers-iso --image-path=virtio-win.iso --insecure
```

```
cat: CDI: No such file or directory`--insecure` cat: No such file or directory

[discrete#vm-configuration:windows-vm-virtio-drivers::_iso_2_windows.iso]
==== 2: WindowsISO

WindowsISODataVolume

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: cdi.kubevirt.io/v1beta1 kind: DataVolume metadata: name:
windows-installation-iso annotations: cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec: storage: resources: requests: storage: 6Gi accessModes: - ReadWriteOnce source: upload: {}
EOF
```

```
WindowsISO\ISO\windows.iso` ISO
[source,bash,role=execute]
```

```
virtctl image-upload dv windows-installation-iso --image-path=windows.iso --insecure
```

```
[discrete#vm-configuration:windows-vm-virtio-drivers::_0000_3_virtio00000000windows_vm0000]
==== 0000 3: VirtIO00000000Windows VM0000

VirtIO0000000000000000Windows000000000000

link:attachment$windows-vm-virtio-drivers/windows-vm-virtio.yaml[windows-vm-
virtio.yaml 00000000,role=unresolved]

[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: windows-vm-
virtio labels: app: windows-vm-virtio spec: runStrategy: Always template: metadata: labels:
kubevirt.io/size: medium kubevirt.io/domain: windows-vm-virtio vm.kubevirt.io/name: windows-
vm-virtio spec: domain: cpu: cores: 2 sockets: 1 threads: 2 resources: requests: memory: 4Gi devices:
disks: - name: rootdisk disk: bus: virtio bootOrder: 2 - name: windows-installation-iso cdrom: bus:
sata bootOrder: 1 - name: virtio-drivers-iso cdrom: bus: sata interfaces: - name: default
masquerade: {} model: virtio tpm: {} rng: {} clock: utc: {} timer: hpet: present: false pit: tickPolicy:
delay rtc: tickPolicy: catchup hyperv: {} features: acpi: {} apic: {} hyperv: relaxed: {} vpic: {}
spinlocks: spinlocks: 8191 vpinde: {} runtime: {} synic: {} stimer: {} reset: {} frequencies: {}
reenlightenment: {} tlbflush: {} ipi: {} smm: enabled: true firmware: bootloader: efi: secureBoot:
true networks: - name: default pod: {} volumes: - name: rootdisk dataVolume: name: windows-vm-
virtio-rootdisk - name: windows-installation-iso dataVolume: name: windows-installation-iso -
name: virtio-drivers-iso dataVolume: name: virtio-drivers-iso dataVolumeTemplates: - metadata:
name: windows-vm-virtio-rootdisk annotations: cdi.kubevirt.io/storage.bind.immediate.requested:
"true" spec: storage: resources: requests: storage: 60Gi accessModes: - ReadWriteOnce source:
blank: {} EOF
```

```
VM000000000000

[source,bash,role=execute]
```

```
oc get vm windows-vm-virtio -w
```

```
`STATUS=Running` 00 `READY=True` 000000000000`Ctrl+C` 000000000000

[discrete#vm-configuration:windows-vm-virtio-
drivers::_0000_4_virtio0000000000000000windows00000000]
==== 0000 4: VirtIO0000000000000000Windows00000000

VM000000000000Windows000000000000

[source,bash,role=execute]
```

```
virtctl vnc windows-vm-virtio
```

```

OpenShift Webconsole: **Virtualization** → **VirtualMachines** → **windows-vm-
virtio** → **Console**

. **Windows**:
+
VMNo bootable device foundUEFI
+
* UEFIboot from other media or DVD
* Boot from DVD/CD
* Windows ISO
* Windows

. **VirtIO**: Windows
+
* ****:
* VirtIOCD D: E:
* `viostor\{windows-version}\amd64\`
* Windows Server 2022: `viostor\2k22\amd64\`
* Windows 11: `viostor\w11\amd64\`
* **Red Hat VirtIO SCSI Controller**

. **: VirtIOWindows

. **: Windows

[discrete#vm-configuration:windows-vm-virtio-drivers::_5_virtio]
=== 5: VirtIO

Windows

. **PC** → ****

. **:
+
*
* ** → **
* VirtIO CD `NetKVM\{windows-version}\amd64\`
*

. **:
+
* **Balloon**: PCI → `Balloon\{windows-version}\amd64\`
* **: PCI → `vioserial\{windows-version}\amd64\`
* **: PCI → `vioinput\{windows-version}\amd64\`

. **QEMU**:
+
VirtIOCD `guest-agent\qemu-ga-x86_64.msi`

[discrete#vm-configuration:windows-vm-virtio-drivers::_6_]

```

```
==== 6: =====

VirtIO=====

. ****:
. ****: ** (services.msc) ** QEMU Guest Agent
. **: OpenShift

[source,bash,role=execute]
```

```
oc get vmi windows-vm-virtio -o jsonpath='{.status.guestOSInfo}'
```

```
VM
```

```
[source,bash,role=execute]
```

`[source,bash,role=execute]`

```
[source,bash,role=execute]
```

```
oc get vmi windows-vm-virtio -o jsonpath='{.status.interfaces[0].ipAddress}'

[discrete#vm-configuration:windows-vm-virtio-drivers::_virtio]
==== VirtIO

VirtIOWindows VM

[cols="1,2"]
|===
| |
|** (NetKVM)**
| 10

|** (viosstor)**
| IOPS

|** (vioser)**
|

|** **
|
|===

[discrete#vm-configuration:windows-vm-virtio-drivers::_]
====

[discrete#vm-configuration:windows-vm-virtio-drivers::_]
=====

[cols="1,1,2"]
|===
| | |
```



```

|**TPM 2.0**
|TPM
|VMWindows 11TPM

|**TPM**
|TPM
|Windows ISO `bootOrder: 1` `bootOrder: 2`

|**TPM**
|VirtIO
|viostor

|**TPM**
|NetKVM
|Windows
|===

[discrete#vm-configuration:windows-vm-virtio-drivers::_]
=====

ISO:

[source,bash,role=execute]

```

□□□□□ --insecure □□□□□□□□□□

```
virtctl image-upload dv <datavolume-name> --image-path=<iso-file> --insecure
```

```
[discrete#vm-configuration:windows-vm-virtio-drivers::_00000000]
```

```
[source,bash,role=execute]
```

```
oc delete vm windows-vm-virtio oc delete dv windows-installation-iso virtio-drivers-iso
```

```
[discrete#vm-configuration:windows-vm-virtio-drivers:::]
```



□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
* DataVolume[VirtIO]Windows[ISO]
```

```
* VirtIOWindows VM
```

```
* WindowsVirtIO
```

```
* 000000000000VirtIO000000000000OEMU000000000000
```

```
* OpenShift Virtualization VM
```

\* □□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
Windows VMVirtIOOpenShift Virtualization
```

```
[discrete#vm-configuration:windows-vm-virtio-drivers:::]
```



\* link:[https://docs.openshift.com/container-platform/latest/virt/vm\\_networking/virt-installing-virtio-drivers.html](https://docs.openshift.com/container-platform/latest/virt/vm_networking/virt-installing-virtio-drivers.html)[OpenShift VirtIO,window= blank]

```
* link:https://kubevirt.io/user-guide/[KubeVirt,window=_blank]
```

```
* xref:vm-configuration:remote-access-windows-vm[Windows VM]
```

```
:!navtitle:
```

```
:docname: infrastructure-node-placement
```

```
:page-module: vm-configuration
```

```
:page-relative-src-path: infrastructure-node-placement.adoc
```

```
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
```

```
:page-origin-start-path:
```

```
:page-origin-refname: main
```

```
:page-origin-reftype: branch
```

```
:page-origin-refhash: (worktree)
```

```
:navtitle: Infrastructure Node Placement
```

```
[#vm-configuration:infrastructure-node-placement]
```

### === Infrastructure Node Placement

```
[discrete#vm-configuration:infrastructure-node-placement::_overview]
==== Overview
```

```
OpenShift Virtualization virt-controller virt-api virt-
handler CDI openshift-
cnv Pod VM

```

HyperConverged OpenShift Virtualization

```
[discrete#vm-configuration:infrastructure-node-placement::_0000]
===== 0000
```

- \* OpenShift Virtualization
- \* HyperConverged CR
- \* Pod
- \*
- \* TaintToleration

```
[discrete#vm-configuration:infrastructure-node-placement::_]
====
```

```
* OpenShift Virtualization OpenShift 4.18
* HyperConverged CR
* \oc\ CLI
*
```

```
[discrete#vm-configuration:infrastructure-node-placement::_000000000000000000000000]
=== 000000000000000000000000
```

# OpenShift Virtualization

```
[cols="1,2,1",options="header"]
|===
| 00000000 | 00 | 000000000000
```

```
|virt-controller
|VM
|Deployment (2)
```

```
|virt-api
|API
|Deployment (2)
```

```
|virt-handler
| VM
|DaemonSet
```

```
|cdi-controller
|Containerized Data Importer|
```

```
|Deployment (10000)
```

```
|cdi-uploadproxy
```

```
|Web0000000000VM0000000000000000
```

```
|Deployment (10000)
```

```
|cdi-apiserver
```

```
|CDI API000000000000
```

```
|Deployment (10000)
```

```
|hco-webhook
```

```
|HyperConverged CR00000000
```

```
|Deployment (10000)
```

```
|===
```

```
[discrete#vm-configuration:infrastructure-node-placement::_000000000000000000]
```

```
===== 0000000000000000000000
```

```
000000000000Pod0000000000000000000000
```

```
[source,bash,role=execute]
```

# View all virtualization infrastructure pods and their nodes

```
oc get pods -n openshift-cnv -o wide
```

```
pod:
[source, text]
```

```
NAME READY STATUS NODE virt-api-6d9f8c7b9d-4xk2l 1/1 Running ip-10-0-24-89.ec2.internal virt-
api-6d9f8c7b9d-8tz5p 1/1 Running ip-10-0-43-58.ec2.internal virt-controller-7c8d9f5b6d-5nvlm 1/1
Running ip-10-0-24-89.ec2.internal virt-controller-7c8d9f5b6d-9kqwz 1/1 Running ip-10-0-43-
58.ec2.internal virt-handler-8zkgp 1/1 Running ip-10-0-24-89.ec2.internal virt-handler-f2nlb 1/1
Running ip-10-0-43-58.ec2.internal virt-handler-q7xmc 1/1 Running ip-10-0-82-27.ec2.internal cdi-
apiserver-5f8c9d7b6c-7nlqx 1/1 Running ip-10-0-24-89.ec2.internal cdi-deployment-7c8f9b5d6e-
2kxmp 1/1 Running ip-10-0-43-58.ec2.internal cdi-uploadproxy-6d9f8c7b9d-5pqrs 1/1 Running ip-10-
0-24-89.ec2.internal
```

```
pod:
[source, bash, role=execute]
```

# View virt-controller pods

```
oc get pods -n openshift-cnv -l kubevirt.io=virt-controller -o wide
```

# View virt-api pods

```
oc get pods -n openshift-cnv -l kubevirt.io=virt-api -o wide
```

# View virt-handler DaemonSet

```
oc get pods -n openshift-cnv -l kubevirt.io=virt-handler -o wide
```



# View CDI components

```
oc get pods -n openshift-cnv -l app.kubernetes.io/component=storage -o wide
```

```
[discrete#vm-configuration:infrastructure-node-placement::_hyperconverged]
==== HyperConverged
```

```
HyperConverged CR
```

```
[source,bash,role=execute]
```

## View current HyperConverged configuration

```
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o yaml
```

```
HyperConverged CR2
[cols="1,2",options="header"]
|===
| |
|`spec.infra.nodePlacement`
|: virt-api virt-controller CDI CDI
|`spec.workloads.nodePlacement`
|: virt-handler DaemonSet
|===

IMPORTANT: HyperConverged CR OpenShift
Virtualization
[discrete#vm-configuration:infrastructure-node-placement::_]
=====
[discrete#vm-configuration:infrastructure-node-placement::_]
=====

[source,bash,role=execute]
```

# Label nodes for virtualization infrastructure

```
oc label node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra= oc label node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=
```

```
HyperConverged CR
```

```
[source,bash,role=execute]
```

```
oc edit hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

```
[source,yaml]
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged metadata: name: kubevirt-hyperconverged namespace: openshift-cnv spec: infra: nodePlacement: nodeSelector: node-role.kubernetes.io/infra: "" workloads: nodePlacement: nodeSelector: node-role.kubernetes.io/worker: ""
```

```
* **virt-api virt-controller CDI `node-role.kubernetes.io/infra`
* **virt-handler DaemonSet `node-role.kubernetes.io/worker`
```

```
[discrete#vm-configuration:infrastructure-node-placement::_]
=====
```

```
[source,yaml]
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged metadata: name: kubevirt-hyperconverged namespace: openshift-cnv spec: infra: nodePlacement: affinity: nodeAffinity: requiredDuringSchedulingIgnoredDuringExecution: nodeSelectorTerms: - matchExpressions: - key: node-role.kubernetes.io/infra operator: Exists preferredDuringSchedulingIgnoredDuringExecution: - weight: 100 preference: matchExpressions: - key: topology.kubernetes.io/zone operator: In values: - us-east-1a - us-east-1b
```

\* □□□□□□□□ \*\*□□ \*\*□□□□□□□□

# Taint infrastructure nodes

```
oc adm taint node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra=:NoSchedule
oc adm taint node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=:NoSchedule
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged CR
```

```
[source,yaml]
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged metadata: name: kubevirt-
hyperconverged namespace: openshift-cnv spec: infra: nodePlacement: nodeSelector: node-
role.kubernetes.io/infra: "" tolerations: - key: node-role.kubernetes.io/infra operator: Exists effect:
NoSchedule workloads: nodePlacement: nodeSelector: node-role.kubernetes.io/worker: ""
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged CR
```

```
[discrete#vm-configuration:infrastructure-node-placement::_pod]
===== Pod
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged CR
```

```
[source,yaml]
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged metadata: name: kubevirt-
hyperconverged namespace: openshift-cnv spec: infra: nodePlacement: affinity: podAntiAffinity:
preferredDuringSchedulingIgnoredDuringExecution: - weight: 100 podAffinityTerm: labelSelector:
matchLabels: kubevirt.io: virt-api topologyKey: topology.kubernetes.io/zone - weight: 100
podAffinityTerm: labelSelector: matchLabels: kubevirt.io: virt-controller topologyKey:
topology.kubernetes.io/zone
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged CR
```

```
[discrete#vm-configuration:infrastructure-node-placement::_pod]
===== Pod
```

```
[discrete#vm-configuration:infrastructure-node-placement::_1_3_6]
===== 1: (3-6)
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged CR
```

```
[source,yaml]
```

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged metadata: name: kubevirt-
hyperconverged namespace: openshift-cnv spec: infra: nodePlacement: nodeSelector: node-
role.kubernetes.io/worker: "" workloads: nodePlacement: nodeSelector: node-
```

role.kubernetes.io/worker: ""

**\*\*注意\*\*:** 部署虚拟机时，需要指定虚拟机配置

[discrete#vm-configuration:infrastructure-node-placement::\_节点\_2\_虚拟机配置]

==== 节点 2: 虚拟机配置

虚拟机配置VM配置

[source,bash,role=execute]

# Label infrastructure nodes

```
oc label node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra= oc label node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=
```

# Taint infrastructure nodes

```
oc adm taint node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra=:NoSchedule oc adm
taint node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=:NoSchedule
```



# Label worker nodes for VMs

oc label node ip-10-0-82-27.ec2.internal kubevirt.io/schedulable=true

HyperConverged::

[source,yaml]

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged metadata: name: kubevirt-
hyperconverged namespace: openshift-cnv spec: infra: nodePlacement: nodeSelector: node-
role.kubernetes.io/infra: "" tolerations: - key: node-role.kubernetes.io/infra operator: Exists effect:
NoSchedule workloads: nodePlacement: nodeSelector: kubevirt.io/schedulable: "true"
```

**\*\*::: VM:::**

```
[discrete#vm-configuration:infrastructure-node-placement::_3_]
===== 3:
```

[source,yaml]

```
apiVersion: hco.kubevirt.io/v1beta1 kind: HyperConverged metadata: name: kubevirt-
hyperconverged namespace: openshift-cnv spec: infra: nodePlacement: nodeSelector: node-
role.kubernetes.io/infra: "" affinity: nodeAffinity:
preferredDuringSchedulingIgnoredDuringExecution: - weight: 100 preference: matchExpressions: -
key: topology.kubernetes.io/zone operator: In values: - us-east-1a - us-east-1b - us-east-1c
podAntiAffinity: preferredDuringSchedulingIgnoredDuringExecution: - weight: 100
podAffinityTerm: labelSelector: matchLabels: kubevirt.io: virt-api topologyKey:
topology.kubernetes.io/zone - weight: 100 podAffinityTerm: labelSelector: matchLabels: kubevirt.io:
virt-controller topologyKey: topology.kubernetes.io/zone tolerations: - key: node-
role.kubernetes.io/infra operator: Exists effect: NoSchedule workloads: nodePlacement:
nodeSelector: kubevirt.io/schedulable: "true"
```

**\*\*::: SLA:::**

```
[discrete#vm-configuration:infrastructure-node-placement::_]
=====
```

```
[discrete#vm-configuration:infrastructure-node-placement::_hyperconverged_cr]
===== HyperConverged CR
```

[source,bash,role=execute]

oc edit hyperconverged kubevirt-hyperconverged -n openshift-cnv

```
[discrete#vm-configuration:infrastructure-node-placement::_hyperconverged_cr]
==== HyperConverged CR
```

```
[source,bash,role=execute]
```

```
oc patch hyperconverged kubevirt-hyperconverged -n openshift-cnv --type=merge -p ' { "spec": {
"infra": { "nodePlacement": { "nodeSelector": { "node-role.kubernetes.io/infra": "" } } } }'
```

```
[discrete#vm-configuration:infrastructure-node-placement::_]
====
```

```
Pod
```

```
[source,bash,role=execute]
```

# Watch pod changes in real-time

```
oc get pods -n openshift-cnv -w
```

# Verify new placement after changes settle

```
oc get pods -n openshift-cnv -o wide
```

# Check specific component placement

```
oc get pods -n openshift-cnv -l kubevirt.io=virt-api -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.nodeName}{"\n"}{end}'
```

Example:

```
* virt-handler Pod
* virt-handler DaemonSet Pod
* virt-handler 25 Pods
```

```
[discrete#vm-configuration:infrastructure-node-placement::_pod]
==== Pod
```

```
[discrete#vm-configuration:infrastructure-node-placement::_pod]
==== Pod
```

```
[source,bash,role=execute]
```

# Group pods by node

```
oc get pods -n openshift-cnv -o jsonpath='{range .items[*]}{.spec.nodeName}{"\t"}{.metadata.name}{"\n"}{end}' | sort | column -t
```

# Count pods per node

```
oc get pods -n openshift-cnv -o jsonpath='{range .items[*]}{.spec.nodeName}{"\n"}{end}' | sort |
uniq -c
```

```
[discrete#vm-configuration:infrastructure-node-placement::_toleration]
```

```
===== Toleration
```

```
[source,bash,role=execute]
```

# Check if virt-api pods have correct tolerations

```
oc get pods -n openshift-cnv -l kubevirt.io=virt-api -o jsonpath='{.items[0].spec.tolerations}' | jq
```

```
[discrete#vm-configuration:infrastructure-node-placement::_xxxxxxxxxxxxxxxxxxxx]
===== xxxxxxxxxxxxxxxxxxxxxxxx

[source,bash,role=execute]
```



# View node affinity for virt-controller

```
oc get pods -n openshift-cnv -l kubevirt.io=virt-controller -o jsonpath='{.items[0].spec.affinity}' | jq
```

```
[discrete#vm-configuration:infrastructure-node-placement::_pod]
==== pod
```

```
[discrete#vm-configuration:infrastructure-node-
placement::_podpending]
===== PodPending
```

```
:: PodPending
```

```
::
```

```
[source,bash,role=execute]
```

# Check pod events

`oc describe pod <pod-name> -n openshift-cnv`

# Look for scheduling errors

```
oc get events -n openshift-cnv --sort-by='.lastTimestamp' | grep -i failedscheduling
```

```
Warning:
```

```
* Warning: FailedScheduling: pod has no feasible execution node
```

```
* Warning: FailedScheduling: pod has no feasible execution node
```

```
* Warning: FailedScheduling: pod has no feasible execution node
```

```
Warning:
```

```
[source,bash,role=execute]
```

# Verify nodes have required labels

```
oc get nodes -L node-role.kubernetes.io/infra
```

# Check node taints

```
oc get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.taints}{"\n"}{end}'
```

# Verify node capacity

oc describe nodes | grep -A 5 "Allocated resources"

```
[discrete#vm-configuration:infrastructure-node-placement::_virt_handler]
==== virt-handler

****: virt-handler DaemonSetPod

***:

[source,bash,role=execute]
```

# Check virt-handler DaemonSet status

```
oc get daemonset virt-handler -n openshift-cnv
```

# Compare desired vs current pods

```
oc describe daemonset virt-handler -n openshift-cnv | grep -E "Desired|Current|Ready"
```

```
:: `spec.workloads.nodePlacement`
```

```
[source,bash,role=execute]
```



# View workloads node selector

```
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o
jsonpath='{.spec.workloads.nodePlacement}' | jq
```

# Verify nodes match the selector

oc get nodes --selector=<label-from-workloads-nodeSelector>

```
[discrete#vm-configuration:infrastructure-node-placement::_xxxxxxxxxxxx]
```

```
===== xxxxxxxxxxxxxxx
```

```
:: HyperConverged CRxxxxxxxxxxxxxxxxPodxxxxxxxx
```

```
HyperConverged CRxxxxxx:
```

```
[source,bash,role=execute]
```

# View HyperConverged status

`oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o yaml | grep -A 10 status:`

# Check for validation errors

```
oc get events -n openshift-cnv --sort-by='.lastTimestamp' | grep HyperConverged
```

```
[]:
```

```
[source,bash,role=execute]
```

# Add annotation to trigger reconciliation

```
oc annotate hyperconverged kubevirt-hyperconverged -n openshift-cnv reconcile-trigger="$(date +%s)" --overwrite
```

```
[discrete#vm-configuration:infrastructure-node-placement::_]
====

1. ****: virt-controller virt-
api CDI CPU 4 vCPU 8GB RAM

2. **: 2

3. **: VM

4. **: /Pod

5. **: Pod

6. **: /

7. **:

[discrete#vm-configuration:infrastructure-node-placement::_]
====

[source,bash,role=execute]
```

# Edit HyperConverged CR and remove nodePlacement sections

```
oc patch hyperconverged kubevirt-hyperconverged -n openshift-cnv --type=json -p='[{"op":
"remove", "path": "/spec/infra/nodePlacement"}, {"op": "remove", "path":
"/spec/workloads/nodePlacement"}]'
```

# Remove infrastructure node labels

```
oc label node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra- oc label node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra-
```

## Remove taints if applied

```
oc adm taint node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra:NoSchedule-
node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra:NoSchedule-
```

```
[discrete#vm-configuration:infrastructure-node-placement::_000]
==== 000
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
* OpenShift Virtualization
* HyperConverged CR
* `spec.infra.nodePlacement` `spec.workloads.nodePlacement`
* Pod
*
*
*
```

```
[discrete#vm-configuration:infrastructure-node-placement::_0000]
==== 0000
```

```
* xref:vm-configuration:node-placement-affinity[VMXXXXXXXXXXXX]
* xref:getting-started:vm-lifecycle-states[VMXXXXXXXXXX]
*
```

Link:[https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/latest/html/virtualization/postinstallation-configuration#virt-about-node-placement-virt-components\\_virt-specifying-nodes-for-virtualization-components](https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virtualization/postinstallation-configuration#virt-about-node-placement-virt-components_virt-specifying-nodes-for-virtualization-components)[XXXXXXXXXXXXXXXXXXXX,window=\_blank]

```
:!navtitle:
```

```
:docname: node-placement-affinity
:page-module: vm-configuration
:page-relative-src-path: node-placement-affinity.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle:
[#vm-configuration:node-placement-affinity]
===
```

```
[discrete#vm-configuration:node-placement-affinity::_]
====
```

OpenShift VM Kubernetes VM

# OpenShift Virtualization VM



```
[discrete#vm-configuration:node-placement-affinity::_0000]
```

==      □□□□

```
[discrete#vm-configuration:node-placement-affinity::_000000000000]
```

== ==      □ □ □ □ □ □ □ □ □ □ □ □ □ □

# OpenShift Virtualization VM 3

```
[cols="1,2,2",options="header"]
```

$$| \quad \equiv \equiv \equiv$$

Diagram showing three groups of boxes. The first group has 5 boxes, the second has 6 boxes, and the third has 2 boxes. Each box contains a number.

| □□□□□□□□

| 00000000000000000000VM000000

10000000000

| □□□□□□□□

00/0000000000000000

100

|■■■■■■■■■■/■■■■■■■■■■|

| VM VM |

| VM□□□□□□□□□□

$$\begin{array}{c} | \\ | \\ \hline \hline \hline \end{array}$$

IMPORTANT: VM VM 'virt-launcher'

# Kubernetes

```
[discrete#vm-configuration:node-placement-affinity::: 0000000000]
```

[illegible]

```
[discrete#vm-configuration:node-placement-affinity::_000000000000]
```

```
[source,bash,role=execute]
```

```
oc get nodes --show-labels
```

```
###:
```

```
[source,text]
```

| NAME                       | STATUS | ROLES                | AGE | VERSION  | LABELS                                                                              |
|----------------------------|--------|----------------------|-----|----------|-------------------------------------------------------------------------------------|
| ip-10-0-8-25.ec2.internal  | Ready  | control-plane,master | 23d | v1.31.14 | beta.kubernetes.io/arch=amd64,kubernetes.io/hostname=ip-10-0-8-25.ec2.internal,...  |
| ip-10-0-24-89.ec2.internal | Ready  | worker               | 22d | v1.31.14 | beta.kubernetes.io/arch=amd64,kubernetes.io/hostname=ip-10-0-24-89.ec2.internal,... |
| ip-10-0-43-58.ec2.internal | Ready  | worker               | 22d | v1.31.14 | beta.kubernetes.io/arch=amd64,kubernetes.io/hostname=ip-10-0-43-58.ec2.internal,... |
| ip-10-0-82-27.ec2.internal | Ready  | worker               | 22d | v1.31.14 | beta.kubernetes.io/arch=amd64,kubernetes.io/hostname=ip-10-0-82-27.ec2.internal,... |

```
[discrete#vm-configuration:node-placement-affinity::_#####]
```

```
===== #####
```

```
#####
```

```
[source,bash,role=execute]
```

# Get worker node names

```
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}') WORKER2=$(oc get nodes -l node-role.kubernetes.io/worker
-o jsonpath='{.items[1].metadata.name}') WORKER3=$(oc get nodes -l node-
role.kubernetes.io/worker -o jsonpath='{.items[2].metadata.name}')
```

# Label nodes by environment

```
oc label node $WORKER1 env=development oc label node $WORKER2 env=production
```

# Label nodes by GPU type

```
oc label node $WORKER1 gpu=nvidia-t4 oc label node $WORKER2 gpu=nvidia-a100
```

# Label nodes by disk type

```
oc label node $WORKER1 storage=ssd oc label node $WORKER3 storage=nvme
```

# Label nodes by workload type

oc label node \$WORKER1 workload=database

```
#####
```

```
[source,bash,role=execute]
```

```
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}') oc get node $WORKER1 -o jsonpath='{.metadata.labels}' | jq
```

```
{}:
```

```
[source,json]
```

```
{ "beta.kubernetes.io/arch": "amd64", "beta.kubernetes.io/os": "linux", "env": "development", "gpu":
"nvidia-t4", "kubernetes.io/arch": "amd64", "kubernetes.io/hostname": "ip-10-0-24-89.ec2.internal",
"kubernetes.io/os": "linux", "node-role.kubernetes.io/worker": "", "storage": "ssd", "workload":
"database" }
```

```
[discrete#vm-configuration:node-placement-affinity::_#####]
```

```
===== #####
```

```
[source,bash,role=execute]
```

```
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}') oc label node $WORKER1 gpu-
```

```
[discrete#vm-configuration:node-placement-affinity::_#####]
```

```
===== #####
```

```
#####VM#####
```

```
[discrete#vm-configuration:node-placement-affinity::_#####]
```

```
===== #####
```

```
#####
```

```
[source,bash,role=execute]
```

oc create namespace vm-placement

```
[discrete#vm-configuration:node-placement-affinity::_#####]
```

```
===== #####
```

```
`storage=ssd` VM
```

```
WARNING: cloud-initSSH
```

```
link:attachment$node-placement-affinity/vm-node-selector.yaml[Download vm-node-selector.yaml,role=unresolved]
```

```
[source,yaml]
```

```
apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: database-vm namespace: vm-
placement spec: runStrategy: Manual template: metadata: labels: app: database spec: nodeSelector:
storage: ssd # VM will only run on nodes with storage=ssd label domain: cpu: cores: 2 memory:
guest: 4Gi devices: disks: - name: rootdisk disk: bus: virtio - name: cloudinitdisk disk: bus: virtio
volumes: - name: rootdisk containerDisk: image: quay.io/containerdisks/fedora:43 - name:
cloudinitdisk cloudInitNoCloud: userData: | #cloud-config user: fedora password: fedora chpasswd:
{ expire: False }
```

```
VM
```

```
[source,bash,role=execute]
```

```
oc apply -f vm-node-selector.yaml virtctl start database-vm -n vm-placement
```

```
[discrete#vm-configuration:node-placement-affinity::_]
=====
```

```
VM
```

```
[source,bash,role=execute]
```

```
oc get vmi database-vm -n vm-placement -o wide
```

```
IP:
[source,text]
```

```
NAME AGE PHASE IP NODENAME READY database-vm 45s Running 10.128.0.200 ocplab02 True
```

```
[source,bash,role=execute]
```

```
oc get vmi database-vm -n vm-placement -o jsonpath='{.status.nodeName}' | xargs oc get node -L
storage
```

```
IP:
```







```
link:attachment$node-placement-affinity/vm-pod-affinity.yaml[Download vm-pod-
affinity.yaml,role=unresolved]
```

```
[source,yaml]
```

```
apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: web-frontend-vm namespace:
vm-placement spec: runStrategy: Manual template: metadata: labels: app: web-frontend tier:
frontend spec: affinity: podAffinity: requiredDuringSchedulingIgnoredDuringExecution: -
labelSelector: matchExpressions: - key: app operator: In values: - database topologyKey:
kubernetes.io/hostname domain: cpu: cores: 2 memory: guest: 2Gi devices: disks: - name: rootdisk
disk: bus: virtio - name: cloudinitdisk disk: bus: virtio volumes: - name: rootdisk containerDisk:
image: quay.io/containerdisks/fedora:43 - name: cloudinitdisk cloudInitNoCloud: userData: |
#cloud-config user: fedora password: fedora chpasswd: { expire: False }
```

```
VM`app=database` /VM`topologyKey: kubernetes.io/hostname`
```

```
:
```

```
[cols="1,2",options="header"]
```

```
|===
```

```
| |
```

```
|`kubernetes.io/hostname`
```

```
|
```

```
|`topology.kubernetes.io/zone`
```

```
|
```

```
|`topology.kubernetes.io/region`
```

```
|
```

```
|===
```

```
[discrete#vm-configuration:node-placement-affinity::_vm]
```

```
==== : VM
```

```
VM
```

```
link:attachment$node-placement-affinity/vm-pod-anti-affinity.yaml[Download vm-pod-
anti-affinity.yaml,role=unresolved]
```

```
[source,yaml]
```

```
apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: cache-vm-01 namespace: vm-
placement spec: runStrategy: Manual template: metadata: labels: app: redis-cache role: cache spec:
affinity: podAntiAffinity: requiredDuringSchedulingIgnoredDuringExecution: - labelSelector:
matchExpressions: - key: app operator: In values: - redis-cache topologyKey:
kubernetes.io/hostname domain: cpu: cores: 2 memory: guest: 4Gi devices: disks: - name: rootdisk
disk: bus: virtio - name: cloudinitdisk disk: bus: virtio volumes: - name: rootdisk containerDisk:
```

image: quay.io/containerdisks/fedora:43 - name: cloudinitdisk cloudInitNoCloud: userData: |  
#cloud-config user: fedora password: fedora chpasswd: { expire: False }

```
#####VM#####

[discrete#vm-configuration:node-placement-affinity::_#####]
===== #####

#####:

[source,yaml]
```

spec: template: spec: affinity: podAntiAffinity: preferredDuringSchedulingIgnoredDuringExecution:  
- weight: 100 podAffinityTerm: labelSelector: matchExpressions: - key: app operator: In values: -  
redis-cache topologyKey: kubernetes.io/hostname

```
[discrete#vm-configuration:node-placement-affinity::_#####]
===== #####

[source,yaml]
```

spec: template: spec: affinity: nodeAffinity: requiredDuringSchedulingIgnoredDuringExecution:  
nodeSelectorTerms: - matchExpressions: - key: env operator: In values: - production podAffinity:  
preferredDuringSchedulingIgnoredDuringExecution: - weight: 100 podAffinityTerm: labelSelector:  
matchExpressions: - key: tier operator: In values: - database topologyKey: kubernetes.io/hostname  
podAntiAffinity: requiredDuringSchedulingIgnoredDuringExecution: - labelSelector:  
matchExpressions: - key: app operator: In values: - web-frontend topologyKey:  
kubernetes.io/hostname

```
#####VM#####:

* #####
* #####VM#####
* #####Web#####VM#####

[discrete#vm-configuration:node-placement-affinity::_#####]
===== #####

[discrete#vm-configuration:node-placement-affinity::_####_1_#####]
===== #### 1: #####

#####VM#####:

[source,yaml]
```

apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: postgres-vm-01 labels: app:  
postgres-cluster instance: "01" spec: template: metadata: labels: app: postgres-cluster spec: affinity:

podAntiAffinity: requiredDuringSchedulingIgnoredDuringExecution: - labelSelector: matchLabels:  
app: postgres-cluster topologyKey: kubernetes.io/hostname

```
#####VM#####
```

```
[discrete#vm-configuration:node-placement-affinity::_2_gpu]
===== 2: GPU
```

```
GPU:
```

```
[source,yaml]
```

spec: template: spec: nodeSelector: gpu: "true" tolerations: - key: "nvidia.com/gpu" operator: "Exists"  
effect: "NoSchedule"

```
#####/Taints/Tolerations#####GPUGPU#####
```

```
[discrete#vm-configuration:node-placement-affinity::_3_]
===== 3:
```

```
:
```

```
[source,yaml]
```



# Check VM status

```
oc get vm -n vm-placement
```

# Check VMI events

oc describe vmi <vm-name> -n vm-placement

```
Warning FailedScheduling 0/3 nodes are available: 3 node(s) didn't match Pod's node affinity/selector
```

Warning FailedScheduling 0/3 nodes are available: 3 node(s) didn't match Pod's node affinity/selector

```
Warning:
* node(s) didn't match Pod's node affinity/selector
* node(s) didn't match Pod's node affinity/selector
* node(s) didn't match Pod's node affinity/selector

Warning:
```

[source,bash,role=execute]



# Verify node labels exist

```
oc get nodes -L storage,env,gpu
```

# Check if nodes have required labels

```
oc get nodes --selector=storage=ssd
```

# Review VM placement constraints

```
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 20 "affinity:"
```

```
nodeSelector:

* nodeSelector
* nodeSelector
* nodeSelector

[discrete#vm-configuration:node-placement-affinity::_vm]
===== VM

:: VM

::

* nodeSelector
* nodeSelector
* VM

::

[source,bash,role=execute]
```

# Check where VM is running

```
oc get vmi <vm-name> -n vm-placement -o jsonpath='{.status.nodeName}'
```

# Check that node's labels

oc get node <node-name> --show-labels

```
~:

* ~~~~~
* ~~~~~
* ~~~~~

[discrete#vm-configuration:node-placement-affinity::_~~~~~]
===== ~~~~~

~: ~~~~~VM~~~~~

~:

[source,bash,role=execute]
```

# List all VMs and their nodes

```
oc get vmi -n vm-placement -o wide
```

# Check anti-affinity configuration

```
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 15 "podAntiAffinity:"
```

```
podAntiAffinity:
```

```
* **podAntiAffinity**: podAntiAffinity: VM podAntiAffinity:
* **podAntiAffinity**: podAntiAffinity:
* **podAntiAffinity VS podAntiAffinity: podAntiAffinity:
* **podAntiAffinity**: podAntiAffinity:
```

```
podAntiAffinity:
```

```
podAntiAffinity:
```

```
[source,bash,role=execute]
```

# Check VM labels

```
oc get vm <vm-name> -n vm-placement -o jsonpath='{.spec.template.metadata.labels}' | jq
```



## Verify anti-affinity selector matches those labels

```
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 5 "labelSelector:"
```

```
[discrete#vm-configuration:node-placement-affinity::_]
=====
**:: VM
**::
[source,bash,role=execute]
```

# View VM spec

```
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 5 "nodeSelector:"
```

# Verify it's under spec.template.spec, not just spec

```
oc get vm <vm-name> -n vm-placement -o jsonpath='{.spec.template.spec.nodeSelector}' | jq
```

```
YAML:
```

```
[source,yaml]
```

# WRONG - nodeSelector at VM spec level

spec: nodeSelector: # This is ignored! storage: ssd template: spec: domain: ...

## CORRECT - nodeSelector at pod spec level

```
spec: template: spec: nodeSelector: # This works! storage: ssd domain: ...
```

[illegible]

# Delete VMs

```
oc delete vm --all -n vm-placement
```

# Remove custom node labels from all workers

```
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o jsonpath='{.items[0].metadata.name}') WORKER2=$(oc get nodes -l node-role.kubernetes.io/worker -o jsonpath='{.items[1].metadata.name}') WORKER3=$(oc get nodes -l node-role.kubernetes.io/worker -o jsonpath='{.items[2].metadata.name}')
```

```
oc label node $WORKER1 storage- env- gpu- workload- oc label node $WORKER2 storage- env- gpu- oc label node $WORKER3 storage-
```

```
#####:
```

```
[source,bash,role=execute]
```

```
oc get vm -n vm-placement oc get nodes --show-labels | grep -E "storage|workload"
```

```
[discrete#vm-configuration:node-placement-affinity::_]
====
```

```
#####:
```

```
* #####/#####
* #####
* #####
* #####
* VM#####/#####
* HA#####
* #####
```

```
[discrete#vm-configuration:node-placement-affinity::_]
====
```

```
* xref:getting-started:vm-lifecycle-states[] - VM#####
* xref:getting-started:virtctl-basics[] - VM#####CLI#####
*
```

```
link:https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virtualization/nodes#virt-node-placement-vm[##### - Red Hat #####,window=_blank]
* link:https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/[##### - Kubernetes #####,window=_blank]
* link:https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/[##### - Kubernetes #####,window=_blank]
```

```
:!navtitle:
```

```
:docname: hotplug-volumes-interfaces
```

```

:page-module: vm-configuration
:page-relative-src-path: hotplug-volumes-interfaces.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/ocp-virt-cookbook_ja
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle:
[#vm-configuration:hotplug-volumes-interfaces]
===

[discrete#vm-configuration:hotplug-volumes-interfaces::_]
====

VM

```

`virtctl addvolume` PVC VM OS `virtctl  
removevolume`

```

[discrete#vm-configuration:hotplug-volumes-interfaces::_]
====

* **OpenShift 4.18** OpenShift Virtualization
* **`system:admin`** (`system:admin`)
* **`oc` CLI** **`virtctl` CLI**
* **StorageClass**

:

```

## OpenShift 4.20, 4.21

```

[discrete#vm-configuration:hotplug-volumes-interfaces::_1_vm]
==== 1: VM

VM

[source,bash,role=execute]

```

## oc create namespace hotplug-demo

```

WARNING: cloud-init
link:attachment$hotplug-volumes-interfaces/vm-hotplug.yaml[vm-
hotplug.yaml,role=unresolved]

[source,bash,role=execute]

```



```
oc apply -f - <<EOF apiVersion: kubevirt.io/v1 kind: VirtualMachine metadata: name: hotplug-vm
namespace: hotplug-demo spec: runStrategy: Always template: metadata: labels:
kubevirt.io/domain: hotplug-vm spec: domain: devices: disks: - name: rootdisk disk: bus: virtio -
name: cloudinitdisk disk: bus: virtio interfaces: - name: default masquerade: {} resources: requests:
memory: 1Gi networks: - name: default pod: {} volumes: - name: rootdisk containerDisk: image:
quay.io/containerdisks/fedora:latest - name: cloudinitdisk cloudInitNoCloud: userData: | #cloud-
config password: redhat chpasswd: expire: false EOF
```

```
VM
[source,bash,role=execute]
```

```
oc get vm -n hotplug-demo -w
```

```
`STATUS=Running` `READY=True` `Ctrl+C`
[source,bash,role=execute]
```

```
oc get vmi hotplug-vm -n hotplug-demo -o jsonpath='{range .status.volumeStatus[*]}{.name}:
{.target}{"\n"}{end}'
```

```
:
```

```
rootdisk: vda cloudinitdisk: vdb
```

```
[discrete#vm-configuration:hotplug-volumes-interfaces::_2_pvc]
=== 2: PVC
VMPersistentVolumeClaim
link:attachment$hotplug-volumes-interfaces/hotplug-pvc.yaml[hotplug-
pvc.yaml,role=unresolved]
[source,bash,role=execute]
```

```
oc apply -f - <<EOF apiVersion: v1 kind: PersistentVolumeClaim metadata: name: hotplug-disk
namespace: hotplug-demo spec: accessModes: - ReadWriteOnce resources: requests: storage: 5Gi
storageClassName: lvms-vg1 volumeMode: Block EOF
```

```
IMPORTANT: `volumeMode: Block` FilesystemPVC`disk.img`
PVC`WaitForFirstConsumer` `Pending`
```

```
[source,bash,role=execute]
```

```
oc get pvc hotplug-disk -n hotplug-demo
```

```
[discrete#vm-configuration:hotplug-volumes-interfaces::_03_000000000000]
==== 03: 000000000000

'virtctl addvolume' 0000000000VM0PVC0000000000

[source,bash,role=execute]
```

```
[source,bash,role=execute]
```

```
virtctl addvolume hotplug-vm --volume-name=hotplug-disk --namespace=hotplug-demo --persist
```

Successfully submitted addvolume request to VM hotplug-vm for volume hotplug-disk

```
'--persist' VM
[discrete#vm-configuration:hotplug-volumes-interfaces::_0_4_]
=== 0 4:
VMI
[source,bash,role=execute]
```

```
[source,bash,role=execute]
```

```
oc get vmi hotplug-vm -n hotplug-demo -o jsonpath='{range .status.volumeStatus[*]}{.name}:
{.target} (hotplugVolume: {.hotplugVolume}){"\n"}{end}'
```

```
rootdisk: vda (hotplugVolume:) cloudinitdisk: vdb (hotplugVolume:) hotplug-disk: sda
(hotplugVolume: {"attachPodName":"...", "attachPodUID":"..."})
```

```
'Ready' SCSI ('sda') hotplugVolume`
Pod

PVC

[source,bash,role=execute]
```

```
oc get pvc hotplug-disk -n hotplug-demo
```

root@localhost:~#

```
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS AGE hotplug-disk Bound pvc-xxxx 5Gi RWO lvms-vg1 1m
```

root@localhost:~# vmtoolsd --add --disk /dev/pvc-xxxx

[source,bash,role=execute]

virtctl console hotplug-vm -n hotplug-demo

root@fedora:~# root@redhat:~# root@redhat:~#

[source,bash]

# lsblk

PVCをフォーマットするコマンド: `sda`をフォーマットする `Ctrl+]` を入力する

**ステップ 5:** ディレクトリを作成する

VMを再起動する

```
sudo mkfs.xfs /dev/sda
```

ディレクトリを作成する

```
sudo mkdir -p /mnt/hotplug
sudo mount /dev/sda /mnt/hotplug
```

ファイルを作成する

```
echo "Hotplug test" | sudo tee /mnt/hotplug/test.txt
```

```
cat /mnt/hotplug/test.txt
```

確認:

```
Hotplug test
```

ディレクトリを削除する

```
sudo umount /mnt/hotplug
```

`Ctrl+]` を入力する

**ステップ 6:** VMを再起動する

VMを再起動する

```
virtctl removevolume hotplug-vm --volume-name=hotplug-disk --namespace=hotplug-demo
```

確認:

```
Successfully submitted removevolume request to VM hotplug-vm for volume hotplug-disk
```



このセクションでは、hotplug-disk を作成し、VM に追加する方法について説明します。

まず、hotplug-disk の作成を確認します。

```
oc get vmi hotplug-vm -n hotplug-demo -o jsonpath='{range
.status.volumeStatus[*]}.{.name}: {.target}}{"\n"}{end}'
```

実行結果:

```
rootdisk: vda
cloudinitdisk: vdb
```

hotplug-disk は、PVC から作成されたディスクです。

```
oc get pvc hotplug-disk -n hotplug-demo
```

hotplug-disk の作成を確認します。

**--persist** オプション

| オプション            | 説明                        |
|------------------|---------------------------|
| <b>--persist</b> | VM spec に VM spec を追加します。 |
| (デフォルト)          | VM spec に VM spec を追加します。 |

hotplug-disk を作成するときに **--persist** オプションを使用すると、ディスクは VM spec に追加されます。

hotplug-disk の作成を確認します。

hotplug-disk の作成を確認します:

- PersistentVolumeClaims (PVCs)
- DataVolumes

hotplug-disk の作成を確認します:

- containerDisk オプション
- cloudInitNoCloud / cloudInitConfigDrive オプション
- emptyDir オプション

hotplug-disk の作成を確認します。

hotplug-disk の作成を確認します。SCSI ディスクは **sda**、**sdb**。

hotplug-disk の作成を確認します。virtio ディスクは PCI ディスクとして SCSI ディスクとして作成されます。



- VM
- `virtctl removevolume`
- 

- [Hotplug Volumes - KubeVirt User Guide](#)
- [Hotplug Network Interfaces - KubeVirt User Guide](#)
- 

VirtualMachines
VirtualMachine

- (vCPU RAM )
- (CPU )
- 
- 
- 

OpenShift Virtualization

VirtualMachine
OpenShift Virtualization
rhel9-server-large
fedora-desktop-medium
VM

VirtualMachine

OpenShift 4.21

- 
- 1:
- 2:
- 3:
- 4:

- [OpenShift](#)
- [OpenShift](#)

## OpenShift

OpenShift is a Kubernetes-based container management platform.

- `bash` shell (or `zsh` if installed)
- OpenShift CLI `oc`
- OpenShift Virtualization (OpenShift Container Engine)
- OpenShift Container Engine IDE (OpenShift Container Engine)

## OpenShift 1: OpenShift Container Engine

OpenShift Container Engine is a Kubernetes-based container management platform.

OpenShift Container Engine `templates` are used to create OpenShift Container Engine

```
oc get templates -n openshift
```

OpenShift Container Engine `templates` are used to create OpenShift Container Engine `templates` OS `T` OpenShift Container Engine

`rhel` OpenShift Container Engine `templates` are used to create OpenShift Container Engine

```
...
rhel8-desktop-tiny Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-large Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-medium Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-small Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-tiny Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-large Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-medium Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-small Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-tiny Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-large Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-medium Template for Red Hat Enterprise Linux 9
```



|                                          |                                         |   |
|------------------------------------------|-----------------------------------------|---|
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-desktop-small                      | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-desktop-tiny                       | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-highperformance-large              | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-highperformance-medium             | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-highperformance-small              | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-highperformance-tiny               | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-server-large                       | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-server-medium                      | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-server-small                       | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| rhel9-server-tiny                        | Template for Red Hat Enterprise Linux 9 |   |
| VM or newer. A PVC with the RHEL disk... | 4 (2 generated)                         | 1 |
| ...                                      |                                         |   |

openshift `rhel9-server-medium` `-o yaml`

```
oc get template rhel9-server-medium -n openshift -o yaml > rhel9-server.template.yaml
```

`rhel9-server.template.yaml` `creationTimestamp` `uid` `resourceVersion`

```
apiVersion: template.openshift.io/v1
kind: Template
metadata:
 annotations:
 defaults.template.kubevirt.io/disk: rootdisk
 description: Template for Red Hat Enterprise Linux 9 VM or newer. A PVC with the
 RHEL disk image must be available.
 iconClass: icon-rhel
 name.os.template.kubevirt.io/rhel9.0: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.1: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.2: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.3: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.4: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.5: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.6: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.7: Red Hat Enterprise Linux 9.0 or higher
 openshift.io/display-name: Red Hat Enterprise Linux 9 VM
 openshift.io/documentation-url: https://github.com/kubevirt/common-templates
 openshift.io/provider-display-name: Red Hat
 openshift.io/support-url: https://github.com/kubevirt/common-templates/issues
```

```

openshift.kubevirt.io/pronounceable-suffix-for-name-expression: "true"
operator-sdk/primary-resource: openshift-cnv/ssp-kubevirt-hyperconverged
operator-sdk/primary-resource-type: SSP.ssp.kubevirt.io
tags: hidden,kubevirt,virtualmachine,linux,rhel
template.kubevirt.io/containerdisks: |
 registry.redhat.io/rhel9/rhel-guest-image
template.kubevirt.io/editable: |
 /objects[0].spec.template.spec.domain.cpu.sockets
 /objects[0].spec.template.spec.domain.cpu.cores
 /objects[0].spec.template.spec.domain.cpu.threads
 /objects[0].spec.template.spec.domain.memory.guest
 /objects[0].spec.template.spec.domain.devices.disks
 /objects[0].spec.template.spec.volumes
 /objects[0].spec.template.spec.networks
template.kubevirt.io/images: |
 https://access.redhat.com/downloads/content/479/ver=/rhel---
9/9.0/x86_64/product-software
template.kubevirt.io/provider: Red Hat
template.kubevirt.io/provider-support-level: Full
template.kubevirt.io/provider-url: https://www.redhat.com
template.kubevirt.io/version: v1alpha1
template.openshift.io/bindable: "false"
labels:
 app.kubernetes.io/component: templating
 app.kubernetes.io/managed-by: ssp-operator
 app.kubernetes.io/name: common-templates
 app.kubernetes.io/part-of: hyperconverged-cluster
 app.kubernetes.io/version: 4.21.1
 flavor.template.kubevirt.io/medium: "true"
 os.template.kubevirt.io/rhel9.0: "true"
 os.template.kubevirt.io/rhel9.1: "true"
 os.template.kubevirt.io/rhel9.2: "true"
 os.template.kubevirt.io/rhel9.3: "true"
 os.template.kubevirt.io/rhel9.4: "true"
 os.template.kubevirt.io/rhel9.5: "true"
 os.template.kubevirt.io/rhel9.6: "true"
 os.template.kubevirt.io/rhel9.7: "true"
 template.kubevirt.io/architecture: amd64
 template.kubevirt.io/type: base
 template.kubevirt.io/version: v0.34.1
 workload.template.kubevirt.io/server: "true"
name: rhel9-server-medium
namespace: openshift
objects:
- apiVersion: kubevirt.io/v1
 kind: VirtualMachine
 metadata:
 annotations:
 vm.kubevirt.io/validations: |
 [
 {

```

```

 "name": "minimal-required-memory",
 "path": "jsonpath::.spec.domain.memory.guest",
 "rule": "integer",
 "message": "This VM requires more memory.",
 "min": 1610612736
 }
]
labels:
 app: ${NAME}
 kubevirt.io/dynamic-credentials-support: "true"
 vm.kubevirt.io/template: rhel9-server-medium
 vm.kubevirt.io/template.revision: "1"
 vm.kubevirt.io/template.version: v0.34.1
name: ${NAME}
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: ${NAME}
 spec:
 sourceRef:
 kind: DataSource
 name: ${DATA_SOURCE_NAME}
 namespace: ${DATA_SOURCE_NAMESPACE}
 storage:
 resources:
 requests:
 storage: 30Gi
runStrategy: Halted
template:
 metadata:
 annotations:
 vm.kubevirt.io/flavor: medium
 vm.kubevirt.io/os: rhel9
 vm.kubevirt.io/workload: server
 labels:
 kubevirt.io/domain: ${NAME}
 kubevirt.io/size: medium
 spec:
 architecture: amd64
 domain:
 cpu:
 cores: 1
 sockets: 1
 threads: 1
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk

```

```

 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 model: virtio
 name: default
 rng: {}
 features:
 smm:
 enabled: true
 firmware:
 bootloader:
 efi: {}
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 terminationGracePeriodSeconds: 180
 volumes:
 - dataVolume:
 name: ${NAME}
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: cloud-user
 password: ${CLOUD_USER_PASSWORD}
 chpasswd: { expire: False }
 name: cloudinitdisk
 parameters:
 - description: VM name
 from: rhel9-[a-z0-9]{16}
 generate: expression
 name: NAME
 - description: Name of the DataSource to clone
 name: DATA_SOURCE_NAME
 value: rhel9
 - description: Namespace of the DataSource
 name: DATA_SOURCE_NAMESPACE
 value: openshift-virtualization-os-images
 - description: Randomized password for the cloud-init user cloud-user
 from: '[a-z0-9]{4}-[a-z0-9]{4}-[a-z0-9]{4}'
 generate: expression
 name: CLOUD_USER_PASSWORD

```

parameters

- **NAME** - 名前空間名 VM 名前空間名[a-z0-9] 名前空間名VM 名前空間名{16}名前空間名
- **DATA\_SOURCE\_NAME** - VM 名前空間名 DataSource名前空間名
- **DATA\_SOURCE\_NAMESPACE** - DataSource 名前空間名 openshift-virtualization-os-images 名前空間名
- **CLOUD\_USER\_PASSWORD** - 名前空間名 cloud-user 名前空間名NAME 名前空間名4 名前空間名3 名前空間名



- 名前空間名
- 名前bash 名前空間名

rhel9-server.template.yaml 名前空間名oc process 名前空間名

```
oc process -f rhel9-server.template.yaml -o yaml
```

名前空間名 yaml 名前空間名 List 名前空間名 VirtualMachine 名前空間名



- OpenShift 名前空間名1 名前空間名 Kubernetes 名前空間名
- OpenShift Virtualization 名前空間名 VirtualMachine 名前空間名

名前空間名 2 名前空間名

名前空間名 2: 名前空間名 VirtualMachine 名前空間名

名前空間名CLI 名前空間名 VirtualMachine 名前空間名oc process 名前空間名 oc create (名前空間名 oc apply) 名前空間名

名前空間名RHEL 9 VirtualMachine 名前空間名

```
oc new-project template-example
```

名前空間名rhel9-server.template.yaml 名前空間名 oc apply 名前空間名 VirtualMachine 名前空間名 oc create 名前空間名

```
oc process -f rhel9-server.template.yaml | oc create -n template-example -f -
```

RHEL 9 名前空間名

```
oc get all -n template-example
```

DataVolume 名前空間名 PendingPopulation 名前空間名VirtualMachine 名前空間名 Stopped 名前空間名

rhel9-server VM

```
export RHEL9_SERVER_VM=$(oc get vm -o custom-columns=:metadata.name --no-headers -n
template-example)
```

VM rhel9-abcdef0987654321

```
echo $RHEL9_SERVER_VM
```

[illegible]

```
virtctl start $RHEL9_SERVER_VM
```

```
oc get vm -w VM Stopped Provisioning Running
```

VM 00000000000000000000000000000000 0000 4 0000000000

□□□□ **3:** □□□□□□□□□□□□□□

**rhe19**

□□□□ 2 □□□□□□□□□□/□□□□□□□□□□□□

oc project template-example

```
rhel9-server.template.yaml ██████████ rhel9-custom.template.yaml ████████████████████
```

```
cp rhel9-{server,custom}.template.yaml
```

```
#####rhel9-custom.template.yaml#####
```

```
apiVersion: template.openshift.io/v1
kind: Template
metadata:
 annotations:
 defaults.template.kubevirt.io/disk: rootdisk
 description: Template for Red Hat Enterprise Linux 9 VM or newer. A PVC with the
 RHEL disk image must be available.
 iconClass: icon-rhel
 name.os.template.kubevirt.io/rhel9.0: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.1: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.2: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.3: Red Hat Enterprise Linux 9.0 or higher
```

```

name.os.template.kubevirt.io/rhel9.4: Red Hat Enterprise Linux 9.0 or higher
name.os.template.kubevirt.io/rhel9.5: Red Hat Enterprise Linux 9.0 or higher
name.os.template.kubevirt.io/rhel9.6: Red Hat Enterprise Linux 9.0 or higher
name.os.template.kubevirt.io/rhel9.7: Red Hat Enterprise Linux 9.0 or higher
openshift.io/display-name: Red Hat Enterprise Linux 9 VM
openshift.io/documentation-url: https://github.com/kubevirt/common-templates
openshift.io/provider-display-name: Red Hat
openshift.io/support-url: https://github.com/kubevirt/common-templates/issues
openshift.kubevirt.io/pronounceable-suffix-for-name-expression: "true"
operator-sdk/primary-resource: openshift-cnv/ssp-kubevirt-hyperconverged
operator-sdk/primary-resource-type: SSP.ssp.kubevirt.io
tags: hidden,kubevirt,virtualmachine,linux,rhel
template.kubevirt.io/containerdisks: |
 registry.redhat.io/rhel9/rhel-guest-image
template.kubevirt.io/editable: |
 /objects[0].spec.template.spec.domain.cpu.sockets
 /objects[0].spec.template.spec.domain.cpu.cores
 /objects[0].spec.template.spec.domain.cpu.threads
 /objects[0].spec.template.spec.domain.memory.guest
 /objects[0].spec.template.spec.domain.devices.disks
 /objects[0].spec.template.spec.volumes
 /objects[0].spec.template.spec.networks
template.kubevirt.io/images: |
 https://access.redhat.com/downloads/content/479/ver=/rhel---
9/9.0/x86_64/product-software
template.kubevirt.io/provider: Red Hat
template.kubevirt.io/provider-support-level: Full
template.kubevirt.io/provider-url: https://www.redhat.com
template.kubevirt.io/version: v1alpha1
template.openshift.io/bindable: "false"
labels:
 app.kubernetes.io/component: templating
 app.kubernetes.io/managed-by: ssp-operator
 app.kubernetes.io/name: common-templates
 app.kubernetes.io/part-of: hyperconverged-cluster
 app.kubernetes.io/version: 4.21.1
 flavor.template.kubevirt.io/medium: "true"
 os.template.kubevirt.io/rhel9.0: "true"
 os.template.kubevirt.io/rhel9.1: "true"
 os.template.kubevirt.io/rhel9.2: "true"
 os.template.kubevirt.io/rhel9.3: "true"
 os.template.kubevirt.io/rhel9.4: "true"
 os.template.kubevirt.io/rhel9.5: "true"
 os.template.kubevirt.io/rhel9.6: "true"
 os.template.kubevirt.io/rhel9.7: "true"
 template.kubevirt.io/architecture: amd64
 template.kubevirt.io/type: base
 template.kubevirt.io/version: v0.34.1
 workload.template.kubevirt.io/server: "true"
name: rhel9-custom
namespace: openshift

```

```

objects:
- apiVersion: kubevirt.io/v1
 kind: VirtualMachine
 metadata:
 annotations:
 vm.kubevirt.io/validations: |
 [
 {
 "name": "minimal-required-memory",
 "path": "jsonpath::.spec.domain.memory.guest",
 "rule": "integer",
 "message": "This VM requires more memory.",
 "min": 1610612736
 }
]
 labels:
 app: ${NAME}
 kubevirt.io/dynamic-credentials-support: "true"
 vm.kubevirt.io/template: rhel9-server-medium
 vm.kubevirt.io/template.revision: "1"
 vm.kubevirt.io/template.version: v0.34.1
 name: ${NAME}
 spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: ${NAME}
 spec:
 sourceRef:
 kind: DataSource
 name: ${DATA_SOURCE_NAME}
 namespace: ${DATA_SOURCE_NAMESPACE}
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: Always
 template:
 metadata:
 annotations:
 vm.kubevirt.io/flavor: medium
 vm.kubevirt.io/os: rhel9
 vm.kubevirt.io/workload: server
 labels:
 kubevirt.io/domain: ${NAME}
 kubevirt.io/size: medium
 spec:
 architecture: amd64
 domain:
 cpu:

```



```

 cores: 1
 sockets: 1
 threads: 1
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 model: virtio
 name: default
 rng: {}
 features:
 smm:
 enabled: true
 firmware:
 bootloader:
 efi: {}
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 terminationGracePeriodSeconds: 180
 volumes:
 - dataVolume:
 name: ${NAME}
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: ${CLOUD_INIT_USERNAME}
 password: ${CLOUD_INIT_PASSWORD}
 chpasswd: { expire: False }
 name: cloudinitdisk
 parameters:
 - description: VM name
 from: rhel9-custom-[a-z0-9]{6}
 generate: expression
 name: NAME
 - description: Name of the DataSource to clone
 name: DATA_SOURCE_NAME
 value: rhel9
 - description: Namespace of the DataSource
 name: DATA_SOURCE_NAMESPACE
 value: openshift-virtualization-os-images
 - description: Name of the user account created by cloud-init

```

```

name: CLOUD_INIT_USERNAME
value: shadowman
- description: Randomized password for the cloud-init user cloud-user
 from: '[a-z0-9]{4}-[a-z0-9]{4}-[a-z0-9]{4}'
 generate: expression
 name: CLOUD_INIT_PASSWORD

```

다음은 `rhel9-server-medium` 템플릿의 예시입니다.

- **metadata.name** - 템플릿의 이름 `rhel9-custom`입니다.
- **objects[0].spec.runStrategy** - 실행 전략으로 `Halted` 또는 `Always`를 지정합니다.
- **NAME** - `NAME` 템플릿의 이름 `rhel9-custom-`입니다.
- **CLOUD\_INIT\_USERNAME** - `cloud-user`로 설정된 사용자 이름입니다. `cloud-init`가 이 사용자 이름으로 로그인합니다.
- **CLOUD\_INIT\_PASSWORD** - `cloud-user`의 비밀번호입니다.

이 예제를 `yaml` 형식으로 `rhel9-custom.template.yaml`에 저장합니다.

다음은 `oc process` 명령을 사용하여 템플릿을 적용하는 예시입니다.

```
oc process -f rhel9-custom.template.yaml -o yaml
```

이 예제를 사용하여 `4`개의 VM을 생성합니다.

**4:** VM을 생성합니다.

다음은 `2`개의 VM을 생성하는 예시입니다.

`rhel9-custom` 템플릿을 사용하여 `VirtualMachine` 객체를 생성합니다.

```
oc process -f rhel9-custom.template.yaml | oc apply -n template-example -f -
```

다음은 `rhel9-custom` VM을 생성하고 `oc get vm` 명령을 사용하여 VM을 확인하는 예시입니다.

```
oc get vm -n template-example
```

다음은 `rhel9-custom` VM을 생성하는 예시입니다.

```
export RHEL9_CUSTOM_VM=$(oc get vm -o custom-columns=:metadata.name --no-headers -n
template-example | grep custom)
```

이 예제를 사용하여 `rhel9` VM을 생성하고 `$RHEL9_CUSTOM_VM` 및 `RHEL9_SERVER_VM` 변수를 설정합니다.

VM의 `yaml` 형식으로 `cloud-init` 데이터를 가져옵니다.

```
oc describe vm $RHEL9_CUSTOM_VM | grep password
```

virtctl VM username VM shadowman cloud-user

```
virtctl console $RHEL9_CUSTOM_VM
```

ENTER [CTRL+]



- OpenShift Virtualization openshift

kube default OpenShift

```
oc project default
```

```
oc delete namespace template-example
```

- VirtualMachine
- OpenShift Virtualization VirtualMachine
- VirtualMachine
- VirtualMachine
- VirtualMachine

# VM 管理の基礎

## 概要

OpenShift Virtualization は、Kubernetes 上で VM を実行するためのプラットフォームです。CPU、メモリ、NUMA などのリソースを適切に管理する必要があります。

## 考慮点

- CPU/メモリ (requests) と (limits) の QoS の設定
- vCPU と CPU の pinning (pinning) の設定
- Hugepage と NUMA の設定
- (worst-case latency) の考慮

## 設定

VM の設定は、Kubernetes の YAML ファイルで行います。

- OpenShift 4.18+ では OpenShift Virtualization を使用します。
- Kubernetes の設定を確認します。
- CLI ツール (`oc`, `virtctl`, `kubectrl`) を使用します。
- CPU Manager と Hugepages の設定を確認します。

## トラブルシューティング

### QoS

CPU/メモリ (requests) と (limits) の QoS を適切に設定し、ResourceQuota と LimitRange を使用します。

### CPU

Kubernetes CPU Manager の `static` モードを使用し、vCPU と CPU の pinning (pinning) を設定し、CPU steal (CPU steal) を監視します。

### NUMA と Hugepages

Hugepage を適切に設定し、NUMA のアーキテクチャと TLB を理解し、NUMA の設定を確認します。

### VM

CPU と NUMA の設定、hugepages の RT (real-time) の設定、SCHED\_FIFO (first-in, first-out) の設定を確認します。

# 

1. **Resource Limits and QoS** — CPU Guaranteed QoS
2. **CPU Pinning** — CPU Manager static vCPU
3. **NUMA and Hugepages** — hugepage NUMA
4. **Real-time VMs** — RT SCHED\_FIFO

## 

- [VM Configuration](#) -
- [Storage Configuration](#) -

## 

- [OpenShift Virtualization](#)
- [KubeVirt](#)

## QoS

OpenShift Virtualization VM CPU requests limits QoS VM

KubeVirt VM Kubernetes QoS VM ResourceQ
 uota LimitRange

- **OpenShift 4.18** OpenShift Virtualization
- (system:admin )
- oc CLI virtctl CLI
- StorageClass VM

OpenShift 4.20, 4.21

## VM

OpenShift Virtualization VM KubeVirt VM VM **virt-launcher**  
requests limits

### CPU

- vCPU  $1/\text{cpuAllocationRatio}$  CPU  $\text{cpuAllocationRatio}$  1001  
vCPU VM 100m CPU
- CPU VM
- VM CPU CPU
- vCPU  $\text{sockets} * \text{cores} * \text{threads}$  1

### 

- VM  $\text{spec.domain.memory.guest}$   $\text{spec.domain.resources.requests.memory}$
- KubeVirt QEMU
- 
- 500 MiB 2%

:

VM OS VM

### 1:

```
oc create namespace resource-qos-demo
```

### 2: QoS VM

Kubernetes Pod 3 QoS 1 VM Pod

#### Burstable QoS

requests limits VM Burstable QoS OpenShift Virtualization VM

: cloud-init SSH

#### vm-burstable.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-burstable
```

```

namespace: resource-qos-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-burstable
 spec:
 domain:
 memory:
 guest: 512Mi
 resources:
 requests:
 memory: 512Mi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false
EOF

```

## Guaranteed QoS

CPU VM Guaranteed

QoS VM evict

**vm-guaranteed.yaml**

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1

```





## BestEffort QoS

VM BestEffort QoS VM

**vm-besteffort.yaml**

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-besteffort
 namespace: resource-qos-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-besteffort
 spec:
 domain:
 memory:
 guest: 512Mi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false
EOF
```

### □□□□ 3: QoS□□□□□□

VM□□□□□□□□

```
oc get vm -n resource-qos-demo
```

□□□□□□□□

| NAME          | AGE | STATUS  | READY |
|---------------|-----|---------|-------|
| vm-besteffort | 30s | Running | True  |
| vm-burstable  | 60s | Running | True  |
| vm-guaranteed | 45s | Running | True  |

□VM□Pod□□□□□□□□QoS□□□□□□□□

Burstable VM□□□□

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-burstable -o
jsonpath='{.items[0].status.qosClass}{"\n"}'
```

□□□□□□□□

Burstable

Guaranteed VM□□□□

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-guaranteed -o
jsonpath='{.items[0].status.qosClass}{"\n"}'
```

□□□□□□□□

Guaranteed

BestEffort VM□□□□

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-besteffort -o
jsonpath='{.items[0].status.qosClass}{"\n"}'
```

□□□□□□□□

Burstable

例: `virt-launcher` KubeVirt `vm-besteffort`  
BestEffort Burstable

`virt-launcher` Pod

Burstable VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-burstable -o yaml
```

Guaranteed VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-guaranteed -o yaml
```

BestEffort VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-besteffort -o yaml
```

## 4: ResourceQuota

ResourceQuota VM Pod ResourceQuota KubeVirt VM

`resource-quota.yaml`

```
oc apply -f - <<EOF
apiVersion: v1
kind: ResourceQuota
metadata:
 name: vm-resource-quota
 namespace: resource-qos-demo
spec:
 hard:
 requests.cpu: "4"
 requests.memory: 8Gi
 limits.cpu: "8"
 limits.memory: 16Gi
 pods: "10"
EOF
```

```
oc describe resourcequota vm-resource-quota -n resource-qos-demo
```

VM CPU 1 vCPU 1 2

2

VM1.5

```
LimitRangePodVM
```

```
oc apply -f - <<EOF
apiVersion: v1
kind: LimitRange
metadata:
 name: vm-limit-range
 namespace: resource-qos-demo
spec:
 limits:
 - type: Pod
 max:
 cpu: "4"
 memory: 8Gi
 min:
 cpu: 100m
 memory: 256Mi
 - type: Container
 default:
 cpu: 500m
 memory: 1Gi
 defaultRequest:
 cpu: 100m
 memory: 512Mi
 max:
 cpu: "4"
 memory: 8Gi
 min:
 cpu: 50m
 memory: 128Mi
EOF
```

```
oc describe limitrange vm-limit-range -n resource-qos-demo
```

LimitRangePodVM  
Pod

virt-launcher

6:

VM Pod

```
oc adm top pods -n resource-qos-demo
```

virt-launcher PodCPU

```
oc describe resourcequota vm-resource-quota -n resource-qos-demo
```

QoS

| QoS        |        |            |    |
|------------|--------|------------|----|
| Guaranteed | CPU == |            |    |
| Burstable  |        | BestEffort | VM |
| BestEffort |        |            |    |

- Burstable: VM
- VM Guaranteed
- 
- oc adm top pods
- ResourceQuota
- LimitRangeVM

```
oc delete namespace resource-qos-demo
```

- VM CPU

- KubeVirt VM
- GuaranteedBurstableBestEffort QoS VM
- ResourceQuota
- LimitRangePod
- VM
- 

- [KubeVirt Resource Requests and Limits \(KubeVirt \)](#)
- [Working with Resource Quotas for VMs - OpenShift](#)
- [Configure QoS for Pods - Kubernetes](#)
- [Resource Quotas Per Project - OpenShift](#)

## CPU

OpenShift

VirtualizationCPUvCPUCPUCPUvCPUCPUCPU

NFV/HPC

- OpenShift VirtualizationOpenShift 4.184.21
- `system:admin`
- `oc virtctl` CLI
- CPU4
- KubernetesQoSQoS

## CPU

OpenShiftKubernetesPodCPUstatic

KubeVirt `dedicatedCpuPlacement` CPU `virt-launcher`  
PodvCPUCPUPodvCPUCPU

- CPU `static`
- VMQoS **Guaranteed** CPUrequestlimitCPU

|                                     |                           |                             |
|-------------------------------------|---------------------------|-----------------------------|
| 名前                                  | vCPU数                     | メモリー                        |
| デフォルト                               | vCPU数制限なしCPU数制限なし         | VM数制限なしメモリー制限なし             |
| 専用<br>(dedicatedCpuPlacement: true) | vCPU数制限ありCPU数制限ありCPU数制限あり | NFV/ハイパバイザーHPC数制限ありメモリー制限あり |

ステップ 1: CPU数制限を有効にする

CPU数制限を有効にするには、KubeletConfig オブジェクトに MachineConfigPool オブジェクトを

MachineConfigPool オブジェクトに

```
oc label machineconfigpool worker custom-kubelet=cpumanager-enabled
```



OpenShift (SNO) では、worker ノードに KubeletConfig オブジェクトを machineConfigPoolSelector で worker ノードに master ノードに

kubeletconfig-static-policy.yaml を適用する

KubeletConfig オブジェクトに CPU 数制限を有効にする

```
oc apply -f - <<EOF
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
 name: cpumanager-enabled
spec:
 machineConfigPoolSelector:
 matchLabels:
 custom-kubelet: cpumanager-enabled
 kubeletConfig:
 cpuManagerPolicy: static
 cpuManagerReconcilePeriod: 5s
EOF
```

Machine Config Operator が kubelet を更新するのを待つ

```
oc get mcp worker -w
```

UPDATING 0 False READYMACHINECOUNT 0 MACHINECOUNT 0

| NAME     | CONFIG | UPDATED | UPDATING |
|----------|--------|---------|----------|
| DEGRADED |        |         |          |

|        |                                                      |      |       |       |
|--------|------------------------------------------------------|------|-------|-------|
| worker | rendered-worker-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx | True | False | False |
|--------|------------------------------------------------------|------|-------|-------|

## CPU

kubelet static

```
oc debug node/<node-name> -- chroot /host cat /etc/kubernetes/kubelet.conf
```

```
cpuManagerPolicy: static
cpuManagerReconcilePeriod: 5s
```

## 2: CPU

CPU VM CPU cpubanager=true

```
oc label node <node-name> cpumanager=true
```

```
oc get nodes -L cpumanager
```

| NAME      | STATUS | ROLES  | AGE | VERSION | CPUMANAGER |
|-----------|--------|--------|-----|---------|------------|
| worker-01 | Ready  | worker | 10d | v1.34.5 | true       |
| worker-02 | Ready  | worker | 10d | v1.34.5 | true       |



KubeVirt CR CPUManager KubeVirt cpubanager=true CPU

## 3:

```
oc create namespace cpu-pinning-demo
```

## 4: CPU VM

VM dedicatedCpuPlacement: true 2 vCPU nodeSelector VM cpubanager=true Guaranteed QoS





cloud-initをインストールしてSSH接続できるようにする

## vm-dedicated-cpu.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-dedicated-cpu
 namespace: cpu-pinning-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-dedicated-cpu
 spec:
 nodeSelector:
 cpumanager: "true"
 domain:
 cpu:
 sockets: 2
 cores: 1
 threads: 1
 dedicatedCpuPlacement: true
 memory:
 guest: 1Gi
 resources:
 requests:
 memory: 2Gi
 limits:
 memory: 2Gi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
```

```
- name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false
```

EOF



QEMU VMs are configured with 2Gi of memory and 1Gi of disk space. KubeVirt VMs are configured with 2Gi of memory and 1Gi of disk space. QoS is configured for the VMs.

VM Running

```
oc get vm vm-dedicated-cpu -n cpu-pinning-demo -w
```

Output

| NAME             | AGE | STATUS  | READY |
|------------------|-----|---------|-------|
| vm-dedicated-cpu | 30s | Running | True  |

## 5: CPU

Output

VM

```
oc get vmi vm-dedicated-cpu -n cpu-pinning-demo -o wide
```

Output

| NAME             | AGE | PHASE   | IP          | NODENAME  | READY |
|------------------|-----|---------|-------------|-----------|-------|
| vm-dedicated-cpu | 60s | Running | 10.128.0.45 | worker-01 | True  |

Output

```
oc debug node/<node-name>
```

Output CPU

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

JSONentries virt-launcher PodUIDPod compute  
CPU ID

```
{ "policyName": "static", "defaultCpuSet": "0-1,3-41,43-79", "entries": { "<pod-uid>": { "compute": "2,42" } }, "checksum": ... }
```

CPU 2 42 VM2vCPU defaultCpuSet  
VM

```
exit
```

VM

```
virtctl console vm-dedicated-cpu -n cpu-pinning-demo
```

fedora redhat OSvCPU

```
lscpu | grep -E "^CPU(s)|^Socket|^Core|^Thread"
```

```
CPU(s): 2
Thread(s) per core: 1
Core(s) per socket: 1
Socket(s): 2
```

2vCPU211

Ctrl+]

QEMU I/O VMvCPUCPU  
isolateEmulatorThread CPU

VMCPU dedicatedCpuPlacement: true  
isolateEmulatorThread: true

```
domain:
cpu:
```

```
sockets: 2
cores: 1
threads: 1
dedicatedCpuPlacement: true
isolateEmulatorThread: true
```

3 KubeVirt CPU 2 vCPU 1



Simultaneous Multithreading (SMT) full-pcpus-only  
CPU vCPU isolateEmulatorThread  
KubeVirt CR AlignCPUs

VM Pending

VM

```
oc describe vmi vm-dedicated-cpu -n cpu-pinning-demo
```

Warning FailedScheduling 0/3 nodes are available: 3 node(s) didn't match Pod's node affinity/selector

- `cpumanager=true` `oc get nodes -L cpumanager`  
CPU
- `CPU`: `oc describe node <node-name>`  
CPU

VM CPU

KubeletConfig

```
oc get kubeletconfig cpumanager-enabled
```

MachineConfigPool

```
oc get mcp worker
```

UPDATED ☐ False ☐ DEGRADED ☐ True ☐ MCO ☐

```
oc logs -n openshift-machine-config-operator -l k8s-app=machine-config-operator
--tail=50
```

VM **"insufficient dedicated CPU resources"** ☐

☐ dedicatedCpuPlacement: ☐ true

☐ QoS ☐ Guaranteed ☐ CPU ☐

virt-launcher Pod ☐ QoS ☐

```
oc get pod -n cpu-pinning-demo -l kubevirt.io/domain=vm-dedicated-cpu -o
jsonpath='{.items[0].status.qosClass}{"\n"}'
```

☐ Guaranteed ☐ Burstable ☐ VM ☐

cpu\_manager\_state ☐ VM ☐

CPU ☐ virt-launcher Pod CPU ☐ kubelet ☐ none

☐

```
oc debug node/<node-name> -- chroot /host cat /etc/kubernetes/kubelet.conf
```

cpuManagerPolicy ☐ ☐ none

☐ KubeletConfig ☐ MachineConfigPool ☐

☐

☐

```
oc delete namespace cpu-pinning-demo
```

☐ cpumanager=true ☐

```
oc label node <node-name> cpumanager-
```

CPU ☐ KubeletConfig ☐ MachineConfigPool ☐

```
oc delete kubeletconfig cpumanager-enabled
```

```
oc label machineconfigpool worker custom-kubelet-
```



## KubeletConfig

このコンフィグは kubelet が Kubernetes MCO から管理される CPU に関する設定を定義する。  
MachineConfigPool が管理するコンフィグ。

概要

このコンフィグは以下を設定する。

- **KubeletConfig** が管理する Kubernetes CPU に関する **static** の設定
- CPU に関する設定
- **dedicatedCpuPlacement: true** が Guaranteed QoS を提供する
- CPU に関する設定
- vCPU に関する設定
- **isolateEmulatorThread: true** が QEMU を提供する

詳細

- **VM に関する QoS** — Guaranteed QoS を提供する
- **VM に関する** — VM に関する
- **VM に関する** — VM に関する
- **KubeVirt** の CPU (2)
- **CPU に関する** - OpenShift (2)
- **CPU に関する** - Kubernetes (2)

## NUMA と Hugepages

概要

このコンフィグは OpenShift

Virtualization の Hugepage を NUMA を VM に関する  
VM に関する 4Ki の OS の NUMA を NFV/HPC に関する  
2 に関する

- **TLB**:  
16Gi の 4Ki の VM の 400 の CPU の TLB Translation Lookaside Buffer
- **NUMA**:  
NUMA の 1.5 3 の NUMA の OS の NUMA

Hugepages の 4Ki の 2Mi の 1Gi の TLB の NUMA の **guestMappingPassthrough**  
NUMA の VM の NUMA の CPU の VM

□ □ □ □

- OpenShift Virtualization OpenShift 4.18 4.21
- `system:admin`
- `oc`virtctl` CLI
- CPU 8 Hugepage RAM
- CPU CPU
- Kubernetes QoS QoS

# Hugepages□□□

Linux 4KiB VM 524,288 TLB C  
PU

Hugepages

| 項目  | 2Gi     | TLB     |    |
|-----|---------|---------|----|
| 4Ki | 524,288 | 524,288 |    |
| 2Mi | 1,024   | 1,024   | VM |
| 1Gi | 2       | 2       | VM |

VM的`memory.hugepages.pageSize` 配置项指定了KubeVirt虚拟机使用的Hugepage配置。配置项`memory.hugepages.pageSize` 指定了Hugepage的大小，单位为KiB。配置项`memory.hugepages.pinned` 指定了Hugepage是否被锁定。配置项`memory.hugepages.pinned` 指定了Hugepage是否被锁定。

□□:

Hugepages VM NUMA Hugepages VM NUMA  
M NUMA VM CPU NUMA Hugepages NUMA  
QEMU VM Hugepages

# NUMA□□□□□□□□

## NUMA□Non-Uniform

## Memory

Access CPU

□□□□2□□□□□□□□□□:

```
NUMA Node 0: CPUs 0-19, 40-59 Local Memory: 128Gi
NUMA Node 1: CPUs 20-39, 60-79 Local Memory: 128Gi
```

NUMA0 OS NUMA0 0 vCPU NUMA1 1

KubeVirt `guestMappingPassthrough` NUMA CPU NUMA NUMA

NUMA 3

1. CPU `dedicatedCpuPlacement: true`
2. Hugepages
3. KubeVirt CR NUMA OpenShift Virtualization 4.18

## 1: Hugepages

Hugepages OpenShift Node  
Operator `Tuned`

Tuning

`Tuned` 2Mi 1024 2Gi

`hugepages-tuned-boottime.yaml`

```
oc apply -f - <<EOF
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
 name: hugepages
 namespace: openshift-cluster-node-tuning-operator
spec:
 profile:
 - data: |
 [main]
 summary=Boot time configuration for hugepages
 include=openshift-node
 [bootloader]
 cmdline_openshift_node_hugepages=hugepagesz=2M hugepages=1024
 name: openshift-node-hugepages
 recommend:
 - machineConfigLabels:
 machineconfiguration.openshift.io/role: "worker"
 priority: 20
 profile: openshift-node-hugepages
EOF
```

: Single Node OpenShift SNO `machineConfigLabels` `worker` `master`

: `priority` Tuned `openshift-control-plane` `openshift-node` 30 Hugepages 20

Node Tuning Operator `hugepagesz=2M hugepages=1024` MachineConfig Machine Config Operator

```
oc get mcp worker -w
```



`UPDATED` `True` `MACHINECOUNT` `READYMACHINECOUNT`

| NAME     | CONFIG                                         | UPDATED | UPDATING |
|----------|------------------------------------------------|---------|----------|
| DEGRADED |                                                |         |          |
| worker   | rendered-worker-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx | True    | False    |

MachineConfigPool

## Hugepage

Hugepage

```
oc get node <node-name> -o jsonpath='{.status.allocatable.hugepages-2Mi}{"\n"}'
```

:

2Gi

`/proc/meminfo`

```
oc debug node/<node-name>
```

:

```
chroot /host grep -i hugepages /proc/meminfo
```

:

```
AnonHugePages: 0 kB
ShmemHugePages: 0 kB
FileHugePages: 0 kB
HugePages_Total: 1024
HugePages_Free: 1024
HugePages_Rsvd: 0
HugePages_Surp: 0
Hugepagesize: 2048 kB
```

`HugePages\_Total` 1024 `HugePages\_Free`

```
exit
```

## 例 2: NUMA環境構築

NUMA環境をVMで構築し、NUMA環境を確認するためのコマンドを実行する。RHCOS環境を確認する。

確認コマンド

```
oc debug node/<node-name>
```

NUMA環境を確認する

```
chroot /host ls /sys/devices/system/node/
```

2ノード環境の場合:

```
node0 node1
```

NUMA環境のCPUを確認する

```
chroot /host cat /sys/devices/system/node/node0/cpulist
```

```
0-9,40-49
```

```
chroot /host cat /sys/devices/system/node/node1/cpulist
```

```
20-29,60-69
```

NUMA環境のメモリを確認する

```
chroot /host cat /sys/devices/system/node/node0/meminfo
```

確認結果:

```
Node 0 MemTotal: 131072000 kB
Node 0 MemFree: 125000000 kB
Node 0 HugePages_Total: 512
Node 0 HugePages_Free: 512
```

NUMA環境のCPU-to-nodeを確認する`lscpu`コマンド

```
chroot /host lscpu | grep -i numa
```

000:

```

NUMA node(s): 2
NUMA node0 CPU(s): 0-9,40-49
NUMA node1 CPU(s): 20-29,60-69

```

OpenShift NUMA 1 NUMA CPU

### 3: Namespace

```
oc create namespace numa-hugepages-demo
```

## 4: Hupages VM

VM NUMA CPU Hugepage TLB VM

□□□□□□□□:

- `dedicatedCpuPlacement: true` — vCPU CPU KubeVirt Hupages
- `memory.hugepages.pageSize: 2Mi` — 2Mi Hugepage
- `requests`limits`` Guaranteed QoS

```
00: 0000cloud-init000000000000000000000000SSH00000000000000000000
```

vm-hugepages.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-hugepages
 namespace: numa-hugepages-demo
spec:
 runStrategy: Always
 template:
```

```

metadata:
 labels:
 kubevirt.io/domain: vm-hugepages
spec:
 nodeSelector:
 cpumanager: "true"
 domain:
 cpu:
 sockets: 2
 cores: 1
 threads: 1
 dedicatedCpuPlacement: true
 memory:
 hugepages:
 pageSize: 2Mi
 resources:
 requests:
 memory: 2Gi
 limits:
 memory: 2Gi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false

```

EOF

VMRunning

```
oc get vm vm-hugepages -n numa-hugepages-demo -w
```

□□□□□□:

| NAME         | AGE | STATUS  | READY |
|--------------|-----|---------|-------|
| vm-hugepages | 45s | Running | True  |

## □□□□ 5: Hugepage□□□□□□□□

□□□□□□

VM□□□□□□□□□□□□□□

```
oc get vmi vm-hugepages -n numa-hugepages-demo -o wide
```

□□□□□□□□□□□□□□□□

```
oc debug node/<node-name>
```

Hugepage□□□□□□□□□□ `HugePages\_Free` □□□VM□□□□□□□□□□□□ `HugePages\_Total` □□□□□□□□□□□□□□

```
chroot /host grep -i hugepages /proc/meminfo
```

2Mi□□□□□□□□2GiVM□□□□1024□□□□□□□□□□□□ `HugePages\_Free` □□□□□□□□1024□□□□□□□□□□

□□□□□□□□□□□□

```
exit
```

□□□□□□

VM□□□□□□□□□□□□

```
virtctl console vm-hugepages -n numa-hugepages-demo
```

□□□□□ `fedora` □□□□□□ `redhat` □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
free -h
```

□□□□□□□□□□□□RAM□□□□□□□□□□Hugepages□QEMU□□□□□□□□□□□□□□□□□□□□□□□□OS□□□□□□□□□□□□□□□□□□□□Hugepages□□□□□□□□□□□□



```

cpu:
 sockets: 1
 cores: 2
 threads: 1
 dedicatedCpuPlacement: true
 isolateEmulatorThread: true
 numa:
 guestMappingPassthrough: {}
memory:
 hugepages:
 pageSize: 2Mi
resources:
 requests:
 memory: 1Gi
 limits:
 memory: 1Gi
devices:
 autoattachMemBalloon: false
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
networks:
 - name: default
 pod: {}
volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false

```

EOF

VMRunning

```
oc get vm vm-numa-hugepages -n numa-hugepages-demo -w
```

:

| NAME              | AGE | STATUS  | READY |
|-------------------|-----|---------|-------|
| vm-numa-hugepages | 60s | Running | True  |

## 7: VM NUMA

### VM NUMA

VM NUMA

```
oc get vmi vm-numa-hugepages -n numa-hugepages-demo -o
jsonpath='{.spec.domain.cpu.numa}{"\n"}'
```

:

```
{"guestMappingPassthrough":{}}
```

### NUMA

VM

```
virtctl console vm-numa-hugepages -n numa-hugepages-demo
```

fedora`redhat`NUMA

```
lscpu | grep -i numa
```

NUMA CPU NUMA NUMA 1 NU  
MA

CPU

```
lscpu
```

NUMA

```
ls /sys/devices/system/node/
```

:

```
node0
```



NUMA CPU

```
cat /sys/devices/system/node/node0/cpulist
```

:

0-1

```
cat /sys/devices/system/node/node0/meminfo | head -5
```

:

```
Node 0 MemTotal: 1048576 kB
Node 0 MemFree: 800000 kB
Node 0 MemUsed: 248576 kB
Node 0 Active: 120000 kB
Node 0 Inactive: 80000 kB
```

NUMA CPU

`Ctrl+J`

NUMA

CPU CPU

```
oc get vmi vm-numa-hugepages -n numa-hugepages-demo -o wide
```

```
oc debug node/<node-name>
```

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

`entries` CPU ID `compute` `isolateEmulatorThread: true` 1 CPU 3 vCPU 2 + 1

```
exit
```

## 2Mi 1Gi Hugepages

| 項目  | 2Mi    | 1Gi         |
|-----|--------|-------------|
| TLB | 2Mi    | 1Gi 512     |
|     | 100Mi  | 1Gi         |
|     |        | 1Gi VM 1Gi  |
|     |        | Hugepages   |
|     | VM TLB | VM 64Gi TLB |

1Gi `Tuned`

```
[bootloader]
cmdline_openshift_node_hugepages=hugepagesz=1G hugepages=16
```

VM

```
memory:
 hugepages:
 pageSize: 1Gi
resources:
 requests:
 memory: 16Gi
 limits:
 memory: 16Gi
```

: Hugepage 1Gi Hugepages 1.5Gi

VM Hugepages

VM `Scheduling` `Pending` VM

```
oc describe vmi vm-hugepages -n numa-hugepages-demo
```

Hugepages Hugepages

```
oc get node <node-name> -o jsonpath='{.status.allocatable.hugepages-2Mi}'
```

VM VM Hugepage

## NUMA VM cannot bind memory to host NUMA nodes

virt-launcher Pod NUMA

- NUMA Hugepages: Hugepages VM CPU NUMA Hugepage
- kubelet none CPU NUMA NUMA single-numa-node restricted

NUMA

1 NUMA

- VM guestMappingPassthrough
- VM CPU NUMA CPU NUMA 1
- NUMA CPU VM vCPU

## MachineConfigPool Updating

Tuned MachineConfigPool

```
oc get mcp worker
```

```
oc describe mcp worker
```

machine-config-daemon

```
oc logs -n openshift-machine-config-operator -l k8s-app=machine-config-daemon --tail=50
```

VM Namespace

```
oc delete namespace numa-hugepages-demo
```

Hugepages Tuned

```
oc delete tuned hugepages -n openshift-cluster-node-tuning-operator
```



| ホスト側      | ゲスト側    | 設定                                  |
|-----------|---------|-------------------------------------|
| NUMAノード   | NUMAノード | VM (guestMappingPassthrough)        |
| Hugepages | TLB     | VM (memory.hugepages)               |
| リアルタイム    | リアルタイム  | PerformanceProfile (realTimeKernel) |
| CPU       | IRQ     | PerformanceProfile (cpu.isolated)   |
| リアルタイム    | vCPU    | VM (cpu.realtime)                   |
| メモリ/バースト  | メモリ     | VM (autoattachMemBalloon: false)    |

3ノード、CPU、NUMA/Hugepages

## 手順

- OpenShift Virtualization OpenShift 4.18/4.21
- `system:admin`
- `oc virtctl` CLI
- 8CPU、16Gi RAM
- CPU
- NUMA、Hugepages
- Node Tuning OpenShift

## インストール

\* VM

V



VM、RT、CPU

## 1: MachineConfigPool

MachineConfigPool PerformanceProfile RT CP

worker-rt

```
oc label node <node-name> node-role.kubernetes.io/worker-rt=
```

worker `worker-rt` MachineConfigPool

```
oc apply -f - <<EOF
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
 name: worker-rt
 labels:
 machineconfiguration.openshift.io/role: worker-rt
spec:
 machineConfigSelector:
 matchExpressions:
 - key: machineconfiguration.openshift.io/role
 operator: In
 values:
 - worker
 - worker-rt
 nodeSelector:
 matchLabels:
 node-role.kubernetes.io/worker-rt: ""
EOF
```

```
oc get mcp worker-rt
```

| NAME      | CONFIG | UPDATED | UPDATING | DEGRADED | MACHINECOUNT | READYMACHINECOUNT |
|-----------|--------|---------|----------|----------|--------------|-------------------|
| worker-rt |        | True    | False    | False    | 1            | 1                 |

## 2: PerformanceProfile

PerformanceProfile Node Tuning

- (kernel-rt)
- CPU 2-7 IRQ
- CPU 0-1
- 4 x 1Gi
- 

performance-profile.yaml

```

oc apply -f - <<EOF
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
 name: realtime-workers
spec:
 cpu:
 isolated: "2-7"
 reserved: "0-1"
 hugepages:
 defaultHugepagesSize: 1G
 pages:
 - size: 1G
 count: 4
 realTimeKernel:
 enabled: true
 additionalKernelArgs:
 - nmi_watchdog=0
 - audit=0
 - mce=off
 - processor.max_cstate=1
 - idle=poll
 - intel_idle.max_cstate=0
 nodeSelector:
 node-role.kubernetes.io/worker-rt: ""
 machineConfigPoolSelector:
 machineconfiguration.openshift.io/role: worker-rt
EOF

```



OpenShift (SNO)  
 MachineConfigPool nodeSelector node-  
 role.kubernetes.io/master: "" machineConfigPoolSelector  
 pools.operator.machineconfiguration.openshift.io/master: ""  
 machineConfigPoolSelector  
 machineConfigSelector MCP metadata.labels  
 VM nodeSelector 1 worker-rt  
 (oc label node <node-name> node-role.kubernetes.io/worker-rt=)



PerformanceProfile API (1Gi, 2Mi) SIO (1G, 2M)  
 VM Kubernetes 1Gi, 2Mi

PerformanceProfile Tuned  
 MachineConfig CPU KubeletConfig Machine Config



PerformanceProfile RT  
 RT

RT

```
oc get mcp worker-rt -w
```

SNO master

```
oc get mcp master -w
```

UPDATED True READYMACHINECOUNT MACHINECOUNT  
RT1020MCP UPDATING: True

| nmi_watchdog=0          | NMI                                |
|-------------------------|------------------------------------|
| audit=0                 |                                    |
| mce=off                 |                                    |
| processor.max_cstate=1  | CPU C-states > C1                  |
| idle=poll               | CPU                                |
| intel_idle.max_cstate=0 | Intel C-state processor.max_cstate |



RT CPU

3: RT

MachineConfigPool

```
oc debug node/<node-name>
```

```
chroot /host uname -r
```

rt

```
5.14.0-427.55.1.el9_4.x86_64+rt
```

CPU





```
kernel.sched_rt_runtime_us = -1
```

## 실습 4: Namespace

```
oc create namespace realtime-demo
```

## 실습 5: RT VM

이 VM은 Fedora RT를 기반으로 합니다. RT 커널을 사용하여 RT 작업을 실행할 수 있습니다.

다음은 VM의 사양입니다.

- `cpu.model: host-passthrough` — CPU를 호스트에 직접 연결합니다.
- `cpu.dedicatedCpuPlacement: true` — vCPU를 전용 CPU에 할당합니다.
- `cpu.isolateEmulatorThread: true` — QEMU의 에뮬레이터 스레드를 CPU에 고립합니다.
- `cpu.ioThreadsPolicy: auto` — I/O 스레드를 CPU에 고립합니다.
- `cpu.numa.guestMappingPassthrough: {}` — NUMA 매핑을 호스트에 그대로 전달합니다.
- `cpu.realtime.mask: "0"` — vCPU가 `SCHED_FIFO` 스케줄링을 받도록 설정합니다.
- `memory.hugepages.pageSize: 1Gi` — 1Gi의 HugePage를 사용합니다.
- `autoattachMemBalloon: false` — 메모리 볼런을 자동으로 부착하지 않습니다.
- `autoattachGraphicsDevice: false` — 그래픽스 장치를 자동으로 부착하지 않습니다.



cloud-init을 사용하여 VM에 SSH 접속할 수 있습니다.

### vm-realtime.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-realtime
 namespace: realtime-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-realtime
 spec:
 nodeSelector:
 node-role.kubernetes.io/worker-rt: ""
```

```

domain:
 cpu:
 model: host-passthrough
 sockets: 1
 cores: 2
 threads: 1
 dedicatedCpuPlacement: true
 isolateEmulatorThread: true
 ioThreadsPolicy: auto
 numa:
 guestMappingPassthrough: {}
 realtime:
 mask: "0"
 memory:
 hugepages:
 pageSize: 1Gi
 resources:
 requests:
 memory: 2Gi
 cpu: 2
 limits:
 memory: 2Gi
 cpu: 2
 devices:
 autoattachSerialConsole: true
 autoattachMemBalloon: false
 autoattachGraphicsDevice: false
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 terminationGracePeriodSeconds: 0
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:

```

```
EOF
 expire: false
```

VMRunning

```
oc get vm vm-realtime -n realtime-demo -w
```

:

| NAME        | AGE | STATUS  | READY |
|-------------|-----|---------|-------|
| vm-realtime | 90s | Running | True  |



VM [kubevirt.io/realtime](https://kubevirt.io/realtime)

KubeVirt VM

VM

cpumanager=true  
realtime  
Scheduling

6:

CPU

VM

```
oc get vmi vm-realtime -n realtime-demo -o wide
```

```
oc debug node/<node-name>
```

CPU CPU

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

entries 2vCPU ID1 CPU3 compute

CPU PerformanceProfile2-7

```
chroot /host grep HugePages /proc/meminfo
```

HugePages\_Free VM1Gi VM2 HugePages\_Total



□□□□□□□□□□

□□□□ 7: VM□□□□□□□□□□□□□□

□ □ □ □ □ □ □ □ □ □

□□□□□□□□rt

CPU□□□□

□□□□□□:

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

(Ctrl+) □□□□□□□□□□□□□□□□

QEMU

552

```
oc debug node/<node-name>
```

QEMU ID

```
chroot /host pgrep qemu-kvm
```

PID `<qemu-pid>`

```
chroot /host ps -T -o tid,cls,rtprio,comm -p <qemu-pid>
```

:

| TID   | CLS | RTPRIO | COMMAND         |
|-------|-----|--------|-----------------|
| 70898 | TS  | -      | qemu-kvm        |
| 70900 | TS  | -      | call_rcu        |
| 70901 | TS  | -      | TC tc-ram-node0 |
| 70941 | TS  | -      | IO mon_iothread |
| 70942 | FF  | 1      | CPU 0/KVM       |
| 70943 | TS  | -      | CPU 1/KVM       |

CLS SCHED\_FIFO FF SCHED\_OTHER TS RTPRIO CPU 0/KVM  
1 FF realtime.mask: "0" SCHED\_FIFO CPU 1/KVM TS

```
exit
```

8:

cyclictest

VM

```
virtctl console vm-realtime -n realtime-demo
```

realtime-tests cyclictest

```
sudo dnf install -y realtime-tests
```

cyclictest 3 60







ラベル `worker-rt` を追加します

```
oc label node <node-name> node-role.kubernetes.io/worker-rt-
```



PerformanceProfileとMachineConfigPoolを組み合わせることで、  
worker MachineConfigPoolを構成します

## 手順

必要なコンポーネントをインストールします

- MachineConfigPoolをインストール
- PerformanceProfileをRT CPU用に設定
- RT CPU用に [kubevirt.io/realtime](https://kubevirt.io/realtime) をインストール
- VMをインストール
- CPUスケジューリングを `SCHED_FIFO` に設定
- `cyclictest` をインストール
- 検証

## 参照

- [CPU](#) — CPUとvCPU
- [NUMAとHugepages](#) — NUMA
- [VMとQoS](#) — Guaranteed QoS
- [KubeVirt Real-Time Workloads](#)
- [Tuning Nodes for Low Latency - OpenShift Documentation](#)
- [Provisioning Real-Time Workloads - OpenShift Documentation](#)
- [KVM Real-Time Tuning - libvirt](#)

# VM

## 

OpenShift

Virtualization  
VM

## 

- qcow2 OpenShift Virtualization
- 
- VM
- VM
- UEFI/Secure Boot

## 

VM

- OpenShift 4.18 OpenShift Virtualization
- ReadWriteMany StorageClass
- CLI: `oc`, `virtctl`, `kubectl`
- PersistentVolumeClaims DataVolumes

## 

qcow2

KVM qcow2 OpenShift Virtualization  
VM

VM

Golden Images

VM

cloud-init

VM

VM

## VM

VM PVC ContainerDisk HTTP UEFI BIOS Secure Boot

## 

- [VM Configuration](#) - VM
- [Storage Configuration](#) - VM
- [Performance Tuning](#) - CPU NUMA

## 

- [OpenShift Virtualization Documentation](#)
- [KubeVirt User Guide](#)

## qcow2

qcow2 KVM OpenShift Virtualization VM OVA OS

qcow2 (QEMU Copy-On-Write version 2) KVM QEMU OpenShift Virtualization

: OpenShift Virtualization OpenShift 4.18.33 AWS EBS gp3-csi

- OpenShift 4.18 [OpenShift Virtualization](#)
- CLI: `oc`, `virtctl`
- `qemu-img` ( ) - qcow2
- HTTP/S3 qcow2
- -
- namespace

qcow2 OpenShift Virtualization 4

| 명칭                                | 설명              | 참고                              |
|-----------------------------------|-----------------|---------------------------------|
| <b>virtctl image-upload</b>       | 이진 파일을 업로드      | 이진 파일을 DataVolume/PVC로 업로드      |
| <b>DataVolume with HTTP/S3</b>    | 이진 파일을 가져옴      | Webhook로 이진 파일을 DataVolume로 가져옴 |
| <b>ContainerDisk</b>              | 이진 파일을 가져옴      | 이진 파일을 qcow2로 가져옴               |
| <b>VM with DataVolumeTemplate</b> | 이진 파일을 VM으로 가져옴 | VM을 DataVolumeTemplate로 가져옴     |

이진 파일을 업로드하는 방법은 **virtctl image-upload** 명령어를 사용하여 다음과 같이 합니다.

## 예제 1: qcow2 이미지

이진 파일을 qcow2로 업로드하는 OpenShift Virtualization 명령어는 다음과 같습니다.

이진 파일을 가져오는 명령어는 다음과 같습니다.

이진 파일을 Fedora Cloud로 가져오는 qcow2 명령어는 다음과 같습니다.

```
Download Fedora Cloud image (example)
curl -LO
https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2

Verify the image
ls -lh Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
```

이진 파일을 가져오는 명령어는 다음과 같습니다.

이진 파일을 **qemu-img** 명령어를 사용하여 가져옵니다.

```
qemu-img info Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
```

결과:

```
image: Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
file format: qcow2
virtual size: 5 GiB (5368709120 bytes)
disk size: 420 MiB
```

이진 파일의 **virtual size**는 DataVolume로 업로드할 때 사용되는 크기입니다.

## ステップ 2: インポート

インポートされた VM を Kubernetes にインポート

```
oc create namespace imported-vms
```

## ステップ 3: virtctl を使用して qcow2 をインポート

virtctl を使用して qcow2 をインポートし、PVC を作成

```
virtctl image-upload pvc fedora-imported-disk \
 --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
 --size=10Gi \
 --namespace=imported-vms \
 --insecure \
 --force-bind
```

オプション:

- **pvc fedora-imported-disk** - インポートする PVC
- **--image-path** - インポートする qcow2 のパス
- **--size** - インポートする PVC のサイズ (qcow2 のサイズを指定することも可能)
- **--namespace** - インポートする namespace
- **--insecure** - TLS を無視してインポート
- **--force-bind** - PVC が `WaitForFirstConsumer` の StorageClass を使用

例: **--force-bind** を使用して `volumeBindingMode: WaitForFirstConsumer` の StorageClass (AWS EBS、Azure Disk) を指定してインポート

例:

```
PVC imported-vms/fedora-imported-disk created
Uploading data to https://cdi-uploadproxy-openshift-cnv.apps...

420.00 MiB / 420.00 MiB [=====] 100.00% 1m30s

Uploading data completed successfully, waiting for processing to complete
Processing completed successfully
```

例: 1GB の VM をインポートし、1GB3 の VM をインポート

例:

例: PVC をインポート

```
oc get pvc fedora-imported-disk -n imported-vms --watch
```

~~~~~PVC~Pending~Bound~~~~~

| NAME                 | STATUS  | VOLUME    | CAPACITY |
|----------------------|---------|-----------|----------|
| ACCESS MODES         |         |           |          |
| fedora-imported-disk | Pending |           |          |
| fedora-imported-disk | Bound   | pvc-xxxxx | 10Gi     |
| RWO                  |         |           |          |

~~~~~ **cdi-upload** ~~~~~

```
oc get pods -n imported-vms
oc logs -n imported-vms cdi-upload-fedora-imported-disk -f
```

~~~~~ **4:** ~~~~~**VM**~~~~~

~~~~~PVC~~~~~**VM**~~~~~

~: ~~~~cloud-init~~~~~SSH~~~~~

**vm-from-imported-qcow2.yaml** ~~~~~

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: fedora-imported-vm
 namespace: imported-vms
 labels:
 app: fedora-imported
spec:
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: fedora-imported-vm
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
```

```

 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 persistentVolumeClaim:
 claimName: fedora-imported-disk
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true

```

VM

```
oc apply -f vm-from-imported-qcow2.yaml
```

VM

```
virtctl start fedora-imported-vm -n imported-vms
```

5: VM

VM

```
oc get vmi -n imported-vms --watch
```

VM:

| NAME               | AGE | PHASE      | IP          | NODENAME  | READY |
|--------------------|-----|------------|-------------|-----------|-------|
| fedora-imported-vm | 15s | Scheduling |             |           | False |
| fedora-imported-vm | 30s | Scheduled  |             | worker-01 | False |
| fedora-imported-vm | 45s | Running    | 10.128.2.54 | worker-01 | True  |

VM

```
virtctl console fedora-imported-vm -n imported-vms
```

cloud-init: **fedora**: **fedora123**

```
Check OS version
cat /etc/os-release

Check disk configuration
lsblk
```

**Ctrl+]**

virtctl image-upload

## 2: HTTP **DataVolume**

qcow2 HTTP/HTTPS **DataVolume** CDI Web

Fedora Cloud:

**datavolume-http-import.yaml**

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: fedora-http-imported
 namespace: imported-vms
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
 source:
 http:
 url:
 "https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2"
 storage:
 resources:
 requests:
 storage: 10Gi
```

: **cdi.kubevirt.io/storage.bind.immediate.requested: "true"** PVC



`WaitForFirstConsumer` StorageClass

DataVolume CDI

```
oc apply -f datavolume-http-import.yaml
oc get dv -n imported-vms --watch
```

| NAME                 | PHASE            | PROGRESS | RESTARTS | AGE   |
|----------------------|------------------|----------|----------|-------|
| fedora-http-imported | ImportScheduled  | N/A      |          | 5s    |
| fedora-http-imported | ImportInProgress | 12.5%    |          | 15s   |
| fedora-http-imported | ImportInProgress | 67.3%    |          | 45s   |
| fedora-http-imported | Succeeded        | 100.0%   |          | 1m30s |

DataVolume **Succeeded** VM

```
volumes:
- name: rootdisk
 dataVolume:
 name: fedora-http-imported
```

: CI/CD S3 URL

### 3: ContainerDisk

qcow2

Containerfile

```
FROM scratch
ADD Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 /disk/
```

```
podman build -t quay.io/youruser/fedora-disk:v1 .
podman push quay.io/youruser/fedora-disk:v1
```

VM

```
volumes:
- name: rootdisk
 containerDisk:
 image: quay.io/youruser/fedora-disk:v1
```

ContainerDisk VM

#### 4: DataVolumeTemplate VM

DataVolumeTemplate VM DataVolume VM VM

VM - GitOps - DataVolume VM

[vm-with-upload-datavolume.yaml](#)

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-with-upload
 namespace: imported-vms
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-with-upload-disk
 spec:
 source:
 upload: {}
 storage:
 resources:
 requests:
 storage: 10Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-with-upload
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
```

```

 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-with-upload-disk
 name: rootdisk

```

VM DataVolume

```
oc apply -f vm-with-upload-datavolume.yaml
```

DataVolume VM

```

virtctl image-upload dv vm-with-upload-disk \
 --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
 --size=10Gi \
 --namespace=imported-vms \
 --insecure \
 --force-bind

```

VM

```
virtctl start vm-with-upload -n imported-vms
```

Infrastructure-as-Code

PVC VM DataVolume/virtctl

- 
- **virt-sparsify** qcow2
- 
- **cloud-init** cloud-init

- VM
-

- `oc get pvc` - `oc get pvc`

oc get pvc

- `oc get pvc` - `oc get pvc`
- **TLS** - `--insecure` `oc get pvc`
- `oc get pvc` - `oc get pvc`
- `oc get pvc` - `oc get pvc`

oc get pvc

- `oc get pvc` - `oc get pvc`
- `oc get pvc` - `oc get pvc`
- `oc get pvc` - `oc get pvc`

oc get pvc

oc get pvc "Waiting for PVC upload pod to be ready" oc get pvc

oc get pvc "Waiting for PVC upload pod to be ready" oc get pvc "WaitForFirstConsumer" oc get pvc StorageClass

PVC oc get pvc

```
oc get pvc -n imported-vms
oc describe pvc fedora-imported-disk -n imported-vms
```

oc get pvc "waiting for first consumer to be created before binding" oc get pvc StorageClass  
volumeBindingMode: WaitForFirstConsumer oc get pvc

oc get pvc: virtctl oc get pvc --force-bind oc get pvc

```
virtctl image-upload pvc fedora-imported-disk \
 --image-path=./image.qcow2 \
 --size=10Gi \
 --namespace=imported-vms \
 --insecure \
 --force-bind
```

oc get pvc PVC oc get pvc

```
oc delete pvc fedora-imported-disk -n imported-vms
```

oc get pvc --force-bind oc get pvc

エラーメッセージ "Unable to get PVC Prime name from PVC" の対処法

OpenShiftのDataVolumeとCDIの互換性に関する問題で、PVCの名前が取得できないというエラーが発生している。

```
Use PVC upload (not DataVolume)
virtctl image-upload pvc <pvc-name> \
 --image-path=./image.qcow2 \
 --size=10Gi \
 --namespace=imported-vms \
 --insecure
```

エラーメッセージ: \* CDIのデプロイメント: `cdi-deployment` の `cdi-operator` のリソースが不足している \* エラーメッセージ: StorageClassのデプロイメントが失敗している

CDIのデプロイメントを確認する

```
oc get pods -n openshift-cnv | grep cdi
```

エラーメッセージを確認する

CDIのデプロイメントを確認する

```
oc get pods -n openshift-cnv | grep cdi-uploadproxy
oc logs -n openshift-cnv <cdi-uploadproxy-pod>
```

エラーメッセージを確認する

PVCのデプロイメント

`--size` パラメータを `qcow2` フォーマットに設定する

```
qemu-img info image.qcow2 | grep "virtual size"
```

エラーメッセージを確認するPVCのデプロイメントを確認する

```
oc delete pvc fedora-imported-disk -n imported-vms

Retry upload with larger --size parameter
virtctl image-upload pvc fedora-imported-disk \
 --image-path=./image.qcow2 \
 --size=20Gi \
 --namespace=imported-vms \
 --insecure
```

VM

VM

```
oc describe vm fedora-imported-vm -n imported-vms
oc get events -n imported-vms --field-selector involvedObject.name=fedora-imported-vm
```

```
virtctl console fedora-imported-vm -n imported-vms
```

: \* - \* - VM **machine.type** \* -  
-

```
Delete the VM
oc delete vm fedora-imported-vm -n imported-vms

Delete the PVC
oc delete pvc fedora-imported-disk -n imported-vms

Delete the namespace
oc delete namespace imported-vms
```

: PVC

- OpenShift Virtualization **qcow2**
- **virtctl image-upload pvc** **qcow2**
- VM
- 
- HTTP, ContainerDisk, DataVolumeTemplate
- 
- 

**qcow2** **KVM** **OpenShift Virtualization**

## 必要項目

- [OpenShift Virtualization](#) - OpenShift VM管理コンテナ
- [IPアドレス](#) - VMのIPアドレス
- [SSH接続](#) - OpenShift VMへの接続

## 手順

- [Containerized Data Importer \(CDI\)](#)
- [virtctl](#) のインストール

## インストールと設定

### 前提

OpenShift Virtualization

Virtualizationは、OpenShift VM管理コンテナのインストールと設定を簡便にするためのツールです。VM管理コンテナは、VMのインストールと設定を簡便にするためのツールです。

### インストール

OpenShift Virtualizationのインストールは、OpenShift VM管理コンテナのインストールと設定を簡便にするためのツールです。VM管理コンテナは、VMのインストールと設定を簡便にするためのツールです。

OpenShift Virtualizationのインストールは、OpenShift VM管理コンテナのインストールと設定を簡便にするためのツールです。VM管理コンテナは、VMのインストールと設定を簡便にするためのツールです。

### 設定

- OpenShift Virtualizationのインストールは、OpenShift 4.18のインストールと設定を簡便にするためのツールです。
- CLIコマンド: `oc`, `virtctl`, `kubectl`
- OpenShift Virtualizationのインストールは、`lvms-vg1` のインストールと設定を簡便にするためのツールです。
- Linuxのインストールと設定を簡便にするためのツールです。
- cloud-initのインストールと設定を簡便にするためのツールです。

### インストールと設定

OpenShift Virtualizationのインストールは、OpenShift VM管理コンテナのインストールと設定を簡便にするためのツールです。VM管理コンテナは、VMのインストールと設定を簡便にするためのツールです。

### ステップ 1: VMのインストール

OpenShift Virtualizationのインストールは、OpenShift VM管理コンテナのインストールと設定を簡便にするためのツールです。

```
oc create namespace golden-images
```

# ○○○○○○○○Fedora VM○○○○○○○○



```

cloud-init
SSH

```

golden-image-base-vm.yaml □□□□□□

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: base-fedora-vm
 namespace: golden-images
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: base-fedora-vm-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: base-fedora-vm
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}

```



```

machine:
 type: pc-q35-rhel9.4.0
memory:
 guest: 4Gi
networks:
 - name: default
 pod: {}
volumes:
 - dataVolume:
 name: base-fedora-vm-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true
 name: cloudinitdisk

```

### base-fedora-vm.yaml ###

```
oc apply -f base-fedora-vm.yaml
```

VM#####DataVolume#####

```
oc get dv -n golden-images
```

#####

| NAME                | PHASE           | PROGRESS | RESTARTS | AGE |
|---------------------|-----------------|----------|----------|-----|
| base-fedora-vm-disk | CloneInProgress | 45.2%    |          | 2m  |

DataVolume **Succeeded** #####VM#####

```
oc get vmi -n golden-images
```

###

| NAME           | AGE | PHASE   | IP           | NODENAME | READY |
|----------------|-----|---------|--------------|----------|-------|
| base-fedora-vm | 4m  | Running | 10.128.1.151 | ocplab01 | True  |

##: #####305#####



```
删除ID
sudo rm -f /etc/machine-id
sudo touch /etc/machine-id

cloud-init清理日志
sudo cloud-init clean --logs --seed

bash历史
history -c
cat /dev/null > ~/.bash_history

同步
sync
exit
```

**Ctrl+] 退出虚拟机**

## 步骤 4: VM

停止虚拟机

```
virtctl stop base-fedora-vm -n golden-images
```

查看虚拟机状态

```
oc get vm -n golden-images
```

输出

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 15m | Stopped | False |

## 步骤 5: 创建DataSource

创建DataSource，用于从虚拟机镜像创建虚拟机

DataSource配置

**golden-image-datasource.yaml** 文件内容

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataSource
metadata:
 name: fedora-nginx-golden
 namespace: golden-images
spec:
```

```
source:
 pvc:
 name: base-fedora-vm-disk
 namespace: golden-images
```

`fedora-golden-datasource.yaml` を作成

```
oc apply -f fedora-golden-datasource.yaml
```

DataSourceを確認

```
oc get datasource -n golden-images
```

結果

| NAME                | AGE |
|---------------------|-----|
| fedora-nginx-golden | 5s  |

ステップ 6: VMを作成

VMを作成する。VMのディスクは、先ほど作成したDataSourceから取得する。

VMの定義

`webserver-vm-from-golden.yaml` を作成

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-vm-1
 namespace: golden-images
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: webserver-vm-1-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora-nginx-golden
 namespace: golden-images
 storage:
 resources:
 requests:
 storage: 30Gi
```

```

runStrategy: RerunOnFailure
template:
 metadata:
 labels:
 kubevirt.io/domain: webserver-vm-1
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: webserver-vm-1-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 hostname: webserver-vm-1
 fqdn: webserver-vm-1.example.com
 name: cloudinitdisk

```

2000VM0000000

webserver-vm-2.yaml 0000000

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-vm-2
 namespace: golden-images
spec:
 dataVolumeTemplates:

```

```

- apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: webserver-vm-2-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora-nginx-golden
 namespace: golden-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: webserver-vm-2
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: webserver-vm-2-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 hostname: webserver-vm-2
 fqdn: webserver-vm-2.example.com

```

```
name: cloudinitdisk
```

VM

```
oc apply -f webserver-vm-1.yaml
oc apply -f webserver-vm-2.yaml
```

```
oc get dv -n golden-images
```

| NAME                | PHASE           | PROGRESS | RESTARTS | AGE |
|---------------------|-----------------|----------|----------|-----|
| base-fedora-vm-disk | Succeeded       | 100.0%   |          | 15m |
| webserver-vm-1-disk | CloneInProgress | 72.5%    |          | 3m  |
| webserver-vm-2-disk | CloneInProgress | 71.8%    |          | 3m  |

DataVolume VM35

## 7: VM

VM

```
oc get vmi -n golden-images
```

| NAME           | AGE | PHASE   | IP           | NODENAME | READY |
|----------------|-----|---------|--------------|----------|-------|
| webserver-vm-1 | 5m  | Running | 10.128.1.152 | ocplab01 | True  |
| webserver-vm-2 | 5m  | Running | 10.128.1.153 | ocplab01 | True  |

VMnginx

```
virtctl console webserver-vm-1 -n golden-images
```

nginx

```
sudo systemctl status nginx
```

```
curl localhost
```

HTML

**Ctrl+]**

1. VM
- 2.
3. VM
4. DataSource: **fedora-nginx-golden-v2**
5. VM

DataSource - **fedora-nginx-golden-v1 - fedora-nginx-golden-v2 - fedora-nginx-golden-2025-10**

- 
- 
- 
- 
- changelog

- 
- SSH
- 
- cloud-init
- bash

- 
- 
-



## 前提条件

- OpenShift Container Platform 4.7 以降のバージョンがインストールされている
- OpenShift Container Platform 4.7 以降のバージョンがインストールされている
- OpenShift Container Platform 4.7 以降のバージョンがインストールされている
- OpenShift Container Platform 4.7 以降のバージョンがインストールされている
- VM 環境に SSH 接続可能で cloud-init がインストールされている

## 手順

OpenShift Container Platform 4.7 以降のバージョンがインストールされていることを確認します。

## 手順

OpenShift Container Platform 4.7 以降のバージョンがインストールされていることを確認します。

```
VM を削除
oc delete vm webserver-vm-1 webserver-vm-2 base-fedora-vm -n golden-images

DataSource を削除
oc delete datasource fedora-nginx-golden -n golden-images

Namespace を削除
oc delete namespace golden-images
```

## 確認

OpenShift Container Platform 4.7 以降のバージョンがインストールされていることを確認します。 - VM を削除 - VM を削除 - DataSource を削除 - VM を削除 - VM を削除

OpenShift Container Platform 4.7 以降のバージョンがインストールされていることを確認します。

## 参照

- [OpenShift Virtualization Documentation](#)
- [KubeVirt DataVolume Documentation](#)
- [Containerized Data Importer](#)

# Golden Images のインストール

OpenShift Container Platform 4.7 以降のバージョンがインストールされていることを確認します。 Virtualization  
OS のインストールを完了します。

□□

- [\[Quick Health Checks\]](#)
- [\[Detailed Verification by Step\]](#)
- [\[Common Issues and Solutions\]](#)

□□□□□□□□□□

□□□□□□□

```
oc get namespace golden-images
```

□□□□□□□:

| NAME          | STATUS | AGE |
|---------------|--------|-----|
| golden-images | Active | 5m  |

□□□□□□□□

```
oc get all -n golden-images
```

**VM** □□□ **VMI** □□□

```
oc get vm,vmi -n golden-images
```

**DataVolume** □□□ **PVC** □□□

```
oc get dv,pvc -n golden-images
```

**DataSource** □□□

```
oc get datasource -n golden-images
```

□□□□□□□□□□

□□□□ **1:** □□□**VM**□□□□□□

**VM** □□□□□□□□

```
oc get vm base-fedora-vm -n golden-images
```

Pod:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 2m  | Running | True  |

VM: Pod

```
oc get vmi base-fedora-vm -n golden-images -o yaml
```

Pod: - status.phase: Running - status.interfaces[].ipAddress (VMIP) - status.guestOSInfo.id: fedora (OS)

:

```
status:
 guestOSInfo:
 id: fedora
 kernelRelease: 6.11.4-301.fc41.x86_64
 kernelVersion: '#1 SMP PREEMPT_DYNAMIC Thu Oct 10 16:16:06 UTC 2024'
 name: Fedora Linux
 prettyName: Fedora Linux 41 (Cloud Edition)
 version: '41'
 versionId: '41'
 interfaces:
 - infoSource: domain, guest-agent, multus-status
 interfaceName: eth0
 ipAddress: 10.128.0.168
 ipAddresses:
 - 10.128.0.168
 - fd02::76
 mac: 02:92:c2:00:00:06
 name: default
 queueCount: 1
 phase: Running
```

VM DataVolume

```
oc get dv base-fedora-vm-disk -n golden-images
```

Pod:

| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
|---------------------|-----------|----------|----------|-----|
| base-fedora-vm-disk | Succeeded | 100.0%   |          | 5m  |

□□□VM□ PVC □□□

```
oc get pvc base-fedora-vm-disk -n golden-images
```

Bound: 0

| NAME                | STATUS       | VOLUME        | CAPACITY |
|---------------------|--------------|---------------|----------|
| ACCESS MODES        | STORAGECLASS | AGE           |          |
| base-fedora-vm-disk | Bound        | pvc-abc123... | 30Gi     |
| RWO                 | lvms-vg1     | 5m            |          |

## □□□□ 2: VM □□□□□□□□

VM 000000000000

```
virtctl console base-fedora-vm -n golden-images
```

00000000000000000000000000000000

VM0000:

```
For nginx
systemctl status nginx
nginx -v

For custom configuration
cat /etc/nginx/nginx.conf
ls -la /usr/share/nginx/html/
```

## Cloud-init 的安装 (安装)

```
cat /var/log/cloud-init.log
cat /var/log/cloud-init-output.log
```

### 3: VM 虚拟机

VM 000000000000

```
oc get vm base-fedora-vm -n golden-images
```

□□□□□□:

| NAME | AGE | STATUS | READY |
|------|-----|--------|-------|
|------|-----|--------|-------|

```
base-fedora-vm 10m Stopped False
```

**VM** □□□□□□□□□□

```
oc get vmi base-fedora-vm -n golden-images 2>&1
```

□□□□□□□:

```
Error from server (NotFound): virtualmachineinstances.kubevirt.io "base-fedora-vm" not found
```

□□□□ **4: DataSource** □□□□□

**DataSource** □□□

```
oc get datasource fedora-nginx-golden -n golden-images
```

□□□□□□□:

| NAME                | AGE |
|---------------------|-----|
| fedora-nginx-golden | 1m  |

**DataSource** □□□□□

```
oc get datasource fedora-nginx-golden -n golden-images -o yaml
```

□□□□□□□□□□:

```
spec:
 source:
 pvc:
 name: base-fedora-vm-disk
 namespace: golden-images
```

□□□□ **5:** □□□□□□□□□□□□□□□□ **VM** □□□

□□□□□□□ **VM** □□□

```
oc get vm webserver-vm-1 webserver-vm-2 -n golden-images
```

□□□□□□□:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| webserver-vm-1 | 2m  | Running | True  |
| webserver-vm-2 | 2m  | Running | True  |

□□□□□□ VM □ **DataVolume** □□□

```
oc get dv webserver-vm-1-disk webserver-vm-2-disk -n golden-images
```

□□□□□□:

| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
|---------------------|-----------|----------|----------|-----|
| webserver-vm-1-disk | Succeeded | 100.0%   |          | 2m  |
| webserver-vm-2-disk | Succeeded | 100.0%   |          | 2m  |

**VM** □□□□□□□□□□□□

```
oc get dv webserver-vm-1-disk -n golden-images -o jsonpath='{.spec.sourceRef}'
```

□□□□□□□□ **DataSource** □□□□□□□□□□:

```
{"kind":"DataSource","name":"fedora-nginx-golden","namespace":"golden-images"}
```

**VM** □□□□□

VM□IP□□□□□□□□□□:

```
oc get vmi webserver-vm-1 webserver-vm-2 -n golden-images -o jsonpath='{range .items[*]}.{metadata.name}: {.status.interfaces[0].ipAddress}{"\n"}{end}'
```

□□□:

```
webserver-vm-1: 10.128.0.169
webserver-vm-2: 10.128.0.170
```

□□Pod□□ **nginx** □□□□□□□:

```
oc run test-curl --image=curlimages/curl:latest --rm -it --restart=Never -- curl http://10.128.0.169
```

## □□□□ 6: □□□□□□□□

### PVC □□□□□□

```
oc get pvc -n golden-images
```

□□□□□□□□□□□□□□□□□□□□□□□□VM□□□□□□□□□□□□PVC□□□□□□□□□□:

| NAME                | STATUS | VOLUME        | CAPACITY | ACCESS MODES |
|---------------------|--------|---------------|----------|--------------|
| base-fedora-vm-disk | Bound  | pvc-abc123... | 30Gi     | RWO          |
| webserver-vm-1-disk | Bound  | pvc-def456... | 30Gi     | RWO          |
| webserver-vm-2-disk | Bound  | pvc-ghi789... | 30Gi     | RWO          |

□□□□□□□□□□□□ (LVM□□□)

```
Get the LVM thin pool usage
oc get lvmvolume group -n openshift-storage -o yaml | grep -A 10 status
```

□□□□□□□□□□

□□ 1: □□□VM□ **DataVolume** □ **ImportInProgress** □□□□□□□□

□□:

```
oc get dv base-fedora-vm-disk -n golden-images
NAME PHASE PROGRESS AGE
base-fedora-vm-disk ImportInProgress 10% 10m
```

□□:

```
Check CDI importer pod
oc get pods -n golden-images | grep importer

Check importer logs
oc logs -n golden-images <importer-pod-name>
```

□□□□□□: - □□□ **DataSource** □□□□□ - □□□□□□□□□□□□□□□□□□□□ - □□□□□□□□□□ - □□□□□□□□□□□□

□□□:

1. □□□ **DataSource** □□□□□□□□□□□□□□□□:

```
oc get datasource fedora -n openshift-virtualization-os-images
```

2. CDI `openshift-cnv` namespace:

```
oc get pods -n openshift-cnv | grep cdi
```

3. `golden-images` namespace:

```
oc get pvc -n golden-images
oc get lvmvolumegroup -n openshift-storage
```

## 2: VM `base-fedora-vm` - `DataVolume` object

Check:

```
oc get vm base-fedora-vm -n golden-images
```

| NAME           | AGE | STATUS                  | READY |
|----------------|-----|-------------------------|-------|
| base-fedora-vm | 5m  | WaitingForVolumeBinding | False |

Check:

```
Check DataVolume status
oc get dv base-fedora-vm-disk -n golden-images

Check events
oc get events -n golden-images --sort-by='.lastTimestamp' | grep -i error
```

Check: `base-fedora-vm` - `DataVolume` `PendingPopulation` status - `base-fedora-vm-disk` - `PVC` status

Check:

1. `WaitForFirstConsumer` status - `base-fedora-vm-disk` `DataVolume` `base-fedora-vm` status

2. `base-fedora-vm-disk` `PVC` status:

```
oc get storageclass
```

3. `base-fedora-vm-disk` `PVC` status:

```
oc get storageclass -o
jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default-
class=="true")].metadata.name}'
```



□□:

□□:

□□□:

2.

## 00: 0000000000000000 VM 0 nginx 0000000000000000

00:

□□□□□□: - DataSource □□□□□□VM□□□□□□□□□□□□□□ - DataSource □□□□□ PVC □□□□□□□□ - VM□□□□□□□□□□□□

000:

588

```
virtctl console base-fedora-vm -n golden-images
Check manually for installed software
```

2. DataSource `base-fedora-vm-disk` PVC `base-fedora-vm-disk`:

```
oc get datasource fedora-nginx-golden -n golden-images -o jsonpath='{.spec.source}'
```

PVC `base-fedora-vm-disk` `base-fedora-vm-disk`

3. `base-fedora-vm-disk` PVC `base-fedora-vm-disk` DataSource `base-fedora-vm-disk`

**5: DataSource `base-fedora-vm-disk` PVC `base-fedora-vm-disk`**

`base-fedora-vm-disk`: DataSource `base-fedora-vm-disk` VM `base-fedora-vm-disk`

`base-fedora-vm-disk`:

```
oc get datasource fedora-nginx-golden -n golden-images -o yaml
```

`spec.source.pvc.name` `base-fedora-vm-disk` `base-fedora-vm-disk`

`base-fedora-vm-disk`:

1. `base-fedora-vm-disk` PVC `base-fedora-vm-disk` DataSource `base-fedora-vm-disk`:

```
oc delete datasource fedora-nginx-golden -n golden-images
oc create -f fedora-golden-datasource.yaml
```

**6: `base-fedora-vm-disk`**

`base-fedora-vm-disk`:

```
Failed to create PVC: insufficient storage
```

`base-fedora-vm-disk`:

```
Check LVM volume group usage
oc get lvmvolumegroup -n openshift-storage -o yaml

Check PVC requests vs available
oc get pvc -n golden-images
```

`base-fedora-vm-disk`:

1. 削除 PVC を確認する:

```
oc delete pvc <unused-pvc> -n golden-images
```

2. LVM を確認する (確認する)
3. VM/DataVolume を確認する

ステップ 7: VM を確認する

ステップ: 確認する VM の DataVolume が **Succeeded** であることを確認する

ステップ:

```
Check DataVolume progress
oc get dv webserver-vm-1-disk -n golden-images -w

Check CDI pod logs
oc logs -n golden-images -l app=containerized-data-importer --tail=100
```

ステップ: - 確認する - 確認する - 確認する (例: DataSource を確認する)

ステップ:

1. 確認する DataSource を確認する:

```
spec:
 sourceRef:
 kind: DataSource
 name: fedora-nginx-golden
```

2. 確認する (例: LVM を確認する)
3. VM を確認する

確認する

確認する

```
Everything in namespace
oc get all,dv,pvc,datasource -n golden-images

With more details
oc get all,dv,pvc,datasource -n golden-images -o wide

Watch for changes
oc get vm,vmi,dv -n golden-images -w
```

□ □ □ □ □ □ □

```
Recent events
oc get events -n golden-images --sort-by='.lastTimestamp' | tail -20

VM events
oc describe vm base-fedora-vm -n golden-images | tail -20

CDI operator logs
oc logs -n openshift-cnv -l app=cdi-operator --tail=50
```

□ □ □ □ □ □ □ □

```
PVC details
oc describe pvc <pvc-name> -n golden-images

Storage class configuration
oc get storageclass -o yaml

LVM status (if using LVM)
oc get lvmvolume group,lvmvolume groupnode status -n openshift-storage
```

□ □ □ □ □ □ □ □

- OpenShift Virtualization
- KubeVirt - DataVolumes
- Containerized Data Importer (CDI)
- LVM

## VM□□□□□□

11

OpenShift

**Virtualization** VM VM VM  
VM VM VM

## VM□□□□□□

VM VM OpenShift Virtualization 2

- **Web**: VM
- **CLI**: PVC

VM0000000000:

- [OpenShift Virtualization](#)
- [OpenShift Web Console](#)
- [OpenShift CLI \(oc\)](#)
- [OpenShift Virtualization](#)
- [OpenShift Virtualization](#)

### OpenShift Virtualization

OpenShift Virtualization VMs are managed by OpenShift Virtualization

| Component | VMs                                 | OpenShift Virtualization             |
|-----------|-------------------------------------|--------------------------------------|
| OpenShift | OpenShift Virtualization VMs        | OpenShift Virtualization VMs         |
| OpenShift | OpenShift Virtualization cloud-init | OpenShift Virtualization             |
| OpenShift | OpenShift Virtualization            | OpenShift Virtualization             |
| OpenShift | OpenShift Virtualization            | OpenShift Virtualization             |
| OpenShift | OpenShift Virtualization            | OpenShift Virtualization             |
| OpenShift | OpenShift Virtualization            | OpenShift Virtualization Web Console |

### OpenShift

- OpenShift 4.18 OpenShift Virtualization
- OpenShift Web Console CLI (oc, virtctl)
- OpenShift Virtualization
- OpenShift VMs
- PersistentVolumeClaim (PVC)

### OpenShift Virtualization

OpenShift Virtualization VMs are managed by OpenShift Virtualization

### Web Console

Web Console is used to manage OpenShift Virtualization

- OpenShift PVC
- OpenShift Virtualization VMs
- OpenShift Virtualization VMs
- VM CPU

### OpenShift CLI

OpenShift CLI is used to manage OpenShift Virtualization

- OpenShift VM PVC

- 使用PVC创建VM
- 使用云盘创建VM
- 使用镜像创建VM

创建VM的步骤

创建VM的步骤如下：

- 创建PVC：使用CSI驱动创建PVC
- 创建VM：使用PVC创建VM

## Web VM 创建步骤

Web VM 创建步骤如下：

创建VM的步骤

创建VM的步骤

```
oc create namespace vm-cloning-demo
```

nginx VM 创建步骤



cloud-init 配置 SSH 连接

source-webserver-vm.yaml 配置

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: source-webserver
 namespace: vm-cloning-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: source-webserver-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
```

```

template:
 metadata:
 labels:
 kubevirt.io/domain: source-webserver
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: source-webserver-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true
 runcmd:
 - dnf install -y nginx
 - systemctl enable --now nginx
 - echo "<h1>Source Webserver VM</h1><p>Hostname: $(hostname)</p>" >
 /usr/share/nginx/html/index.html
 - firewall-cmd --permanent --add-service=http
 - firewall-cmd --reload
 name: cloudinitdisk

```

□□□VM□□□□□□□□□□

```
oc apply -f source-webserver-vm.yaml
```

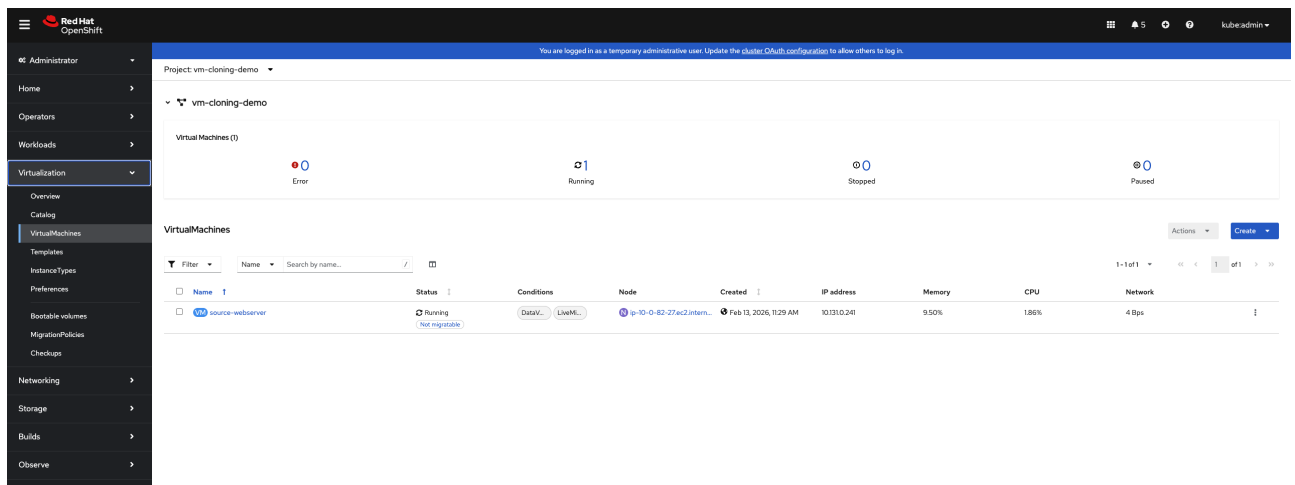
VM

```
oc get vmi -n vm-cloning-demo
```

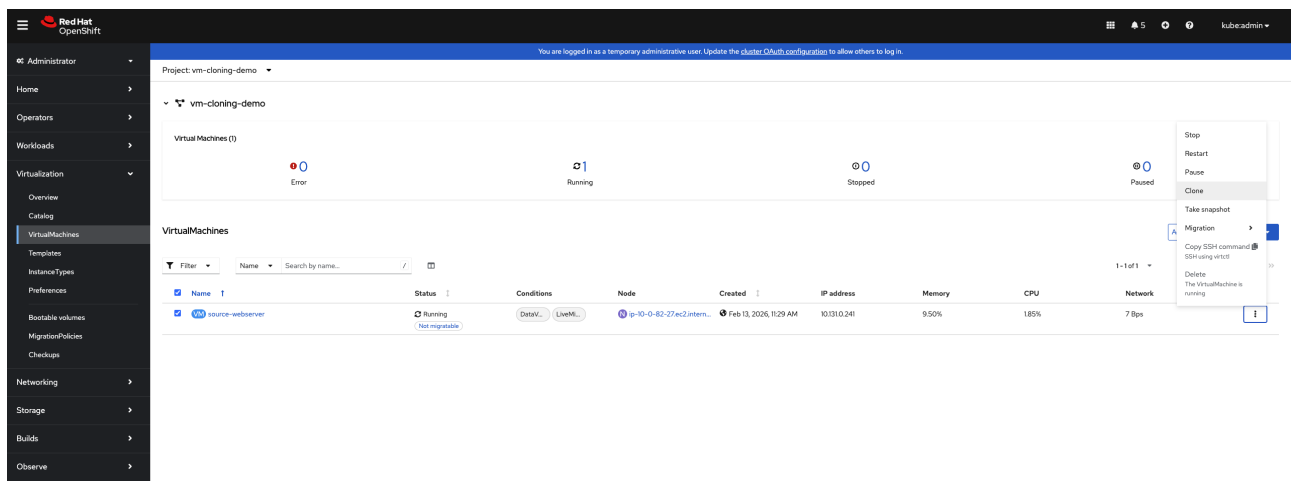
Web

OpenShift Web VM:

1. →



2. VM (source-webserver)
3. VM
- 4.



5. :
  - VM: webserver-clone-1
  - 
  - CPU NIC



6. 仮想マシンをクローンする

Clone VirtualMachine

Name \*

webserver-clone-1

☒ Start VirtualMachine once created

Configuration

Operating system  
Fedora Linux 43 (Cloud Edition)

Flavor  
2 CPU | 4 GiB Memory

Workload profile  
Not available

NICs  
default

Disks  
rootdisk  
cloudinitdisk

Request sent

In process

Created

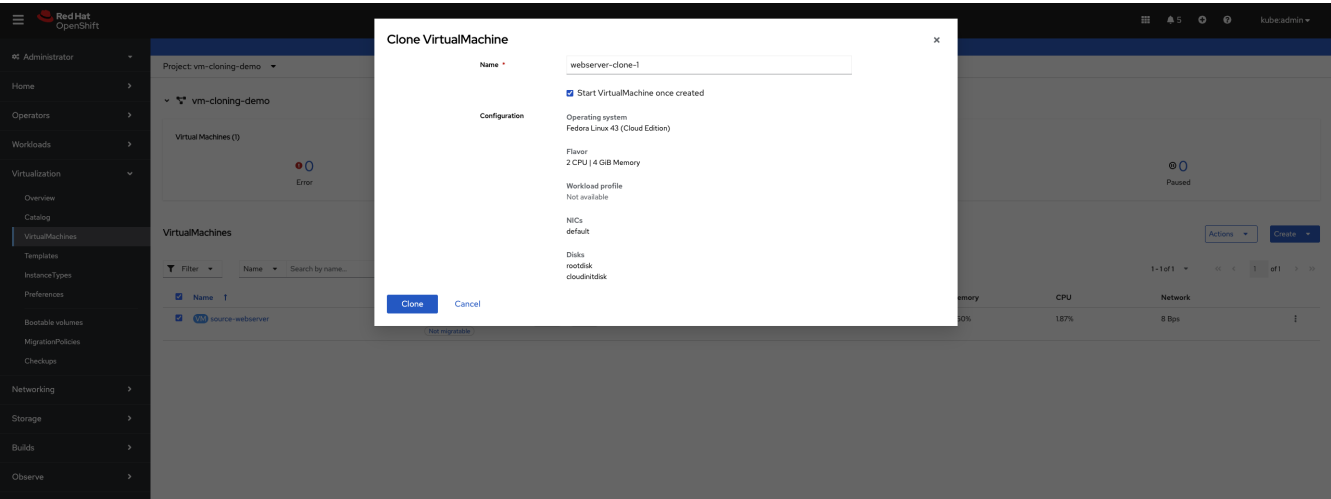
Clone

Cancel

Webコンテナをクローンする

- 仮想VMをPVCにクローン
- 仮想マシンをクローンして仮想VMを作成
- VMのCPUをクローンして仮想VMを作成
- 仮想マシンをクローンして仮想VMを作成

仮想マシンをクローンして仮想VMを作成するPVCをクローンして仮想VMを作成する



Webコンテナをクローンする

仮想マシンをクローンして仮想VMを作成する

```
oc get vm -n vm-cloning-demo
oc get vmi -n vm-cloning-demo
```

~~~~~VM~~~~~

```
virtctl console webserver-clone-1 -n vm-cloning-demo
```

~~~~~VM~~~nginx~~~~~

## CLI~~~~~VM~~~~~

~~~~~CLI~~~~~

1PVC~~~~~VM~~~~~

CLI~~~~~VM~PVC~~~~~

```
oc get pvc -n vm-cloning-demo
```

~~~:

| NAME                  | STATUS       | VOLUME                                   | CAPACITY |
|-----------------------|--------------|------------------------------------------|----------|
| source-webserver-disk | Bound        | pvc-1234abcd-5678-efgh-9012-ijklmnop3456 | 30Gi     |
| ACCESS MODES          | STORAGECLASS | AGE                                      |          |
| RWO                   | lvms-vg1     | 10m                                      |          |

~~~PVC~~~~~DataVolume~~~~~

**clone-datavolume.yaml** ~~~~

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: webserver-clone-2-disk
 namespace: vm-cloning-demo
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true" ①
spec:
 source:
 pvc:
 name: source-webserver-disk
 namespace: vm-cloning-demo
 storage:
 accessModes:
 - ReadWriteOnce
resources:
```

```
requests:
 storage: 30Gi
storageClassName: gp3-csi ②
```

① `WaitForFirstConsumer` ②

② `PVC`



`cdi.kubevirt.io/storage.bind.immediate.requested:` `"true"`  
`WaitForFirstConsumer` `AWS EBS gp3-csi Azure`  
`Disk` `VM` `DataVolume` `PendingPopulation`

`DataVolume`

```
oc apply -f clone-datavolume.yaml
```

```
oc get dv webserver-clone-2-disk -n vm-cloning-demo -w
```

| NAME                   | PHASE                             | PROGRESS | RESTARTS | AGE |
|------------------------|-----------------------------------|----------|----------|-----|
| webserver-clone-2-disk | CloneFromSnapshotSourceInProgress | N/A      |          | 4s  |
| webserver-clone-2-disk | PrepClaimInProgress               | N/A      |          | 68s |
| webserver-clone-2-disk | RebindInProgress                  | N/A      |          | 78s |
| webserver-clone-2-disk | Succeeded                         | 100.0%   |          | 79s |

`DataVolume` `Succeeded` `VM`

`vm-from-cloned-pvc.yaml`

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-clone-2
 namespace: vm-cloning-demo
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
 kubevirt.io/domain: webserver-clone-2
 spec:
 domain:
 cpu:
```

```

 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 persistentVolumeClaim:
 claimName: webserver-clone-2-disk

```

VM

```
oc apply -f vm-from-cloned-pvc.yaml
```

VM

```
virtctl start webserver-clone-2 -n vm-cloning-demo
```

2 VM



1

VM

```
virtctl stop source-webserver -n vm-cloning-demo
```

VM

```
oc get vm source-webserver -n vm-cloning-demo
```

VM

```
oc get vm source-webserver -n vm-cloning-demo -o yaml > clone-base.yaml
```

다음과 같이 clone-base.yaml 파일을 생성합니다

### 1. VM파일:

```
metadata:
 name: source-webserver # VM: webserver-manual-clone
 namespace: vm-cloning-demo
```

### 2. DataVolume파일PVC파일:

```
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: source-webserver-disk # VM: webserver-manual-clone-disk
 spec:
 # VM sourceRef
 #
 source:
 pvc:
 name: source-webserver-disk
 namespace: vm-cloning-demo
 storage:
 resources:
 requests:
 storage: 30Gi
```

### 3. VM파일:

```
spec:
 template:
 metadata:
 labels:
 kubevirt.io/domain: source-webserver # VM: webserver-manual-clone
 spec:
 domain:
 # ...
 volumes:
 - dataVolume:
 name: source-webserver-disk # VM: webserver-manual-clone-disk
 name: rootdisk
```

### 4. VM파일:

```
metadata.uid * metadata.resourceVersion *
metadata.creationTimestamp * metadata.generation * status
```

## 5. MAC

macAddress

```
spec:
 template:
 spec:
 domain:
 devices:
 interfaces:
 - masquerade: {}
 name: default
 macAddress: "52:54:00:xx:xx:xx" #
```

Failed to allocate mac to the vm object: failed to allocate requested mac address

YAML

```
oc apply -f clone-base.yaml
```

## Cloud-init VM

cloud-init

DataVolume2

```
cat <<EOF | oc apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: webserver-custom-disk
 namespace: vm-cloning-demo
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
 source:
 pvc:
 name: source-webserver-disk
 namespace: vm-cloning-demo
 storage:
 accessModes:
 - ReadWriteOnce
```

```
resources:
 requests:
 storage: 30Gi
EOF
```

[illegible]

```
oc get dv webserver-custom-disk -n vm-cloning-demo -w
```

cloud-init VM

cloned-vm-with-cloudinit.yaml

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-custom
 namespace: vm-cloning-demo
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
 kubevirt.io/domain: webserver-custom
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
```

```

persistentVolumeClaim:
 claimName: webserver-custom-disk
- cloudInitNoCloud:
 userData: |-
 #cloud-config
 hostname: webserver-custom
 fqdn: webserver-custom.local
 manage_etc_hosts: true
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true
 runcmd:
 - hostnamectl set-hostname webserver-custom
 - echo "<h1>Customized Clone</h1><p>Hostname: $(hostname)</p>" >
 /usr/share/nginx/html/index.html
 - systemctl restart nginx
 name: cloudinitdisk

```

cloud-init:

- (webserver-custom) VM
- fedora VM
- nginx VM
- VM nginx VM

cloud-init

```

oc apply -f cloned-vm-with-cloudinit.yaml
virtctl start webserver-custom -n vm-cloning-demo

```

cloud-init

```

virtctl console webserver-custom -n vm-cloning-demo

```

VM:

```

hostname
: webserver-custom

systemctl status nginx
nginx VM

curl localhost
HTML

```



[illegible]

VM

□ □ □ □ □ □ □ □ □ □ □ □

```
oc create namespace vm-cloning-production
```

□□□□□□□□□□□□□□□□**RBAC**□□

PVC

## cross-namespace-clone-rbac.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: datavolume-cloner
 namespace: vm-cloning-demo
rules:
- apiGroups: [""]
 resources: ["persistentvolumeclaims"]
 verbs: ["get", "list"]
- apiGroups: ["cdi.kubevirt.io"]
 resources: ["datavolumes"]
 verbs: ["get", "list"]

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: allow-clone-from-demo
 namespace: vm-cloning-demo
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: Role
 name: datavolume-cloner
subjects:
- kind: ServiceAccount
 name: default
 namespace: vm-cloning-production
```

RBAC□□□□□□□□

```
oc apply -f cross-namespace-clone-rbac.yaml
```

■■■■■■■■■■■■■■■■■■■■

■■■■■■■■■■■■■■■■■■■■DataVolume■■■■■■■■■■■■■■■■■■■■

**cross-namespace-clone-datavolume.yaml** ■■■■■■■■

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: webserver-production-disk
 namespace: vm-cloning-production
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
 source:
 pvc:
 name: source-webserver-disk
 namespace: vm-cloning-demo
 storage:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 30Gi
```

DataVolume■■■■■■■■

```
oc apply -f cross-namespace-clone-datavolume.yaml
```

■■■■■■■■■■■■■■■■■■■■

```
oc get dv -n vm-cloning-production -w
```

■■VM■■■■■■■■■■■■■■■■■■■■

**cloned-webserver-production.yaml** ■■■■■■■■

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-production
 namespace: vm-cloning-production
 labels:
 environment: production
spec:
 runStrategy: Manual
 template:
```

```

metadata:
 labels:
 kubevirt.io/domain: webserver-production
spec:
 domain:
 cpu:
 cores: 4
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 8Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 persistentVolumeClaim:
 claimName: webserver-production-disk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 hostname: webserver-prod
 fqdn: webserver-prod.production.local
 manage_etc_hosts: true
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true
 runcmd:
 - hostnamectl set-hostname webserver-prod
 - echo "<h1>Production Webserver</h1><p>Environment: Production</p>" >
 /usr/share/nginx/html/index.html
 - systemctl restart nginx
 name: cloudinitdisk

```

□□□□□□□□:

- □□□□□□
- □□□□□□□□□□4□□□8Gi RAM

- □□□□□□
- □□□□□□□□□□
- □□□□□□□□

[illegible]

```
oc apply -f cloned-webserver-production.yaml
virtctl start webserver-production -n vm-cloning-production
```

□ □ □ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □

□□□□□□□□□□□□□□□□□□□□

- **Storage Provisioning Class (StorageClass):** Provisioning class
  - Ceph RBD, AWS EBS, Azure Disk, GCP PD
  - **Annotation:** `SnapshotForSmartCloneInProgress` → `SmartClonePVCInProgress`
- **Storage Provisioner:** Provisioner
  - Provisioner plugin
  - **Annotation:** `SmartCloneInProgress`

[illegible]

```
oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType
```

□□:

```
cdi.kubevirt.io/cloneType: snapshot # 00000000
000
cdi.kubevirt.io/cloneType: copy # 0000000000
```

□ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- 00000000000000000000000000000000
- 00000000000000000000000000000000
- 000000: 000000 × 000000000000
- 00000000000000000000000000000000

□□□□□□□□□□□□□□□□

```
oc get sc
oc describe sc <storage-class-name>
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- [illegible]

□ □ □ □ □ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □

DataVolume□□□□□□□□□□

```
oc get dv -n vm-cloning-demo
```

Success

| NAME                  | PHASE     | PROGRESS | AGE |
|-----------------------|-----------|----------|-----|
| webserver-custom-disk | Succeeded | N/A      | 5m  |

□ □ □ □ □ □ □ □ □ □ □ □

100

```
oc describe dv <datavolume-name> -n <namespace>
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
Events:
 Type Reason Age From Message
 ---- -
 Normal CloneInProgress 5m datavolume-controller Cloning from vm-cloning-
demo/source-webserver-disk in progress
 Normal CloneSucceeded 2m datavolume-controller Successfully cloned from vm-
cloning-demo/source-webserver-disk
```

```
Events:
 Type Reason Age From Message
 ---- -
 Normal CloneInProgress 5m datavolume-controller Cloning from vm-cloning-
demo/source-webserver-disk in progress
 Normal CloneSucceeded 2m datavolume-controller Successfully cloned from vm-
cloning-demo/source-webserver-disk
```

## VM

```
oc get vmi -n vm-cloning-demo
```

VM

```
virtctl console <vm-name> -n <namespace>
```

VM:

```

hostname

VM
systemctl status nginx

cloud-init
cloud-init status

cloud-init
sudo cat /var/log/cloud-init-output.log

ID
cat /etc/machine-id
```

CloneInProgress

oc: DataVolume CloneInProgress

: \* \* \*

:

```

oc describe dv <datavolume-name> -n <namespace>

CDI
oc get pods -n <namespace> | grep cdi

CDI
oc logs <cdi-pod> -n <namespace>
```



□□□:

```
PVC
oc get pvc -n <source-namespace>

DataVolume
oc get dv <datavolume-name> -n <namespace> -o yaml
```

```
spec.source.pvc.name spec.source.pvc.namespace
```

□□□□□□□□□□**VM**□□□□□□

**00: DataVolume**

□□□□□□: \* PVC□□□□□□□□□□ \* □□□□□□ \* □□□□□□□□□□□□

000:

```
PVC
oc get pvc -n <namespace>

VM
oc describe vm <vm-name> -n <namespace>

VMI
oc describe vmi <vm-name> -n <namespace>
```

□ □ □ □ □ □ □

[illegible]

```
删除VM
oc delete vm --all -n vm-cloning-demo

删除DataVolume
oc delete dv --all -n vm-cloning-demo

删除VM
oc delete vm webserver-production -n vm-cloning-production
oc delete dv --all -n vm-cloning-production

RBAC
oc delete role datavolume-cloner -n vm-cloning-demo
oc delete rolebinding allow-clone-from-demo -n vm-cloning-demo

删除命名空间
oc delete namespace vm-cloning-demo
```



```
oc delete namespace vm-cloning-production
```

□□□□□□□□□□□□□□

```
oc get namespace | grep vm-cloning
```

111

- OpenShift Virtualization VM
- Web CLI
- OpenShift Web VM
- DataVolume CLI
- cloud-init
- RBAC VM
- 
- 
- 
- ID
- 
- VM

# VM OpenShift Virtualization

11

- PVC (Polyvinyl Chloride) VM (Virtual Machine)

VM□□□□□□□□

11

**VM**

# OpenShift Virtualization VM

□ □ □ □

- VM
- Web

- 仮想マシンをVMとして
- 仮想マシンを仮想マシンとして

仮想マシン:

OpenShift 4.20

仮想マシン

- OpenShift VirtualizationをインストールしてOpenShift 4.18
- 仮想マシンを仮想マシンとしてVM
- CSIをインストールして仮想マシンとして
- 仮想マシンを `oc` から `virtctl` CLI

仮想マシンを仮想マシンとして

仮想マシンを仮想マシンとしてCSIをインストールして仮想マシンとして

`oc get volumesnapshotclass`

仮想マシン:

| NAME                                   | DRIVER                             | DELETIONPOLICY | AGE |
|----------------------------------------|------------------------------------|----------------|-----|
| csi-hostpath-snapclass                 | hostpath.csi.k8s.io                | Delete         | 30d |
| ocs-storagecluster-rbdplugin-snapclass | openshift-storage.rbd.csi.ceph.com | Delete         | 30d |

VolumeSnapshotClassをインストールして仮想マシンとして

VMを仮想マシンとして

仮想マシンを仮想マシンとして

OpenShift Virtualizationをインストールして仮想マシンとして2

**VirtualMachineSnapshot**

VMを仮想マシンとして仮想マシンとしてVMを仮想マシンとして

**VirtualMachineSnapshotContent**

仮想マシンを仮想マシンとしてVirtualMachineSnapshotを仮想マシンとしてVolumeSnapshotを仮想マシンとして

仮想マシンを仮想マシンとして仮想マシンとして

| VM 状態   | VM 種類   | VM 名前   |
|---------|---------|---------|
| VM 起動中  | VM 起動中  | VM 起動中  |
| VM 停止中  | VM 停止中  | VM 停止中  |
| VM 起動失敗 | VM 起動失敗 | VM 起動失敗 |



VM 起動失敗の原因は、VM 起動中に発生したエラーによる可能性があります。QEM

VM 起動失敗の原因は、VM 起動中に発生したエラーによる可能性があります。Failed  
thaw VM 起動中に発生したエラーによる可能性があります。VirtualMachineSnapshot spec failureDeadline  
failureDeadline: 10m

VM 起動失敗の原因は、VM 起動中に発生したエラーによる可能性があります。

VM 起動失敗の原因は、VM 起動中に発生したエラーによる可能性があります。status.indications

| VM 状態         | VM 種類    |
|---------------|----------|
| Online        | VM 起動中   |
| GuestAgent    | QEMU 起動中 |
| NoGuestAgent  | QEMU 起動中 |
| QuiesceFailed | VM 起動失敗  |

VM 起動失敗の原因は、VM 起動中に発生したエラーによる可能性があります。

```
oc get virtualmachinesnapshot <snapshot-name> -n vm-snapshots -o json | jq '.status.indications'
```

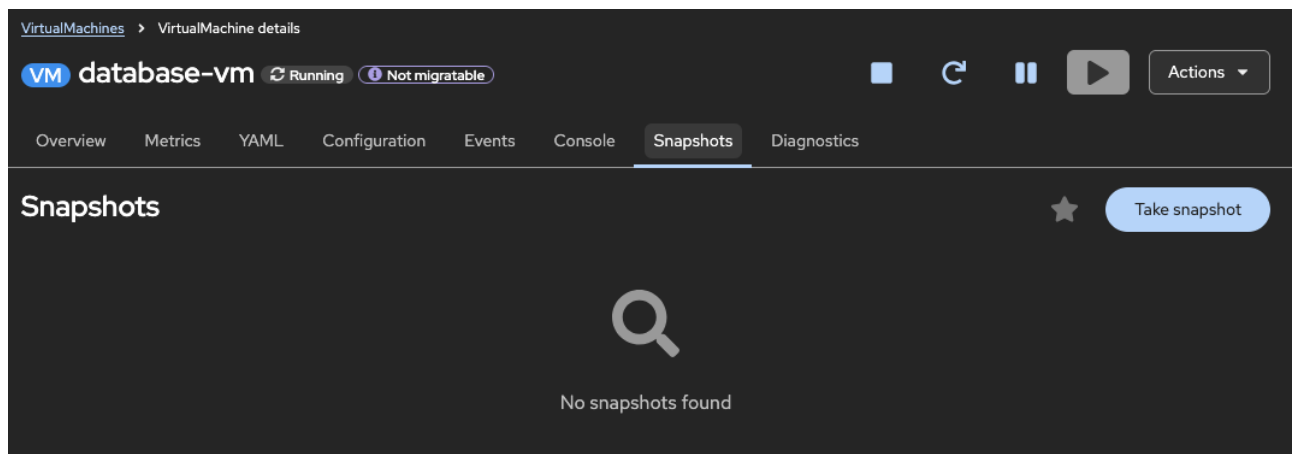
VM 状態:

```
[
 "GuestAgent",
 "Online"
]
```

VM 起動失敗の原因は、VM 起動中に発生したエラーによる可能性があります。

### Web 管理コンソール

1. Virtualization → VirtualMachines
2. VM 起動失敗
3. Snapshots



#### 4. Take snapshot

Take snapshot

×

ⓘ

Taking snapshot of running VirtualMachine.

Name \*

database-vm-snapshot-20260304-155755

Description

Deadline

600

Seconds (s) ▾

▾ Disks included in this snapshot (1)

rootdisk

⚠ The following disk will not be included in the snapshot

cloudinitdisk - Snapshot is not supported for this volumeSource type [cloudinitdisk]

Edit the disk or contact your cluster admin for further details.

[Learn more about snapshots](#)

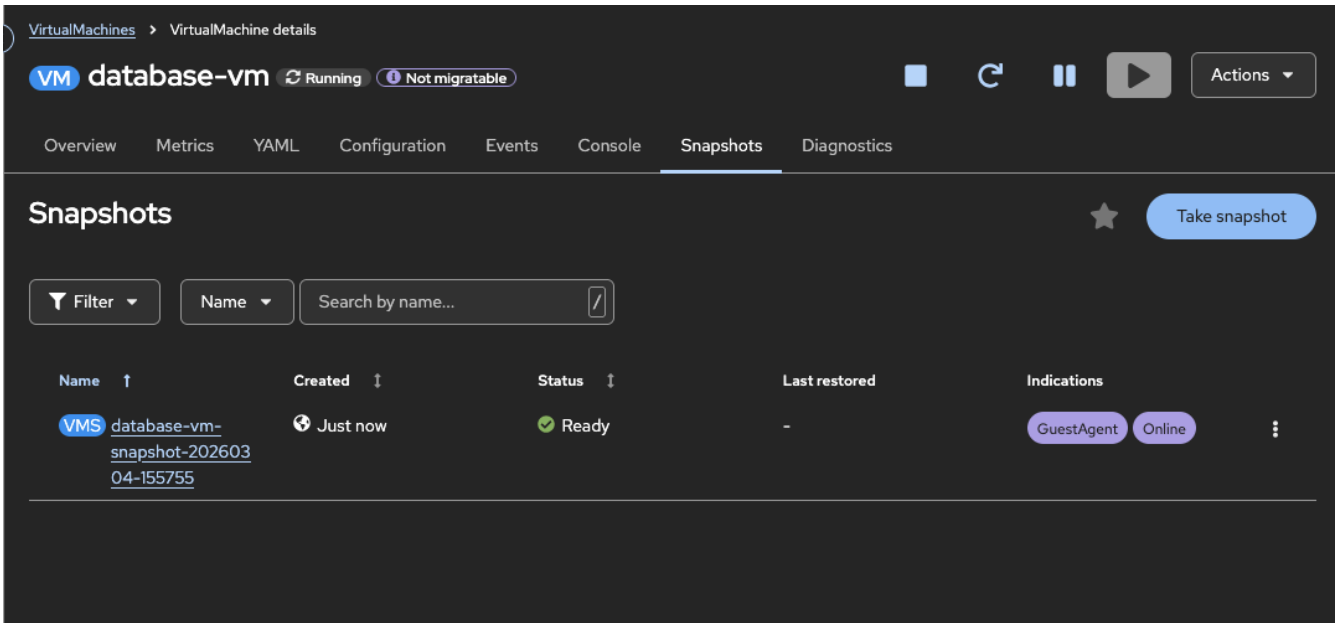
Save

Cancel

5.

6. **Save**

Snapshots



CLI

VirtualMachineSnapshot

vm-snapshot.yaml

```
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
 name: database-vm-snapshot-01
 namespace: vm-snapshots
spec:
 source:
 apiGroup: kubevirt.io
 kind: VirtualMachine
 name: database-vm
```

```
oc apply -f vm-snapshot.yaml
```

```
oc get virtualmachinesnapshot -n vm-snapshots
```

:

| NAME | SOURCEKIND | SOURCENAME | PHASE |
|------|------------|------------|-------|
|------|------------|------------|-------|

| READYTOUSE                           | CREATIONTIME   | ERROR                      |
|--------------------------------------|----------------|----------------------------|
| database-vm-snapshot-20260304-155755 | VirtualMachine | database-vm Succeeded true |
| 8m                                   |                |                            |

□□□□□□□□□□□□□□:

```
oc describe virtualmachinesnapshot <snapshot-name> -n vm-snapshots
```

□□□:

```
Name: database-vm-snapshot-20260304-155755
Namespace: vm-snapshots
API Version: snapshot.kubevirt.io/v1beta1
Kind: VirtualMachineSnapshot
Spec:
 Failure Deadline: 600s
 Source:
 API Group: kubevirt.io
 Kind: VirtualMachine
 Name: database-vm
Status:
 Conditions:
 Reason: Operation complete
 Status: False
 Type: Progressing
 Reason: Ready
 Status: True
 Type: Ready
 Indications:
 GuestAgent
 Online
 Phase: Succeeded
 Ready To Use: true
 Snapshot Volumes:
 Excluded Volumes:
 cloudinitdisk
 Included Volumes:
 rootdisk
```

□□□□□□□□□□□□□□:

- **Phase:** InProgress□Succeeded□□□□ Failed
- **ReadyToUse:** true □□□□□□□□□□□□Web□□□□□□Status□□□ Ready □□□□
- **Error:** □□□□□□□□□□□□□□□□□□□□□□□□





| NAME                                                    | READYTOUSE | CREATIONTIME |
|---------------------------------------------------------|------------|--------------|
| ERROR                                                   |            |              |
| vmsnapshot-content-cd166db8-00be-4aa9-972e-dea669527bfd | true       | 8m           |

```
oc get volumesnapshot -n vm-snapshots
```

🔍:

| NAME                                                            | READYTOUSE |
|-----------------------------------------------------------------|------------|
| SOURCEPVC                                                       | AGE        |
| vmsnapshot-cd166db8-00be-4aa9-972e-dea669527bfd-volume-rootdisk | true       |
| database-vm-disk                                                | 30Gi       |
| lvms-vg1                                                        | 8m         |

📄📄📄📄📄📄📄📄

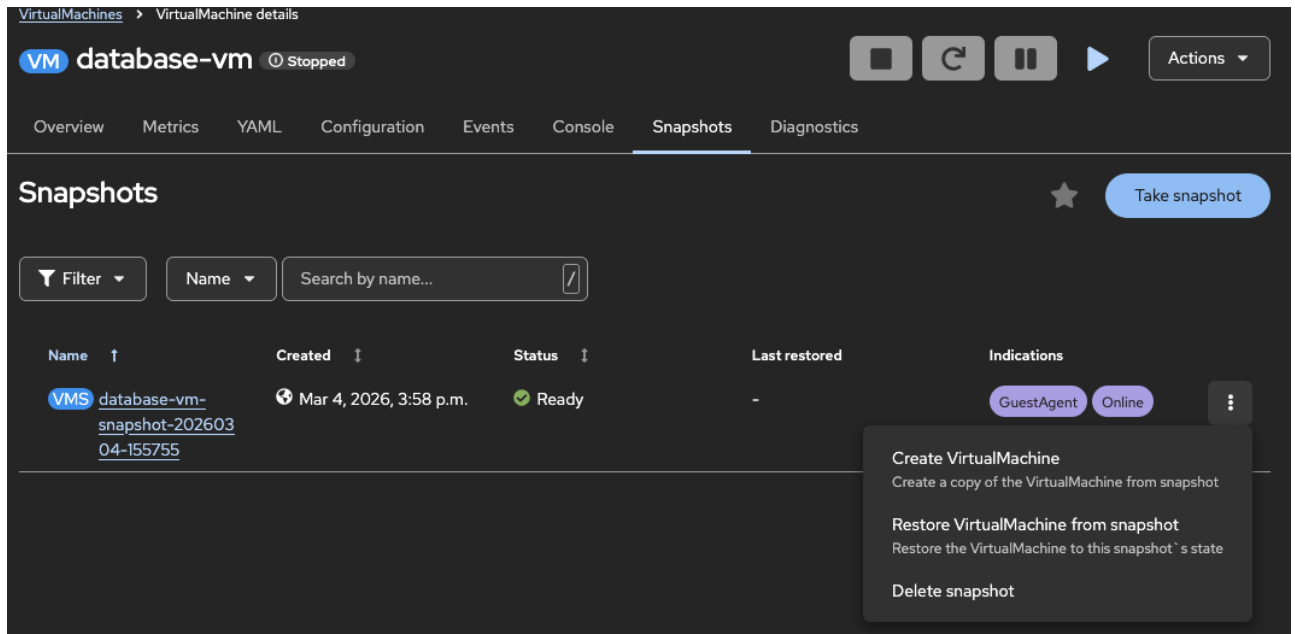
📄📄📄📄📄📄VM📄📄📄📄VM📄📄📄📄📄📄📄📄📄📄📄📄📄📄



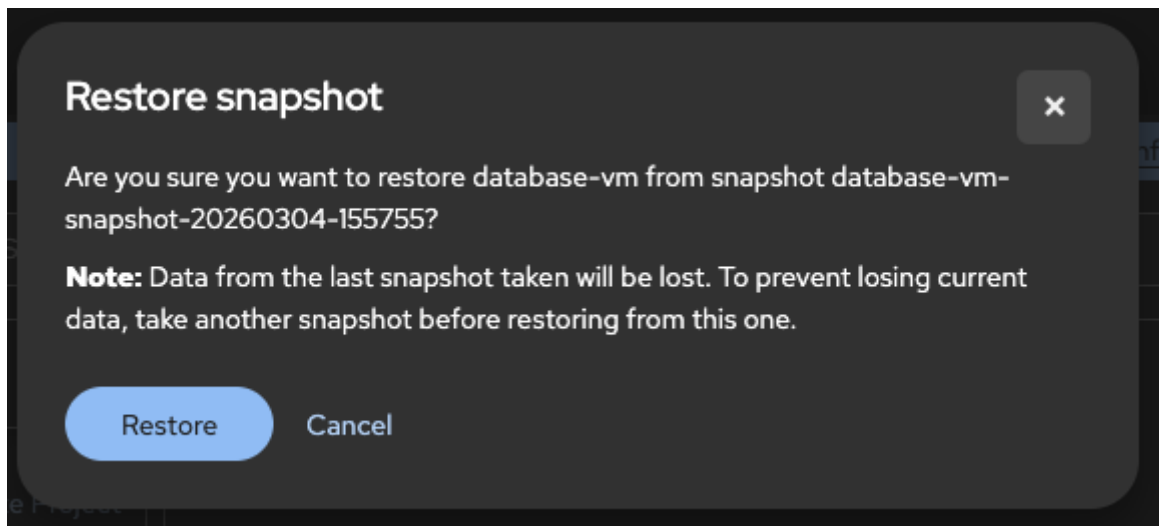
📄📄📄📄📄📄📄📄VM📄📄📄📄📄📄VirtualMachineRestore📄spec📄  
 targetReadinessPolicy: StopTarget 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄  
 FailImmediate📄📄📄📄📄WaitGracePeriod📄📄5📄📄📄📄WaitEventually📄📄📄📄📄📄📄📄  
 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

### Web📄📄📄📄📄📄📄📄

1. VM📄📄📄📄📄📄📄📄
2. VM📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄
3. **Snapshots** 📄📄📄📄📄📄
4. 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄3📄📄📄📄📄📄📄📄
5. **Restore VirtualMachineSnapshot** 📄📄📄📄📄



6. 確認



CLI

VirtualMachineRestore

vm-restore.yaml

```
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineRestore
metadata:
 name: database-vm-restore-01
 namespace: vm-snapshots
spec:
 target:
 apiGroup: kubevirt.io
 kind: VirtualMachine
 name: database-vm
 virtualMachineSnapshotName: database-vm-snapshot-01
```

確認コマンド

```
oc apply -f vm-restore.yaml
```

確認

仮想マシンリストを確認

```
oc get virtualmachinerestore -n vm-snapshots
```

結果:

| NAME                   | TARGETKIND     | TARGETNAME  | COMPLETE | RESTORETIME | ERROR |
|------------------------|----------------|-------------|----------|-------------|-------|
| database-vm-restore-01 | VirtualMachine | database-vm | true     | 2m          |       |

仮想マシンの詳細を確認:

```
oc describe virtualmachinerestore database-vm-restore-01 -n vm-snapshots
```

仮想マシンVMの起動

```
virtctl start database-vm -n vm-snapshots
```

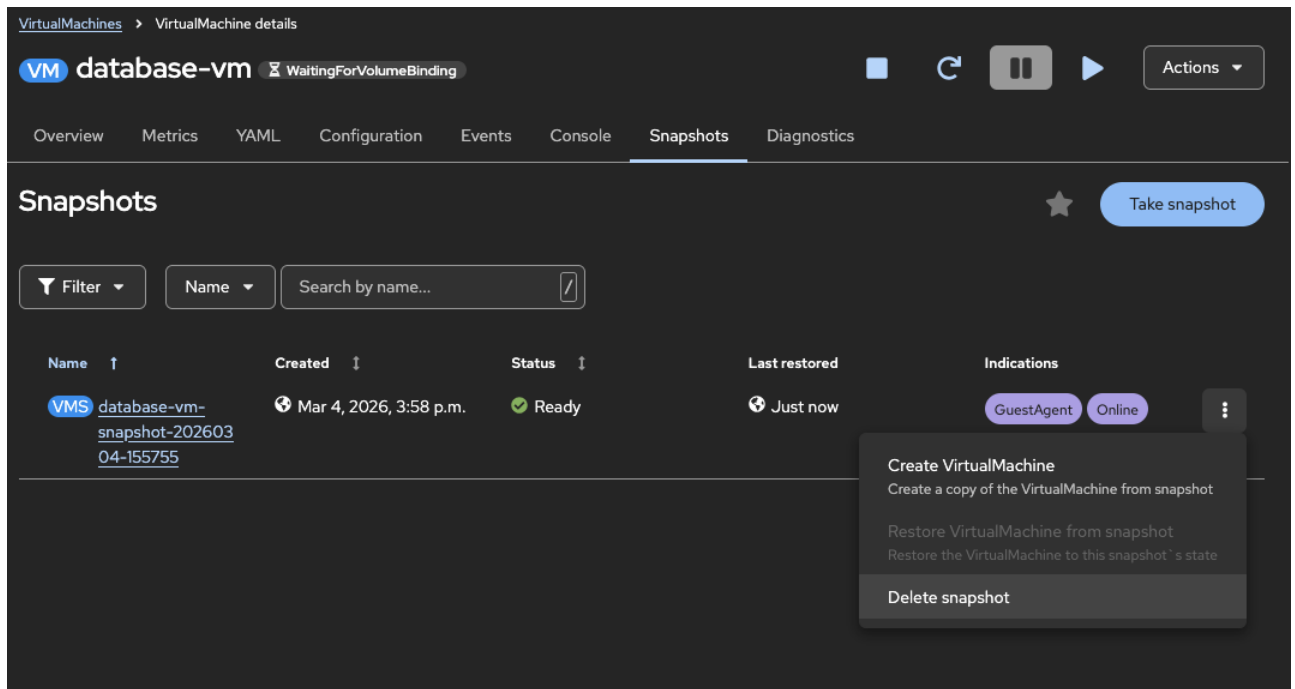
VMの起動を確認

```
oc get vm database-vm -n vm-snapshots
```

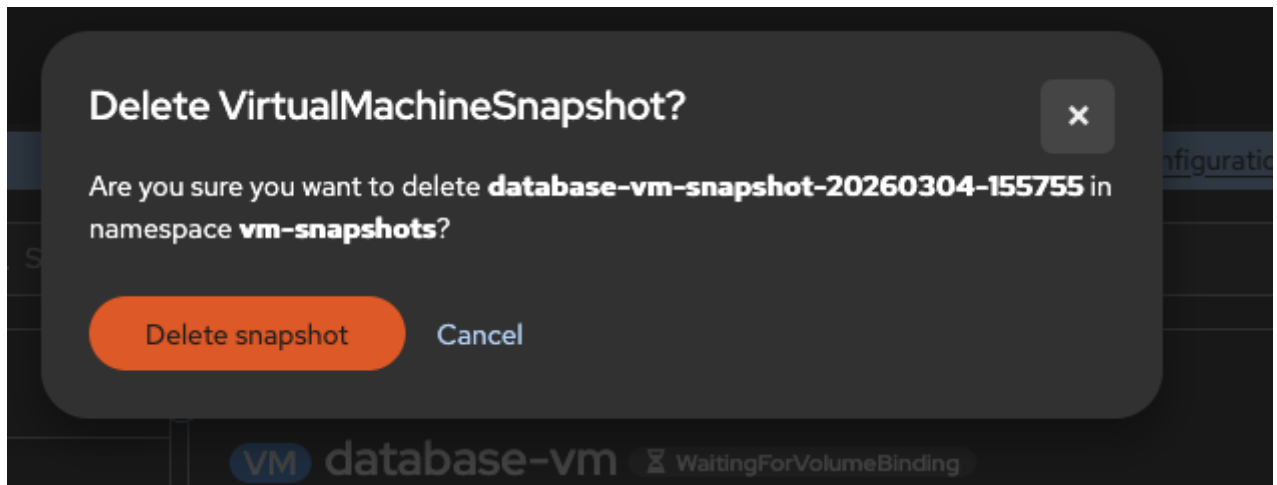
確認コマンド

**Web**操作手順

1. VM **Snapshots** を確認
2. 仮想マシンを再起動
3. **Delete snapshot** を実行



#### 4. 削除



#### CLI

```
oc delete virtualmachinesnapshot database-vm-snapshot-01 -n vm-snapshots
```



VirtualMachineSnapshotはVirtualMachineSnapshotContentとVolumeSnapshotの組み合わせで構成されています。

#### 関連項目

#### CSI

VMはCSI Container InterfaceによってStorage

Storage

- Red Hat OpenShift Data Foundation (ODF)

- AWS EBS CSI
- Microsoft Azure Disk CSI
- Google Cloud Persistent Disk CSI
- LVM Storage (LVMS) OpenShift 4.10 이상에서 지원
- VMware vSphere CSI
- NetApp Trident
- Pure Storage
- Dell CSI OpenShift 4.10 이상에서 지원

## VolumeSnapshotClass

VolumeSnapshotClass는 OpenShift 4.10 이상에서 지원하는 스토리지 클래스를 사용하여 볼륨 스냅샷을 생성하는 데 사용됩니다.

```
스토리지 클래스 확인
oc get storageclass

볼륨 스냅샷 클래스 확인
oc get volumesnapshotclass
```

볼륨 스냅샷 클래스는 스토리지 클래스와 유사하지만, 볼륨 스냅샷을 생성하는 데 사용됩니다.

볼륨 스냅샷

볼륨 스냅샷 클래스는 스토리지 클래스와 유사하지만, 볼륨 스냅샷을 생성하는 데 사용됩니다.

- 볼륨 스냅샷: **CSI** 스토리지 클래스를 사용하여 볼륨 스냅샷을 생성하는 데 사용됩니다.
- 볼륨 스냅샷: **VM** 스토리지 클래스를 사용하여 볼륨 스냅샷을 생성하는 데 사용됩니다.
- 볼륨 스냅샷 클래스: 볼륨 스냅샷을 생성하는 데 사용됩니다.
- 볼륨 스냅샷 클래스: 볼륨 스냅샷을 생성하는 데 사용됩니다.

```
oc get volumesnapshot -n vm-snapshots -o wide
```

결과:

| NAME                                                            | RESTORESIZE | SNAPSHOTCLASS | CREATIONTIME | READYTOUSE |
|-----------------------------------------------------------------|-------------|---------------|--------------|------------|
| SOURCEPVC                                                       |             |               | AGE          |            |
| vmsnapshot-cd166db8-00be-4aa9-972e-dea669527bfd-volume-rootdisk |             |               |              | true       |
| database-vm-disk                                                | 30Gi        | lvms-vg1      | 9m           | 9m         |

볼륨 스냅샷



例: VolumeSnapshotClassを指定する

```
oc get storageclass -o jsonpath='{range
.items[*]}{.metadata.name}{"\t"}{.provisioner}{"\n"}{end}'
oc get volumesnapshotclass -o jsonpath='{range
.items[*]}{.metadata.name}{"\t"}{.driver}{"\n"}{end}'
```

VolumeSnapshotClassのdriverとStorageClassのprovisioner  
を一致させる必要がある

削除

削除コマンド

```
oc delete virtualmachinerestore --all -n vm-snapshots
oc delete virtualmachinesnapshot --all -n vm-snapshots
```

確認

```
oc get virtualmachinesnapshot -n vm-snapshots
oc get volumesnapshot -n vm-snapshots
```

参考

関連リンク

- [OpenShift VirtualizationのVM管理](#)
- [OpenShift Virtualizationのインストールとアップグレード](#)
- [WebコンソールCLI](#)
- [OpenShift Virtualizationのインストール](#)
- [OpenShift VirtualizationのVM管理](#)
- [OpenShift Virtualizationのバックアップと復元](#)
- [OpenShift Virtualizationのデバッグ](#)

参考

- [Virtctl](#) - VM管理CLI
- [OpenShift Virtualization](#) - OpenShift VM管理
- [VM管理](#) - VM管理のベストプラクティス
- [VM管理のベストプラクティス](#) - Red Hat
- [Snapshot Restore API](#) - KubeVirt

# VM

OpenShift  
VirtualizationVMOS

VMBIOSUEFIOpenShift  
Virtualization

- OpenShift VirtualizationOpenShift 4.18
- CLI: `oc`, `virtctl`, `kubectl`
- VM
- 
- : Quay.ioDocker Hub

## OpenShift Virtualization

OpenShift Virtualization

| PersistentVolume Claim (PVC) |                                   | VMOS    |
|------------------------------|-----------------------------------|---------|
| ContainerDisk                |                                   | VMCI/CD |
| DataVolume                   | Containerized Data Importer (CDI) | HTTPS3  |
| CloudInitNoCloud             |                                   | VM      |

OpenShift Virtualization2

### BIOS ( )

- PC BIOS
- 
- MBR2TB
-



- Linux

## UEFI (Unified Extensible Firmware Interface)

- 
- 
- GPT
- Windows 11
- Windows Server
- 

VMKubeVirtbootOrder

:

- bootOrder 12
- 
- bootOrder
- bootOrder: 1

## 1: PersistentVolumeClaim (PVC)

VM

```
oc create namespace boot-demo
```



cloud-initSSH

vm-boot-from-pvc.yaml

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-pvc
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-pvc-disk
```

```

spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
runStrategy: RerunOnFailure
template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-pvc
spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-boot-pvc-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

□□□□□□:

- rootdisk **bootOrder: 1** `rootdisk`
- DataVolume `DataVolume`
- Cloud-init `cloudinit`
- `vm`

□□□□□□:

```
oc apply -f vm-boot-from-pvc.yaml
oc get vm vm-boot-pvc -n boot-demo
oc get vmi vm-boot-pvc -n boot-demo
```

□□□□□□:

- 00000000000000000000000000000000
- 00000000000000000000000000000000VM
- 00000000000000000000000000000000
- 00000000000000000000000000000000

## 2: ContainerDisk

[illegible]

## vm-boot-from-containerdisk.yaml

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-containerdisk
 namespace: boot-demo
spec:
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-containerdisk
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: containerdisk
 bootOrder: 1
```

```

- disk:
 bus: virtio
 name: cloudinitdisk
interfaces:
- masquerade: {}
 name: default
rng: {}
machine:
 type: pc-q35-rhel9.4.0
memory:
 guest: 4Gi
networks:
- name: default
 pod: {}
volumes:
- name: containerdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
- cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

□□□□□□:

- ContainerDisk□□□□□□□□□□VM□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□CI/CD□□□□□□□□□□
- quay.io/containerdisks □□□□□□□□□□□□□□□□□□□□□□

□□□□:

```
oc apply -f vm-boot-from-containerdisk.yaml
```

□□□□□□:

- □□□□□□□□□□
- CI/CD□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

### 例3: HTTP DataVolume

DataVolume HTTP/HTTPS VM

**vm-boot-from-http.yaml**

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-http
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-http-disk
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
 spec:
 source:
 http:
 url:
 "https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2"
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-http
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
```

```

machine:
 type: pc-q35-rhel9.4.0
memory:
 guest: 4Gi
networks:
 - name: default
 pod: {}
volumes:
 - dataVolume:
 name: vm-boot-http-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

□□□□:

- **CDI** `cdi.kubevirt.io/storage.bind.immediate.requested: "true"`
- `cdi.kubevirt.io/storage.bind.immediate.requested: "true"` **PVC** `cdi.kubevirt.io/storage.bind.immediate.requested: "true"`
- `cdi.kubevirt.io/storage.bind.immediate.requested: "true"` **PVC** `cdi.kubevirt.io/storage.bind.immediate.requested: "true"`
- `cdi.kubevirt.io/storage.bind.immediate.requested: "true"` `cdi.kubevirt.io/storage.bind.immediate.requested: "true"`

□ □ □ □ □ □ □ □ □ □ □ :

```
oc apply -f vm-boot-from-http.yaml
oc get dv vm-boot-http-disk -n boot-demo --watch
```

□□□□□□:

- 物理環境を仮想化する
- 物理環境を仮想化する
- 物理環境を仮想化する
- **Web**環境を仮想化する

□□□□4: □□□□□□□□□□□□□□□□□□

[illegible]

## vm-multiple-boot-devices.yaml

```
apiVersion: kubevirt.io/v1
```

```

kind: VirtualMachine
metadata:
 name: vm-multi-boot
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-multi-boot-primary
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-multi-boot-secondary
 spec:
 source:
 blank: {}
 storage:
 resources:
 requests:
 storage: 20Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-multi-boot
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: primarydisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: secondarydisk
 bootOrder: 2
 - cdrom:
 bus: sata

```

```

 name: installedcdrom
 bootOrder: 3
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-multi-boot-primary
 name: primarydisk
 - dataVolume:
 name: vm-multi-boot-secondary
 name: secondarydisk
 - name: installedcdrom
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

□□□□□□□□□□:

1. **bootOrder: 1** - □□□□□□□□□□□□□□□□□□
2. **bootOrder: 2** - □□□□□□□□□□□□□□□□□□□□□□□□
3. **bootOrder: 3** - CD-ROM□□□□□□□□□□□□□□□□

□□□□:

```
oc apply -f vm-multiple-boot-devices.yaml
```

□□□□□□:

- □□□□□DR□□□□□



- 32位OS
- 64位OS
- 32位OS/64位OS

## 例5: UEFI

BIOSをUEFIに切り替えるVMをWindowsにインストール

**vm-boot-uefi.yaml**

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-uefi
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-uefi-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-uefi
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
```

```
- masquerade: {}
 name: default
 rng: {}
firmware:
 bootloader:
 efi:
 secureBoot: false
machine:
 type: pc-q35-rhel9.4.0
memory:
 guest: 4Gi
networks:
- name: default
 pod: {}
volumes:
- dataVolume:
 name: vm-boot-uefi-disk
 name: rootdisk
- cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk
```

□□□□:

- `firmware.bootloader.efi` ☐ UEFI ☐
- `secureBoot: false` - ☐ OS ☐
- ☐ `pc-q35-rhel9.4.0` ☐ UEFI ☐

□□□□:

```
oc apply -f vm-boot-uefi.yaml
```

□□□□□:

- Windows 11 Windows Server VM
- 
- 2TB
- 
- UEFI Linux

## 例6: UEFIセキュアブートの設定

この例では、UEFIセキュアブートを有効にするための設定を示しています。

**vm-boot-uefi-secureboot.yaml** の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-secureboot
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-secureboot-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-secureboot
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 firmware:
 bootloader:
 efi:
```

```
secureBoot: true
machine:
 type: pc-q35-rhel9.4.0
memory:
 guest: 4Gi
features:
 smm:
 enabled: true
networks:
 - name: default
 pod: {}
volumes:
 - dataVolume:
 name: vm-boot-secureboot-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk
```

□□□□:

- `secureBoot: true` 安全启动
- `features.smm.enabled: true` System Management Mode
- 启动模式: Linux/Windows
- 启动模式: Linux/Windows

□□□□:

```
oc apply -f vm-boot-uefi-secureboot.yaml
```

□□□□□□:

- 環境構築
- Windows 11環境構築
- 環境構築
- 環境構築



```

#####VM#####UEFI#####
#####

```

## 例7: PXE (PXE)

PXE (Preboot Execution Environment)  
仮想マシンOSの起動環境

vm-boot-pxe.yaml

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-pxe
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-pxe-disk
 spec:
 source:
 blank: {}
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-pxe
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 2
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 - bridge: {}
 name: pxe-net
 bootOrder: 1
 rng: {}
```





```

 rng: {}
 firmware:
 bootloader:
 bios:
 useSerial: true
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-boot-menu-disk
 name: rootdisk
 - name: installedcdrom
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

□□□□□□□□□□□□:

```

oc apply -f vm-boot-menu.yaml

VNC□□□□□□□□□□□□□□□□
virtctl vnc vm-boot-menu -n boot-demo

□□□□□□□□□□
virtctl console vm-boot-menu -n boot-demo

```

- □□□□ ESC □□□ F12 □□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□
- **useSerial: true** □□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□□□□□□



## VM操作

```
VM操作
oc get vm -n boot-demo

VM操作
oc get vmi -n boot-demo
```

```
VM操作vm-boot-pvc操作VM操作
oc describe vm vm-boot-pvc -n boot-demo
```

## 操作

```
操作VM操作vm-boot-pvc操作VM操作
oc get vm vm-boot-pvc -n boot-demo -o yaml | grep -A 5 bootOrder

操作操作vm-boot-uefi操作VM操作
oc get vm vm-boot-uefi -n boot-demo -o yaml | grep -A 10 firmware
```

## VM操作

```
操作操作vm-boot-pvc操作VM操作
virtctl console vm-boot-pvc -n boot-demo

VNC操作操作vm-boot-menu操作VM操作
virtctl vnc vm-boot-menu -n boot-demo
```

## 操作

### VM操作:

```
操作
dmesg | head -50

操作
systemd-analyze

操作
journalctl -b

UEFI操作BIOS操作
[-d /sys/firmware/efi] && echo "UEFI" || echo "BIOS"
```

## VM 启动顺序

用户: 如何配置 VM 启动顺序

用户: VM 的 `bootOrder`: 1 如何配置

助手:

- 如何配置 VM 启动顺序
- 如何配置 VM 启动顺序
- VM 启动顺序:

```
oc describe vmi vm-boot-pvc -n boot-demo
```

- 如何配置 VM 启动顺序:

```
virtctl console vm-boot-pvc -n boot-demo
```

用户: 如何配置 VM 启动顺序

用户: `secureBoot: true` 如何配置 VM 启动顺序

助手:

- OS 启动顺序
- 如何配置 `smm.enabled: true` 启动顺序
- 如何配置 VM 启动顺序
- 如何配置 VM 启动顺序

```
VM 启动顺序
oc edit vm vm-boot-secureboot -n boot-demo
secureBoot: true | secureBoot: false
```

用户: **PXE** 启动顺序

用户: PXE 启动顺序 VM 启动顺序

助手:

- PXE 启动顺序
- DHCP 启动顺序
- VM 启动顺序
- 如何配置 PXE 启动顺序 DHCP TFTP 启动顺序
- 如何配置 VM 启动顺序 `bootOrder` 启动顺序







□ □ □ □ □ □

```
#####YAML#####
```



| □□□□             | □  | □□                       |
|------------------|----|--------------------------|
| Storage          | 7  | LVM Operator□□□□□□□□□□   |
| Networking       | 8  | Localnet□UDN□□□□□□□□□□□□ |
| Virtual Machines | 14 | □□□□□□□□□□VM□            |
| Services         | 3  | VM□□□□□□Kubernetes□□□□□  |

## Storage

LVM Storage OperatorXXXXXXXXXXXXXXXXXXXX

| 項目                    | Kind              | 説明                    | インストール方法        | インストールファイル |
|-----------------------|-------------------|-----------------------|-----------------|------------|
| lvm-namespace         | Namespa<br>ce     | openshift-lvm-storage | LVM<br>Operator | YAML       |
| lvm-operatorgroup     | Operator<br>Group | LVM Operator          | LVM<br>Operator | YAML       |
| lvm-subscription      | Subscript<br>ion  | OperatorHub           | LVM<br>Operator | YAML       |
| lvmcluster-with-wipe  | LVMClust<br>er    | forceWipe             | LVM<br>Operator | YAML       |
| lvmcluster-clean-disk | LVMClust<br>er    | LVM                   | LVM<br>Operator | YAML       |
| test-pvc              | PVC               | PVC                   | LVM<br>Operator | YAML       |
| test-pod              | Pod               | PVC                   | LVM<br>Operator | YAML       |

## Networking

localnetUDN

| 名前                           | Kind      | 名前               | 参照先                             | 参照先  |
|------------------------------|-----------|------------------|---------------------------------|------|
| localnet-secondary-namespace | Namespace | localnetのサブネットVM | [networking:localnet-secondary] | YAML |

| 名前                                | Kind      | 説明                                               | 関連リンク                                           | ファイル                 |
|-----------------------------------|-----------|--------------------------------------------------|-------------------------------------------------|----------------------|
| localnet-bridge-mapping-nncp      | NNCP      | localnetとbridge-mappingのlocalnet1のbridge-mapping | <a href="#">[networking:localnet-secondary]</a> | <a href="#">YAML</a> |
| cludn-localnet                    | CUDN      | localnetとClusterUserDefinedNetwork               | <a href="#">[networking:localnet-secondary]</a> | <a href="#">YAML</a> |
| localnet-vlan-namespace           | Namespace | VLAN localnet VM                                 | <a href="#">[networking:localnet-vlan]</a>      | <a href="#">YAML</a> |
| localnet-vlan-bridge-mapping-nncp | NNCP      | VLANとbridge-mappingのlocalnet-vlanのbridge-mapping | <a href="#">[networking:localnet-vlan]</a>      | <a href="#">YAML</a> |
| localnet-vlan-100-nad             | NAD       | VLAN 100 localnetのNetworkAttachmentDefinition    | <a href="#">[networking:localnet-vlan]</a>      | <a href="#">YAML</a> |
| udn-primary-namespace             | Namespace | UDN                                              | <a href="#">UDN</a>                             | <a href="#">YAML</a> |
| udn-primary                       | UDN       | VMとUserDefinedNetwork2                           | <a href="#">UDN</a>                             | <a href="#">YAML</a> |

## Virtual Machines

Virtual Machines (VM) の一覧

| 名前                       | Kind           | 説明                  | 関連リンク                                           | ファイル                 |
|--------------------------|----------------|---------------------|-------------------------------------------------|----------------------|
| fedora-localnet-vm       | VirtualMachine | localnetとFedora VM  | <a href="#">[networking:localnet-secondary]</a> | <a href="#">YAML</a> |
| fedora-vlan-vm           | VirtualMachine | VLAN 100とFedora VM  | <a href="#">[networking:localnet-vlan]</a>      | <a href="#">YAML</a> |
| fedora-vm-with-udn       | VirtualMachine | UDNとFedora VM       | <a href="#">UDN</a>                             | <a href="#">YAML</a> |
| fedora-dhcp-vm           | VirtualMachine | DHCPとFedora VM      | <a href="#">IP</a>                              | <a href="#">YAML</a> |
| fedora-multi-network-vm  | VirtualMachine | DHCP + IPとFedora VM | <a href="#">IP</a>                              | <a href="#">YAML</a> |
| fedora-vlan-static-ip-vm | VirtualMachine | VLANとIPとFedora VM   | <a href="#">IP</a>                              | <a href="#">YAML</a> |
| golden-image-base-vm     | VirtualMachine | VM                  | <a href="#">Image</a>                           | <a href="#">YAML</a> |
| webserver-vm-from-golden | VirtualMachine | VM                  | <a href="#">Image</a>                           | <a href="#">YAML</a> |
| webserver-vm-2           | VirtualMachine | 2つのVM               | <a href="#">Image</a>                           | <a href="#">YAML</a> |

| 名前                      | Kind           | 説明                        | 参照                                                         | 形式   |
|-------------------------|----------------|---------------------------|------------------------------------------------------------|------|
| fedora-access-vm        | VirtualMachine | SSH接続可能なFedora VM         | <a href="#">[vm-configuration:remote-access-linux-vms]</a> | YAML |
| fedora-vlan-access-vm   | VirtualMachine | SSH接続可能なVLAN Fedora VM    | <a href="#">[vm-configuration:remote-access-linux-vms]</a> | YAML |
| fedora-app-vm           | VirtualMachine | DNS可能なFedora VM           | <a href="#">[vm-configuration:internal-dns-for-vms]</a>    | YAML |
| golden-image-datasource | DataSource     | GoldenImagePVC DataSource | <a href="#">[golden-image-datasource]</a>                  | YAML |

## Services

VMはKubernetesで管理されます。

| 名前                    | Kind    | 説明                      | 参照                                                         | 形式   |
|-----------------------|---------|-------------------------|------------------------------------------------------------|------|
| fedora-vm-web-service | Service | VM Webサービス LoadBalancer | <a href="#">UDN サービス</a>                                   | YAML |
| ssh-nodeport-service  | Service | VM SSH NodePort         | <a href="#">[vm-configuration:remote-access-linux-vms]</a> | YAML |
| headless-service      | Service | DNS Headless            | <a href="#">[vm-configuration:internal-dns-for-vms]</a>    | YAML |

## Usage

インストール

1. YAMLファイルを準備します。
2. 必要なリソースをインストールします。
3. `oc apply -f <filename>.yaml` を実行します。

デバッグ

```
oc apply --dry-run=client -f <filename>.yaml
```



## See Also

- [VM 配置](#)
- [VM 性能优化](#)
- [VM 安全](#)
- [\[vm-configuration:index\]](#)
- [VM 性能优化](#)
- [VM 安全](#)

# API

## 

OpenShift Virtualization API API

API OpenShift Virtualization OpenShift Container Platform

## 

- API
- API
- API
- API
- API
- VM
- VM

## 

- OpenShift Virtualization OpenShift v4.18
- kubeadmin
- REST API HTTP CRUD GET PUT POST PATCH DELETE JSON
- REST API Postman cURL

## 

API

OpenShift Virtualization API

## 

OpenShift Virtualization



- KubeVirt API 00000000
- KubeVirt 00000000

## API □□□□□□□□



OpenShift 版本: v4.19, v4.20



OpenShift Virtualization API  
API

API

OpenShift Virtualization KubeVirt Kubernetes OpenShift (Kubernetes) json yaml API OpenShift Virtualization API

API

[Virtualization](#)
[yml](#)
[KubeVirt](#)
[API](#)
[KubeVirt API reference](#)
[OpenShift](#)

□ □ □ □ □ □ □ □ □ □ □ □ □ □

- **API**
  - VirtualMachines
  - VirtualMachineInstances
  - [VirtualMachineClusterInstances](#)
  - VirtualMachineClusterInstanceTypes
  - VirtualMachineInstanceTypes
  - VirtualMachinePreferences [VirtualMachineClusterPreferences](#)
  - PreferredCPUTopology
  - vCPU SpreadOptions
- **Network API**
  - Networks
  - Interfaces
  - [NetworkInterfaceAttachments](#)
  - Multus CNI [Network](#)
  - NMState Operator

- [Storage API](#)
  - [PersistentVolumes](#)
  - [PersistentVolumeClaims](#)
  - [VolumeSnapshots](#)
  - [DataVolumes](#)
  - [DataSources](#)
  - [StorageClasses](#)
  - [VirtualMachineSnapshots](#)
  - [VirtualMachineRestores](#)
- [Virtualization API](#)
  - [VirtualMachineClones](#)
  - [Virtual VM Images](#)
  - [Virtual VM Snapshots](#)
  - [Virtualization StorageProfiles](#)
- [Export API](#)
  - [VirtualMachineExports](#)

## VirtualMachine API

### VirtualMachines

VirtualMachine is an API that is implemented by Kubernetes, OpenShift Virtualization, and other Kubernetes distributions. VirtualMachine is a resource that is used to create and manage virtual machines.

VirtualMachine is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.

VirtualMachine is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.

- **instanceType** - VirtualMachine instance type. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.
- **volumes** - List of volumes attached to the VM. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.
  - **dataVolume** - DataVolume resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.
  - **dataVolumeTemplates** - List of DataVolumeTemplates that are used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.
  - **persistentVolumeClaim** - PersistentVolumeClaim resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.
- **networks** - List of networks attached to the VM. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.
- **interfaces** - List of interfaces attached to the VM. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.
- **dataVolumeTemplates** - List of DataVolumeTemplates that are used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines. It is a resource that is used to create and manage virtual machines.

VirtualMachine の spec.runStrategy は

- **Always** - VM が失敗したときに Always として再実行される
- **RerunOnFailure** - Always と同じように再実行される
- **Once** - VM が失敗したときに Once として再実行される
- **Manual** - VM が失敗したときに POST によって再実行される
- **Halted** - spec.runStrategy が Halted である VM は start によって再実行される



spec.running は spec.runStrategy が Always である VM に対してのみ有効である

virtctl は VirtualMachine を start、stop、restart するための CLI ツールである。KubeVirt の VM の status API は

VirtualMachine の runStrategy は

- **start** - VM が Halted であるときに Always、RerunOnFailure、Manual のいずれかで再実行される
- **restart** - VM が Halted であるときに Halted VM が再実行される
- **stop** - VM が Always、Halted、RerunOnFailure、Manual のいずれかで再実行される

API は Manual VM の stateChangeRequests[] は VirtualMachine の

```
status:
 stateChangeRequests:
 - action: <Stop|Start> ①
 uid: <uid-of-the-virtual-machine-instance> ②
```

① (Stop, Start)

② VirtualMachineInstance (VirtualMachine の UID) の UID、VM の VMI の Start

VirtualMachine の API は VM の status は

- **status.printableStatus** - VM の status (Running (Starting), Stopped)
- **status.runStrategy** - spec.runStrategy (Always, RerunOnFailure, Once, Manual, Halted)
- **status.conditions["type": "Ready"]** - conditions: type - Ready, status - Reason, Message -

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
```

```

metadata:
 name: vm-status-example
spec:
 ...
status:
 conditions:
 - lastProbeTime: "2025-11-10T15:00:46Z"
 lastTransitionTime: "2025-11-10T15:00:46Z"
 message: Guest VM is not reported as running
 reason: GuestNotRunning
 status: "False"
 type: Ready

```

## VirtualMachineInstances

VirtualMachineInstance (VMI) is OpenShift Virtualization's representation of a VM instance.

VirtualMachineInstances are managed by Kubernetes Pods. Each Pod contains a QEMU / KVM virtual machine instance. Each VMI is associated with a PersistentVolumeClaim (PVC) for its storage.

VirtualMachine is a Kubernetes resource that represents a VM template. It contains a `spec.template` field that defines the VMI's configuration in YAML (JSON) format.

VMs are managed by VMIs. Each VMI is associated with a VM template. Each VMI is associated with a PersistentVolumeClaim (PVC) for its storage. Each VMI is associated with a DataVolume (DV) for its storage.

VMs are managed by VMIs. Each VMI is associated with a VM template. Each VMI is associated with a PersistentVolumeClaim (PVC) for its storage. Each VMI is associated with a DataVolume (DV) for its storage.

VirtualMachine

OpenShift Virtualization supports various CPU architectures. The default architecture is x86\_64. Other supported architectures include aarch64, ppc64le, and s390x. The architecture is specified in the VMI's `spec.architecture` field. The architecture can be `u1`, `T`, `small`, `medium`, `large`, `xlarge`, `2xlarge`, or `u1.xlarge`. The architecture is specified in the VMI's `spec.architecture` field. The architecture can be `u1`, `T`, `small`, `medium`, `large`, `xlarge`, `2xlarge`, or `u1.xlarge`. The architecture is specified in the VMI's `spec.architecture` field.

OpenShift Virtualization supports various CPU architectures. The default architecture is x86\_64. Other supported architectures include aarch64, ppc64le, and s390x. The architecture is specified in the VMI's `spec.architecture` field.

- **Network (N)** - Supports DPDK for high-performance networking. CPU/RAM ratio is 1:2.
- **Overcommitted (O)** - Supports overcommitting RAM. CPU/RAM ratio is 1:4.
- **Compute Exclusive (CX)** - CPU is dedicated to the VM. CPU/RAM ratio is 1:2.
- **General Purpose (U)** - Supports general-purpose workloads. CPU/RAM ratio is 1:4.
- **Memory Intensive (M)** - Supports memory-intensive workloads. CPU/RAM ratio is 1:8.

VirtualMachineClusterInstanceType is a Kubernetes resource that represents a VM cluster instance type.

## VirtualMachineClusterInstanceTypes

VirtualMachineClusterInstanceType is a Kubernetes resource that represents a VM cluster instance type. OpenShift

Virtualization 虚拟机 VM 配置参数

VirtualMachineClusterInstanceTypes 虚拟机集群实例类型配置参数

- **annotations** - 虚拟机注解
- **cpu** (CPU) - CPU 配置
  - **dedicatedCPUPlacement** - VM 是否 vCPU 独占 CPU 池/物理 CPU 池 **false**
  - **guest** (CPU) - 虚拟机 vCPU 数量 **0**
  - **isolateEmulatorThread** - VMI 是否隔离虚拟机 CPU 池/物理 CPU 池 **false**
  - **maxSockets** - 虚拟机最大 socket 数
  - **model** - VMI 虚拟机 CPU 模型 **host-model**
  - **numa** - 虚拟机 NUMA 配置
    - **guestMappingPassthrough** - 是否 NUMA 映射虚拟机 CPU 池/物理 CPU 池 **{}**
  - **realtime** - virt-launcher Pod 是否实时虚拟机 VMI 配置
    - **mask** - 虚拟机 vCPU 掩码 **"0-3,^1"**, **"1,2,3"**, **"1-2"** 虚拟机 libvirt 配置
- **gpu** - 虚拟机 GPU 配置
  - **deviceName** - 虚拟机名称 **""**
  - **name** (CPU) - 虚拟机 GPU 名称 **""**
  - **tag** - 虚拟机标签
  - **virtualGPUOptions** - vGPU 配置
    - **display** - vGPU 配置
      - **enabled** - vGPU 是否启用 **true**
      - **ramFB** - 是否 OS 使用 GPU 内存 **enabled** - 是否 **true**
- **hostDevices** - 虚拟机主机设备配置
  - **deviceName** (CPU) - 虚拟机设备名称 **""**
  - **name** (CPU) - 虚拟机名称 **""**
  - **tag** - 虚拟机标签
- **memory** (CPU) - 虚拟机内存配置
  - **guest** (CPU) - 虚拟机 OS 内存 **"4096Mi"** 或 **"6Gi"** 内存
  - **hugepages** - 虚拟机 VMI 大页内存配置
    - **pageSize** - 虚拟机大页内存大小 **amd64** 虚拟机 **1Gi** 或 **2Mi** 内存
  - **maxGuest** - 虚拟机最大内存 **maxGuest** 或 **guest** 内存
  - **overcommitPercent** - 虚拟机内存超卖百分比 virt-launcher Pod 配置 **0**
- **nodeSelector** - 虚拟机节点选择器 Kubernetes 配置

◦ **schedulerName** - Kubernetes K8s  
schedulerName

## VirtualMachineInstanceTypes

VirtualMachineInstanceType VirtualMachineClusterInstanceType  
VirtualMachineInstanceType VirtualMachineClusterInstanceType  
/VirtualMachineInstanceType



**specinstancetype** OpenShift Virtualization Web  
spec.template.spec.domain.cpu spec.template.spec.domain.memory  
CPU VirtualMachine  
VM VirtualMachineInstance  
2

## VirtualMachinePreferences VirtualMachineClusterPreferences

VirtualMachineInstanceTypes VirtualMachinePreferences  
VirtualMachineInstanceTypes VirtualMachinePreferences  
VirtualMachineInstanceTypes VirtualMachineInstanceTypes

VirtualMachineInstanceTypes spec VirtualMachineClusterPreferences

OpenShift Virtualization VirtualMachineClusterPreferences OS OS  
fedora rhel10 win11 OS UEFI BIOS  
OS VM VirtualMachineClusterPreference

SecureBoot EFI VirtualMachineClusterPreference

```
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachineClusterPreference
metadata:
 name: rhel10-custom
spec:
 firmware:
 preferredUseEfi: true
 preferredUseSecureBoot: true
```



VirtualMachinePreference spec VM BIOS preferredUseEfi  
spec.firmware.preferredUseBios SecureBoot UEFI

## PreferredCPUTopology

VirtualMachinePreferences 1 VirtualMachineInstanceType vCPU OS  
CPU CPU vCPU  
VM CPU CPU



preferredCPUTopology

- **sockets** (sockets) - vCPU sockets OS
- **cores** - vCPU cores OS
- **threads** - vCPU threads OS
- **spread** - vCPU spreadOptions
- **any** - vCPU OS

## vCPU SpreadOptions

preferredCPUTopology spread spreadOptions vCPU

spreadOptions 2

- **across** - vCPU
- **ratio** (ratio 2) - N

across

- **SocketsCores** (SocketsCores) - vCPU 1 N
- **SocketsCoresThreads** - vCPU 1 N 1 2
- **CoresThreads** (2 vCPU) - vCPU 1 2

preferredCPUTopology VirtualMachinePreference

```
apiVersion:instancetype.kubevirt.io/v1beta1
kind:VirtualMachinePreference
metadata:
 name:cpu-topology-example
spec:
 cpu:
 preferredCPUTopology:spread
 spreadOptions:
 across:SocketsCoresThreads
 ratio:6
```

## API

VirtualMachine VirtualMachineInstances API Multus CNI NMState Operator VM/VMI

VirtualMachine 2 networks interfaces

## Networks

VirtualMachine 对象包含 Pod 配置 (spec.template.spec) 的 networks[] 属性。该属性是一个包含 Kubernetes 网络配置对象的数组。VirtualMachine 对象包含以下属性：

spec.template.spec.networks[] 包含 2 个网络配置对象。

- **pod** - 配置 Kubernetes 网络。
- **multus** - Multus CNI 网络。

## Interfaces

VirtualMachine 对象包含 VM 配置。NIC 配置包含 Pod 配置。VM 配置包含 virtio 配置。Windows 配置包含 OpenShift Virtualization 配置。NIC 配置包含：

VirtualMachine 对象包含 spec.domain.devices.interfaces[] 属性。该属性是一个包含 network 对象的数组。

该属性包含 2 个网络配置对象。

- **name** (字符串) - VirtualMachine 的 spec.template.spec.networks[] 属性中的 network 对象。
- (字符串) - 网络配置对象的 binding 属性：
  - **bridge** - Linux 网络。
  - **sriov** - SR-IOV VF 的 vfio 网络。
  - **masquerade** - NAT 网络。IP 地址由 nftables 配置。
  - **binding** - 网络配置对象的 binding 属性：
    - **name** - 网络配置对象的 name 属性：
      - **l2bridge** - VM interface 网络。network 属性包含 2 个网络配置对象。
      - **passt** (Plug a Simple Socket Transport) - Pod 网络。OpenShift 网络。
- **model** - 网络配置对象的 model 属性 (10/100 网络)：
  - **e1000** - Intel 82545EM PCI 网络。
  - **e1000e** - Intel 82574 PCI-E 网络。
  - **virtio** - Linux 网络。
- **macAddress** - MAC 地址。
- **ports** - VM 网络配置对象的 ports 属性：
  - **name** - 网络配置对象的 name 属性。
  - **port** - 网络配置对象的 port 属性。范围 0 到 65535。
- **pciAddress** - 0000:81:00.1 网络配置对象的 PCI 地址。

VM 网络配置对象的 network 属性。

OpenShift での Pod 設定

OpenShift での Pod 設定は、Kubernetes の Pod 設定と同様ですが、`pod: {}` の代わりに `masquerade: {}` を指定する必要があります。

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: default-network-example
 namespace: my-namespace
spec:
 template:
 spec:
 domain:
 devices:
 interfaces:
 - name: default ①
 masquerade: {} ②
 networks:
 - name: default ①
 pod: {} ③
```

① `domain.devices.interfaces` (`spec.template.spec` の中) は、VirtualMachine の `networks` セクションで定義されたネットワークに接続するための設定です。

② IP アドレスを NAT 経由で NIC にマスカレードするための設定です。

③ Kubernetes Pod の IP アドレスを指定するための設定です。

OpenShift での Pod 設定の例

- OpenShift での Pod 設定の例
- OpenShift での Pod 設定の例
- OpenShift での Pod 設定の例
- OpenShift での Pod 設定の例

OpenShift での Pod 設定の例

- VM の IP アドレスを指定するための設定
- OpenShift での Pod 設定の例
- VLAN の設定
- OpenShift での Pod 設定の例

## Multus CNI の設定

Multus は、OpenShift での Pod 設定に CNI を指定するための設定です。Multus は、OpenShift での Pod 設定に CNI を指定するための設定です。

Networks は、OpenShift での Pod 設定に Multus を指定するための設定です。CNI は、OpenShift での Pod 設定に Multus を指定するための設定です。

OpenShiftのネットワーク

- **bridge** - Podのネットワークをbridgeで接続する
- **host-device** - Podのネットワークをhostのネットワークデバイスに接続する
- **ipvlan** - PodにMACアドレスを割り当て、VPCのMACアドレスを共有する
- **macvlan** - PodにMACアドレスを割り当て、VPCのMACアドレスを共有する
- **SR-IOV** - I/Oのオーバーヘッドを削減するPodのネットワーク

OpenShiftはKubernetesのMultusとNetworkAttachmentDefinitionを使用して、Podのネットワークをカスタマイズできます。

OpenShiftのKubernetes clusterのNetwork specにspec.additionalNetworks[]を使用して、NetworkAttachmentDefinitionを指定できます。

## NMState Operator

NMState ("NM" は NetworkManager) は RHEL/Fedora Linux のネットワーク管理のための API です。

NMState Operator は API (NodeNetworkConfigurationPolicy) を Kubernetes Operator として NMState を Kubernetes (または OpenShift) に統合します。

Multus は linux-bridge を Pod の VM のネットワークを管理するための NMState Operator のプラグインです。

Multus は NetworkAttachmentDefinition (NetworkAttachmentDefinition) を Pod の VM の Pod/VM のネットワークを管理するための API です。

## API

OpenShift Virtualization は Kubernetes の Kind を使用して、Virtualization の API を提供します。

Virtualization の API は、Virtualization の API を提供します。

- **VolumeSnapshotContents** - VolumeSnapshotContents の API
- **VirtualMachineSnapshotContents** - VolumeSnapshotContents の API
- **VolumeSnapshotClasses** - StorageClasses の API

OpenShift Virtualization は Kubernetes の API を使用して、Virtualization の API を提供します。

Virtualization の API は、Virtualization の API を提供します。

OpenShift Container Platform 4.7

## PersistentVolumes

PersistentVolume (PV) は Kubernetes の PersistentVolumeClaim (PVC) を Pod にマウントするための PersistentVolume を提供します。ReadWriteOnce、ReadWriteMany、ReadOnlyMany などのアクセスモードをサポートします。

## PersistentVolumeClaims

PersistentVolumeClaim (PVC) は Pod に PersistentVolume を要求するための Kubernetes の PersistentVolumeClaim (PVC) を提供します。PVC は 1:1 の割合で PV にマウントされ、ReadWriteMany アクセスモードをサポートします。

OpenShift は Kubernetes の PersistentVolumes を SAN / NAS として提供します。PersistentVolumes は StorageClasses を使用して管理されます。PersistentVolumes は StorageClasses を使用して管理されます。

OpenShift Virtualization は DataSources と DataVolumes を kube-apiserver PV にマウントする PVC を提供します。VirtualMachines は PersistentVolumes を使用して管理されます。

StorageClasses は PersistentVolumeClaim を dataSource で管理します。VolumeSnapshot は PVC を dataSource で管理します。PVC は dataSource を使用して管理されます。

## VolumeSnapshots

VolumeSnapshots は PersistentVolume を VM にマウントする PersistentVolume を提供します。

VolumeSnapshots は VirtualMachine に VirtualMachineSnapshots を提供します。KubeVirt は DataVolume を DataSource にマウントする PersistentVolume を提供します。

## StorageClasses

StorageClass は PersistentVolumeClaim を PersistentVolumes にマウントする OpenShift の Kubernetes を提供します。

StorageClasses は CSI を使用して OpenShift を提供します。Amazon Elastic Block Store API (PV と PVC) を提供します。

StorageClasses は OpenShift Virtualization を提供します。OpenShift Virtualization は PersistentVolumes を提供します。

## DataVolumes

DataVolumes は PersistentVolumeClaims を参照するオブジェクトです。

DataVolumes は VM を参照するオブジェクトです。PVC は DataVolumes を参照するオブジェクトです。Containerized Data Importer (CDI) は DataVolumes を参照するオブジェクトです。

DataVolumes (CDI) は 2 つの形式 (qcow2 または raw) の PVC を参照するオブジェクトです。3 つの形式 (qcow2, raw, vmdk) の PVC を参照するオブジェクトです。

- **blank** - 空の DataVolume
- **web** - Web を参照する DataVolume
- **pvc** - PVC を参照する DataVolume

CDI は DataVolumes を参照するオブジェクトです。

- **blank** - 空の DataVolume
- **gcs** - GCS を参照する DataVolume
- **http** - HTTP/S URL を参照する DataVolume
- **imageio** - ovirt-imageio を参照する DataVolume
- **pvc** - PVC を参照する DataVolume
- **registry** - Registry を参照する DataVolume
- **s3** - AWS S3 (MinIO) を参照する DataVolume
- **snapshot** - VolumeSnapshot を参照する DataVolume
- **upload** - Upload を参照する DataVolume
- **vddk** - VMware を参照する DataVolume

## DataSources

DataSources は PersistentVolumeClaim、VolumeSnapshot、OpenShift、Virtualization、Web を参照するオブジェクトです。Bootable Volumes は DataSources を参照するオブジェクトです。

DataSource **spec** は 1 つのオブジェクトです。

- **source** - **pvc** または **snapshot** を参照する DataVolume
  - **pvc** - **snapshot** - PersistentVolumeClaim を参照する DataVolume
  - **name** - pvc または snapshot の名前
  - **namespace** - pvc または snapshot の名前空間

## VirtualMachineSnapshots

VirtualMachineSnapshot は VM を参照するオブジェクトです。

VirtualMachineSnapshot      VolumeSnapshot      VirtualMachineSnapshot      Status     

VirtualMachineSnapshot      source      VM      1m3600s1h30m      -1h      failureDeadline      5m     

## VirtualMachineRestores

     VM      VirtualMachineRestore     

VirtualMachineRestore      VM      apiGroupkindname      target      VM      virtualMachineSnapshotName     

## API

VM      3     

- VM      PVC      VolumeSnapshots      VM      VirtualMachine
- VirtualMachineSnapshot      VirtualMachine      VM/      MAC
- API      VirtualMachine      VirtualMachineSnapshot      VirtualMachineClone

VirtualMachine      VirtualMachineClones     

## VirtualMachineClones

     API      VirtualMachineClone      VirtualMachine      VM      VM     

     VirtualMachineClone      source      target      VirtualMachine      VM      apiGroupkindname     

     VM     

VM      (labels      annotations)      MAC      BIOS / UEFI      VM      CPU      RAM      VM     

- **labelFilters** -      VM      (metadata.labels      )      VM
- **annotationFilters** -      VM      (metadata.annotations      )      VM
- **template** - VM / VMI **template**      2
  - **labelFilters** -      VM      template.metadata.labels      VM
  - **annotationFilters** -      VM      template.metadata.annotations      VM





```
- '{"op": "replace", "path": "/spec/instanceType/name", "value": "u1.xlarge"}'
```

□□□□ **VM** □□□□□□

# VirtualMachine

[illegible]

{} add

```
apiVersion: clone.kubevirt.io/v1beta1
kind: VirtualMachineClone
metadata:
 name: missing-parent-patch-example
 namespace: my-namespace
spec:
 source:
 apiGroup: kubevirt.io
 kind: VirtualMachine
 name: example-source-vm
 patches:
 - '{ "op": "add", "path": "/spec/template/spec/domain/cpu", "value": {} }'
```



- VirtualMachine の cpu と memory を spec.instanceType で指定
- spec.instanceType は VM の UI から選択可能
- CPU が InstanceTypes にあるもののみで、VirtualMachine[Cluster]Preferences から選べる
- spec.instanceType が VM の cpu と memory を指定する。VM の仕様は AWS ドキュメントを参照。

## StorageProfiles

OpenShift Virtualization cloneStrategy OpenShift/Kubernetes StorageClass  
OpenShift Virtualization CDI StorageClass CSI  
StorageProfile StorageProfile  
StorageClass



```

CDI StorageClass StorageProfile
StorageClass DataVolumes PVC

```

○ OpenShift Virtualization ○ VirtualMachine ○ VM ○ VolumeSnapshot  
 ○ PVC ○ DataVolumes ○

[illegible]

□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□ OpenShift/Kubernetes □□□□□□□□ CSI

このセクションでは、CoW (Copy on Write) の仕組みについて説明します。

AWS EBS の詳細については、[GitHub](#) の [StorageProfile](#) を参照してください。

StorageProfile の cloneStrategy の設定方法について説明します。

- **snapshot** (デフォルト) - 既存の CSI VolumeSnapshotClass を参照して CDI を作成します。
- **copy** - 既存の Pod から Pod へデータをコピーします。
- **csi-clone** - CSI API を介して StorageClass から StorageProfile を作成します。

cloneStrategy を csi-clone に設定する StorageProfile の例です。

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: StorageProfile
metadata:
 name: example-provisioner
spec:
 claimPropertySets:
 - accessModes:
 - ReadWriteOnce
 volumeMode: Filesystem
 cloneStrategy: csi-clone
```

## API

OpenShift Virtualization の API については、[VM API](#) を参照してください。

VM の OS については、[OpenShift API](#) の [IngressRoute](#) または [NodePort](#) を参照してください。



Ingress Route の詳細については、[virtctl port-forward](#) を参照してください。

このセクションでは、

VM の詳細については、[VM](#) の [K8s](#) の [\(x-kubevirt-export-token\)](#) を参照してください。



このセクションでは、12 の詳細について説明します。

このセクションでは、

```

apiVersion: v1
kind: Secret
metadata:
 name: export-token
stringData:
 token: abc098765421def

```

VirtualMachineExport 是 VirtualMachineExport 资源

## VirtualMachineExports

VirtualMachineExport 资源可以导出 VirtualMachine 和 VirtualMachineSnapshot 以及 PersistentVolumeClaim 资源

VirtualMachineExport 资源包含 `tokenSecretRef` 和 `source` 两个属性

VirtualMachine 资源包含 `VM` 属性，`yaml` 属性包含 `VM` 属性，`PVC` 属性包含 `DataVolumes` 属性，`MemoryDump` 属性包含 `VirtualMachineSnapshot` 属性，`PVC` 属性包含 `VM` 属性

VirtualMachineExport 资源包含 `status` 属性，`URL` 属性包含 `VM` 属性

VirtualMachineExport 资源包含 4 个属性

- **raw** - 导出 qcow2 格式
- **gzip** - gzip 压缩格式
- **dir** - PVC 导出为目录，URL 为 `/dir` 或 `/path/to/file.txt`
- **tar.gz** - gzip 压缩 tar 格式，PVC 导出

VirtualMachineExport 资源包含 2 个属性

- **KubeVirt content-type** - PVC 为 KubeVirt 导出，`raw` 或 `gzip` 格式
- **Archive content-type** - PVC 为 KubeVirt 导出，`dir` 或 `tar.gz` 格式

VirtualMachineExport 资源示例

```

apiVersion: export.kubevirt.io/v1beta1
kind: VirtualMachineExport
metadata:
 name: export-example ①
spec:
 tokenSecretRef: export-token ②
 source: ③
 apiGroup: "kubevirt.io"
 kind: VirtualMachine
 name: virtual-machine-example

```

- ① 指定する VirtualMachineExport の名前
- ② 指定する VirtualMachineExport の名前
- ③ 指定する VirtualMachineExport/Kind の kind の名前

指定する VirtualMachineExport の名前

| apiGroup             | kind                   |
|----------------------|------------------------|
| kubevirt.io          | VirtualMachine         |
| snapshot.kubevirt.io | VirtualMachineSnapshot |
| ""                   | PersistentVolumeClaim  |

# Glossary

## OpenShift Virtualization

### CNV

Container Native Virtualization Red Hat OpenShift Virtualization

### NAD

NetworkAttachmentDefinition Pod VM

### NNCP

NodeNetworkConfigurationPolicy NMState (CR)

### UDN

User Defined Network 2 3

### VLAN

Virtual LAN LAN VLAN

### OVS

Open vSwitch

### IPAM

IP Address Management IP IP

### PVC

PersistentVolumeClaim

### DataVolume

VM CDI (CR)

### VMI

VirtualMachineInstance VM

### SCC

SecurityContextConstraint OpenShift Pod

### RBAC

Role-Based Access Control Kubernetes

### Localnet

### Cloud-init

VM

## Virtctl

VM 管理 CLI 工具

### 简介

- [简介](#) - OpenShift Virtualization 简介
- [安装](#) - 安装 OpenShift Virtualization